

# Essential F# 한국어 학습 노트

---

이 문서는 Ian Russell 의 **Essential F#** 을 읽는 한국인 독자를 위한 학습 노트다. 원서의 번역이 아니다. 각 절의 개념을 한국어로 요약하고, 예제 코드는 같은 개념을 보여 주는 새로 작성한 F# 코드로 채웠다. 절 제목마다 원서 인쇄 페이지를 달아 두었으니 원서를 옆에 두고 대조하며 읽을 수 있다.

원서: Ian Russell, *Essential F#*, Leanpub (2023-01-30 판). 원서 자체를 대신하지 않는다. 원서에서 직접 읽어야 하는 대목은 페이지를 가리켜 두었다.

**코드 실행:** 본문의 F# 코드 블록은 `dotnet fsi` 로 실행해 검증한 것이다. 같은 절의 블록들은 위에서 아래로 이어 붙여 하나의 스크립트가 된다. 일부 블록은 실행되지 않는 조각이다 — 시그니처 표기, 일부러 컴파일 오류를 내는 예시, NuGet 패키지나 `namespace` 선언 때문에 스크립트로 돌 수 없는 코드다. 본문에 그때마다 밝혀 두었다.

**환경:** .NET SDK 10.0.111 / F# 10 에서 확인했다. 원서는 2023년 1월 판이므로 그 사이 달라진 것은 실측해 본문에 적었다.

## 차례

---

- 00 - 서문과 시작하기 (원서 pp.1-7)
  - 01 - 도메인 모델링 입문 연습 (원서 pp.8-25)
  - 02 - 함수 (원서 pp.26-38)
  - 03 - `null` 과 예외 처리 (원서 pp.39-51)
  - 04 - 코드 구성과 테스트 (원서 pp.52-62)
  - 05 - 컬렉션 입문 (원서 pp.63-79)
  - 06 - 파일에서 데이터 읽기 (원서 pp.80-89)
  - 07 - 액티브 패턴 (원서 pp.90-102)
  - 08 - 함수형 검증 (원서 pp.103-117)
  - 09 - 단일 케이스 판별 유니온 (원서 pp.118-129)
  - 10 - 객체 프로그래밍 (원서 pp.130-140)
  - 11 - 재귀 (원서 pp.141-152)
  - 12 - 계산 식 (원서 pp.153-165)
  - 13 - Giraffe 로 웹 프로그래밍 시작하기 (원서 pp.166-177)
  - 14 - Giraffe 로 API 만들기 (원서 pp.178-184)
  - 15 - Giraffe 로 웹 페이지 만들기 (원서 pp.185-191)
  - 16 - 맷음말과 다음 걸음 (원서 pp.192-194)
  - 17 - 부록 1: VS Code 에서 솔루션과 프로젝트 만들기 (원서 pp.195-196)
-

# 00 - 서문과 시작하기 (원서 pp.1-7)

---

이 챕터는 F# 문법을 가르치지 않는다. 책이 무엇을 겨냥하는지, 열다섯 챕터가 어떤 순서로 쌓이는지, 그리고 코드를 실제로 돌려 보려면 무엇을 설치해야 하는지를 알려 주는 안내문이다. 여기서 얻어 갈 것은 두 가지다. 하나는 챕터 구성 지도이고, 다른 하나는 동작하는 개발 환경이다. 원서는 2023년 1월 판이라 환경 설정 부분의 버전 정보가 지금과 맞지 않는다. 이 노트는 그 부분을 2026년 현재 기준으로 바꿔 적고, 원서와 달라진 점을 함께 표시했다.

## Preface — 이 책이 겨냥하는 것 (원서 pp.1-2)

---

- 대상 독자는 F# 도 함수형 프로그래밍도 처음인 사람이다. C#이나 [VB.NET](#) 경험이 있으면 도움이 되지만 전제 조건은 아니다.
- 저자는 F#을 순수 함수형 언어로 소개하지 않는다. 범주론, 람다 대수, 모나드 같은 이론은 다루지 않겠다고 미리 선을 긋는다. 함수형 개념은 문제를 풀다가 필요해서 딸려 나오는 것으로 취급한다.
- 대신 반복해서 강조하는 표현이 함수 우선(functional-first)이다. 함수형 스타일이 기본값이지만 명령형과 객체 지향도 쓸 수 있고, 실무에서는 F#이 원하는 방식으로 쓰는 편이 순수주의를 고집하는 것보다 오히려 쉽다는 주장이다.
- "F#은 무엇에 좋은가"라는 물음에 저자는 커뮤니티 멤버 Dave Thomas의 답을 인용한다. 웹, 클라우드, 머신러닝, AI, 데이터 과학 같은 목록을 늘어놓기보다 그냥 프로그래밍에 좋다는 쪽이 더 정확하다는 것이다.
- F#은 C#, [VB.NET](#)과 나란히 .NET 플랫폼용 범용 언어다. 2010년부터 Visual Studio에, 이후 .NET SDK에 함께 실려 왔고 그 사이 언어가 크게 흔들리지 않았다. 2010년에 쓴 코드가 지금도 유효할 뿐 아니라 스타일까지 비슷하다는 점을 저자는 안정성의 근거로 든다.
- F#을 고를 이유로 저자가 꼽는 것은 다섯 가지다. 표현력 있는 타입 시스템, 정방향 파이프 연산자(forward pipe operator)로 하는 합성, 패턴 매칭, 컬렉션, 그리고 REPL. 다만 저자는 개별 기능보다 이들이 서로 맞물리는 방식이 F#의 강점이라고 덧붙인다. 다른 언어에 있으니 따라 넣은 기능이 아니라 설계된 기능이라는 뜻이다.

이 다섯 가지는 이후 챕터에서 하나씩 다시 나온다. 타입 시스템은 1챕터와 9챕터, 합성은 2챕터, 패턴 매칭은 1챕터와 7챕터, 컬렉션은 5챕터, REPL은 이 챕터의 F# Interactive(FSI) 절이 각각 담당한다.

## Contents — 열다섯 챕터의 구성 (원서 pp.2-4)

---

- 원서의 뿌리는 저자가 2020~2021년 Trustbit 블로그에 쓴 연재 두 편이다. 책으로 옮기면서 설명을 늘리고, VS Code + Ionide 환경에서 코드가 돌아가도록 손보고, F# 5와 6에서 들어온 기능을 반영했다.
- 다루는 범위는 Trustbit에서 실제로 만드는 종류의 업무용 애플리케이션(LOB, Line of Business)에 필요한 핵심으로 한정된다. 타입과 함수 합성, 패턴 매칭, 테스트에서 출발해 Giraffe 라이브러리로 만든 간단한 웹사이트와 API로 끝난다.
- 원서는 15챕터 뒤에 부록 둘을 더 붙인다. 부록 1은 VS Code에서 솔루션과 프로젝트를 만드는 절차(원서 pp.195-196), 부록 2는 15챕터에서 쓰는 CSS와 JavaScript 코드다.

| 원서<br>챕터 | 원서 제목                                        | 다루는 주제                                                                                                    |
|----------|----------------------------------------------|-----------------------------------------------------------------------------------------------------------|
| 1        | An Introductory Domain Modelling Exercise    | 업무 문제 하나를 잡아 레코드, 판별 유니온, 패턴 매칭으로 도메인을 모델링해 본다                                                            |
| 2        | Functions                                    | 함수의 규칙, 함수 합성, 커링, 부분 적용을 다루고 시그니처를 읽는 법을 익힌다                                                             |
| 3        | Null and Exception Handling                  | <code>Option</code> 과 <code>Result</code> 로 <code>null</code> 오류를 없애고 예외 발생을 줄인다                          |
| 4        | Organising Code and Testing                  | 솔루션과 프로젝트, 네임스페이스와 모듈, 파일 순서를 정리하고 xUnit 과 FsUnit 으로 테스트를 처음 써 본다                                         |
| 5        | Introduction to Collections                  | 컬렉션 함수를 파이프라인으로 이어 데이터를 변환한다. <code>Seq</code> 와 <code>Array</code> 는 종류만 소개하고 실습은 <code>List</code> 로 한다 |
| 6        | Reading Data From a File                     | CSV 파일을 읽어 F# 타입으로 파싱한다. 짧은 챕터                                                                            |
| 7        | Active Patterns                              | 패턴 매칭을 확장해 직접 만든 매처로 코드를 단순하게 만든다                                                                         |
| 8        | Functional Validation                        | 6챕터에서 읽은 데이터에 검증을 붙이고, 계산 식(computation expression)을 처음 다룬다                                               |
| 9        | Single-Case Discriminated Unions             | 원시 타입 의존을 줄이고 도메인 중심 타입으로 바꾼다                                                                             |
| 10       | Object Programming                           | 함수 우선 언어인 F# 이 지원하는 객체 프로그래밍 기능을 살펴본다                                                                     |
| 11       | Recursion                                    | 누적값을 쓴 꼬리 재귀, <code>List.fold</code> 로 다시 쓰기, 퀵소트와 트리 순회를 다룬다                                             |
| 12       | Computation Expressions                      | <code>Option</code> , <code>Result</code> , <code>Async</code> 같은 효과를 다루는 일반적 장치로서 계산 식을 파고든다             |
| 13       | Introduction to Web Programming With Giraffe | Giraffe 와 Giraffe View Engine 으로 API 경로 하나와 웹 페이지 하나를 만든다                                                 |
| 14       | Creating an API With Giraffe                 | 13챕터의 API 부분을 확장한다                                                                                        |
| 15       | Creating Web Pages With Giraffe              | Giraffe View Engine 의 기능을 더 살펴본다. 짧은 챕터                                                                   |

표의 원서 제목은 서문 Contents 절 표기를 따랐다. 9챕터는 본문 제목이 단수형 Single-Case Discriminated Union 이고, 13~15챕터는 본문 제목에서 with 가 소문자다.

읽는 순서를 고르는 데 참고할 만한 갈래는 이렇다. 1~5챕터는 건너뛰기 어려운 토대다. 6챕터와 8챕터는 이어지는 한 묶음이고, 7챕터는 8챕터의 준비물이다. 8챕터에서 맛만 본 계산 식을 12챕터가 제대로 파고들므로 두 챕터는 짝으로 읽는 편이 낫다. 12챕터를 끝낸 뒤 8챕터의 검증 예제로 돌아가 보라는 것은 원서 자신의 권고다(원서 p.153). 13~15챕터는 웹 실습이라 언어 자체를 익히는 것이 목적이면 뒤로 미뤄도 무리가 없다.

## F# Software Foundation, 저자, 감사의 말 (원서 pp.4-5)

- 저자는 F# Software Foundation 가입을 권한다. 무료이며, 가입하면 입문자용 채널이 잘 갖춰진 전용 Slack 에 들어갈 수 있다.
- 저자 소개(Ian Russell, 25년 넘게 일한 영국 개발자, Trustbit 근무)와 감사의 말(리뷰어들, Giraffe 와 Ionide 개발자들, F# 을 만든 Don Syme)은 원서 pp.4-5 에 있다. 요약할 내용은 아니다.

## Getting Started — 환경 설정 (원서 p.6)

원서가 제시하는 순서는 네 단계다. .NET SDK 설치, VS Code 설치, Ionide F# 확장 설치, 그리고 챕터별 폴더 만 들기.

- 원서와 달라진 점: 원서는 출간 당시 최신인 .NET SDK 6.0.x 를 적었다. 2026년 현재 최신 LTS 는 .NET 10 이고, 이 저장소에는 SDK 10.0.111 이 설치돼 있다. F# 언어 버전은 F# 10 이다. 원서 코드는 F# 5~6 기준으 로 쓰였지만 그 사이 F# 은 하위 호환을 지켜 왔으므로 원서의 F# 문법과 코어 라이브러리 예제는 .NET 10 에 서도 그대로 돌아간다. 13~15챕터의 웹 실습만 Giraffe 패키지 버전과 [ASP.NET Core](#) 호스팅 API 가 달라져 있어 그 챕터에서 버전을 맞춰 적는다.
- F# 을 따로 설치하는 일은 없다. .NET SDK 를 깔면 컴파일러와 FSI 가 함께 들어온다.

설치 여부와 버전은 이렇게 확인한다.

```
# 현재 디렉터리에서 쓰이는 SDK 버전 (global.json 이 있으면 그 값)
dotnet --version           # 이 저장소: 10.0.111

# 설치된 SDK 목록과 런타임 목록
dotnet --list-sdks
dotnet --list-runtimes
```

- 편집기는 VS Code 와 Ionide F# 확장 조합이 원서 기준이고 지금도 유효하다. 확장 마켓플레이스에서 `Ionide.Ionide-fsharp` 를 찾아 설치한다. 명령줄에서는 `code --install-extension Ionide.Ionide-fsharp` 로 설치한다. 설치 후 창을 다시 로드해야 할 수 있다.
- Visual Studio 나 JetBrains Rider 를 쓰던 사람은 그쪽을 그대로 써도 된다. 원서 설명은 VS Code 단축키 기 준이므로 다른 편집기라면 FSI 전송 단축키만 각자 환경에 맞게 찾으면 된다.
- 폴더 구성은 원서 방식대로 책 폴더 하나를 만들고 그 안에 챕터별 폴더를 두는 것으로 충분하다. 이 저장소는 노트가 `docs/ko/` 에 있고, 실습 코드는 검증 스크립트가 시스템 임시 경로로 뱉아 실행한다. 뱉아낸 스크립 트를 직접 보고 싶으면 `KEEP=1` 을 붙여 실행하면 경로를 알려 준다.

프로젝트를 만들어 돌려 보는 명령은 다음과 같다. `-lang "F#"` 을 빼면 C# 프로젝트가 생기므로 반드시 붙인다.

```
# F# 콘솔 프로젝트 생성
dotnet new console -lang "F#" -o HelloFSharp

# 실행
dotnet run --project HelloFSharp    # 출력: Hello from F#
```



- 생성된 `.fsproj` 의 `TargetFramework` 는 `net10.0` 이고, `ItemGroup` 안에 `<Compile Include="Program.fs" />` 가 들어 있다. C# 과 달리 F# 프로젝트는 컴파일 대상 파일을 프로젝트 파일에 순서대로 나열한다. 이 순서가 왜 중요한지는 4챕터에서 다룬다.
- 4챕터의 테스트 실습에 필요한 템플릿도 SDK 에 함께 들어 있다. `dotnet new xunit -lang "F#"` 으로 만든다.
- 4챕터는 솔루션 하나에 코드 프로젝트와 테스트 프로젝트를 나눠 담는 구성에서 출발한다. 그 절차는 원서 본문이 아니라 부록 1(원서 pp.195-196)에 있다. `dotnet new sln` 으로 솔루션을 만들고 `dotnet sln add` 로 두 프로젝트를 넣는다. 그다음 테스트 프로젝트에서 `dotnet add reference` 로 코드 프로젝트를 참조하고 `dotnet add package FsUnit.xUnit` 을 더한다.

## F# Interactive (FSI) (원서 p.6)

- FSI 는 F# 의 REPL 이다. C# 의 Interactive Window 와 비슷한 위치이지만 쓰는 방식의 무게가 다르다. 저자는 F# 개발에서 중단점을 걸고 디버깅한 일이 한 번도 없다고 적는다. 코드가 맞는지 확인하는 수단이 디버거가 아니라 FSI 라는 것이다.
- VS Code 에서는 터미널 패널에 F# Interactive 창이 하나 더 생긴다. 여기에 직접 입력할 수도 있지만, 실제 작업은 파일에 쓴 코드를 골라 FSI 로 보내는 방식으로 한다.
- 코드를 골라 `ALT+ENTER` 를 누르면 그 부분이 FSI 로 넘어가 컴파일되고 실행된다. FSI 는 컴파일한 결과를 메모리에 남겨 두므로, 앞서 보낸 정의를 뒤에서 이어 쓸 수 있다.
- FSI 창에 직접 타이핑할 때는 입력 끝에 세미콜론 두 개 `;;` 를 붙여야 실행된다. 파일에서 보내는 경우에는 필요 없다.
- 원서와 달라진 점: 원서는 명령줄 FSI 를 범위 밖이라고 적었지만, 이 저장소의 검증 스크립트는 `dotnet fsi <파일>.fsx` 를 쓴다. `id` 가 붙은 코드 블록은 전부 이 명령으로 실행되므로 알아 두는 편이 낫다.

```
# 스크립트 파일을 명령줄에서 실행
dotnet fsi smoke.fsx

# 표준 입력으로 밀어 넣어 실행 (정의마다 val 줄이 보인다)
dotnet fsi --nologo < smoke.fsx

# 대화형 세션으로 진입 (입력 끝에 ;; 필요, #quit;; 으로 종료)
dotnet fsi
```

아래는 설치 직후 환경이 제대로 동작하는지 확인하는 최소 스크립트다. 파일로 저장해 `dotnet fsi smoke.fsx` 로 돌리거나, VS Code 에서 전체를 골라 `ALT+ENTER` 로 FSI 에 보내거나, 명령 팔레트에서 `FSI: Send File` 을 쓰면 된다.

```
// 이 단위가 보여주는 것: 값 바인딩, 함수 바인딩, FSI 의 정의 유지, 런타임 버전 확인

// 값 바인딩: string
let greeting = "F# 환경 확인 완료"
printfn "%s" greeting // 기대: F# 환경 확인 완료
```

`let` 은 이름과 값을 묶는 바인딩(binding)이다. 매개변수가 없으면 값 바인딩, 하나 이상 있으면 함수 바인딩이다. 아래 `add` 는 매개변수가 둘이라 함수 바인딩이다.

```
// 함수 바인딩: int -> int -> int
let add a b = a + b
printfn "add 19 23 = %d" (add 19 23)    // 기대: add 19 23 = 42
```

- `add` 의 시그니처를 FSI 로 실측하면 `val add: a: int -> b: int -> int` 가 나온다. 타입 주석(type annotation)을 하나도 적지 않았는데 `int` 로 정해진 것은 `+` 의 타입 추론(type inference) 결과다. 화살표가 두 개 보이는 이유, 곧 매개변수가 둘인 함수가 왜 `int -> int -> int` 로 적히는지 2챕터의 커링(currying) 절에서 다룬다.
- FSI 는 대화형 세션에서 값이나 함수를 정의할 때마다 `val <이름>: <시그니처>` 를 되돌려 준다. VS Code 에서 `ALT+ENTER` 로 보낼 때, 그리고 `dotnet fsi` 로 진입한 세션에 직접 입력할 때가 그렇다. 반면 `dotnet fsi <파일>.fsx` 로 스크립트를 실행하면 `printfn` 출력만 나오고 `val` 줄은 나오지 않는다. 스크립트 파일의 시그니처를 보려면 `dotnet fsi --nologo < smoke.fsx` 처럼 표준 입력으로 밀어 넣어 대화형 세션으로 실행한다.
- 이 줄을 읽는 습관이 F# 학습의 절반이다. 노트에서 시그니처를 주석으로 적을 때도 손으로 예상한 값이 아니라 FSI 가 출력한 값을 그대로 옮긴다.

FSI 가 정의를 세션에 남겨 둔다는 것은 이렇게 확인한다. 앞 블록에서 만든 두 정의가 그대로 살아 있다.

```
// 앞 블록의 greeting 과 add 를 그대로 쓸 수 있다. FSI 는 정의를 세션에 남겨 둔다
printfn "%s (%d)" greeting (add 40 2)    // 기대: F# 환경 확인 완료 (42)
```

마지막으로 실행 중인 런타임을 찍어 설치 상태를 확인한다.

```
// 실행 중인 런타임. SDK 와 메이저·마이너 버전이 같은지 눈으로 확인한다
printfn "런타임 = %s" System.Runtime.InteropServices.RuntimeInformation.FrameworkDescription
// 이 저장소 출력 예: 런타임 = .NET 10.0.11 (SDK 10.0.111 과 앞의 10.0 이 같다)
```

위 블록들이 오류 없이 돌아가면 환경 설정은 끝났다. 같은 `id` 를 쓴 블록은 문서 순서대로 이어 붙어 하나의 스크립트로 실행된다.

## Script Files vs Code Files (원서 p.6)

- F# 파일은 두 종류다. 스크립트 파일 `.fsx` 와 코드 파일 `.fs`. 원서는 두 종류를 모두 쓴다.
- 가장 큰 차이는 프로젝트 파일이 필요한지에 있다. `.fsx` 는 혼자 실행된다. `dotnet fsi <파일>.fsx` 로 바로 돌아가고, 필요한 패키지와 다른 스크립트를 `#r "nuget: ..."` 와 `#load "다른.fsx"` 로 자기 안에서 직접 끌어온다. `.fs` 는 의존성을 `.fsproj` 에서 받는다.
- 이 두 지시문(compiler directive)은 컴파일러가 확장자를 보고 막는다. `.fs` 에 `#r` 을 쓰면 `error FS0076` 이 난다. `dotnet fsi` 로 `.fs` 파일을 넘기는 것 자체는 되지만, 그 파일이 모듈이나 네임스페이스 선언으로 시작해야 하고 그렇지 않으면 `error FS0222` 가 난다. `.fsx` 는 그런 선언 없이 그냥 돌아간다.
- 빌드 쪽은 관행의 문제다. SDK 템플릿이 `.fs` 만 `Compile` 항목에 넣기 때문에 보통 `.fsx` 만 `.exe` 나 `.dll` 로 빌드된다. `.fsx` 를 `Compile` 에 직접 적으면 빌드 자체는 된다.
- 어느 쪽 파일이든 코드를 골라 `ALT+ENTER` 로 FSI 에 보내 실행할 수 있다. 파일 확장자가 FSI 전송을 막지는 않는다.
- 실무 감각으로 정리하면 이렇다. 개념을 확인하거나 데이터를 한 번 훑어보는 작업은 `.fsx` 에서 하고, 빌드 해 배포할 코드는 `.fs` 에 넣는다. 이 노트에서 `id` 가 붙은 코드 블록은 모두 `.fsx` 로 뽐혀 검증된다.

지시문의 모양만 미리 보면 이렇다. 13챕터부터 Giraffe 패키지를 끌어올 때 이 두 줄을 쓴다.

```
// .fsx 에서만 쓸 수 있는 지시문. .fs 에 쓰면 error FS0076 이다
#r "nuget: FsUnit.xUnit"
#load "Helpers.fsx"
```

## VS Code 와 Ionide, 그리고 마지막 한마디 (원서 p.7)

- VS Code + Ionide 조합은 Visual Studio 나 Rider 만큼 기능이 많지는 않다. 다만 저자는 F# 코드베이스를 다루는 데 그렇게 많은 기능이 필요하지 않다고 본다. 규모가 큰 코드베이스도 마찬가지라는 것이다.
- Ionide 는 VS Code 확장 하나로 끝나지 않는 프로젝트다. 원서는 Ionide 쪽 문서를 한 번 둘러보라고 권하고, VS Code + F# 단축키를 소개하는 Compositional-IT 의 YouTube 재생목록도 링크로 걸어 둔다.
- 저자가 마지막에 붙인 당부는 하나다. 책에 코드가 많이 나오는데 복사해 붙이지 말고 직접 타이핑하라는 것이다. 손으로 쳐 보는 쪽이 더 많이 남는다는 경험담이다.

이 노트의 코드도 같은 태도로 다루는 편이 좋다. 블록을 그대로 실행해 출력만 확인하고 넘어가기보다, 값을 바꿔 보고 타입을 일부러 틀리게 만들어 컴파일 오류 메시지를 읽어 보는 쪽이 남는 것이 많다.

## 정리

- 이 챕터에서 실제로 챙길 것은 챕터 구성 지도와 동작하는 환경, 이 둘뿐이다. F# 문법은 1챕터부터 시작한다.
- .NET SDK 하나를 설치하면 F# 컴파일러와 FSI 가 함께 들어온다. F# 을 별도로 설치하는 절차는 없다.
- 원서의 .NET 6 기준 서술은 2026년 현재 .NET 10 으로 읽으면 된다. F# 문법과 코어 라이브러리 예제는 하위 호환이 지켜져 그대로 동작하고, 13~15챕터의 웹 실습만 패키지 버전을 맞춰야 한다.
- FSI 는 F# 개발의 기본 도구다. 중단점 디버깅 대신 코드를 골라 FSI 로 보내 확인하는 흐름이 표준이다. VS Code 에서는 `ALT+ENTER` , 명령줄에서는 `dotnet fsi` 다.
- FSI 가 대화형 세션에서 되돌려 주는 `val <이름>: <시그니처>` 줄을 읽는 습관을 여기서부터 들여 두는 것이 좋다. 2챕터부터는 이 줄이 설명의 중심이 된다.
- `.fsx` 는 프로젝트 파일 없이 혼자 돌고 `#r · #load` 지시문을 쓸 수 있다. `.fs` 는 프로젝트에 담아 빌드한다. FSI 전송은 둘 다 된다.

## 원서 대조 표

| 절                                 | 원서 페이지 | 실행 단위                          |
|-----------------------------------|--------|--------------------------------|
| Preface — 이 책이 겨냥하는 것             | pp.1-2 | —                              |
| Contents — 열다섯 챕터의 구성             | pp.2-4 | —                              |
| F# Software Foundation, 저자, 감사의 말 | pp.4-5 | —                              |
| Getting Started — 환경 설정           | p.6    | —                              |
| F# Interactive (FSI)              | p.6    | <code>00-fsi-smoke-test</code> |
| Script Files vs Code Files        | p.6    | —                              |
| VS Code 와 Ionide, 그리고 마지막 한마디     | p.7    | —                              |

## 01 - 도메인 모델링 입문 연습 (원서 pp.8-25)

이 챕터는 이론이나 정의로 시작하지 않는다. 흔한 업무 요구사항 하나를 놓고, 그것을 F# 코드로 옮기는 과정을 처음부터 끝까지 보여 준다. 처음 버전은 순진하다. 참/거짓 플래그 몇 개로 도메인 개념을 표현하고, 그래서 명세상 있을 수 없는 상태까지 코드에서는 만들어진다. 그다음부터가 이 챕터의 본론이다. 레코드(record), 판별 유니온(discriminated union), 패턴 매칭(pattern matching)을 써서 같은 문제를 여러 번 다시 푼다. 그때마다 도메인의 단어가 타입 이름으로 올라오고, 잘못된 상태는 아예 표현할 수 없게 된다. 예제는 F# Interactive(FSI)로 돌려 가며 확인한다.

원서는 고객 등급별 할인 계산을 예제로 쓴다. 이 노트는 같은 구조를 전기차 충전 요금 도메인으로 바꿔 새로 짰다. 코드는 다르지만 각 절이 보여 주는 개념과 개선의 순서는 원서와 같게 맞췄다.

## The Problem — 명세 안에 이미 도메인이 있다 (원서 pp.8-9)

- 원서는 행위 주도 개발(BDD) 스타일로 쓰인 명세 하나를 출발점으로 삼는다. 명세에는 검증용 예시 표가 붙어 있고, 그 안에 도메인 고유의 단어와 개념이 이미 들어 있다.
- 이 노트가 쓸 명세는 전기차 충전소 요금 정산이다.
  - 충전 단가는 1 kWh 당 300원이다.
  - 요금제에 가입한 운전자 중 요금제가 유효한 사람만 감면 대상이다.
  - 감면 대상이 25 kWh 이상 충전하면 요금의 12퍼센트를 감면한다.
  - 비회원은 요금제 자체가 없으므로 감면 대상이 될 수 없다.
- 검증용 예시는 이렇다.

| 운전자    | 상태     | 충전량    | 청구액   |
|--------|--------|--------|-------|
| yujin  | 요금제 유효 | 30 kWh | 7920원 |
| minho  | 요금제 유효 | 20 kWh | 6000원 |
| soyeon | 요금제 만료 | 30 kWh | 9000원 |
| taeho  | 비회원    | 30 kWh | 9000원 |

- 명세를 읽으면 “요금제 가입 / 미가입”, “유효 / 만료” 같은 갈림이 보인다. 이 챕터가 계속 묻는 질문은 하나다. 그 갈림을 코드에서 어떻게 드러낼 것인가.
- 먼저 순진한 해법을 만들고, F# 의 타입 시스템으로 그것을 점점 도메인 중심으로 바꿔 간다. 그 과정에서 버그가 끼어들 자리도 함께 줄어든다.

## Getting Started — 튜플과 레코드로 데이터 모양 잡기 (원서 pp.9-11)

- F# 에는 `string`, `decimal`, `bool` 같은 원시 타입 말고도 대수적 타입 시스템(Algebraic Type System, ATS)이 있다. 작은 데이터 구조를 조립해 큰 구조를 만드는 재료로 보면 된다.
- 재료는 크게 두 종류다. 하나는 여러 값을 동시에 담는 AND 타입이고 튜플(tuple)과 레코드가 여기에 든다. 다른 하나는 여러 경우 중 하나만 담는 OR 타입이고 판별 유니온이 여기에 든다.
- 타입은 `type` 키워드로 정의한다. 타입 이름에는 첫 글자를 대문자로 쓰는 파스칼 표기(Pascal case)를, 그 밖의 대부분에는 첫 글자를 소문자로 쓰는 카멜 표기(camel case)를 쓴다.

```
// 이 단위가 보여주는 것: 튜플과 레코드, 첫 계산 함수, 커링된 매개변수와 튜플 매개변수, FSI 검증

// 타입 약어: string 하나와 bool 두 개를 묶은 AND 타입에 RawDriver 라는 별명을 붙인다
type RawDriver = string * bool * bool

// 타입 표기는 * 로 잇고, 값 표기는 , 로 잇는다. 값 쪽 괄호는 생략해도 된다
let rawYujin = ("yujin", true, true)

// 값 바인딩에 타입 주석을 붙일 수도 있다: RawDriver
let rawTaeho: RawDriver = ("taeho", false, false)
```

`type RawDriver = ...` 는 새 타입을 만드는 것이 아니라 타입 약어(type abbreviation)다. 기존 타입에 별명을 붙이는 것일 뿐이다. `RawDriver` 를 요구하는 자리에 `string * bool * bool` 을 그대로 넣을 수 있고 그 반대도 된다. 별명이 아니라 진짜 다른 타입을 만드는 방법은 9챕터에서 다룬다.

`let` 은 오른쪽 값을 왼쪽 이름에 묶는다. F# 의 데이터에는 기본적으로 불변성(immutability)이 적용되므로 `rawYujin` 을 변수로 생각하지 않는 편이 낫다. 값이 이름에 붙었을 뿐이고 그 값은 바뀌지 않는다.

`let` 은 튜플을 분해하는 데도 쓴다.

```
// 튜플을 세 이름으로 분해한다
let rawSoyeon = ("soyeon", true, false)
let (rawId, rawSubscribed, rawPlanActive) = rawSoyeon
printfn "%s / 가입=%b / 유효=%b" rawId rawSubscribed rawPlanActive // 기대: soyeon / 가입=true / 유효=false

printfn "%A" rawTaeho // 기대: ("taeho", false, false)
```

- 튜플의 문제는 부분에 이름이 없다는 점이다. 두 번째와 세 번째 이름을 뒤집어 `let (rawId, rawPlanActive, rawSubscribed) = rawSoyeon` 으로 묶어도 컴파일은 그대로 된다. `string * bool * bool` 어디에도 어느 쪽이 가입 여부인지 적혀 있지 않으니 컴파일러가 막아 줄 근거가 없다. 결국 뜻이 뒤집힌 값을 그대로 쓰게 되고, 쓰는 사람이 순서를 짐작해야 한다.
- 그래서 실제 모델에는 레코드를 쓴다. 레코드도 AND 타입이지만 각 부분에 이름이 붙는다.

```
// 레코드: 부분마다 이름이 붙은 AND 타입
type Driver = {
    DriverId: string
    IsSubscribed: bool
    IsPlanActive: bool
}
```

한 줄로 적을 수도 있다. 이때는 필드 사이에 세미콜론이 필요하다.

```
type Driver = { DriverId: string; IsSubscribed: bool; IsPlanActive: bool }
```

- 줄을 나눠 쓰면 세미콜론이 필요 없다. 대신 들여쓰기가 맞아야 한다. F# 은 유의미한 공백(significant whitespace)으로 스코프(scope)를 판단하므로 정렬이 문법의 일부다. 컴파일 오류가 났을 때 가장 먼저 볼 곳도 정렬이다.
- 탭 문자는 아예 지원하지 않는다. 탭을 넣으면 경고가 아니라 오류 FS1161 이 나고, 문구는 `#indent "off"` 옵션을 사용하지 않는 한 TAB은 F# 코드에서 허용되지 않습니다. 다. 편집기가 탭을 공백으로 바꾸도록 설정해 두는 것이 좋다.
- 레코드 식은 필드를 하나도 빠뜨릴 수 없어서, 인스턴스를 만들 때 모든 필드를 채워야 한다. 그리고 레코드 필드는 기본이 불변이라 만든 뒤에는 값을 바꿀 수 없다. 필드에 `mutable` 을 붙이면 이 규칙에서 벗어날 수 있지만 이 노트에서는 쓰지 않는다.

```
// 필드를 줄마다 나눠 쓰는 방식. 필드가 많을 때 읽기 좋다
let yujin = {
    DriverId = "yujin"
    IsSubscribed = true
    IsPlanActive = true
}

// 세미콜론으로 한 줄에 쓰는 방식. 타입 주석은 붙여도 되고 생략해도 된다
let minho: Driver = { DriverId = "minho"; IsSubscribed = true; IsPlanActive = true }
let soyeon = { DriverId = "soyeon"; IsSubscribed = true; IsPlanActive = false }
let taeho = { DriverId = "taeho"; IsSubscribed = false; IsPlanActive = false }
```

- 타입 주석을 생략해도 컴파일러가 필드 이름을 보고 `Driver` 로 추론한다. 단, 구조가 똑같은 레코드 타입이 여러 정의돼 있으면 컴파일러는 가장 나중에 선언된 타입을 고른다. 다른 쪽을 원한다면 타입 주석을 붙여야 한다.
- F# 컴파일러는 파일 위에서 아래로 읽는다. 그래서 어떤 이름을 쓰려면 그 이름이 쓰는 자리보다 위에 정의돼 있어야 한다. 프로젝트 단위에서도 같은 규칙이 적용되므로 `.fs` 파일도 알파벳 순이 아니라 의존 순서대로 배열한다. C#이나 Java 를 쓰던 사람에게는 처음에 낯설지만, 이 제약 덕분에 코드를 읽는 순서와 검증하는 순서가 일치한다.

## Getting Started — 계산 함수와 함수 시그니처 (원서 pp.11-13)

- 이제 `Driver` 와 충전량을 받아 청구액을 내는 함수를 만든다. 타입 정의 아래에 놓아야 한다.

```
// Driver -> decimal -> decimal
let chargeFeeAnnotated (driver: Driver) (kwh: decimal) : decimal =
    let gross = kwh * 300.0M
    let waiver =
        if driver.IsPlanActive && kwh >= 25.0M
        then gross * 0.12M else 0.0M
    let net = gross - waiver
    net
```

숫자 뒤의 `M` 접미사는 그 숫자가 `decimal` 이라는 표시다. 금액 계산에는 부동소수점보다 `decimal` 이 맞다.

이 함수에서 짚을 것들.

- 함수 정의도 `let` 이고, 함수 안에서 `gross`, `waiver`, `net` 을 묶은 것도 `let` 이다. 같은 키워드다.
- `gross`, `waiver`, `net` 은 함수 스코프 안에만 있다. 밖에서는 보이지 않는다.
- 함수를 담은 클래스 같은 껍데기를 적지 않는다. 모듈이나 스크립트 최상위에 `let` 을 그대로 놓을 수 있다.
- F# 에서 함수는 일급 함수(first-class function)다. 값과 똑같이 다룰 수 있어서 다른 함수에 넘기거나 함수에서 반환받을 수도 있다.
- 반환 타입 주석은 매개변수 오른쪽에 붙는다.
- `return` 키워드가 없다. 마지막 줄의 값이 그대로 반환된다.
- 들여쓰기가 스코프를 만든다. 탭은 쓸 수 없다.
- `if` 는 식(expression)이므로 참 쪽과 거짓 쪽이 같은 타입을 내야 한다. 값을 내지 않는 문(statement)과 달리 식은 항상 출력이 있어서 다른 식과 조합하기 쉽고 테스트하기도 쉽다. F# 코드가 식으로 가득한 이유가 이것이다.
- 주석에 적은 `Driver -> decimal -> decimal` 이 이 함수의 시그니처(signature)다. 마지막 화살표 뒤가 반환 타입이다. 편집기에서 함수 이름에 마우스를 올리면 볼 수 있다. 시그니처를 읽는 습관은 F# 을 읽는 데 결정적이다.
- 매개변수를 `(driver: Driver) (kwh: decimal)` 처럼 나란히 적은 형태를 커링된 매개변수(curried parameters)라 하고, 튜플 하나로 묶어 받는 형태를 튜플 매개변수(tupled parameter)라 한다. 커링된 매개변수로 정의하면 시그니처에 매개변수 개수만큼 화살표가 생긴다. 다만 화살표 개수만 보고 거꾸로 판정할 수는 없다. 튜플 매개변수 하나를 받고 함수를 반환하는 함수도 화살표가 둘이다. 어느 쪽인지는 정의를 봐야 구분된다.

```
// Driver * decimal -> decimal - 본문은 그대로고 매개변수만 튜플 하나로 바뀌었다
let chargeFeeTupled (driver: Driver, kwh: decimal) : decimal =
    let gross = kwh * 300.0M
    let waiver =
        if driver.IsPlanActive && kwh >= 25.0M
        then gross * 0.12M else 0.0M
    gross - waiver
```

시그니처가 `Driver -> decimal -> decimal` 에서 `Driver * decimal -> decimal` 로 바뀐 것을 보면 된다. `*` 가 `->` 보다 강하게 묶이므로 튜플 매개변수는 하나다. 원서는 함수 대부분을 커링된 매개변수 형태로 쓰라고 권한다. 그 이유는 다음 챕터에서 다룬다.

- 타입 추론(type inference) 덕분에 위의 주석은 대부분 지울 수 있다. 컴파일러가 쓰임새를 보고 타입을 알아낸다.



```
// Driver -> decimal -> decimal - 타입 주석을 모두 지웠는데도 시그니처는 같다
let chargeFee driver kwh =
    let gross = kwh * 300.0M
    let waiver =
        if driver.IsPlanActive && kwh >= 25.0M then gross * 0.12M
        else 0.0M
    gross - waiver

printfn "yujin 30kwh -> %.0f원" (float (chargeFee yujin 30.0M)) // 기대: 7920원
printfn "minho 20kwh -> %.0f원" (float (chargeFee minho 20.0M)) // 기대: 6000원
printfn "soyeon 30kwh -> %.0f원" (float (chargeFee soyeon 30.0M)) // 기대: 9000원
printfn "taeho 30kwh -> %.0f원" (float (chargeFee taeho 30.0M)) // 기대: 9000원
```

- `net` 바인딩은 읽기에 도움이 되지 않아 없었다. `kwh` 가 `decimal` 로 정해지는 근거는 `kwh * 300.0M` 과 `kwh >= 25.0M` 처럼 `decimal` 리터럴과 나란히 쓰인 자리다.
- 타입 추론이 안 되는 자리도 있다. `DateTime.TryParse` 처럼 오버로드가 여럿인 .NET 함수를 쓰면 컴파일러가 어느 것인지 고를 수 없어 매개변수에 타입 주석을 달아 줘야 한다.

## Getting Started — FSI 로 검증하고, `=` 의 세 가지 얼굴 (원서 pp.13-15)

- 정식 단위 테스트는 4챕터에서 다룬다. 그전까지는 FSI 에서 `bool` 바인딩으로 간단히 확인하면 된다. 명세의 예시 표를 그대로 옮기는 것으로 충분하다.

```
// 명세의 예시를 그대로 검증한다. 괄호는 없어도 된다
let assertYujin = (chargeFee yujin 30.0M = 7920.0M)
let assertMinho = chargeFee minho 20.0M = 6000.0M
let assertSoyeon = chargeFee soyeon 30.0M = 9000.0M
let assertTaeho = chargeFee taeho 30.0M = 9000.0M

printfn "%b %b %b %b" assertYujin assertMinho assertSoyeon assertTaeho // 기대: true true true true
```

- F# 에는 `==` 도 `===` 도 없다. `=` 하나가 바인딩, 필드 값 지정, 동등성 비교를 모두 맡는다. 레코드 필드뿐 아니라 뒤에서 볼 케이스 데이터 필드에도 같은 `=` 를 쓴다. 문맥이 어느 쪽인지 결정한다.
- 값을 바꿔 쓰려면 바인딩을 `mutable` 로 명시해야 하고, 대입에는 `<-` 를 쓴다.

```
// mutable 을 명시해야 값을 바꿔 쓸 수 있다. 대입은 <-, 비교는 =
let mutable sessionCount = 0
let beforeAssign = (sessionCount = 1) // 여기서 = 는 동등성 비교
sessionCount <- 1 // 여기서 대입
printfn "대입 전 비교=%b, 대입 후 비교=%b" beforeAssign (sessionCount = 1) // 기대: 대입 전 비교=false, 대입 후 비교=true
```

`mutable` 은 그 이름에 다른 값을 다시 대입할 수 있게 해 주는 것이지, 담긴 데이터를 가변으로 만드는 것이 아니다. `mutable` 로 묶은 이름이 레코드를 담고 있어도 그 레코드의 필드는 여전히 바꿀 수 없다.

- 검증 의도를 더 드러내려면 비교를 함수로 뽑아도 된다. 이 함수는 특정 타입에 묶이지 않고 제네릭(generic)으로 일반화된다. 같은 타입의 두 값이면 `=` 로 비교할 수 있다는 사실만 필요하기 때문이다.

```
// 'a -> 'a -> bool (when 'a : equality)
let areEqual expected actual =
    actual = expected

printfn "%b %b" (areEqual 7920.0M (chargeFee yujin 30.0M)) (areEqual 6000.0M (chargeFee minho 20.0M)) // 기대: true true
```

'a' 처럼 작은따옴표로 시작하는 이름은 어떤 타입이든 들어갈 수 있는 자리, 곧 타입 매개변수(type parameter)다. 여기서 컴파일러는 'a' 에 동등성 비교가 가능해야 한다는 제약까지 함께 붙였다.

```
// 레코드에는 구조적 동등성이 기본으로 붙는다. 따로 만든 두 값도 = 가 참이다
let sameSoyeon = { DriverId = "soyeon"; IsSubscribed = true; IsPlanActive = false }
printfn "%b" (sameSoyeon = soyeon) // 기대: true
printfn "%b" (areEqual sameSoyeon soyeon) // 기대: true
```

따로 만든 두 레코드인데도 = 가 참이다. 레코드와 판별 유니온에는 구조적 동등성(structural equality)이 기본으로 붙는다. 참조가 아니라 담긴 값을 비교하므로 값이 같으면 = 가 참이다. areEqual 이 'a : equality' 하 나만 요구하고도 도메인 타입에 그대로 쓸 수 있는 이유가 이것이다.

- 여기까지의 코드는 동작한다. 그러나 도메인 개념을 bool 플래그로 표현한 것이 약점이다. IsSubscribed = false 이면서 IsPlanActive = true 인 값, 곧 가입하지도 않았는데 요금제가 유효한 운전자를 만들 수 있다. 명세상 있을 수 없는 상태인데 타입이 막아 주지 않는다.
- driver.IsSubscribed 검사를 조건에 추가하면 되지만, 그런 검사는 빼먹기 쉽다. 대신 타입 시스템으로 “가입” 과 “미가입” 이라는 개념 자체를 드러내는 쪽이 낫다.

## Making the Implicit Explicit — 판별 유니온과 패턴 매칭 (원서 pp.16-18)

- 먼저 가입 운전자와 비회원을 각각의 레코드 타입으로 나눈다. 담은 데이터가 다르기 때문이다.
- 그다음 “운전자는 가입자이거나 비회원이다” 를 표현할 재료가 필요하다. 대수적 타입 시스템의 OR 타입인 판별 유니온이 그것이다. 줄여서 DU 라고도 부른다.

```
// 이 단위가 보여주는 것: 판별 유니온, match 식, 케이스 데이터 분해, when 가드, 와일드카드

type PlanHolder = {
    DriverId: string
    IsPlanActive: bool
}

type Visitor = {
    DriverId: string
}

// "운전자는 PlanHolder 를 담은 Subscribed 이거나, Visitor 를 담은 Walkup 이다"
type Driver =
    | Subscribed of PlanHolder
    | Walkup of Visitor
```

- | 로 나열한 항목을 유니온 케이스(union case)라 한다. 각 케이스는 케이스 식별자(case identifier)와 선택적인 케이스 데이터(case data)로 이뤄진다. 여기서는 Subscribed 와 walkup 이 케이스 식별자이고, of 뒤에 붙는 타입이 케이스 데이터다. 케이스 데이터로는 어떤 타입이든 붙일 수 있고, 여러 타입을 섞어도 되고, 아예 붙이지 않아도 된다.
- 판별 유니온은 닫힌 집합이다. 타입 정의에 적힌 케이스만 존재하고, 케이스를 추가할 수 있는 곳은 타입 정의 한 곳뿐이다.
- 값을 만들 때는 케이스 식별자를 함수처럼 앞에 붙인다. Driver 자체를 직접 만들 수는 없고, 반드시 Subscribed 나 walkup 중 하나여야 한다.

```
// 케이스 식별자를 앞에 붙여 만든다. 네 값 모두 타입은 Driver 다
let taeho = Walkup { DriverId = "taeho" }
let yujin = Subscribed { DriverId = "yujin"; IsPlanActive = true }
let minho = Subscribed { DriverId = "minho"; IsPlanActive = true }
let soyeon = Subscribed { DriverId = "soyeon"; IsPlanActive = false }

printfn "%A" taeho // 기대: Walkup { DriverId = "taeho" }
printfn "%A" yujin // 기대: 레코드가 담긴 케이스는 %A 가 여러 줄로 출력한다
//           Subscribed { DriverId = "yujin"
//           IsPlanActive = true }
```

- 타입이 레코드에서 판별 유니온으로 바뀌었으니 계산 함수도 바뀐다. 어느 케이스인지 확인하는 도구가 패턴 매칭이고, 그 문법이 `match` 식이다.

```
// Driver -> decimal -> decimal
let chargeFee driver kwh =
    let gross = kwh * 300.0M
    let waiver =
        match driver with
        | Subscribed h ->
            if h.IsPlanActive && kwh >= 25.0M then gross * 0.12M else 0.0M
        | Walkup _ -> 0.0M
    gross - waiver
```

- `Subscribed h` 를 값을 만들 때 쓴 `Subscribed { DriverId = "yujin"; IsPlanActive = true }` 와 나란히 놓고 보면 이해가 쉽다. 만들 때 케이스 데이터를 넣었던 자리에, 매칭할 때는 그 데이터를 받을 이름 `h` 를 놓는다.
- `Walkup _` 의 밑줄은 와일드카드(wildcard)다. 그 케이스 데이터를 쓰지 않겠다는 표시다.
- 판별 유니온을 상대로 하는 패턴 매칭은 빠짐없는 패턴 매칭(exhaustive pattern matching)이어야 한다. 모든 케이스를 처리해야 하고, 빠뜨리면 컴파일러가 경고 FS0025 를 낸다. 아래처럼 `Walkup` 을 빼면 그 경고가 난다.

```
match driver with
| Subscribed h -> ... // Walkup 이 빠졌다 -> warning FS0025
```

편집기와 FSI 가 내놓는 문구는 이렇다. 이 노트가 “빠짐없는 패턴 매칭” 이라 부르는 것을 컴파일러는 “완전하지 않습니다” 로 말한다.

warning FS0025: 이 식의 패턴 일치가 완전하지 않습니다. 예를 들어, 값 'Walkup (\_)'은(는) 패턴에 포함되지 않은 케이스를 나타낼 수 있습니다.

어느 케이스가 빠졌는지까지 짚어 준다. 위 문구는 F# 10 한국어 로케일에서 실측한 것이고, SDK 버전과 로케일에 따라 표현이 달라질 수 있다.

FS0025 는 오류가 아니라 경고다. 그래서 코드는 그대로 컴파일되고, 빠뜨린 케이스에 해당하는 값이 실제로 들어오면 그 `match` 식이 실행 시점에 `MatchFailureException` 을 던진다. 그때 보게 되는 문구는 `Microsoft.FSharp.Core.MatchFailureException: 일치하는 케이스가 완전하지 않습니다.` 다. 경고를 무시하는 것은 컴파일러가 잡아 준 문제를 실행 시점으로 미루는 일일 뿐이다.

- 중첩된 `if` 는 가드 절(guard clause)인 `when` 으로 펼 수 있다. 단, 컴파일러는 가드의 참/거짓을 계산하지 않고, 가드가 붙은 케이스는 그 케이스 전체를 덮지 못한 것으로 본다. 케이스를 하나도 빠뜨리지 않고 적어도 가드를 붙인 쪽이 있으면 같은 FS0025 가 난다. 그래서 요금제가 만료된 가입자를 처리하는 케이스를 따로 적어 줘야 모든 경우가 채워진다.

```
// Driver -> decimal -> decimal - when 가드로 중첩 if 를 없앴다
let chargeFeeGuarded driver kwh =
    let gross = kwh * 300.0M
    let waiver =
        match driver with
        | Subscribed h when h.IsPlanActive && kwh >= 25.0M -> gross * 0.12M
        | Subscribed _ -> 0.0M
        | Walkup _ -> 0.0M
    gross - waiver
```

- 같은 값을 내는 뒤쪽 두 케이스는 와일드카드 하나로 합칠 수 있다.

```
// Driver -> decimal -> decimal - 나머지 케이스를 와일드카드로 합쳤다
let chargeFeeTerse driver kwh =
    let gross = kwh * 300.0M
    let waiver =
        match driver with
        | Subscribed h when h.IsPlanActive && kwh >= 25.0M -> gross * 0.12M
        | _ -> 0.0M
    gross - waiver

// 세 버전 모두 명세를 만족한다
printfn "%b" (chargeFee yujin 30.0M = 7920.0M
               && chargeFeeGuarded yujin 30.0M = 7920.0M
               && chargeFeeTerse yujin 30.0M = 7920.0M) // 기대: true
printfn "%b" (chargeFeeTerse minho 20.0M = 6000.0M
               && chargeFeeTerse soyeon 30.0M = 9000.0M
               && chargeFeeTerse taeho 30.0M = 9000.0M) // 기대: true
```

- 다만 이런 식의 와일드카드에는 대가가 있다. 나중에 판별 유니온에 케이스를 추가해도 컴파일러가 알려 주지 않는다. 새 케이스가 조용히 `_` 로 흘러 들어가 엉뚱한 결과를 낼 수 있다.
- 검증 코드는 손덜 필요가 없었다. 처음 버전보다 로직이 읽기 쉬워졌고, 잘못된 상태를 만들 수 없게 됐다. 그렇다면 감면 자격까지 타입으로 올리면 더 나아질까.

## Going Further — 자격을 타입으로 올리기 (원서 pp.18-20)

- `IsPlanActive` 라는 `bool` 필드를 없애고, 요금제가 유효한 상태를 판별 유니온의 케이스로 올린다. 이렇게 하면 “유효” 가 플래그 값이 아니라 도메인 개념이 된다.

```
// 이 단위가 보여주는 것: bool 플래그를 유니온 케이스로 올려 자격을 타입으로 표현하기

type PlanHolder = {
    DriverId: string
}

type Visitor = {
    DriverId: string
}

// 유효한 요금제 / 만료된 요금제 / 비회원 - 세 갈림이 케이스 이름으로 드러난다
type Driver =
    | ActivePlan of PlanHolder
    | ExpiredPlan of PlanHolder
    | Walkup of Visitor
```

- `PlanHolder` 와 `Visitor` 는 구조가 똑같다. 이럴 때 `{ DriverId = "yujin" }` 만 놓으면 컴파일러는 오류를 내지 않고 가장 나중에 선언된 타입, 곧 `Visitor` 를 고른다. 원한 타입이 아니어도 그 자리에서는 아무 신호가 없고, 그 값을 쓰는 자리에서 타입이 맞지 않는다는 오류 FS0001 이 뒤늦게 난다. 반면 케이스 식별자 뒤에 놓으면 그 케이스가 요구하는 타입이 정해져 있어 이런 일이 없다.

```
let yujin = ActivePlan { DriverId = "yujin" }
let minho = ActivePlan { DriverId = "minho" }
let soyeon = ExpiredPlan { DriverId = "soyeon" }
let taeho = Walkup { DriverId = "taeho" }
```

- 함수에서 `IsPlanActive` 검사가 사라진다. 케이스 데이터도 필요 없으니 이름 대신 와일드카드를 놓는다.

```
// Driver -> decimal -> decimal
let chargeFee driver kwh =
    let gross = kwh * 300.0M
    let waiver =
        match driver with
        | ActivePlan _ when kwh >= 25.0M -> gross * 0.12M
        | _ -> 0.0M
    gross - waiver

printfn "%b %b %b %b"
    (chargeFee yujin 30.0M = 7920.0M)
    (chargeFee minho 20.0M = 6000.0M)
    (chargeFee soyeon 30.0M = 9000.0M)
    (chargeFee taeho 30.0M = 9000.0M) // 기대: true true true true
```

- 감면 조건이 `ActivePlan` 케이스와 충전량 하나로 줄었다. 읽기도 쉬워졌고, 잘못된 상태도 만들 수 없다.
- 남은 개선 여지도 있다. `DriverId` 가 여전히 그냥 `string` 이다. 원시 타입을 도메인 개념으로 감싸는 방법은 9챕터에서 다루고, 여기서 손으로 만든 검증을 xUnit 단위 테스트로 옮기는 방법은 4챕터에서 다룬다.

`PlanHolder` 와 `Visitor` 가 필드 하나뿐이라면, 레코드를 없애고 `string` 을 케이스 데이터로 직접 붙일 수도 있다.

```
// 이 단위가 보여주는 것: 레코드 없이 원시 타입을 케이스 데이터로 직접 붙이기

// 케이스 데이터에 이름을 달아 두면 그 이름이 문서 역할을 한다
type Driver =
    | ActivePlan of DriverId: string
    | ExpiredPlan of DriverId: string
    | Walkup of DriverId: string
```

레코드를 없애도 세 갈림은 그대로 남는다. `of` 뒤에 `DriverId: string` 이라고 이름을 달았기 때문에 `string` 하나만 붙였을 때보다 뜻이 분명하다. 함수 쪽은 앞 절의 버전을 그대로 쓸 수 있다.

```
// Driver -> decimal -> decimal - 함수 본문은 앞의 버전과 똑같다
let chargeFee driver kwh =
    let gross = kwh * 300.0M
    let waiver =
        match driver with
        | ActivePlan _ when kwh >= 25.0M -> gross * 0.12M
        | _ -> 0.0M
    gross - waiver

let yujin = ActivePlan "yujin"
let minho = ActivePlan "minho"
let soyeon = ExpiredPlan "soyeon"
let taeho = Walkup "taeho"

printfn "%b %b %b %b"
    (chargeFee yujin 30.0M = 7920.0M)
    (chargeFee minho 20.0M = 6000.0M)
    (chargeFee soyeon 30.0M = 9000.0M)
    (chargeFee taeho 30.0M = 9000.0M) // 기대: true true true true
```

- 원서는 이 방식을 최종안으로 고르지 않는다. 실제 시스템이라면 가입자 쪽 케이스 데이터에 필드가 더 붙을 가능성이 높기 때문이다. 필드가 늘어날 일이 없는 비회원 쪽만 이렇게 펴는 절충도 가능하다.
- 판별 유니온으로 이 도메인을 모델링하는 방법이 하나뿐인 것은 아니다. 다음 두 절은 같은 문제를 다른 구성으로 다시 푼다.

## Alternative Approaches (1 of 2) — 판별 유니온을 레코드 필드로 (원서 pp.21-22)

- 앞의 방식은 판별 유니온을 최상위 타입으로 놓고 그 안에 레코드를 담았다. 순서를 뒤집을 수도 있다. 레코드를 최상위로 놓고, 그 필드 하나의 타입으로 판별 유니온을 쓰는 것이다.

```
// 이 단위가 보여주는 것: 레코드 필드의 타입으로 판별 유니온을 쓰는 구성

// 케이스 데이터가 없는 케이스(Walkup)와 있는 케이스(Subscribed)를 섞을 수 있다
type Membership =
    | Subscribed of IsPlanActive: bool
    | Walkup

type Driver = { DriverId: string; Membership: Membership }

// Driver -> decimal -> decimal
let chargeFeeVerbose driver kwh =
    let gross = kwh * 300.0M
    let waiver =
        match driver.Membership with
        | Subscribed (IsPlanActive = isActive) when isActive && kwh >= 25.0M -> gross * 0.12M
        | _ -> 0.0M
    gross - waiver
```

- `Subscribed (IsPlanActive = isActive)` 는 케이스 데이터의 `IsPlanActive` 필드를 `isActive` 라는 이름으로 꺼내는 패턴이다. 꺼낸 값을 `when` 절에서 검사한다.
- 값 쪽에서도 같은 문법으로 필드 이름을 지정해 만들 수 있다.

```
let yujin = { DriverId = "yujin"; Membership = Subscribed (IsPlanActive = true) }
let minho = { DriverId = "minho"; Membership = Subscribed (IsPlanActive = true) }
let soyeon = { DriverId = "soyeon"; Membership = Subscribed (IsPlanActive = false) }
let taeho = { DriverId = "taeho"; Membership = Walkup }
```

- 값을 꺼낸 뒤 `when` 에서 검사하는 대신, 패턴 자체에 값을 박아 걸러낼 수도 있다. `IsPlanActive = true` 인 것만 이 케이스에 걸린다.

```
// Driver -> decimal -> decimal - 패턴에 값을 직접 박아 걸러낸다
let chargeFee driver kwh =
    let gross = kwh * 300.0M
    let waiver =
        match driver.Membership with
        | Subscribed (IsPlanActive = true) when kwh >= 25.0M -> gross * 0.12M
        | _ -> 0.0M
    gross - waiver

printfn "%b %b %b %b"
    (chargeFee yujin 30.0M = 7920.0M)
    (chargeFee minho 20.0M = 6000.0M)
    (chargeFee soyeon 30.0M = 9000.0M)
    (chargeFee taeho 30.0M = 9000.0M) // 기대: true true true true
printfn "verbose 버전과 결과가 같은가: %b"
    (chargeFeeVerbose yujin 30.0M = chargeFee yujin 30.0M) // 기대: true
```

- 이 구성도 잘못된 상태를 막아 준다는 점에서는 문제가 없다. 그러나 `Membership` 이라는 이름은 원래 명세에 없던 말이다. 모델링을 하다가 이런 중간 이름을 발명해야 한다면, 도메인 중심 코드에서 한 발 멀어졌다는 신호로 볼 만하다.

## Alternative Approaches (2 of 2) — 케이스 데이터에 이름 붙이기 (원서 pp.22-24)

- 앞의 두 절에서도 케이스 데이터에 이름을 하나씩 달아 썼다. 이번 절은 여러 필드에 이름을 붙이고 그것을 꺼내는 방식을 정리한다. 판별 유니온의 케이스 데이터는 튜플처럼 `*` 로 이어 쓰지만 튜플이 아니다. 컴파일된 뒤에도 필드가 따로 놓이고, 패턴에서 `Subscribed t` 처럼 튜플 하나로 받으려 하면 오류 FS0727 이 난다. 그리고 튜플과 달리 각 부분에 이름을 붙일 수 있다.

```
// 이 단위가 보여주는 것: 이름 붙은 케이스 데이터, 세 가지 매칭 방식, 액티브 패턴 맛보기

// 튜플처럼 * 로 이어 쓰지만 패턴에서 튜플 하나로 받을 수는 없다. 각 부분에 이름이 붙는다
type Driver =
    | Subscribed of DriverId: string * IsPlanActive: bool
    | Walkup of DriverId: string

let yujin = Subscribed (DriverId = "yujin", IsPlanActive = true)
let minho = Subscribed (DriverId = "minho", IsPlanActive = true)
let soyeon = Subscribed (DriverId = "soyeon", IsPlanActive = false)
let taeho = Walkup (DriverId = "taeho")
```

- 매칭 방식은 여러 가지다. 첫째, 위치대로 이름을 나열해 꺼낸다.

```
// Driver -> decimal -> decimal - 위치 순서대로 꺼낸다. 쓰지 않는 자리는 와일드카드로 둔다
let chargeFeeByPosition driver kwh =
    let gross = kwh * 300.0M
    let waiver =
        match driver with
        | Subscribed (_, isActive) when isActive && kwh >= 25.0M -> gross * 0.12M
        | _ -> 0.0M
    gross - waiver
```

- 둘째, 필드 이름을 써서 값을 박아 걸러낸다. 앞 절과 같은 문법이다. 위치를 셀 필요가 없어 필드가 늘어나도 흔들리지 않는다.

```
// Driver -> decimal -> decimal - 필드 이름으로 걸러내기 위치를 셀 필요가 없다
let chargeFee driver kwh =
  let gross = kwh * 300.0M
  let waiver =
    match driver with
    | Subscribed (IsPlanActive = true) when kwh >= 25.0M -> gross * 0.12M
    | _ -> 0.0M
  gross - waiver

printfn "%b %b %b %b"
  (chargeFee yujin 30.0M = 7920.0M)
  (chargeFee minhoo 20.0M = 6000.0M)
  (chargeFee soyeon 30.0M = 9000.0M)
  (chargeFee taeho 30.0M = 9000.0M) // 기대: true true true true
printfn "위치 방식과 결과가 같은가: %b"
  (chargeFeeByPosition yujin 30.0M = chargeFee yujin 30.0M) // 기대: true
```

- 셋째, 걸러내면서 다른 필드 값도 함께 꺼낼 수 있다. 이때 필드 사이 구분자는 쉼표가 아니라 세미콜론이다.

```
// Driver -> decimal -> string - 걸러내는 동시에 DriverId 도 꺼낸다
let waiverNote driver kwh =
  match driver with
  | Subscribed (DriverId = id; IsPlanActive = true) when kwh >= 25.0M -> $"{id}: 12퍼센트 감면"
  | Subscribed (DriverId = id) -> $"{id}: 감면 없음"
  | Walkup (DriverId = id) -> $"{id}: 비회원"

printfn "%s" (waiverNote yujin 30.0M) // 기대: yujin: 12퍼센트 감면
printfn "%s" (waiverNote minhoo 20.0M) // 기대: minhoo: 감면 없음
printfn "%s" (waiverNote taeho 30.0M) // 기대: taeho: 비회원
```

- 이렇게 반복되는 필터를 재사용 가능한 조각으로 뺄 수 있다. 그 기능이 액티브 패턴(active pattern)이고 7챕터에서 따로 다룬다. 여기서는 맛만 본다.

```
// Driver -> unit option - (| ... |_) 로 감싸면 패턴 자리에서 쓸 수 있는 이름이 된다
let (|OnActivePlan|_|) driver =
  match driver with
  | Subscribed (IsPlanActive = true) -> Some ()
  | _ -> None

// Driver -> decimal -> decimal - 필터가 케이스 이름처럼 읽힌다
let chargeFeeWithPattern driver kwh =
  let gross = kwh * 300.0M
  let waiver =
    match driver with
    | OnActivePlan when kwh >= 25.0M -> gross * 0.12M
    | _ -> 0.0M
  gross - waiver

printfn "%b %b %b %b"
  (chargeFeeWithPattern yujin 30.0M = 7920.0M)
  (chargeFeeWithPattern minhoo 20.0M = 6000.0M)
  (chargeFeeWithPattern soyeon 30.0M = 9000.0M)
  (chargeFeeWithPattern taeho 30.0M = 9000.0M) // 기대: true true true true
```

`Option`, `Some`, `None`, `unit` 은 3챕터에서, 액티브 패턴은 7챕터에서 제대로 다룬다.

- 원서가 제시하는 도메인 모델링 지침은 두 문장으로 요약된다. 최대한 도메인의 언어를 쓸 것, 그리고 잘못된 상태를 표현조차 할 수 없게 만들 것.
- 모델링에 정답은 없다. 여러 버전을 만들어 보면 처음 떠올린 것보다 나은 구성이 나올 때가 많다. 이 챕터가 같은 문제를 다섯 번 다시 푼 이유가 그것이다.



## Summary — 원서의 챕터 요약 (원서 pp.24-25)

- 원서는 이 챕터에서 타입을 여러 방식으로 조합해 하나의 업무 문제를 풀었다. 자료는 AND 타입(튜플, 레코드)과 OR 타입(판별 유니온)뿐인데도, 이것들을 겹쳐 쌓으면 꽤 다양한 도메인을 정확하게 표현할 수 있다.
- 다른 항목은 FSI, 대수적 타입 시스템(튜플, 레코드, 판별 유니온, 타입 조합), 패턴 매칭( `match` 식, 가드 절), `let` 바인딩, 함수, 함수 시그니처다.
- 다음 챕터에서는 함수 합성을 살펴본다. 작은 함수를 이어 붙여 큰 함수를 만드는 방법이다.

## Postscript — 개념은 언어를 넘는다 (원서 p.25)

- 원서는 마지막에 동료가 작성한 Scala 판 해법을 실어, 이 챕터에서 다른 개념이 F# 전용이 아니라는 점을 보여 준다.
- 봉인된 트레이트(sealed trait)와 케이스 클래스는 F# 의 판별 유니온에 대응하고, Scala 의 `match` 는 F# 의 `match` 식에 거의 그대로 대응한다. 아래는 같은 대응을 이 노트의 도메인으로 옮겨 본 스케치다.
- 아래 스케치는 원서에 실린 Scala 코드를 따라 `Double` 을 쓴다. 금액 계산에 `decimal` 이 맞다는 앞의 판단은 그대로 유효하고, 원서와 대조하기 쉽게 숫자 타입만 원서대로 두었다.

```
sealed trait Driver
case class ActivePlan(driverId: String) extends Driver
case class ExpiredPlan(driverId: String) extends Driver
case class Walkup(driverId: String) extends Driver

def chargeFee(driver: Driver)(kwh: Double) = {
  val gross = kwh * 300.0
  val waiver = driver match {
    case ActivePlan(_) if kwh >= 25.0 => gross * 0.12
    case _ => 0.0
  }
  gross - waiver
}
```

- 문법이 낯설어도 앞에서 F# 으로 짰 해법의 골격이 그대로 보인다. 배운 것은 F# 문법이 아니라 모델링 방식이다.

## 정리 — 이 노트의 요약

- 대수적 타입 시스템의 재료는 두 가지다. 여러 값을 동시에 담는 AND 타입(튜플, 레코드)과, 여러 경우 중 하나만 담는 OR 타입(판별 유니온).
- 튜플은 부분에 이름이 없어 뜻을 짐작해야 한다. 레코드와 이름 붙은 케이스 데이터가 그 문제를 없앤다.
- `bool` 플래그로 도메인 개념을 표현하면 명세상 있을 수 없는 상태까지 코드에서는 만들어진다. 그 갈림을 판별 유니온 케이스로 올리면 잘못된 상태가 표현 자체로 불가능해진다.
- 판별 유니온을 `match` 식으로 다룰 때는 모든 케이스를 빠짐없이 적어야 한다. 빠뜨리면 컴파일러가 경고 FS0025 를 낸다. `_` 로 뭉개면 그 경고를 잃는다는 점을 기억해야 한다.
- `when` 가드는 케이스 안에서 조건을 더 좁힌다. 필드 이름을 쓴 패턴( `Subscribed (IsPlanActive = true)` )은 조건을 패턴 자체로 옮겨 준다.
- F# 은 파일 위에서 아래로 컴파일한다. 타입이 함수보다 위에, 함수가 사용처보다 위에 있어야 한다. 프로젝트의 `.fs` 파일 순서도 같은 규칙을 따른다.
- 같은 도메인을 판별 유니온 안에 레코드를 담는 구성, 레코드 필드에 판별 유니온을 담는 구성, 케이스 데이터에 이름을 붙이는 구성으로 각각 모델링할 수 있다. 판단 기준은 도메인의 언어를 그대로 쓰는지, 그리고 잘못된 상태를 막아 주는지다.
- `=` 는 바인딩, 필드 값 지정, 동등성 비교를 겸한다. 대입은 `<-` 이고, 그 전에 바인딩을 `mutable` 로 명시해야 한다.

## 원서 대조 표

| 절                                                      | 원서 페이지   | 실행 단위                                                 |
|--------------------------------------------------------|----------|-------------------------------------------------------|
| The Problem — 명세 안에 이미 도메인이 있다                         | pp.8-9   | —                                                     |
| Getting Started — 튜플과 레코드로 데이터 모양 잡기                   | pp.9-11  | <code>01-record-fee</code>                            |
| Getting Started — 계산 함수와 함수 시그니처                       | pp.11-13 | <code>01-record-fee</code>                            |
| Getting Started — FSI 로 검증하고, <code>=</code> 의 세 가지 얼굴 | pp.13-15 | <code>01-record-fee</code>                            |
| Making the Implicit Explicit — 판별 유니온과 패턴 매칭           | pp.16-18 | <code>01-du-explicit</code>                           |
| Going Further — 자격을 타입으로 올리기                           | pp.18-20 | <code>01-du-eligible</code> , <code>01-du-flat</code> |
| Alternative Approaches (1 of 2) — 판별 유니온을 레코드 필드로      | pp.21-22 | <code>01-du-in-record</code>                          |
| Alternative Approaches (2 of 2) — 케이스 데이터에 이름 붙이기      | pp.22-24 | <code>01-du-named-fields</code>                       |
| Summary — 원서의 챕터 요약                                    | pp.24-25 | —                                                     |
| Postscript — 개념은 언어를 넘는다                               | p.25     | —                                                     |

## 02 - 함수 (원서 pp.26-38)

이 챕터는 F# 프로그래밍의 중심에 있는 함수를 다룬다. 함수의 규칙은 놀랄 만큼 단순하다. 입력 하나를 받아 출력 하나를 낸다. 이 단순한 규칙 위에서 작은 함수를 이어 붙여 큰 일을 하게 만드는 함수 합성(function composition), 여러 매개변수를 한 입력/한 출력의 사슬로 바꿔 주는 커링(currying), 그리고 그 사슬 덕분에 가능해지는 부분 적용(partial application)이 나온다. 이 챕터가 반복해서 말하는 것은 하나다. F#에서는 함수 시그니처(function signature)를 읽는 능력이 곧 코드를 읽는 능력이다.

### Getting Started — 함수의 규칙과 순수 함수 (원서 p.26)

- F# 함수의 규칙은 하나다. 입력 하나를 받고 출력 하나를 낸다. 1챕터에서 쓴, 매개변수를 두 개 받는 함수도 이 규칙과 충돌하지 않는다. 그 이유는 뒤의 커링 절에서 다룬다.
- 이 챕터는 그중 순수 함수(pure function)에 집중한다. 순수 함수는 두 가지 성질을 만족한다. 첫째, 결정적이다. 같은 입력에는 언제나 같은 출력을 낸다. 둘째, 부수 효과(side effect)를 만들지 않는다.

```
// 이 단위가 보여주는 것: 순수 함수, 시그니처 맞물림과 합성, 어댑터 함수, 레코드와 튜플

// string -> int
let charCount (text: string) = text.Length

// int -> float
let toDensity (n: int) = float n / 10.0

// float -> string
let describe (d: float) = sprintf "밀도 %.2f" d

// 세 함수 모두 결정적이고 부수 효과가 없다. 같은 입력이면 몇 번 불러도 같은 출력이 나온다
printfn "charCount \"hello\" 두 번: %d, %d" (charCount "hello") (charCount "hello") // 기대: 5, 5
```

- 부수 효과란 데이터베이스 접근, 메일 발송, 사용자 입력 처리, 난수 생성, 현재 시각 조회 같은 활동이다. 부수 효과가 하나도 없는 프로그램은 현실에서 만들 수 없다. 목표는 완전히 제거하는 것이 아니라 순수한 부분과 분리하는 것이다.
- 순수 함수는 테스트하기 쉽다. 결과를 캐시해 두거나 여러 개를 병렬로 실행하기에도 유리하다.
- 한 함수에 모든 로직을 몰아넣을 수도 있지만, 작은 함수를 조합하면 재사용성이 훨씬 좋아진다. 이 조합을 함수 합성이라 부른다.

### Theory — 시그니처가 맞물려야 합성된다 (원서 pp.26-27)

- 합성의 조건은 단순하다. 앞 함수의 출력 타입이 뒤 함수의 입력 타입과 같아야 한다.
- `f1 : 'a -> 'b` 와 `f2 : 'b -> 'c` 가 있으면 `'b` 가 맞물리므로 `f1 >> f2` 로 `'a -> 'c` 인 새 함수를 만들 수 있다. `'a` 처럼 작은따옴표로 시작하는 이름은 어떤 타입이든 들어갈 수 있는 자리, 곧 타입 매개변수(type parameter)를 뜻한다.

```
// charCount 의 출력 int 가 toDensity 의 입력 int 와 맞물린다: string -> float
let density = charCount >> toDensity
printfn "density \"hello\" = %f" (density "hello") // 기대: 0.500000

// 맞물리기만 하면 몇 개든 이어 붙는다: string -> string
let report = charCount >> toDensity >> describe
printfn "report \"functional\" = %s" (report "functional") // 기대: 밀도 1.00
```

- `>>` 는 특정 타입 전용 도구가 아니라 임의의 두 함수를 이어 붙이는 범용 연산자로 보면 된다.
- 타입이 맞지 않을 때( `f1 : 'a -> 'b` , `f2 : 'c -> 'd` , `'b <> 'c` )는 사이에 끼울 어댑터 함수(adaptor function) `'b -> 'c` 를 새로 만들거나 이미 있는 함수를 찾아 넣는다. 그러면 전체가 다시 `'a -> 'd` 로 이어진다.

아래 두 함수는 `int` 와 `string` 이 달라 바로 합성되지 않는다.

```
// string -> int 과 string -> string 은 맞물리지 않는다
wordCount >> shout // 컴파일 오류 FS0001
```

`int -> string` 어댑터를 사이에 끼우면 다시 이어진다.

```
// string -> int
let wordCount (text: string) = text.Split(' ').Length

// string -> string
let shout (text: string) = text.ToUpper() + "!"

// 어댑터 함수: int -> string
let toDashes (n: int) = String.replicate n "-"

// 어댑터를 끼우면 전체가 다시 이어진다: string -> string
let banner = wordCount >> toDashes >> shout
printfn "banner \"%f sharp is fun\" = %s" (banner "f sharp is fun") // 기대: ----!
```

- 이렇게 하면 함수를 몇 개든 계속 이어 붙일 수 있다.

## In Practice — 네 가지 표기, 같은 결과 (원서 pp.27-29)

- 원서는 레코드(record) 타입 하나와 시그니처가 맞물리는 함수 세 개로 실제 합성을 보여 준다. 이 노트는 같은 구조를 게임 캐릭터 도메인으로 다시 짜서 예제로 실었다.
- 여기서 새로 등장하는 문법이 두 가지다. 하나는 튜플(tuple)이다. 함수 사이에서 값 여러 개를 한 덩어리로 옮길 때 쓴다. 이 챕터 뒤쪽에서는 매개변수 자리에 쓰인 튜플도 다룬다. 타입을 적을 때는 `(Character * int)` 처럼 `*` 로 쓰고, 값을 쓸 때나 분해할 때는 `(character, exp)` 처럼 쉼표로 쓴다는 점을 구분해야 한다.

```
type Character = { Name: string; Level: int; Hp: int }

// Character -> (Character * int)
// 튜플은 함수 사이에서 값 여러 개를 한 덩어리로 옮길 때 쓴다
let gainExp character =
    let exp = if character.Name.Length % 2 = 0 then 150 else 40
    character, exp // 괄호는 생략 가능
```

- 다른 하나는 복사-수정 레코드 식(copy-and-update record expression)인 `{ record with Field = value }` 다. 기존 레코드를 바탕으로 일부만 바꾼 새 인스턴스를 만든다. 레코드와 그 필드는 기본적으로 바뀌지 않는다. 이 성질을 불변성(immutability)이라 하며, 그래서 원본은 그대로 남는다.

```
// (Character * int) -> Character
// 매개변수 자리에서 튜플을 분해했다.
// 매개변수를 하나로 받아 함수 안에서 `let (character, exp) = pair` 로 풀어도 같다
let levelUpIfEnough (character, exp) =
    if exp >= 100 then { character with Level = character.Level + 1 } // 복사-수정 레코드 식
    else character

// Character -> Character
let refillHp character = { character with Hp = character.Level * 20 }
```

- 함수를 이어 붙이는 방법은 크게 네 가지다. 함수 합성 연산자(function composition operator) `>>` 로 쓰기, 호출을 중첩해서 쓰기( `h (g (f x))` ), 중간 값에 이름을 붙여 절차적으로 쓰기, 정방향 파이프 연산자(forward pipe operator) `|>` 로 흘러보내기. 네 방식 모두 시그니처가 같고 같은 입력에 같은 결과를 낸다.

```
// 네 표기 모두 시그니처가 Character -> Character 다
let trainComposed = gainExp >> levelUpIfEnough >> refillHp

let trainNested character = refillHp (levelUpIfEnough (gainExp character))

let trainStepwise character =
    let withExp = gainExp character
    let leveled = levelUpIfEnough withExp
    refillHp leveled

let trainPiped character =
    character
    |> gainExp
    |> levelUpIfEnough
    |> refillHp
```

- `>>` 와 `|>` 의 차이는 놓이는 자리다. `>>` 는 함수와 함수 사이에, `|>` 는 값과 함수 사이에 놓인다. `|>` 로 쓴 파이프라인은 절차적 표기와 같은 일을 하면서 중간 값에 이름을 붙이는 수고를 없애 준다.
- 원서는 기본 스타일로 `|>` 를 쓰라고 권한다.
- 레코드에는 구조적 동등성(structural equality)이 있다. 담긴 값이 같으면 `=` 연산자가 참을 내므로 FSI 에서 결과를 검증하기 편하다.

```
let hero = { Name = "aria"; Level = 3; Hp = 10 } // 이름 길이 4(짝수) -> exp 150 -> 레벨업
let rookie = { Name = "ken"; Level = 3; Hp = 10 } // 이름 길이 3(홀수) -> exp 40 -> 유지

printfn "trainComposed hero = %A" (trainComposed hero)
printfn "네 표기의 결과가 모두 같은가: %b"
    (trainComposed hero = trainNested hero
    && trainNested hero = trainStepwise hero
    && trainStepwise hero = trainPiped hero) // 기대: true

// 담긴 값이 같으면 `=` 가 true
printfn "hero 검증: %b" (trainComposed hero = { Name = "aria"; Level = 4; Hp = 80 }) // 기대: true
printfn "rookie 검증: %b" (trainComposed rookie = { Name = "ken"; Level = 3; Hp = 60 }) // 기대: true
```

## Unit — 입력도 출력도 없을 때 (원서 pp.30-31)

- 모든 함수는 입력 하나를 받고 출력 하나를 내야 한다. 그런데 받을 것이 없거나 돌려줄 것이 없는 함수도 필요하다. F# 은 이 자리를 채우려고 `unit` 이라는 특별한 타입을 둔다.
- 시그니처에는 `unit` 으로 나타나지만 코드에서는 `()` 로 쓴다. `let now () = DateTime.UtcNow` 의 시그니처는 `unit -> DateTime` 이다.
- `unit` 을 유일한 입력으로 받거나 유일한 출력으로 내는 함수는 대개 부수 효과를 일으키고 있다는 신호다. 시각 조회, 로그 기록, 난수 생성이 전형적이다.

```
// 이 단위가 보여주는 것: `unit` 의 두 자리, 그리고 함수 바인딩과 값 바인딩의 차이
open System

// unit -> int
// 호출할 때마다 새로 평가되므로 결과가 달라질 수 있다(부수 효과 있음)
let ticks () = int (DateTime.UtcNow.Ticks % 1000L)

printfn "ticks() 1회: %d" (ticks ())
printfn "ticks() 2회: %d" (ticks ()) // 위와 다른 값이 나올 수 있다
```

`unit` 이 출력 자리에 오는 쪽은 돌려줄 값이 없는 작업이다.

```
// 'a -> unit
// 화면 출력, 로그 기록처럼 돌려줄 값이 없는 작업이 여기 해당한다
let record label =
    printfn "[기록] %A" label
    () // `printfn` 이 이미 `unit` 을 돌려주므로 이 줄은 생략할 수 있다

record "저장 완료" // 기대: [기록] "저장 완료"
record 42 // 기대: [기록] 42
```

- 다만 확정 판정이 아니라 신호일 뿐이다. `ignore : 'a -> unit` 은 `unit` 을 돌려주면서도 부수 효과가 없는 순수 함수다.
- .NET 을 다뤄 온 독자는 `unit` 을 `void` 와 혼동하기 쉽다. `unit` 은 값이 `()` 하나뿐인 실제 타입이라 `Async<unit>`, `Result<unit, string>` 처럼 다른 타입의 타입 인자로 넣을 수 있고 `let x = printfn "hi"` 처럼 값으로 받을 수도 있다. `void` 는 반환값이 없다는 표지여서 타입 인자로 쓸 수 없다. 3챕터 이후 `Option`, `Async` 를 다룰 때 이 구분이 필요해진다.
- `()` 를 빼고 함수 이름만 쓰면 실행되지 않는다. 실행 결과 대신 `unit -> DateTime` 인 함수 자체가 값으로 남는다. 함수 이름 자체가 값이라는 뜻이다. 함수를 값처럼 다룰 수 있다는 이 성질을 일급 시민(first-class citizen)이라 하며, 익명 함수 절에서 다시 다룬다.

```
// () 를 빼고 부르면 실행되지 않고 함수 자체가 값으로 남는다
let notYetCalled = ticks
printfn "notYetCalled 는 아직 함수다. 인자를 주면 실행된다: %d" (notYetCalled ())
```

- 정의할 때 `()` 를 빼면 값 바인딩(value binding)이 된다. 오른쪽 식은 그 줄에서 한 번만 평가되고, 이후에는 그 결과가 계속 재사용된다.

```
// unit -> int : 함수 바인딩
let freshNumber () = int (DateTime.UtcNow.Ticks % 1000L)

// int : 값 바인딩. 이 줄이 평가된 순간의 값으로 고정된다
let frozenNumber = int (DateTime.UtcNow.Ticks % 1000L)

// 잠깐 시간을 흘려보낸다
Threading.Thread.Sleep 50

printfn "함수 바인딩 재호출: %d, %d" (freshNumber ()) (freshNumber ())
printfn "값 바인딩 재사용: %d, %d (두 값이 같다)" frozenNumber frozenNumber
```

- 시그니처에 화살표 `->` 가 있으면 그 값은 호출할 수 있는 함수이며, 매개변수가 `unit` 하나뿐이어도 마찬가지다.
- 다만 화살표가 있다고 반드시 `let f x = ...` 형태의 함수 바인딩(function binding)인 것은 아니다. 값 바인딩에 함수를 담아도 화살표가 보인다. FSI 는 값 바인딩 쪽에 괄호를 붙여 `val f: (unit -> int)` 로 구분해 준다.

```
// (unit -> int) : 값 바인딩이지만 담긴 값이 함수라서 시그니처에 화살표가 있다.
// FSI 는 값 바인딩 쪽에 괄호를 붙여 val sharedNumber: (unit -> int) 로 구분해 준다
let sharedNumber =
    let captured = int (DateTime.UtcNow.Ticks % 1000L)    // 이 줄은 한 번만 평가된다
    fun () -> captured

Threading.Thread.Sleep 50

printfn "sharedNumber() 1회: %d" (sharedNumber ())
printfn "sharedNumber() 2회: %d (불잡아 둔 값이라 같다)" (sharedNumber ())
```

## Anonymous Functions — 이름 없는 함수 (원서 pp.31-32)

- 지금까지는 이름 있는 함수만 다뤘지만, 이름 없이 만드는 익명 함수(anonymous function)도 있다. `fun x y -> x + y` 처럼 `fun` 과 화살표로 쓰며 람다(lambda)라고 부른다.
- `let add x y = x + y` 와 `let add = fun x y -> x + y` 는 시그니처가 똑같이 `int -> int -> int` 다. 표기만 다르고 같은 함수다. 타입 주석(type annotation)을 달지 않으면 `+` 의 기본 대상인 정수로 추론된다.

```
// 이 단위가 보여주는 것: 익명 함수, 함수를 매개변수로 받는 함수, 와일드카드, 클로저
open System

// int -> int -> int
let joinNamed x y = x * 10 + y

// int -> int -> int : 시그니처가 위와 완전히 같다
let joinLambda = fun x y -> x * 10 + y

printfn "joinNamed 3 7 = %d" (joinNamed 3 7)    // 기대: 37
printfn "joinLambda 3 7 = %d" (joinLambda 3 7)  // 기대: 37
```

- F# 에서 함수는 일급 시민이다. 다른 값처럼 함수의 인자로 넘길 수 있다. `let apply f x y = f x y` 를 쓰면 컴파일러가 `('a -> 'b -> 'c) -> 'a -> 'b -> 'c` 라는 제네릭(generic) 시그니처를 추론한다. 시그니처가 맞는 이름 있는 함수든 익명 함수든 넘길 수 있다.

```
// ('a -> 'b -> 'c) -> 'a -> 'b -> 'c
// f 가 함수 자리다. 타입 주석이 없으므로 컴파일러가 제네릭으로 일반화한다
let runWith f a b = f a b

printfn "runWith joinNamed 3 7 = %d" (runWith joinNamed 3 7)    // 기대: 37
printfn "runWith (fun x y -> x * 10 + y) 3 7 = %d" (runWith (fun x y -> x * 10 + y) 3 7)    // 기대: 37
printfn "runWith (fun a b -> a + b) 3 7 = %d" (runWith (fun a b -> a + b) 3 7)    // 기대: 10

// 제네릭이므로 문자열에도 같은 함수를 쓸 수 있다
printfn "runWith (+) \"F\" \"#\" = %s" (runWith (+) "F" "#")    // 기대: F#
```

- 한 번 쓰고 버릴 간단한 작업까지 작은 함수로 만들어 이름을 붙이는 수고를 익명 함수가 덜어 준다. 3챕터의 고차 함수(higher-order function)에서 본격적으로 쓴다.
- 밑줄 `_` 은 와일드카드(wildcard)라고 부르며, 그 값을 쓰지 않겠다고 컴파일러에 알리는 표시다. `List.init 50 (fun _ -> rnd ())` 에서 인덱스를 버리는 용도로 쓰인다.
- 스코프(scope) 규칙을 이용하면 무거운 객체를 한 번만 만들어 재사용할 수 있다. `let rnd () = let r = Random() in r.Next(100)` 은 호출마다 새 인스턴스를 만든다. 반면 `let rnd = let r = Random() in fun () -> r.Next(100)` 은 인스턴스를 한 번만 만들고, 반환된 람다가 그 인스턴스를 계속 붙잡아 쓴다. 이렇게 정의 시점의 값을 붙잡아 두는 함수 값을 클로저(closure)라 한다.

```
// unit -> int : 호출할 때마다 Random 인스턴스를 새로 만든다
let dieRollFresh () =
    let generator = Random(20260904)
    generator.Next(1, 7)

// (unit -> int) : Random 을 한 번만 만들고 람다가 그것을 계속 붙잡아 쓴다(클로저)
let dieRollShared =
    let generator = Random(20260904)
    fun () -> generator.Next(1, 7)
```

- 뒤쪽 `rnd` 는 값 바인딩이다. 담긴 값이 함수라서 시그니처에 화살표가 보이고, FSI 출력에는 괄호가 붙는다.
- 예제는 이 차이를 눈에 보이게 하려고 시드를 고정했다. 시드를 주지 않으면 .NET Core 는 인스턴스마다 다른 시드를 쓰므로 두 버전의 출력이 비슷해져 차이가 드러나지 않는다.

```
// 인덱스 값이 필요 없으므로 `_` 로 버린다
let freshRolls = List.init 8 (fun _ -> dieRollFresh ())
let sharedRolls = List.init 8 (fun _ -> dieRollShared ())

printfn "매번 새로 만든 경우: %A" freshRolls // 시드가 같으니 같은 값만 반복된다
printfn "하나를 재사용한 경우: %A" sharedRolls // 난수 수열이 이어지므로 값이 매번 달라진다
// 기대: [6; 6; 6; 6; 6; 6; 6; 6] / [6; 2; 3; 2; 3; 1; 4; 6]
```

## Multiple Parameters — 커링 (원서 p.32)

- 챕터 앞에서 함수는 입력 하나를 받고 출력 하나를 낸다고 했는데, 1챕터에서 만든 `calculateTotal customer spend` 는 매개변수가 둘이었다. 이 모순은 시그니처를 다시 읽으면 풀린다.
- `Customer -> decimal -> decimal` 은 실제로 `Customer -> (decimal -> decimal)` 이다. 즉 `Customer` 하나를 입력으로 받아 `decimal -> decimal` 인 함수를 출력으로 내는 함수다. `->` 는 오른쪽 결합이라 두 표기의 뜻이 같다. 매개변수를 두 줄로 쪼개 `let calculateTotal customer = fun spend -> ...` 로 써 보면 같은 뜻임이 눈에 보인다.

```
// 이 단위가 보여주는 것: 커링된 매개변수의 사슬, 그리고 튜플 매개변수와와의 차이

// float -> float -> float
// 매개변수를 나란히 적는 보통 표기다
let applyTax rate amount = amount * (1.0 + rate)

// float -> float -> float
// 첫 인자를 받고 "함수"를 돌려주는 형태로 직접 써도 위와 같은 함수다
let applyTaxExplicit rate =
    fun amount -> amount * (1.0 + rate)

printfn "applyTax 0.1 1000.0 = %.1f" (applyTax 0.1 1000.0) // 기대: 1100.0
printfn "applyTaxExplicit 0.1 1000.0 = %.1f" (applyTaxExplicit 0.1 1000.0) // 기대: 1100.0
```

사슬 구조를 눈으로 보려면 시그니처에 괄호를 넣어 읽으면 된다. FSI 출력에는 괄호가 없다.

```
// FSI 가 실제로 출력하는 것은 괄호 없는 형태다.
// 아래 괄호는 사슬 구조를 눈으로 보려고 넣은 것이고, `->` 는 오른쪽 결합이라 뜻이 같다
applyTax : float -> (float -> float)
applyTaxAndFee : float -> (float -> (float -> float))
```



- 이렇게 한 입력/한 출력 함수를 자동으로 사슬로 엮어 주는 것을 커링이라 한다. 미국 수학자 해스켈 커리 (Haskell Curry)의 이름에서 왔다.
- 덕분에 겉보기에는 매개변수가 여러 개인 함수를 쓰면서 실제로는 한 입력/한 출력 함수의 연쇄를 다루게 된다. 그리고 이 사슬 구조가 다음 절의 부분 적용을 가능하게 한다.

```
// float -> float -> float -> float
// 매개변수가 세 개여도 결국 한 입력/한 출력의 사슬이다
let applyTaxAndFee rate fee amount = amount * (1.0 + rate) + fee

printfn "applyTaxAndFee 0.1 500.0 1000.0 = %.1f" (applyTaxAndFee 0.1 500.0 1000.0) // 기대: 1600.0

// 인자를 하나만 주면 아직 함수다. 시그니처: float -> float
let withVat = applyTax 0.1
printfn "withVat 2000.0 = %.1f" (withVat 2000.0) // 기대: 2200.0
printfn "withVat 3000.0 = %.1f" (withVat 3000.0) // 기대: 3300.0

// 인자를 하나씩 차례로 적용해도 결과는 같다
let step1 = applyTaxAndFee 0.1 // float -> float -> float
let step2 = step1 500.0 // float -> float
printfn "step2 1000.0 = %.1f" (step2 1000.0) // 기대: 1600.0
```

## Partial Application (Part 1) — 인자를 나눠서 주기 (원서 p.33)

- 커링된 함수에 필요한 인자 전부가 아니라 앞쪽 일부만 주면, 남은 인자를 기다리는 새 함수가 결과로 나온다. 이것이 부분 적용이다.

```
// 이 단위가 보여주는 것: 부분 적용, 판별 유니온으로 만든 전용 로거, 튜플 매개변수의 한계

// string -> int -> string
let padCode (prefix: string) (number: int) =
    sprintf "%s-%04d" prefix number

// 첫 인자만 주면 시그니처가 int -> string 인 함수가 남는다
let orderCode = padCode "ORD"
let refundCode = padCode "RFD"

printfn "padCode \"ORD\" 7 = %s" (padCode "ORD" 7) // 기대: ORD-0007
printfn "orderCode 7 = %s" (orderCode 7) // 기대: ORD-0007
printfn "refundCode 42 = %s" (refundCode 42) // 기대: RFD-0042
```

- `Customer -> decimal -> decimal` 인 함수에 첫 인자만 주면 결과의 시그니처는 `decimal -> decimal` 이다. 여기에 마지막 인자를 주면 원래 함수가 완성되어 최종 값을 낸다.
- 인자는 왼쪽에서 오른쪽 순서로 채워야 한다. 하나씩 채워도 되고 여러 개를 한꺼번에 채워도 되지만, 순서를 건너뛰면 타입이 맞지 않아 컴파일되지 않는다.

```
// 순서를 바꿔서 줄 수는 없다. 첫 인자는 string 자리다
padCode 7 // 컴파일 오류 FS0001
```

- 처음에는 이런 기능이 왜 필요한지 의문이 들 수 있다. 가장 가까운 답은 이 챕터에서 이미 쓰고 있는 `|>` 가 부분 적용 위에서 동작한다는 점이다.

## The Forward Pipe Operator — `|>` 의 내부 동작 (원서 pp.33-36)

- `|>` 는 원서 전체에서 계속 쓰이므로 원리를 한 번 짚어 둘 만하다. 핵심은 부분 적용이다.
- `let complete = 100.0M |> calculateTotal john` 은 부분 적용된 함수에 이름을 붙이는 단계를 생략한 것이다. 연산자 왼쪽의 값이 오른쪽 함수에서 아직 채워지지 않은 첫 인자 자리로 적용된다.

```
// 이 단위가 보여주는 것: `|>` 가 값을 어느 자리에 넣는지, 그리고 그 정의와 `>>` 와의 차이

// float -> float -> float
let applyDiscount rate price = price * (1.0 - rate)

// 부분 적용으로 한 단계씩
let halfOff = applyDiscount 0.5 // float -> float
let step = halfOff 4000.0
printfn "부분 적용: %.1f" step // 기대: 2000.0

// 같은 일을 파이프로. 왼쪽 값이 아직 채워지지 않은 인자 자리로 들어간다
printfn "파이프: %.1f" (4000.0 |> applyDiscount 0.5) // 기대: 2000.0
```

왼쪽 값이 언제나 “마지막” 인자로 들어가는 것은 아니다. 오른쪽 함수에 인자가 둘 다 남아 있으면 첫 인자 자리로 들어간다.

```
// 반례: 0.5 가 rate, 즉 첫 인자로 들어가고 결과는 아직 함수다
let stillAFunction = 0.5 |> applyDiscount // (float -> float)
printfn "0.5 |> applyDiscount 는 아직 함수: %.1f" (stillAFunction 4000.0) // 기대: 2000.0
```

- 검증 코드를 읽기 좋게 만드는 데도 쓸 만하다. `areEqual 90.0M (calculateTotal john 100.0M)` 은 기대값이 앞에 나와 읽기 어색하다. 도우미 함수 이름을 `isEqualTo` 로 바꾸고 `calculateTotal john 100.0M |> isEqualTo 90.0M` 로 쓰면 영어 문장처럼 읽힌다. 이때 `isEqualTo` 는 인자 두 개 중 하나만 받은 부분 적용 상태이고, 남은 인자는 파이프가 넘겨 준다.

```
// 'a -> 'a -> bool (when 'a : equality)
// `=` 를 썼으므로 컴파일러가 동등성 제약을 붙인다
let isEqualTo expected actual = (expected = actual)

// 도우미 함수를 그냥 쓰면 기대값이 앞에 온다
printfn "읽기 어려운 순서: %b" (isEqualTo 2000.0 (applyDiscount 0.5 4000.0)) // 기대: true

// 파이프를 쓰면 "계산 결과가 2000.0과 같다" 순서로 읽힌다
printfn "문장처럼 읽는 순서: %b" (applyDiscount 0.5 4000.0 |> isEqualTo 2000.0) // 기대: true

// 'a -> 'a -> bool (when 'a : comparison) : 인자 순서가 결과를 바꾸는 비대칭 도우미
let isGreaterThan limit actual = actual > limit
printfn "결과가 1000.0 보다 크다: %b" (applyDiscount 0.5 4000.0 |> isGreaterThan 1000.0) // 기대: true
```

- `|>` 의 정의는 `FSharp.Core` 에 있고 대략 `let (|>) v f = f v` 형태다. 시그니처는 `'a -> ('a -> 'b) -> 'b` 다. 실제 정의에는 `inline` 이 붙어 `let inline (|>) arg func = func arg` 이며, 그 덕분에 호출마다 함수 값을 새로 만들지 않는다.

```
// 표준 `|>` 를 덮어쓰지 않도록 같은 동작의 별칭 연산자를 만들어 확인한다
// 'a -> ('a -> 'b) -> 'b
let (|>~) value func = func value

printfn "직접 만든 연산자: %.1f" (4000.0 |>~ applyDiscount 0.5) // 기대: 2000.0
```

- `|>` 는 컴파일러에 특별 취급된 문법이 아니다. 사용자 정의 연산자(custom operator)와 똑같은 방식으로 `FSharp.Core` 에 정의된 보통 함수다.
- 정의할 때 `(|>)` 처럼 괄호를 씌우는 것은 기호로 된 이름을 식별자 자리에 놓기 위한 문법이다. 괄호로 감싼 이름은 보통 함수처럼 앞에 놓고 적용할 수 있다. 이것이 연산자의 함수 형태(operator function form)다: `(|>) (calculateTotal john 100.0M) (isEqualTo 90.0M)`.

```
// 연산자 이름을 괄호로 감싸면 보통 함수처럼 앞에 놓고 적용할 수 있다
printfn "함수 형태: %.1f" ((|>) 4000.0 (applyDiscount 0.5)) // 기대: 2000.0
// 표준 연산자도 마찬가지로
printfn "표준 `|>` 함수 형태: %.1f" ((|>) 4000.0 (applyDiscount 0.5)) // 기대: 2000.0
```

- 괄호를 벗기면 중위 형태(infix form)로 쓰인다: `calculateTotal john 100.0M |> isEqualTo 90.0M`. 두 표기는 같은 함수를 부른다.
- `|>` 는 중위 연산자다. `--` 처럼 값 앞에 붙는 접두 형태(prefix form)로는 쓸 수 없다(`|> 3` 은 컴파일 오류다).

```
// 중위 연산자를 값 앞에 붙일 수는 없다
let x = |> 3 // 컴파일 오류 FS0010: 예기치 않은 중위 연산자
```

- `|>` 는 함수를 매개변수로 받으므로 고차 함수다. 고차 함수는 함수를 하나 이상 매개변수로 받거나 함수를 출력으로 내는 함수를 말한다.

```
let addShipping price = price + 3000.0
let toLabel (price: float) = sprintf "결제 금액 %.0f원" price

// `>>`: 함수와 함수 사이. 결과는 아직 함수다
let checkout = applyDiscount 0.2 >> addShipping >> toLabel // float -> string
printfn "합성 함수 결과: %s" (checkout 10000.0) // 기대: 결제 금액 11000원

// `|>`: 값과 함수 사이. 결과는 바로 값이다
let receipt =
    10000.0
    |> applyDiscount 0.2
    |> addShipping
    |> toLabel
printfn "파이프라인 결과: %s" receipt // 기대: 결제 금액 11000원

// 파이프는 함수를 매개변수로 받으므로 `List.map` 같은 고차 함수와 잘 붙는다
[ 5000.0; 12000.0; 30000.0 ]
|> List.map (applyDiscount 0.1) // 부분 적용된 함수를 그대로 넘긴다
|> List.map toLabel
|> List.iter (printfn " %s") // 기대: 결제 금액 4500원 / 10800원 / 27000원
```

- 연산자 왼쪽의 값은 오른쪽 함수에서 아직 채워지지 않은 첫 인자 자리로 들어간다. 오른쪽에 인자를 하나만 남겨 두는 것이 관용적인 쓰임이므로, 실무에서는 "마지막 인자로 들어간다"라고 이해해도 무리가 없다.

## Partial Application (Part 2) — 로거 예제 (원서 pp.36-38)

- 원서는 `log` 함수로 부분 적용을 다시 보여 준다. 이 함수는 판별 유니온(discriminated union)으로 정의한 로그 레벨과 문자열 메시지를 받으며, 시그니처는 `LogLevel -> string -> unit` 이다.
- 모든 F# 함수는 출력을 내야 하므로 원서는 처음에 `()` 를 마지막 줄에 적는다. 그러나 `printfn` 자체가 `unit` 을 돌려주므로 그 줄은 지울 수 있다.

```

type LogLevel =
  | Error
  | Warning
  | Info

// LogLevel -> string -> unit
// message 에 타입 주석이 없어도 `s` 때문에 string 으로 추론된다.
// `printfn` 이 `unit` 을 돌려주므로 함수 본문이 이 한 줄로 끝난다
let log (level: LogLevel) message =
  printfn "[%A] %s" level message

log Info "커링된 함수로 직접 호출" // 기대: [Info] 커링된 함수로 직접 호출

```

- `printfn` 과 문자열 보간(string interpolation) 둘 다 서식 지정자(format specifier)를 지원한다. 지정자와 실제 타입이 맞지 않으면 런타임 오류가 아니라 컴파일 오류가 난다. 보간 문자열에서는 지정자를 생략할 수도 있지만, 그러면 지정자가 주던 타입 안전성을 잃는다. `printfn $"[{level}]: {message}"` 처럼 쓰면 메시지 쪽이 제네릭 'a 로 추론된다.

```

// LogLevel -> string -> unit
// 문자열 보간으로도 같은 출력을 낼 수 있다. 여기서는 보간 안에 서식 지정자를 함께 썼다
let logInterpolated (level: LogLevel) message =
  printfn $"[%A{level}] %s{message}"

// LogLevel -> 'a -> unit
// 지정자를 생략하면 메시지가 제네릭으로 추론된다(타입 안전성을 잃는다)
let logGeneric (level: LogLevel) message =
  printfn $"[{level}] {message}"

logInterpolated Info "보간 문자열 버전" // 기대: [Info] 보간 문자열 버전
logGeneric Info "문자열도" // 기대: [Info] 문자열도
logGeneric Info 42 // 지정자가 없으니 int 도 통과한다. 기대: [Info] 42

```

- 레벨만 미리 채운 `let logError = log Error` 는 `string -> unit` 인 함수에 이름을 붙인 것이다. 이후에는 레벨을 매번 적지 않고 `logError "..."` 로 호출한다.

```

// 레벨만 고정한 전용 함수: string -> unit
let logError = log Error
let logWarning = log Warning
let logInfo = log Info

logError "부분 적용된 함수로 호출" // 기대: [Error] 부분 적용된 함수로 호출
logWarning "레벨을 매번 적지 않아도 된다" // 기대: [Warning] 레벨을 매번 적지 않아도 된다

```

부분 적용의 결과는 보통 값과 다르지 않으므로 다른 함수에 그대로 넘길 수 있다.

```

// ('a -> unit) -> 'a list -> unit
// string 으로 고정하지 않았으므로 컴파일러가 자동 일반화해 제네릭 함수가 된다
// logError 를 넘기는 순간 'a 가 string 으로 정해진다
let logAll writer messages =
  messages |> List.iter writer

// 로거 자체를 매개변수로 받는다. 부분 적용 결과가 그대로 값처럼 쓰인다
logAll logError [ "디스크 없음"; "연결 끊김" ] // 기대: [Error] 디스크 없음 / [Error] 연결 끊김

```

- 반환 타입이 `unit` 이면 결과를 `let` 으로 바인딩할 필요가 없다. 호출문만 적으면 된다.

```

// 반환 타입이 `unit` 이므로 결과를 `let` 으로 바인딩하지 않아도 된다
logInfo "let 바인딩 없이 그냥 호출" // 기대: [Info] let 바인딩 없이 그냥 호출

```

- 부분 적용은 커링된 매개변수(curried parameters)가 있어야 가능하다. `(LogLevel * string) -> unit`처럼 튜플 매개변수(tupled parameter) 하나를 받는 형태는 튜플 전체를 한꺼번에 줘야 하므로 부분 적용을 할 수 없다.

```
// float -> float -> float : 커링된 매개변수. 하나씩 줄 수 있다
let curriedArea width height = width * height

// (float * float) -> float : 튜플 하나를 받는다. 반드시 한꺼번에 줘야 한다
let tupledArea (width, height) = width * height

printfn "curriedArea 3.0 4.0 = %.1f" (curriedArea 3.0 4.0) // 기대: 12.0
printfn "tupledArea (3.0, 4.0) = %.1f" (tupledArea (3.0, 4.0)) // 기대: 12.0

// 커링된 매개변수는 인자를 하나씩 채워 부분 적용할 수 있다
let heightOf3 = curriedArea 3.0
printfn "heightOf3 4.0 = %.1f, heightOf3 5.0 = %.1f" (heightOf3 4.0) (heightOf3 5.0) // 기대: 12.0, 15.0
```

```
// (LogLevel * string) -> unit : 로거도 튜플 매개변수로 바꾸면 부분 적용을 잃는다
let logTupled (level: LogLevel, message: string) =
    printfn "[%A] %s" level message

logTupled (Warning, "튜플은 한꺼번에 줘야 한다") // 기대: [Warning] 튜플은 한꺼번에 줘야 한다
```

튜플은 절반만 채울 수 없다. 아래 두 줄은 모두 타입 불일치로 컴파일되지 않는다.

```
tupledArea 3.0 // 컴파일 오류 FS0001
logTupled Error // 컴파일 오류 FS0001
```

## Summary — 원서의 챕터 요약 (원서 p.38)

- 원서는 이 챕터에서 순수 함수, 익명 함수, 함수 합성, 튜플, 복사-수정 레코드 식, 커링된 매개변수와 튜플 매개변수, 커링과 부분 적용을 다뤘다.
- 여기까지 오면 F# 프로그래밍의 기본 재료가 갖춰진다. 타입과 함수의 조합, 식, 불변성이 그 재료다.
- 다음 챕터에서는 F# 이 `null` 과 예외를 다루는 방법을 살펴본다.

## 정리 — 이 노트의 요약

- 함수의 규칙은 입력 하나, 출력 하나다. 매개변수가 여러 개로 보이는 함수는 커링된 한 입력/한 출력 함수의 사슬이다.
- 시그니처를 읽으면 그 함수를 어떻게 쓸 수 있는지가 드러난다. 합성 가능성, 부분 적용 가능성, 함수인지 값인지가 모두 시그니처에 적혀 있다.
- `>>` 는 함수와 함수를 잇고, `|>` 는 값을 함수에서 아직 채워지지 않은 첫 인자 자리로 밀어 넣는다. 오른쪽에 인자를 하나만 남겨 두는 관용적인 쓰임에서는 그 자리가 마지막 인자와 같아진다. 기본 스타일은 `|>` 다.
- `|>` 는 마법이 아니라 `let (|>) v f = f v` 로 정의된 고차 함수이며, 부분 적용에 의존한다.
- 부분 적용은 커링된 매개변수 사슬에서 일어난다. 튜플 매개변수는 그 자체가 한 개의 인자여서 절반만 채울 수 없다.
- `unit` 은 입력이나 출력이 없는 자리를 채우는 타입이고, `unit` 이 유일한 입력이나 유일한 출력으로 보이면 대개 부수 효과를 의심할 만하다.
- 레코드는 불변이고 구조적 동등성이 있으므로, 복사-수정 식으로 새 값을 만들고 `=` 로 검증하는 흐름이 자연스럽다.

## 원서 대조 표

| 절                                                      | 원서 페이지   | 실행 단위                                                          |
|--------------------------------------------------------|----------|----------------------------------------------------------------|
| Getting Started — 함수의 규칙과 순수 함수                        | p.26     | <code>02-signatures-and-composition</code>                     |
| Theory — 시그니처가 맞물려야 합성된다                               | pp.26-27 | <code>02-signatures-and-composition</code>                     |
| In Practice — 네 가지 표기, 같은 결과                           | pp.27-29 | <code>02-signatures-and-composition</code>                     |
| Unit — 입력도 출력도 없을 때                                    | pp.30-31 | <code>02-unit</code>                                           |
| Anonymous Functions — 이름 없는 함수                         | pp.31-32 | <code>02-anonymous-functions</code>                            |
| Multiple Parameters — 커링                               | p.32     | <code>02-currying</code>                                       |
| Partial Application (Part 1) — 인자를 나눠서 주기              | p.33     | <code>02-partial-application</code>                            |
| The Forward Pipe Operator — <code> &gt;</code> 의 내부 동작 | pp.33-36 | <code>02-forward-pipe</code>                                   |
| Partial Application (Part 2) — 로거 예제                   | pp.36-38 | <code>02-partial-application</code> , <code>02-currying</code> |
| Summary — 원서의 챕터 요약                                    | p.38     | —                                                              |

## 03 - null 과 예외 처리 (원서 pp.39-51)

이 챕터는 "값이 없을 수도 있다"와 "실패할 수도 있다"를 F# 이 어떻게 다루는지 보여 준다. 답은 둘 다 타입이다. 없음은 `Option`, 실패는 `Result` 로 표현한다. 두 타입 모두 판별 유니온(discriminated union)이므로 특별한 문법을 새로 배울 필요 없이 1챕터에서 배운 도구로 이해할 수 있다. 여기서 함께 나오는 `map` 과 `bind` 는 시그니처가 맞물리지 않는 함수를 합성에 끼워 넣는 장치이며, 뒤 챕터 전체가 이 두 함수에 기댄다. 8챕터의 검증 파이프라인과 12챕터의 계산 식(computation expression)은 사실 이 챕터의 `Result.map` / `Result.bind` 를 더 읽기 좋게 감싼 것이다.

시작하기 전에 이 챕터에서 새로 등장하는 용어 하나를 정리해 둔다.

- 고차 함수(higher-order function)는 함수를 매개변수로 받거나 함수를 반환하는 함수다. `Option.map`, `Result.bind` 가 모두 여기 해당한다. 2챕터에서 본 `|>` 도 고차 함수였다.

### Null Handling — `Option` 으로 없음을 타입에 적기 (원서 pp.39-41)

- F# 코드를 쓰는 동안에는 `null` 을 거의 만지지 않는다. 값이 있을 수도 없을 수도 있는 자리에는 `Option` 을 쓴다.
- `Option` 은 언어에 내장된 판별 유니온이며 개념적으로 아래 모양이다. 값이 있으면 `Some`, 없으면 `None` 이다.
- `'T` 처럼 이름 앞에 붙은 작은따옴표는 타입 매개변수(type parameter) 표시다. 덕분에 어떤 타입이든 "있을 수도 없을 수도 있는 값"으로 감쌀 수 있다.
- 이 정의는 언어에 이미 있으므로 직접 선언하면 안 된다. 아래 블록에 `id` 를 붙이지 않은 이유가 그것이다.

```
// 개념 확인용. 실제로 선언하면 내장 Option 과 충돌한다
// 실제 정의는 None 이 먼저다
type Option<'T> =
    | Some of 'T
    | None
```

- `Option` 이 처음 등장하는 자리는 .NET 의 `TryParse` 계열이다. F# 에는 `out` 매개변수를 선언하는 문법이 없어서, 상호운용에서는 컴파일러가 `out` 매개변수를 반환값 쪽으로 옮겨 붙여 준다. 그 결과 `DateTime.TryParse` 는 `bool * DateTime` 튜플을 돌려준다. `let mutable` 값을 만들어 `&value` 로 직접 넘길 수도 있지만, 튜플로 받는 쪽이 관용적이다.

```
// 이 단위가 보여주는 것: Option 을 만드는 방법과 Option 을 다루는 모듈 함수
open System

// string -> DateTime option
let tryParseDate (text: string) =
    let parsed, value =
        DateTime.TryParse(text, Globalization.CultureInfo.InvariantCulture, Globalization.DateTimeStyles.None)
    if parsed then Some value else None

// 날짜 값 자체를 출력하면 실행 환경의 문화권 설정에 따라 모양이 달라지므로 서식을 고정해 찍는다
printfn "%A" (tryParseDate "2024-03-18" |> Option.map (fun d -> d.ToString("yyyy-MM-dd"))) // 기대: Some "2024-03-18"
printfn "%A" (tryParseDate "내일" |> Option.map (fun d -> d.ToString("yyyy-MM-dd"))) // 기대: None
```

`if` 식 대신 `match` 식(match expression)으로 튜플을 분해해도 결과는 같다. 원서는 네 가지 표기를 차례로 보여 주는데, 갈라지는 지점은 실패 쪽 패턴을 얼마나 줄여 적는지뿐이다. `false, _` 로 명시하거나 `_, _` 로 두 자리를 모두 와일드카드(wildcard)로 두거나, `_` 하나로 튜플 전체를 받을 수 있다.

```
// string -> DateTime option
let tryParseDateByMatch (text: string) =
    match DateTime.TryParse(text, Globalization.CultureInfo.InvariantCulture, Globalization.DateTimeStyles.None) with
    | true, value -> Some value
    | false, _ -> None
    // | _ -> None 으로 줄여도 같다. 튜플 전체가 와일드카드 하나에 걸린다

printfn "%b" (tryParseDateByMatch "2024-03-18" = tryParseDate "2024-03-18") // 기대: true
```

- 어느 표기든 옳다. 의도가 가장 빨리 읽히는 것은 `if` 식과 `true, value / false, _` 쌍이다. 와일드카드를 넓게 쓸수록 짧아지지만 무엇을 버리는지가 흐려진다.
- `Option` 의 두 번째 용도는 선택적 데이터다. 사람의 중간 이름, 곡의 부제처럼 원래 없을 수 있는 필드를 레코드(record)에 담을 때 쓴다.
- 타입을 적는 방법이 두 가지다. `Option<string>` 처럼 제네릭 표기를 쓰거나 `string option` 처럼 뒤에 붙이는 표기를 쓴다. 뜻은 같고, 실무에서는 뒤쪽 표기를 더 자주 본다.

```
// Subtitle 은 있을 수도 없을 수도 있다. Option<string> 이라고 적어도 뜻은 같다
type Track = { Title: string; Subtitle: string option; Seconds: int }

let plain = { Title = "Aurora"; Subtitle = None; Seconds = 214 }

// 2행터의 복사-수정 레코드 식(copy-and-update record expression)으로 부제만 채운다
let annotated = { plain with Subtitle = Some "Live at Oslo" }

// Track -> string
let describe track =
    match track.Subtitle with
    | Some sub -> $"%s{track.Title} (%s{sub})"
    | None -> track.Title

printfn "%s / %s" (describe plain) (describe annotated) // 기대: Aurora / Aurora (Live at Oslo)
```

`Option` 을 꺼내 쓸 때마다 `match` 식을 적는 것은 손이 많이 간다. 자주 쓰는 조합은 `Option` 모듈에 이미 함수로 들어 있다. 네 함수의 시그니처를 FSI 로 실측한 값은 다음과 같다.

```
Option.map          : ('a -> 'b) -> 'a option -> 'b option
Option.bind         : ('a -> 'b option) -> 'a option -> 'b option
Option.defaultValue : 'a -> 'a option -> 'a
Option.defaultWith  : (unit -> 'a) -> 'a option -> 'a
```

- `Option.map` 은 `Some` 안의 값에 보통 함수를 적용하고 다시 `Some` 으로 감싼다. `None` 이면 함수를 부르지 않고 `None` 을 그대로 흘린다.
- `Option.bind` 는 함수 자체가 `Option` 을 돌려줄 때 쓴다. `map` 을 썼다면 `'b option option` 이 되어 겹쳐 버리는데, `bind` 는 한 겹으로 눌러 준다.
- `Option.defaultValue` 는 `Option` 을 벗겨 평범한 값으로 되돌린다. `None` 이면 미리 준 기본값이 나온다.
- `Option.defaultValue` 는 기본값을 먼저 평가한다. 기본값을 만드는 데 비용이 들면 `Option.defaultWith` 를 쓴다. 8행터의 검증 파이프라인에서 다시 만난다.



```
// DateTime -> DateTime option (주말이면 없음으로 취급한다)
let onlyWeekday (d: DateTime) =
    match d.DayOfWeek with
    | DayOfWeek.Saturday | DayOfWeek.Sunday -> None
    | _ -> Some d

// onlyWeekday 가 Option 을 내므로 bind 로 잇는다. map 을 쓰면 두 겹이 된다
// 2024-03-18 은 월요일, 2024-03-17 은 일요일이다
let monday = tryParseDate "2024-03-18" |> Option.bind onlyWeekday |> Option.map (fun d -> d.DayOfWeek.ToString())
let sunday = tryParseDate "2024-03-17" |> Option.bind onlyWeekday |> Option.map (fun d -> d.DayOfWeek.ToString())

printfn "%A %A" monday sunday // 기대: Some "Monday" None
printfn "%s | %s"
    (monday |> Option.defaultValue "평일 아님")
    (sunday |> Option.defaultValue "평일 아님") // 기대: Monday | 평일 아님
```

- 파이프라인 어디서든 `None` 이 한 번 나오면 뒤 단계는 계산되지 않고 `None` 이 끝까지 흐른다. `null` 검사를 단계마다 손으로 넣던 코드가 사라지는 지점이다.
- F# 코드만으로 이루어진 세계에서는 `null` 이 끼어들 틈이 거의 없다. 문제는 다른 .NET 언어와 맞닿는 경계다.

## Interop With .NET — `null` 이 넘어오는 유일한 통로 (원서 pp.42-43)

- C# 등 다른 .NET 언어로 쓰인 코드와 주고받는 값에는 `null` 이 섞일 수 있다. .NET 플랫폼 자체의 API 도 마찬가지다.
- 경계에서 곧바로 `Option` 으로 바꿔 놓고, 안쪽에서는 `Option` 만 다루는 것이 기본 전략이다.
- `null` 은 두 갈래로 온다. 참조 타입의 `null` 과, 값 타입을 감싼 `Nullable<'T>` 의 빈 상태다. 변환 함수도 갈래별로 따로 있다.

```
// 이 단위가 보여주는 것: .NET 경계에서 null 을 Option 으로 바꾸고 되돌리는 길
open System

// 참조 타입의 null
let missingNickname : string = null

// 값 타입을 감싼 Nullable. 인자를 주지 않으면 빈 상태다
let missingScore = Nullable<int>()

printfn "%b %b" (isNull missingNickname) missingScore.HasValue // 기대: true false
```

`Option` 모듈의 변환 함수 네 개다. 시그니처는 FSI 로 실측한 값이며, 제약이 붙어 있다는 점이 중요하다. `ofObj / toObj` 는 참조 타입에만, `ofNullable / toNullable` 은 값 타입에만 쓸 수 있다. `'a | null` 은 F# 9 에서 들어온 null 허용 참조 타입(nullable reference types) 표기이고, FSharp.Core 시그니처에 nullness 정보가 붙어 있어 이렇게 보인다. `'a` 또는 `null` 로 읽으면 된다. 검사 자체는 기본으로 꺼져 있어 앞 블록의 `missingNickname` 바인딩이 경고 없이 통과한다. 커려면 컴파일러나 FSI 에 `--checknulls+` 옵션을 준다.

```
Option.ofObj      : 'a | null -> 'a option (when 'a : not struct and 'a : not null)
Option.toObj      : 'a option -> 'a | null (when 'a : not struct)
Option.ofNullable : System.Nullable<'a> -> 'a option (when 'a : (new : unit -> 'a) and 'a : struct and 'a :> System.ValueTy
Option.toNullable : 'a option -> System.Nullable<'a> (when 'a : (new : unit -> 'a) and 'a : struct and 'a :> System.ValueTy
```

- 뒤쪽 두 제약은 `Nullable<'T>` 가 요구하는 값 타입 조건을 컴파일러가 풀어 쓴 것이다. 읽을 때는 `'a : struct` 하나만 보면 된다.

```
let presentNickname = "kestrel"
let presentScore = Nullable<int>(42)

// .NET -> Option
let nickNone = Option.ofObj missingNickname
let nickSome = Option.ofObj presentNickname
let scoreNone = Option.ofNullable missingScore
let scoreSome = Option.ofNullable presentScore

printfn "%A %A %A %A" nickNone nickSome scoreNone scoreSome
// 기대: None Some "kestrel" None Some 42
```

되돌리는 방향도 짝이 맞는다. `None` 은 `null` 또는 빈 `Nullable` 로, `Some` 은 안의 값으로 돌아간다.

```
// Option -> .NET
let backToNull = Option.toObj nickNone
let backToNullable = Option.toNullable scoreSome

printfn "%b %A" (isNull backToNull) backToNullable // 기대: true 42
```

- `null` 대신 자리표시자 문자열을 기대하는 상대와 맞닿을 때도 있다. 그럴 때는 `Option` 이 판별 유니온이라는 사실을 이용해 `match` 식으로 꺼내거나, `Option.defaultValue` 를 쓴다.

```
// string option -> string
let byMatch input =
    match input with
    | Some value -> value
    | None -> "(닉네임 없음)"

printfn "%s" (byMatch nickNone) // 기대: (닉네임 없음)
printfn "%s" (Option.defaultValue "(닉네임 없음)" nickNone) // 기대: (닉네임 없음)
printfn "%s" (nickNone |> Option.defaultValue "(닉네임 없음)") // 기대: (닉네임 없음)
```

- 세 줄이 모두 같은 값을 낸다. 셋째 줄은 파이프 연산자로 적은 것이라 파이프라인 중간에 그대로 끼워 넣을 수 있다.
- 같은 기본값을 여러 곳에서 쓴다면 2챕터의 부분 적용(partial application)이 잘 맞는다. `Option.defaultValue` 는 매개변수가 커링되어 있으므로 기본값만 먼저 주면 `Option` 을 기다리는 함수가 남는다.

```
// FSI 실측: val orAnonymous: (string option -> string)
let orAnonymous = Option.defaultValue "(닉네임 없음)"

printfn "%s / %s" (orAnonymous nickNone) (orAnonymous nickSome) // 기대: (닉네임 없음) / kestrel

// 기본값의 타입이 곧 Option 의 내용 타입을 정한다
printfn "%d" (scoreNone |> Option.defaultValue 0) // 기대: 0
```

- 경계에서 이 변환을 성실히 해 두면 실행 중에 `NullReferenceException` 을 볼 일이 사실상 없어진다.

## Handling Exceptions — 시그니처가 거짓말하지 않게 (원서 pp.43-45)

- 예외를 던지는 함수의 문제는 시그니처가 사실을 다 말하지 않는다는 점이다.
- 아래 나눗셈 함수의 시그니처는 `decimal -> decimal -> decimal` 이다. 그런데 분모가 0 이면 값을 내지 않고 예외를 던진다. 이 사실은 구현을 열어 보거나 실행해서 터뜨려 봐야 안다. `try/with` 를 본 뒤에 실제로 터뜨려 확인한다.

```
// 이 단위가 보여주는 것: try/with 식, 예외를 던지는 함수의 시그니처, Result 로 실패를 드러내기
open System

// FSI 실측: decimal -> decimal -> decimal
// 시그니처만 보면 언제나 decimal 을 낸다고 읽히지만 사실이 아니다
let divideUnsafe (numerator: decimal) (denominator: decimal) =
    numerator / denominator
```

- F# 에서 `try/with` 는 문(statement)이 아니라 값을 내는 식(expression)이다. C# 의 `try-catch` 와 역할은 같지만, 두 갈래가 모두 같은 타입의 값을 내야 한다는 제약이 붙는다.
- 아래 함수를 FSI 에 넣으면 `text: string -> int` 로 나온다. `try` 쪽의 `Int32.Parse text` 와 `with` 쪽의 `0` 이 모두 `int` 이므로 식 전체가 `int` 다.

```
// FSI 실측: text: string -> int
// try 갈래와 with 갈래가 같은 타입을 내야 하나의 식이 된다
let parsedOrZero (text: string) =
    try Int32.Parse text
    with :? FormatException -> 0

printfn "%d %d" (parsedOrZero "17") (parsedOrZero "열일곱") // 기대: 17 0
```

```
// 시그니처가 말하지 않은 실패를 눈으로 확인한다. decimal 나눗셈은 0 으로 나누면 예외다
// float 는 예외 없이 infinity 를 내므로 이 예제는 decimal 을 쓴다
try printfn "%M" (divideUnsafe 7M 0M)
with :? DivideByZeroException as ex -> printfn "%s" (ex.GetType().Name) // 기대: DivideByZeroException
```

- `raise` 와 `failwith` 는 이 규칙에서 빠져나가는 것처럼 보인다. FSI 로 확인하면 `raise` 는 `System.Exception -> 'a` , `failwith` 는 `string -> 'a` 다. 반환 타입이 타입 매개변수인 이유는 정상적으로 값을 돌려주는 일이 없기 때문이다. 값을 내지 않으니 어떤 타입 자리에는 끼울 수 있고, 그래서 `with` 갈래에 `failwith` 를 두어도 타입이 어긋나지 않는다.
- 실패를 시그니처에 드러내려면 성공과 실패 중 하나를 담는 타입이 필요하다. 원서는 아래와 같은 직접 정의를 먼저 보여 주고, 곧 F# 4.1 부터 언어에 들어 있는 `Result` 로 넘어간다. 내장 타입의 두 케이스 식별자는 `ok` 와 `Error` 다.

```
// 개념 확인용. 실제로는 내장 Result<'T, 'TError> 를 쓴다
type Result<'TSuccess, 'TFailure> =
    | Success of 'TSuccess
    | Failure of 'TFailure
```

- `Result` 의 타입 매개변수는 두 개다. 앞이 성공값 타입, 뒤가 실패값 타입이다. `Option` 과 달리 실패에도 값이 실린다는 것이 핵심 차이이다. 왜 실패했는지를 전달할 수 있다.
- `try/with` 안의 `?:` 는 타입 테스트 패턴(type test pattern)이다. 던진 예외가 `?:` 뒤에 적어 둔 타입이거나 그 하위 타입인지를 런타임에 검사할 뿐, 값을 변환하지는 않는다. 일치하면 `as ex` 가 그 타입으로 좁혀진 예외 인스턴스를 `ex` 에 바인딩하고, 그 값을 `Error` 의 케이스 데이터로 넘긴다.
- 여기에 걸리지 않은 예외는 그대로 호출 사슬 위로 올라간다. 다른 .NET 코드와 동작이 같다.
- 먼저 반환 타입 주석을 붙이지 않은 판을 본다. 실패 타입은 컴파일러가 `with` 갈래에서 잡은 예외 타입으로 좁혀 추론한다.

```
// 반환 타입 주석이 없는 판. 실패 타입이 잡은 예외 타입으로 좁혀진다
// FSI 실측: x: decimal -> y: decimal -> Result<decimal, System.DivideByZeroException>
let tryDivideNarrow (x: decimal) (y: decimal) =
    try Ok (x / y)
    with :? DivideByZeroException as ex -> Error ex

printfn "%b" (tryDivideNarrow 7M 2M = Ok 3.5M)    // 기대: true
```

- 실패 타입을 좁게 두면 그 함수 하나를 볼 때는 정보가 많아 좋다. 하지만 여러 단계를 이어 붙이려면 단계마다 실패 타입이 같아야 하므로, 파이프라인 전체가 공유하는 실패 타입을 미리 정해 두는 편이 편하다.
- 넓히는 방법은 두 가지다. 반환 타입을 `Result<decimal, exn>` 으로 주석하거나, `Error` 에 답을 때 `ex` `:> exn` 으로 상향 변환한다.

```
// 반환 타입 주석으로 실패 타입을 exn 으로 넓힌 판
// FSI 실측: numerator: decimal -> denominator: decimal -> Result<decimal,exn>
let tryDivide (numerator: decimal) (denominator: decimal) : Result<decimal, exn> =
    try
        Ok (numerator / denominator)
    with
        | :? DivideByZeroException as ex -> Error ex

// FSI 실측: label: string -> result: Result<decimal,exn> -> unit
let show label result =
    match result with
    | Ok value -> printfn "%-5s Ok %M" label value
    | Error (ex: exn) -> printfn "%-5s Error %s" label (ex.GetType().Name)

show "7/2" (tryDivide 7M 2M)    // 기대: 7/2    Ok 3.5
show "7/0" (tryDivide 7M 0M)    // 기대: 7/0    Error DivideByZeroException
```

- 원서 주석에는 `Result<decimal,exn>` 이라 적혀 있다. F# 10 컴파일러는 반환 타입 주석이 없으면 실패 타입을 `DivideByZeroException` 으로 좁히므로, 대조하며 읽을 때 혼동하지 않도록 두 판을 나란히 남겼다.
- 원서 p.45 의 `// Some 1M` 주석은 `// Ok 1M` 의 오타다. 그 시점 `tryDivide` 는 `Option` 이 아니라 `Result` 를 낸다.

```
// 명시적 상향 변환으로 넓힌 판. 반환 타입 주석 없이도 실패 타입이 exn 이 된다
// FSI 실측: x: decimal -> y: decimal -> Result<decimal,exn>
let tryDivideWide (x: decimal) (y: decimal) =
    try Ok (x / y)
    with :? DivideByZeroException as ex -> Error (ex :> exn)

printfn "%b" (tryDivideWide 7M 2M = Ok 3.5M)    // 기대: true
```

- 세 판은 같은 계산이고 다른 것은 실패 타입뿐이다. 파이프라인에 넣을 것은 실패 타입이 `exn` 으로 통일된 판이다.

## Function Composition With Result — map 과 bind (원서 pp.45-51)

- 2챕터에서 배운 합성 조건은 앞 함수의 출력 타입과 뒤 함수의 입력 타입이 같아야 한다는 것이었다. `Result` 를 도입하면 이 조건이 자주 깨진다.
- 원서는 관측소 예제가 아닌 고객 예제를 쓰지만 구조는 같다. 세 단계를 잇는데 가운데 단계만 `Result` 를 모른다.
- 스콧 블라신(Scott Wlaschin)은 이 구조를 나란한 두 선로에 비유해 철도 지향 프로그래밍(Railway Oriented Programming)이라 부른다. `ok` 선로를 달리다가 실패가 나면 `Error` 선로로 갈아타고 끝까지 그 선로로 간다. 가운데 단계처럼 `Result` 를 내지 않는 함수는 한쪽 선로만 다니는 함수다.

```
// 이 단위가 보여주는 것: Result 가 끼면 합성이 끊기는 지점과 map/bind 로 다시 잇는 법
open System

type Reading = { StationId: int; Celsius: decimal; Verified: bool }

// 이력을 읽어 판독값과 결측 일수를 함께 돌려준다. 데이터베이스에서 읽어 오는 상황을 가정한 코드다
// FSI 실측: reading: Reading -> Result<(Reading * int), exn>
// FSI 는 성공값이 튜플이면 Result<(Reading * int), exn> 처럼 괄호를 넣어 표시한다
// 소스에 적은 Result<Reading * int, exn> 과 같은 타입이고 표시 방식만 다르다
let loadHistory reading : Result<Reading * int, exn> =
    try
        let missingDays = if reading.StationId % 3 = 0 then 7 else 1
        Ok (reading, missingDays)
    with ex -> Error ex

// 한쪽 선로만 다니는 함수. Result 를 받지도 내지도 않는다
// FSI 실측: reading: Reading * missingDays: int -> Reading
let markVerified (reading, missingDays) =
    if missingDays <= 3 then { reading with Verified = true } else reading

// 다시 Result 를 내는 함수
// FSI 실측: reading: Reading -> Result<Reading, exn>
let calibrate reading : Result<Reading, exn> =
    try
        let offset = if reading.Verified then 0.2M else 1.5M
        Ok { reading with Celsius = reading.Celsius + offset }
    with ex -> Error ex
```

세 함수를 그냥 이어 붙이면 두 곳에서 타입이 어긋난다. `loadHistory` 는 `Result<Reading * int, exn>` 을 내는데 `markVerified` 는 `Reading * int` 를 기다린다. `calibrate` 도 `Reading` 을 기다리지만, 앞 단계를 지난 결과는 이미 `Result` 로 감싸여 있다.

```
// 컴파일 오류. Result<Reading * int, exn> 을 Reading * int 자리에 넣을 수 없다
let refineBroken reading =
    reading
    |> loadHistory
    |> markVerified
    |> calibrate
```

- 첫 번째 틸드는 익명 함수(anonymous function)에 `match` 식을 넣어 메울 수 있다. `ok` 안의 튜플을 꺼내 `markVerified` 에 주고, 결과를 다시 `ok` 로 감싼다. `Error` 는 손대지 않고 그대로 넘긴다.
- 두 번째 틸드도 같은 방식인데 `calibrate` 가 이미 `Result` 를 내므로 `ok` 로 감싸지 않는다. 이 차이가 곧 `map` 과 `bind` 의 차이다.

```
// 익명 함수와 match 식으로 직접 메운 형태
let refineRaw reading =
  reading
  |> loadHistory
  |> fun result ->
    match result with
    | Ok pair -> Ok (markVerified pair)      // Ok 로 다시 감싼다
    | Error ex -> Error ex
  |> fun result ->
    match result with
    | Ok r -> calibrate r                    // calibrate 가 이미 Result 를 낸다
    | Error ex -> Error ex
```

`fun result -> match result with` 처럼 받은 값을 바로 `match` 식에 넘길 때는 `function` 키워드로 줄여 쓸 수 있다. 둘 중 어느 쪽을 써도 좋다.

```
let refineFn reading =
  reading
  |> loadHistory
  |> function
    | Ok pair -> Ok (markVerified pair)
    | Error ex -> Error ex
  |> function
    | Ok r -> calibrate r
    | Error ex -> Error ex
```

- 다음 단계는 이 익명 함수를 이름 있는 함수로 빼내는 것이다. 빼내는 순간 매개변수가 두 개인 함수가 되는데, 첫 매개변수가 함수다. 즉 고차 함수다.
- 그리고 함수 본문에서 `Reading` 이나 `exn` 같은 구체 타입을 하나도 쓰지 않으므로, 타입 주석을 떼면 컴파일러가 자동 일반화(auto-generalization)로 제네릭 함수를 만들어 준다.

```
// FSI 실측: f: ('a -> 'b) -> result: Result<'a,'c> -> Result<'b,'c>
// 성공값만 바꾸고 실패 타입 'c' 는 건드리지 않는다
// | Error err -> Error err 를 | e -> e 로 줄일 수 없다
// 들어오는 타입은 Result<'a,'c>, 나가는 타입은 Result<'b,'c> 라서 성공 타입이 다르다
// 같은 값을 그대로 흘리면 'a' 와 'b' 가 같은 타입으로 묶여 제네릭이 무너진다
let mapOk f result =
  match result with
  | Ok value -> Ok (f value)
  | Error err -> Error err

// FSI 실측: f: ('a -> Result<'b,'c>) -> result: Result<'a,'c> -> Result<'b,'c>
// 받는 함수 자체가 Result 를 내므로 겹치지 않게 그대로 흘린다
let bindOk f result =
  match result with
  | Ok value -> f value
  | Error err -> Error err

let refineMine reading =
  reading
  |> loadHistory
  |> mapOk markVerified
  |> bindOk calibrate
```

- 시그니처를 나란히 놓고 보면 차이가 한 군데뿐이다. `mapOk` 의 첫 매개변수는 `'a -> 'b`, `bindOk` 의 첫 매개변수는 `'a -> Result<'b,'c>` 다. 넘기는 함수가 `Result` 를 내느냐로 갈린다.
- 실패 타입 `'c` 는 두 함수 모두 그대로 통과시킨다. 제네릭이 된 순간 `'c` 는 예외일 필요가 없다. 문자열이든 판별 유니온이든 원하는 타입을 쓸 수 있다.
- 이 두 함수를 굳이 손으로 만들 필요는 없다. `Result` 모듈에 `Result.map` 과 `Result.bind` 라는 이름으로 똑같은 것이 이미 들어 있다. 원서가 앞의 두 함수에 `map / bind` 라는 이름을 붙인 것도 그래서다. 아래 시그니처는 FSI 실측값이다.

```
Result.map      : ('a -> 'b) -> Result<'a,'c> -> Result<'b,'c>
Result.bind     : ('a -> Result<'b,'c>) -> Result<'a,'c> -> Result<'b,'c>
Result.mapError : ('a -> 'b) -> Result<'c,'a> -> Result<'c,'b>
```

- `Result.mapError` 의 타입 매개변수 순서를 잘 봐야 한다. 성공 타입 `'c` 가 유지되고 실패 타입이 `'a` 에서 `'b` 로 바뀐다. `map` 의 거울상이다.
- `Option` 쪽 쪽과 비교해 두면 기억하기 쉽다. `Option.map` 은 `('a -> 'b) -> 'a option -> 'b option`, `Option.bind` 는 `('a -> 'b option) -> 'a option -> 'b option` 이다. 감싸는 타입만 다르고 역할 분담은 같다.

```
// 직접 만든 mapOk/bindOk 를 표준 함수로 바꾼 최종 형태
let refine reading =
  reading
  |> loadHistory
  |> Result.map markVerified
  |> Result.bind calibrate

let quiet = { StationId = 1; Celsius = 21.0M; Verified = false } // 결측 1일 -> 검증 통과
let gappy = { StationId = 3; Celsius = 21.0M; Verified = false } // 결측 7일 -> 검증 실패

printfn "%b" (refine quiet = Ok { StationId = 1; Celsius = 21.2M; Verified = true }) // 기대: true
printfn "%b" (refine gappy = Ok { StationId = 3; Celsius = 22.5M; Verified = false }) // 기대: true
```

- 파이프라인이 마음에 들지 않으면 중간 결과를 `let` 으로 하나씩 받아도 된다. 같은 계산이다. 파이프라인 쪽이 짧고, 중간 이름을 지어 줄 필요가 없다.

```
let refineSteps reading =
  let loaded = loadHistory reading
  let marked = Result.map markVerified loaded
  Result.bind calibrate marked

// 지금까지 만든 다섯 가지 표기가 모두 같은 값을 낸다
printfn "%b %b %b %b"
  (refineRaw quiet = refine quiet)
  (refineFn quiet = refine quiet)
  (refineMine quiet = refine quiet)
  (refineSteps quiet = refine quiet) // 기대: true true true true
```

## 실패 타입을 직접 정하기 — `Result.mapError` (원서 p.48 확장)

- 제네릭이 된 `Result` 의 실패 자리에는 어떤 타입이든 들어간다. 실무에서는 `exn` 대신 도메인 오류를 나열한 판별 유니온을 두는 편이 훨씬 유용하다. 실패 종류가 타입에 적히므로 `match` 식에서 빠뜨린 케이스를 컴파일러가 잡아 준다.
- 문제는 단계마다 실패 타입이 달라질 수 있다는 점이다. `Result.bind` 로 이으려면 실패 타입이 같아야 한다. `Result.mapError` 가 그 어긋남을 메운다.

```
// 이 단위가 보여주는 것: 도메인 오류 판별 유니온, Result.mapError 로 실패 타입 맞추기
open System

type UploadError =
    | EmptyPayload
    | TooLarge of limitKb: int
    | Unexpected of message: string

// FSI 실측: bytes: int -> Result<int,exn>
let readSize (bytes: int) : Result<int, exn> =
    try
        if bytes < 0 then raise (ArgumentException "음수 크기")
        Ok bytes
    with ex -> Error ex

// FSI 실측: ex: exn -> UploadError
let toUploadError (ex: exn) = Unexpected ex.Message

// FSI 실측: bytes: int -> Result<int,UploadError>
let checkSize bytes =
    if bytes = 0 then Error EmptyPayload
    elif bytes > 4096 then Error (TooLarge 4)
    else Ok bytes
```

`readSize` 의 실패 타입은 `exn` , `checkSize` 의 실패 타입은 `UploadError` 다. 그대로는 `Result.bind` 로 이어지지 않는다. `Result.mapError toUploadError` 를 사이에 끼워 앞 단계의 실패 타입을 뒤 단계에 맞춘다. 2 챕터의 어댑터 함수와 발상이 같은데, 이번에는 실패 선로 쪽에 끼우는 것이다.

```
// FSI 실측: bytes: int -> Result<string,UploadError>
let upload bytes =
    bytes
    |> readSize
    |> Result.mapError toUploadError           // exn 을 UploadError 로 맞춘다
    |> Result.bind checkSize
    |> Result.map (fun b -> $"업로드 %d{b} 바이트")

// FSI 실측: result: Result<string,UploadError> -> string
let describeUpload result =
    match result with
    | Ok message -> message
    | Error EmptyPayload -> "실패: 빈 파일"
    | Error (TooLarge limit) -> $"실패: %d{limit}KB 초과"
    | Error (Unexpected msg) -> $"실패: %s{msg}"

for input in [ 1024; 0; 9000; -1 ] do
    printfn "%6d -> %s" input (describeUpload (upload input))
// 기대:
// 1024 -> 업로드 1024 바이트
// 0 -> 실패: 빈 파일
// 9000 -> 실패: 4KB 초과
// -1 -> 실패: 음수 크기
```

- `Option` 과 `Result` 사이를 옮겨야 할 때도 있다. `Option` 은 왜 없는지를 담지 못하므로, `Result` 로 옮길 때 실패 이유를 새로 붙여 준다. 아래 변환 함수는 F# 표준 모듈에 없어 직접 만든 것이다.

```
// FSI 실측: error: 'a -> opt: 'b option -> Result<'b,'a>
let ofOption error opt =
    match opt with
    | Some value -> Ok value
    | None -> Error error

printfn "%A" (Some 3 |> ofOption EmptyPayload) // 기대: Ok 3
printfn "%A" (None |> ofOption EmptyPayload |> Result.map (fun (v: int) -> v * 2)) // 기대: Error EmptyPayload
```

- 반대 방향에는 표준 함수가 있다. `Result.toOption` 은 `Ok` 안의 값을 `Some` 으로 옮기고, `Error` 는 이유를 버려 `None` 으로 바꾼다. 실패 이유를 버리는 것이 의도일 때만 쓴다.



```
// FSI 실측: Result.toOption : Result<'a, 'b> -> 'a option
// Error 의 이유는 버려진다
printfn "%A" (upload 1024 |> Result.toOption)    // 기대: Some "업로드 1024 바이트"
printfn "%A" (upload 0 |> Result.toOption)      // 기대: None
```

## Summary — 원서의 챕터 요약 (원서 p.51)

- 원서는 이 챕터에서 `null` 처리, `Option` 타입과 모듈, 예외 처리, `Result` 타입과 모듈, 고차 함수를 다뤘다.
- 철도 지향 프로그래밍과 도메인 주도 설계를 더 보려면 스콧 블라신의 저서 “Domain Modeling Made Functional” 을 권한다고 적혀 있다.
- 다음 챕터에서는 코드를 프로젝트로 나누는 방법과 단위 테스트를 살펴본다.

## 정리 — 이 노트의 요약

- 없음은 `Option`, 실패는 `Result` 로 타입에 적는다. 둘 다 판별 유니온이므로 `match` 식으로 다룰 수 있고, 모듈 함수로 더 짧게 다룰 수 있다.
- `Option` 과 `Result` 의 차이는 실패 쪽에 값이 실리는지다. `Option` 의 `None` 은 이유를 담지 못하고, `Result` 의 `Error` 는 담는다. 이유가 필요하다면 `Result` 다.
- `null` 은 다른 .NET 코드와 맞닿는 경계에서만 들어온다. `Option.ofObj / ofNullable` 로 즉시 `Option` 으로 바꾸고, 내보낼 때 `toObj / toNullable` 로 되돌린다.
- `try/with` 는 값을 내는 식이다. 두 갈래가 같은 타입을 내야 한다. `raise` 와 `failwith` 의 반환 타입이 `'a` 인 것은 정상적으로 값을 돌려주지 않기 때문이며, 그래서 어느 타입 자리에도 놓을 수 있다.
- 예외를 던지는 함수는 시그니처가 사실을 다 말하지 않는다. 실패를 반환값으로 옮기면 시그니처만 읽고 그 함수를 신뢰할 수 있다.
- `map` 은 성공값에 보통 함수를 적용하고 다시 감싸며, `bind` 는 이미 감싼 값을 내는 함수를 이어 겹침을 막는다. 넘길 함수의 반환 타입만 보면 어느 쪽을 쓸지 정해진다.
- `Result.mapError` 는 실패 선로에 끼우는 어댑터다. 단계마다 실패 타입이 다르면 이것으로 맞춘 뒤 `bind` 로 잇는다.
- 실패 타입을 도메인 판별 유니온으로 정해 두면 처리하지 않은 실패 종류를 컴파일러가 찾아 준다. 8챕터의 검증과 12챕터의 계산 식이 모두 이 형태 위에 올라간다.

## 원서 대조 표

| 절                                                                       | 원서 페이지   | 실행 단위                          |
|-------------------------------------------------------------------------|----------|--------------------------------|
| Null Handling — <code>Option</code> 으로 없음을 타입에 적기                       | pp.39-41 | <code>03-option</code>         |
| Interop With .NET — <code>null</code> 이 넘어오는 유일한 통로                     | pp.42-43 | <code>03-interop</code>        |
| Handling Exceptions — 시그니처가 거짓말하지 않게                                    | pp.43-45 | <code>03-exceptions</code>     |
| Function Composition With Result — <code>map</code> 과 <code>bind</code> | pp.45-51 | <code>03-result-compose</code> |
| 실패 타입을 직접 정하기 — <code>Result.mapError</code>                            | p.48 확장  | <code>03-result-errors</code>  |
| Summary — 원서의 챕터 요약                                                     | p.51     | —                              |

## 04 - 코드 구성과 테스트 (원서 pp.52-62)

앞의 세 챕터가 언어 자체를 다뤘다면 이 챕터는 도구를 다룬다. 스크립트 파일 하나에 코드를 몰아넣는 단계를 벗어나는 방법이 나온다. 솔루션(solution)과 프로젝트(project)로 경계를 긋고, 네임스페이스(namespace)와 모듈(module)로 이름을 정리하고, xUnit 으로 단위 테스트(unit test)를 짠다. F# 에만 있는 규칙이 하나 끼어 있어서 다른 .NET 언어를 쓰던 독자가 가장 자주 걸려 넘어지는 지점이 여기다. F# 은 파일의 컴파일 순서가 고정되어 있고, 그 순서를 프로젝트 파일이 결정한다.

### Getting Started — 솔루션과 프로젝트 만들기 (원서 p.52)

- 원서 본문은 이 절에서 부록 1(원서 pp.195-196)의 스크립트를 실행하라고만 하고 넘어간다. 실제 명령은 부록에 있으므로 여기서는 그 명령을 현재 SDK 기준으로 정리한다.
- 만들 구조는 솔루션 하나에 프로젝트 둘이다. 코드용 콘솔 프로젝트와 테스트용 xUnit 프로젝트다.
- 코드 프로젝트는 `src/` 아래, 테스트 프로젝트는 `tests/` 아래에 둔다. 이것이 .NET 생태계의 관례이고, 이렇게 두면 나중에 프로젝트가 늘어나도 구조가 유지된다.

```
dotnet new sln -o ShopSolution
cd ShopSolution
mkdir src tests
dotnet new console -lang "F#" -o src/Shop
dotnet new xunit -lang "F#" -o tests/ShopTests
dotnet sln add src/Shop/Shop.fsproj tests/ShopTests/ShopTests.fsproj
```

- 다음표를 빼도 셸은 낱말 첫 글자가 아닌 `#` 을 주석으로 보지 않으므로 명령은 그대로 동작한다. 그래도 `-lang "F#"` 처럼 붙여 두는 편이 안전하다.
- `dotnet sln add` 는 인자를 여러 개 받으므로 프로젝트마다 따로 실행할 필요가 없다.

테스트 프로젝트가 코드 프로젝트를 볼 수 있게 참조를 걸고, 어서션(assertion) 라이브러리인 FsUnit 을 넣는다.

```
cd tests/ShopTests
dotnet add reference ../../src/Shop/Shop.fsproj
dotnet add package FsUnit.xUnit
cd ../../
dotnet build
dotnet test
```

- 참조 방향은 한쪽뿐이다. 테스트 프로젝트가 코드 프로젝트를 참조한다. 반대 방향 참조까지 함께 걸면 두 프로젝트가 서로를 참조하게 되어, 복원 단계에서 순환 참조 오류 MSB4006 이 난다. 코드 프로젝트가 테스트 프로젝트를 참조하도록 방향만 뒤집어 걸면 빌드는 되지만, 테스트 프로젝트가 대상 코드를 볼 수 없어 의미가 없다.
- 원서 부록은 FsUnit 과 FsUnit.xUnit 두 패키지를 모두 넣지만, xUnit 만 쓸 때는 FsUnit.xUnit 하나로 충분하다. FsUnit.xUnit 은 FsUnit 을 의존하지 않는 독립 패키지이고, FsUnit 단독 패키지는 NUnit 용 어서션이다.
- dotnet test 를 솔루션 디렉터리에서 실행하면 솔루션에 속한 테스트 프로젝트 전부를 빌드해 실행한다. 처음에는 템플릿이 만들어 둔 테스트 하나만 통과한다. 아래 인용은 DOTNET\_CLI\_UI\_LANGUAGE=en 을 붙여 얻은 영어 출력이다. CLI 메시지는 셀 로케일을 따라 번역돼 나오므로 한국어 로케일에서는 통과! - 실패: 0, ... 로 찍힌다.

```
Passed! - Failed: 0, Passed: 1, Skipped: 0, Total: 1, Duration: < 1 ms - ShopTests.dll (net10.0)
```

코드 프로젝트를 실행할 때는 그 프로젝트 디렉터리에서 dotnet run 을 쓴다.

```
cd src/Shop
dotnet run
```

- 이때 나오는 Hello from F# 은 템플릿이 만든 Program.fs 한 줄이 출력한 것이다.
- 원서(2023년 1월판)와 현재 SDK(.NET 10.0.111)의 차이는 두 군데다. 첫째, dotnet new sln 이 기본으로 XML 형식인 .slnx 를 만든다. 예전 .sln 형식이 필요하면 dotnet new sln -f sln 을 쓴다. 둘째, 대상 프레임워크가 net5.0 / net6.0 대신 net10.0 이다. 명령 이름과 흐름 자체는 원서와 같다.
- dotnet sln add 가 src/ , tests/ 경로를 보고 같은 이름의 솔루션 폴더를 만들어 주는 것은 SDK 10 에서 새로 생긴 동작이 아니다. 이 동작을 끄는 옵션이 --in-root 이고 기본값이 끄지 않는 쪽이며, -f sln 으로 만든 .sln 에도 같은 폴더가 SolutionFolder 항목과 NestedProjects 절로 들어간다. 형식에 따라 적는 표기만 다르다. 이 폴더는 편집기 솔루션 탐색기의 분류일 뿐이고 디스크의 디렉터리도 컴파일 순서도 아니다.

## Solutions and Projects — 경계를 나누는 두 단위 (원서 p.52)

- 프로젝트는 컴파일 단위다. 프로젝트 하나가 어셈블리( .dll ) 하나로 컴파일되고, 콘솔 프로젝트라면 그것을 띄우는 실행 파일이 함께 만들어진다.
- 솔루션은 프로젝트를 묶어 한 번에 빌드하고 테스트하기 위한 목록일 뿐이다. 솔루션 자체가 컴파일되는 것은 아니다.
- 그래서 "무엇을 별도 프로젝트로 뺄까"의 기준은 배포 단위다. 따로 배포하거나 따로 참조할 이유가 없으면 프로젝트를 늘리지 않는 편이 낫다. 코드 정리는 다음 절의 모듈로 하는 것이 가법다.

## Adding a Source File — 파일을 추가하면 순서를 적어야 한다 (원서 pp.52-53)

- F# 프로젝트에서 파일을 추가하는 일은 두 단계다. 파일을 만드는 것과 그 파일을 `.fsproj` 의 컴파일 목록에 적는 것이다. 둘째 단계를 빼먹으면 그 파일은 없는 것과 같다.
- C# 프로젝트는 디렉터리의 `.cs` 파일을 자동으로 전부 컴파일하지만 F# 은 그렇지 않다. `.fsproj` 의 `<Compile Include=... />` 가 적힌 순서가 곧 컴파일 순서다.

`src/Shop` 에 `Learners.fs` 를 만들었다면 `Shop.fsproj` 를 이렇게 고친다.

```
<ItemGroup>
  <Compile Include="Learners.fs" />
  <Compile Include="Program.fs" />
</ItemGroup>
```

- `Program.fs` 가 `Learners.fs` 의 함수를 쓴다면 `Learners.fs` 가 위에 있어야 한다. 순서를 뒤집으면 `Program.fs` 는 아직 존재하지 않는 이름을 참조하는 셈이 되어 컴파일에 실패한다.
- VS Code 에서 Ionide 확장을 쓰면 F# 솔루션 탐색기에서 기존 파일을 마우스 오른쪽 버튼으로 클릭해 그 위나 아래에 새 파일을 만드는 메뉴를 쓸 수 있다. 이러면 파일 생성과 `.fsproj` 등록이 한 번에 된다. 일반 탐색기로 만들었다면 `.fsproj` 를 직접 고쳐야 한다.

이 "위에서 아래로만 보인다"는 규칙은 파일 사이에만 적용되는 것이 아니다. 파일 안에서도 똑같다.

```
// 이 단위가 보여주는 것: 이름은 위에서 아래로만 보인다(파일 안이든 파일 사이든)

module Discount =
  // 아래에서 쓰는 값은 반드시 위에 있어야 한다
  let baseRate = 0.05

  // string -> float
  let rateFor grade = if grade = "gold" then baseRate * 2.0 else baseRate
```

`Discount` 를 먼저 정의했으므로 그 아래의 `Order` 는 `Discount` 를 볼 수 있다.

```
module Order =
  // 앞에서 정의한 Discount 는 이 아래에서 보인다
  // string -> float -> float
  let total grade amount = amount * (1.0 - Discount.rateFor grade)

  printfn "gold 1000.0 -> %.1f" (Order.total "gold" 1000.0) // 기대: 900.0
  printfn "silver 1000.0 -> %.1f" (Order.total "silver" 1000.0) // 기대: 950.0
```

두 모듈의 순서를 바꾸면 컴파일되지 않는다. 파일 순서를 잘못 적었을 때 나오는 오류와 같은 것이다.

```
module Order =
  // 아래에 정의될 Discount 는 여기서 보이지 않는다
  let total grade amount = amount * (1.0 - Discount.rateFor grade)
  // error FS0039: The value, namespace, type or module 'Discount' is not defined

module Discount =
  let baseRate = 0.05
  let rateFor grade = if grade = "gold" then baseRate * 2.0 else baseRate
```

- 이 순서가 필요한 까닭은 F# 의 타입 추론이 위에서 아래로 한 번만 훑기 때문이다. 어떤 이름을 쓰는 지점에서 그 이름의 타입이 이미 확정되어 있어야 하므로, 정의가 사용보다 앞에 와야 한다.
- 이 제약은 불편해 보이지만 얻는 것이 있다. 의존 방향이 파일 목록에 그대로 드러나므로 순환 의존이 애초에 생길 수 없고, `.fsproj` 만 봐도 프로젝트의 계층을 읽을 수 있다.
- 한 파일 안의 두 모듈이 서로를 참조해야 하는 드문 경우에는 `module rec` 이나 `namespace rec` 로 그 파일에 한해 규칙을 풀 수 있다. 파일 사이에는 같은 수단이 없다.

선언 없이 만든 새 `.fs` 파일을 등록하고 빌드하면 순서와는 별개의 오류를 먼저 만난다.

```
error FS0222: Files in libraries or multiple-file applications must begin with a namespace
or module declaration, e.g. 'namespace SomeNamespace.SubNamespace' or
'module SomeNamespace.SomeModule'. Only the last source file of an application may omit
such a declaration.
```

- 파일이 둘 이상이면 각 파일은 네임스페이스나 모듈 선언으로 시작해야 한다. 예외는 애플리케이션의 마지막 파일 하나뿐이고, 그래서 템플릿이 만든 `Program.fs` 는 선언 없이 `printfn` 한 줄로 시작할 수 있다. 이 예외는 테스트 프로젝트에서는 통하지 않는다. 그 이유는 Writing Tests 절에서 다룬다.
- 다음 절이 이 선언을 다룬다.

## Namespaces and Modules — 이름을 정리하는 두 장치 (원서 pp.53-56)

- 모듈은 프로젝트 안에서 코드를 논리적으로 묶는 장치다. 네임스페이스는 다른 프로젝트의 코드를 참조할 때 이름이 충돌할 확률을 줄이는 장치다. 역할이 다르므로 둘 다 쓴다.
- 네임스페이스에 담을 수 있는 것은 타입 선언, `open` 선언, 모듈뿐이다. 값이나 함수를 네임스페이스 바로 아래 두면 오류 FS0201 이 난다. 한편 같은 네임스페이스를 여러 파일에 걸쳐 쓸 수 있고, 모듈 이름은 파일이 달라도 한 네임스페이스 안에서 유일해야 한다. 겹치면 오류 FS0248 이 난다.
- 모듈은 네임스페이스를 뺀 무엇이든 담을 수 있고 모듈 안에 모듈을 넣을 수도 있다. 네임스페이스 대신 최상위 모듈을 둘 수도 있지만, 그 이름은 프로젝트의 모든 파일에 걸쳐 유일해야 한다. 겹치면 오류 FS0239 가 난다.

파일 첫 줄에 쓸 수 있는 선언 형태는 세 가지다.

```
// (1) 네임스페이스만. 함수는 파일 안의 모듈에 넣는다
namespace Shop.Inventory

// (2) 네임스페이스를 두고 그 아래에 모듈을 중첩한다
namespace Shop
module Inventory =
    // ...

// (3) 최상위 모듈. 점 앞은 네임스페이스, 마지막 조각이 모듈 이름이 된다
module Shop.Inventory
```

- (1)과 (3)의 차이가 헛갈리기 쉽다. (1)은 `Shop.Inventory` 전체가 네임스페이스이므로 파일 안에 모듈을 하나 더 만들어야 함수를 둘 수 있다. (3)은 `Shop` 이 네임스페이스, `Inventory` 가 모듈이므로 함수를 파일 최상위에 바로 쓸 수 있다.
- 실무에서 무난한 출발점은 각 파일 첫 줄에 프로젝트 이름으로 네임스페이스를 적고, 나머지 정리는 모듈로 하는 것이다.
- 모듈 하나마다 파일 하나로 쪼개려 애쓸 필요는 없다. 함께 바뀌는 코드는 같은 파일에 두는 편이 낫다. 기술적 계층(컨트롤러, 서비스, 리포지터리)이 아니라 기능이나 도메인 개념을 기준으로 묶으라는 것이 원서의 조언이다.

아래 실행 단위는 중첩 모듈과 정규화된 접근을 보여 준다. `namespace` 와 최상위 `module X.Y` 는 스크립트 파일에서 쓸 수 없어 F# Interactive(FSI) 로 검증할 수 없으므로, 여기서는 `module x =` 형태의 중첩 모듈로 같은 구조를 만든다.

```
// 이 단위가 보여주는 것: 중첩 모듈, 정규화된 접근, open, 이름 가림, RequireQualifiedAccess

module Inventory =

    // 두 하위 모듈이 함께 쓰는 타입은 바깥 모듈 스코프에 둔다
    type Item = { Sku: string; Qty: int }

    module Create =
        // string -> int -> Item
        let item sku qty = { Sku = sku; Qty = qty }

    module Format =
        // Item -> string
        let label (item: Item) = $"{item.Sku} / 재고 {item.Qty}"
```

- 하위 모듈은 바깥 모듈의 타입을 `open` 없이 그대로 쓴다. 원서가 네임스페이스 스코프에 `Customer` 타입을 두고 두 모듈이 그것을 쓰게 한 것과 같은 배치다.
- 바깥에서는 모듈 이름을 앞에 붙여 정규화된 이름으로 접근한다.

```
let bolt = Inventory.Create.item "BOLT-8" 12
printfn "%s" (Inventory.Format.label bolt) // 기대: BOLT-8 / 재고 12

// open 하면 그 모듈 안의 이름을 한 단계 짧게 쓸 수 있다
open Inventory

let nut = Create.item "NUT-8" 40
printfn "%s" (Format.label nut) // 기대: NUT-8 / 재고 40
```

- `open` 은 이름을 짧게 만들어 주지만 대가가 있다. 같은 이름이 여러 모듈에 있으면 나중에 `open` 한 쪽이 앞의 것을 가린다. 이것이 이름 가림(shadowing)이다. 오류나 경고 없이 조용히 가려지므로 주의할 만하다.

```
module Metric =
    // int -> string
    let describe (n: int) = $"{n} 개(미터법)"

module Imperial =
    // int -> string
    let describe (n: int) = $"{n} 개(야드파운드법)"

open Metric
open Imperial

// 나중에 open 한 Imperial 의 describe 가 이긴다
printfn "%s" (describe 3) // 기대: 3 개(야드파운드법)
```

가려지면 곤란한 모듈에는 특성(attribute) [`<RequireQualifiedAccess>`] 를 붙인다. 그러면 그 모듈은 `open` 대상이 되지 못하고(오류 FS0892) 항상 모듈 이름을 앞에 붙여 써야 한다.

```
[<RequireQualifiedAccess>]
module Warehouse =
    // int -> string
    let describe (n: int) = $"{n} 상자"

    printfn "%s" (Warehouse.describe 3) // 기대: 3 상자
```

- F# 코어의 `List`, `Array`, `Seq`, `Map`, `Set`, `String` 모듈에도 이 특성이 붙어 있다. 그래서 `open List` 는 오류이고 `List.map` 과 `Array.map` 이 섞일 일이 없다. 반면 `Option`, `Result` 모듈에는 붙어 있지 않아 `open` 이 가능하다.
- 이 특성은 모듈뿐 아니라 판별 유니온 타입에도 붙는다. `Tier` 에 붙이면 `Pro` 대신 `Tier.Pro` 로만 쓸 수 있어 케이스 식별자가 다른 이름과 부딪히지 않는다.
- 반대로 [`<AutoOpen>`] 을 붙인 모듈은 그 어셈블리를 참조하기만 하면 `open` 없이 이름이 보인다. 편하지만 이름이 어디서 왔는지 추적하기 어려워지므로 아껴 쓴다.
- `open` 은 모듈 안에도 쓸 수 있다. 특정 모듈에서만 필요한 `System.IO` 같은 것은 파일 맨 위가 아니라 그 모듈 안에 두면 영향 범위가 좁아진다.

## Writing Tests — 첫 테스트와 이중 백틱 이름 (원서 pp.56-57)

- xUnit 템플릿이 만든 `Tests.fs` 는 최상위 모듈 선언, `open System`, `open Xunit`, [`<Fact>`] 가 붙은 함수 하나로 되어 있다. 아래 조각에서는 쓰이지 않는 `open System` 을 빼고 옮겼다.
- 테스트 함수는 `unit` 을 받아 `unit` 을 반환한다. 매개변수 자리의 `()` 를 빼면 값 바인딩이 되어 특성을 붙일 자리가 사라진다. 이때 경고 FS0842 가 나고 xUnit 은 그 바인딩을 테스트로 수집하지 않는다.
- 테스트 파일도 네임스페이스나 모듈 선언으로 시작해야 한다. 다만 네임스페이스를 둘 필요는 없고 모듈 하나로 충분하다. 배포되지 않는 코드라서 다른 어셈블리와 이름이 충돌할 일이 없다.
- 테스트 프로젝트에는 마지막 파일 예외가 통하지 않는다. 테스트 SDK 가 진입점 파일을 컴파일 목록 맨 뒤에 붙이므로 사용자가 만든 파일은 어느 것도 마지막이 아니다. 파일이 하나뿐이어도 선언을 빼면 오류 FS0222 가 난다.
- F# 이 테스트 작성에서 특히 편한 이유는 이중 백틱 이름이다. 식별자를 ```` 로 감싸면 공백과 한글을 포함한 문장을 그대로 이름으로 쓸 수 있다.

```
module Tests

open Xunit

[<Fact>]
let ``My test`` () =
    Assert.True(true)
```

모듈에도 이중 백틱 이름을 쓸 수 있다. 모듈 이름을 상황, 테스트 이름을 기대 결과로 잡으면 출력이 그대로 문장이 된다. 아래 조각은 네임스페이스 아래에 모듈을 둔 형태다. 테스트 파일에 네임스페이스가 필수는 아니지만, 실패 출력에 네임스페이스와 모듈이 어떻게 함께 찍히는지 보이려고 이렇게 적었다.

```
namespace ShopTests

open Xunit

module ``테스트를 모듈로 묶으면`` =

    [<Fact>]
    let ``첫 번째 테스트가 통과한다`` () =
        Assert.True(true)

    [<Fact>]
    let ``두 번째 테스트도 통과한다`` () =
        Assert.True(true)
```

- 테스트가 깨지면 출력에 `어셈블리.모듈.테스트이름` 이 그대로 찍힌다. 이름을 문장으로 지어 두면 실패 목록만 읽어도 무엇이 어긋났는지 알 수 있다.

```
[xUnit.net 00:00:01.24]      ShopTests.테스트를 모듈로 묶으면.두 번째 테스트도 통과한다 [FAIL]
```

- 일부러 `Assert.True(false)` 로 바꿔 한 번 실패시켜 보면 이 출력 형식을 확인할 수 있다.
- 원서는 이중 백틱 이름에 `[ , | - ; .` 같은 문자를 넣으면 Ionide 확장이 죽는다고 경고한다(원서 기준 2022년 5월 확인). 오래된 정보이므로 지금은 다를 수 있으나, 특수문자를 피하는 습관 자체는 손해가 없다. 공백과 한글은 현재 SDK 에서 문제없이 동작했다.

## Writing Real Tests — 테스트할 대상을 두고 짜기 (원서 pp.57-61)

- 원서는 2챕터에서 만든 고객 등급 파이프라인을 테스트 대상으로 되돌려 쓴다. 이 노트는 같은 모양의 파이프라인을 온라인 강의 수강생 도메인으로 새로 짜서 쓴다.
- 대상 코드는 순수 함수 파이프라인이다. 같은 입력을 주면 언제나 같은 출력이 나오고 부수 효과(side effect)가 없으므로, 테스트가 준비 코드 없이 한 줄로 끝난다. 테스트하기 쉬운 코드는 대개 테스트 기법이 좋아서가 아니라 대상이 순수해서 쉬운 것이다.

`src/Shop/Learners.fs` 에 들어갈 코드다. 실제 프로젝트라면 첫 줄이 `module Shop.Learners` 이지만, FSI 검증을 위해 여기서는 중첩 모듈 형태로 적는다.



```
// 이 단위가 보여주는 것: 테스트 대상이 될 순수 함수 파이프라인과 그 결과 확인

module Learners =

    type Tier =
        | Basic
        | Pro

    type Learner = { Id: int; Tier: Tier; Points: int }

    // Learner -> Learner * int
    // 수수료 강의 수를 조회한다고 가정한다. 실제로는 데이터베이스를 읽을 자리다
    let findCompleted learner =
        let completed = if learner.Id % 3 = 0 then 12 else 4
        learner, completed

    // Learner * int -> Learner (튜플 매개변수)
    let promoteIfEligible (learner, completed) =
        if completed >= 10 then { learner with Tier = Pro } else learner

    // Learner -> Learner
    let awardPoints learner =
        let bonus =
            match learner.Tier with
            | Pro -> 300
            | Basic -> 100
        { learner with Points = learner.Points + bonus }

    // Learner -> Learner
    let settleMonth learner =
        learner
        |> findCompleted
        |> promoteIfEligible
        |> awardPoints
```

- `promoteIfEligible` 의 매개변수는 두 개가 아니라 튜플 하나다. 시그니처가 `Learner -> int -> Learner` 가 아니라 `Learner * int -> Learner` 인 것이 그 표시다. `findCompleted` 가 돌려준 튜플이 이 매개변수 하나에 그대로 들어가므로 파이프가 맞물리는 것이고, `|>` 가 튜플을 두 인자로 풀어 주는 것은 아니다. 튜플은 절반만 채울 수 없으므로 이 함수에는 부분 적용을 쓸 수 없다.
- `findCompleted` 는 원래 데이터베이스를 읽을 자리다. 여기서는 `Id` 로 대신 계산해 결정적으로 만들었다. 실제 프로젝트에서 이 함수만 바깥에서 주입받도록 바꾸면 나머지 세 함수는 계속 순수하게 남는다.
- 테스트로 확인할 경로는 세 갈래다. 이미 `Pro` 인 수강생, 조건을 채워 승급하는 `Basic` 수강생, 조건을 못 채운 `Basic` 수강생이다.

```
open Learners

let proLearner = { Id = 1; Tier = Pro; Points = 0 }
let basicLearner = { Id = 3; Tier = Basic; Points = 500 }

// Learner -> string
let describe learner =
    sprintf "Id=%d Tier=%A Points=%d" learner.Id learner.Tier learner.Points

printfn "%-13s %s" "pro" (describe (settleMonth proLearner))
printfn "%-13s %s" "basic" (describe (settleMonth basicLearner))
printfn "%-13s %s" "basic(Id=4)" (describe (settleMonth { basicLearner with Id = 4 })))
// 기대:
// pro           Id=1 Tier=Pro Points=300
// basic         Id=3 Tier=Pro Points=800
// basic(Id=4)   Id=4 Tier=Basic Points=600
```

- 세 번째 줄이 `Points=600` 인 이유는 `Id = 4` 라 `4 % 3 = 1` 이므로 수수료 수가 4 에 머물고, 승급 없이 `Basic` 보너스 100 점만 더해지기 때문이다.

원서가 2챕터에서 쓴 검증 방식은 `let 이름 = 실제값 = 기댓값` 처럼 비교 결과를 값 바인딩에 담아 FSI 출력으로 확인하는 것이었다. 그 방식을 테스트로 옮기기 전에 한 단계 다듬으면 형태가 그대로 옮겨진다. 비교를 `areEqual` 이라는 이름의 함수로 빼고, 기댓값에 `expected` 라는 이름을 붙이는 것이다.

```
// 'a -> 'a -> bool (when 'a : equality)
// 매개변수 순서를 expected, actual 로 잡아 xUnit 의 Assert.Equal 과 같게 맞췄다
let areEqual expected actual = actual = expected

let checkProBonus =
    let expected = { proLearner with Points = 300 }
    areEqual expected (settleMonth proLearner)

let checkPromotion =
    let expected = { basicLearner with Tier = Pro; Points = 800 }
    areEqual expected (settleMonth basicLearner)

let checkNoPromotion =
    let expected = { basicLearner with Id = 4; Points = 600 }
    areEqual expected (settleMonth { basicLearner with Id = 4 })

printfn "%-15s %b" "ProBonus" checkProBonus           // 기대: ProBonus      true
printfn "%-15s %b" "Promotion" checkPromotion         // 기대: Promotion      true
printfn "%-15s %b" "NoPromotion" checkNoPromotion     // 기대: NoPromotion    true
```

- 여기까지 오면 각 검증은 기댓값 한 줄, 실제값 한 줄, 비교 한 줄이 된다. 이 세 줄이 그대로 [`<Fact>`] 함수 본문이 된다.
- 레코드는 구조적 동등성(structural equality)이 있어 `=` 로 필드 전체를 한 번에 비교할 수 있다. 필드를 하나씩 확인하는 어서션을 쓸 이유가 없다.

이제 `tests/ShopTests/LearnerTests.fs` 에 옮긴다. xUnit 의 `Assert.Equal` 을 쓴다.

```
namespace ShopTests

open Xunit
open Shop.Learners

module ``월말 정산을 하면`` =

    let proLearner = { Id = 1; Tier = Pro; Points = 0 }
    let basicLearner = { Id = 3; Tier = Basic; Points = 500 }

    [<Fact>]
    let ``Pro 수강생은 보너스 300점을 받는다`` () =
        let expected = { proLearner with Points = 300 }
        let actual = settleMonth proLearner
        Assert.Equal(expected, actual)

    [<Fact>]
    let ``조건을 채운 Basic 수강생은 Pro 로 승급한다`` () =
        let expected = { basicLearner with Tier = Pro; Points = 800 }
        let actual = settleMonth basicLearner
        Assert.Equal(expected, actual)

    [<Fact>]
    let ``조건을 못 채운 Basic 수강생은 등급이 그대로다`` () =
        let learner = { basicLearner with Id = 4 }
        let expected = { learner with Points = 600 }
        let actual = settleMonth learner
        Assert.Equal(expected, actual)
```

- `open Shop.Learners` 하나로 타입과 함수가 다 들어온다. 코드 프로젝트 쪽 파일 첫 줄이 `module Shop.Learners` 였으므로 `Shop` 이 네임스페이스, `Learners` 가 모듈이다.
- 이 파일도 `.fsproj` 에 등록해야 실행된다. 테스트 프로젝트에서도 컴파일 순서 규칙은 똑같다.

```
<ItemGroup>
  <Compile Include="Tests.fs" />
  <Compile Include="LearnerTests.fs" />
</ItemGroup>
```

```
dotnet test
```

- `Assert.Equal` 이 실패하면 기댓값과 실제값을 레코드 모양 그대로 찍어 준다. 어느 필드가 다른지 눈으로 바로 찾을 수 있다.

```
Assert.Equal() Failure: Values differ
Expected: { Id = 1
           Tier = Pro
           Points = 100 }
Actual:   { Id = 1
           Tier = Pro
           Points = 300 }
```

- xUnit 은 선택지 중 하나일 뿐이다. NUnit, MSTest, Expecto 같은 다른 프레임워크를 써도 이 챕터의 구조는 그대로 통한다.

## Using FsUnit for Assertions — 어서션 문장을 파이프라인 (원서 pp.61-62)

- `Assert.Equal(expected, actual)` 은 .NET 관례에 맞는 표기이지만 F# 의 파이프 흐름과는 어긋난다. 인자 순서를 헷갈리기 쉽다.
- FsUnit 은 같은 검증을 `actual |> should equal expected` 로 쓰게 해 준다. 실제값을 왼쪽에서 오른쪽으로 흘러보내는 모양이라 앞 절에서 짰 파이프라인 코드와 읽는 방향이 맞는다.

```
open FsUnit.Xunit

actual |> should equal expected
```

- `open` 할 이름은 `FsUnit.Xunit` 이다. 원서 본문은 `open FsUnit` 이라고 적었지만 그것은 NUnit 용 모듈이다. xUnit 과 함께 쓸 때는 `FsUnit.Xunit` 을 열어야 `should` 가 보인다.
- `FsUnit.xUnit` 만 넣은 프로젝트에서 `open FsUnit` 은 그 자체로는 오류가 아니다. `FsUnit` 이라는 네임스페이스가 있으므로 `open` 은 통과하고, `should` 를 쓰는 줄에서 오류 FS0039 가 난다. 원서처럼 `FsUnit` 패키지까지 함께 넣으면 `should` 는 보인다. 다만 이때 쓰이는 것은 NUnit 쪽 어서션이라, xUnit 으로 돌린 테스트인데도 실패 메시지가 `NUnit.Framework.AssertionException : Assert.That(, )` 로 찍히고 검증한 식이 괄호 안에 비어 나온다.
- 테스트 러너는 여전히 xUnit 이다. FsUnit 이 바꾸는 것은 어서션을 적는 문법뿐이고, `[<Fact>]` 와 `dotnet test` 는 그대로다.

앞 절의 세 테스트에서 `Assert.Equal` 만 바꾼 모습이다.

```

namespace ShopTests

open Xunit
open FsUnit.Xunit
open Shop.Learners

module ``월말 정산을 하면`` =

    let proLearner = { Id = 1; Tier = Pro; Points = 0 }
    let basicLearner = { Id = 3; Tier = Basic; Points = 500 }

    [<Fact>]
    let ``Pro 수강생은 보너스 300점을 받는다`` () =
        let expected = { proLearner with Points = 300 }
        let actual = settleMonth proLearner
        actual |> should equal expected

    [<Fact>]
    let ``조건을 채운 Basic 수강생은 Pro 로 승급한다`` () =
        let expected = { basicLearner with Tier = Pro; Points = 800 }
        let actual = settleMonth basicLearner
        actual |> should equal expected

    [<Fact>]
    let ``조건을 못 채운 Basic 수강생은 등급이 그대로다`` () =
        let learner = { basicLearner with Id = 4 }
        let expected = { learner with Points = 600 }
        let actual = settleMonth learner
        actual |> should equal expected

```

- `should` 뒤에는 `equal` 외에도 `not'` (`equal x`), `be True`, `be Empty`, `haveLength 3`, `be (greaterThan 1)`, `throw typeof<...>` 같은 조합이 온다. 문장처럼 읽히는 어서션을 만드는 것이 이 라이브러리의 목적이다.
- 실패 메시지는 xUnit 의 것을 그대로 쓰되 기댓값 쪽에 `Equals` 가 붙어 나온다.

```

Assert.Equal() Failure: Values differ
Expected: Equals { Id = 3
                  Tier = Pro
                  Points = 900 }
Actual:   { Id = 3
          Tier = Pro
          Points = 800 }

```

- xUnit 어서션과 FsUnit 어서션은 한 파일 안에 섞어 써도 된다. 둘 중 무엇을 골라도 되지만, 팀 안에서 하나로 통일하는 편이 읽기에 낫다.

## Summary — 원서의 챕터 요약 (원서 p.62)

- 원서는 이 챕터에서 솔루션, 프로젝트, 네임스페이스, 모듈로 코드를 구성하는 방법과 xUnit 단위 테스트, FsUnit 어서션을 다뤘다.
- 코드베이스가 커져도 작업을 감당할 수 있는 것은 이 기능들 덕분이라는 말로 원서는 챕터를 맺는다.
- 다음 챕터에서는 컬렉션을 처음으로 살펴본다.

## 정리 — 이 노트의 요약

- F#의 컴파일 순서는 `.fsproj`의 `<Compile Include=... />` 순서다. 파일을 만드는 것과 등록하는 것은 별개의 일이며, 등록하지 않은 파일은 컴파일되지 않는다.
- 이름은 위에서 아래로만 보인다. 파일 안에서도, 파일 사이에서도 그렇다. 파일 사이에는 이 규칙을 우회하는 수단이 없어 순환 의존이 생길 수 없고, 그래서 프로젝트의 계층이 파일 목록에 그대로 드러난다.
- 파일이 둘 이상이면 각 파일은 네임스페이스나 모듈 선언으로 시작해야 한다(오류 FS0222). 애플리케이션의 마지막 파일만 예외이고, 테스트 프로젝트에는 그 예외조차 통하지 않는다.
- 네임스페이스는 타입 선언과 `open` 선언, 모듈만 담고 여러 파일에 걸칠 수 있다. 모듈은 무엇이든 담고 중첩할 수 있다. 함수를 파일 최상위에 바로 쓰고 싶으면 `module Namespace.Module` 형태를 쓴다.
- `open`은 편하지만 같은 이름을 조용히 가린다. 가려지면 곤란한 모듈에는 `[<RequireQualifiedAccess>]`를 붙인다.
- 이중 백틱 이름은 테스트 이름을 문장으로 만들어 준다. 실패 출력이 `어셈블리.모듈.테스트이름`이라 모듈 이름을 상황, 테스트 이름을 기대 결과로 잡으면 그대로 읽힌다.
- 테스트하기 쉬운 코드의 조건은 대상이 순수한 것이다. 순수 함수 파이프라인은 기댓값 한 줄, 실제값 한 줄, 비교 한 줄로 검증이 끝난다.
- 레코드의 구조적 동등성 덕분에 `=`나 `Assert.Equal` 하나로 필드 전체를 비교할 수 있다.
- 어서션은 xUnit의 `Assert.Equal(expected, actual)`이나 FsUnit의 `actual |> should equal expected` 중 무엇을 써도 된다. 테스트 러너는 어느 쪽이든 xUnit이다.

## 원서 대조 표

| 절                                          | 원서 페이지                      | 실행 단위       |
|--------------------------------------------|-----------------------------|-------------|
| Getting Started — 솔루션과 프로젝트 만들기            | p.52 (명령은 부록 1, pp.195-196) | —           |
| Solutions and Projects — 경계를 나누는 두 단위      | p.52                        | —           |
| Adding a Source File — 파일을 추가하면 순서를 적어야 한다 | pp.52-53                    | 04-order    |
| Namespaces and Modules — 이름을 정리하는 두 장치     | pp.53-56                    | 04-modules  |
| Writing Tests — 첫 테스트와 이중 백틱 이름            | pp.56-57                    | —           |
| Writing Real Tests — 테스트할 대상을 두고 찌기        | pp.57-61                    | 04-learners |
| Using FsUnit for Assertions — 어서션 문장을 파이프로 | pp.61-62                    | —           |
| Summary — 원서의 챕터 요약                        | p.62                        | —           |

## 05 - 컬렉션 입문 (원서 pp.63-79)

이 챕터부터 다루는 대상이 값 하나에서 값의 묶음으로 바뀐다. F# 이 데이터 중심 작업에 강하다는 평을 듣는 이유가 여기서 드러난다. 컬렉션(collection) 타입 자체가 불변이고, 그 타입마다 붙은 모듈에 미리 만들어 둔 고차 함수(higher-order function)가 잔뜩 들어 있어서 `for` 루프와 가변 누적 변수로 쓰던 코드가 파이프 라인 몇 줄로 줄어든다. 원서는 세 가지 주요 컬렉션 중 `List` 하나에 집중하고, 마지막 절에서 배운 함수들을 묶어 실제 업무 로직 하나를 처음부터 끝까지 만든다. 이 노트도 같은 범위를 지킨다.

### Before We Start — 준비 (원서 p.63)

- 원서는 이 챕터용 폴더를 새로 만들고 `lists.fsx` 스크립트 파일을 하나 두라고만 한다.
- 마지막 실전 예제 절만 4챕터에서 만든 것과 같은 솔루션 구조를 쓰고, 나머지 절은 FSI 에서 조각조각 실행하며 읽는 내용이다. 이때 코드 프로젝트는 콘솔이 아니라 클래스 라이브러리(`dotnet new classlib -lang "F#"`)로 만든다. 실행할 진입점이 필요 없고 테스트 프로젝트가 참조할 대상만 필요하기 때문이다.
- 이 노트의 코드는 전부 스크립트 하나로 돌아가게 짰다. 실전 예제 절의 xUnit 테스트만 실행 대상에서 빼고, 같은 검증을 FSI 에서 비교식으로 대신한다.

### The Basics — 세 가지 컬렉션과 그 차이 (원서 p.63)

- F# 에서 쓸 수 있는 컬렉션은 여럿이지만 중심이 되는 것은 세 개다. `Seq`, `Array`, `List`.
- `Seq` 는 지연 평가(lazy evaluation)되는 시퀀스다. 필요할 때 한 원소씩 만들어 내므로 무한 시퀀스나 파일 스트림처럼 끝을 모르는 데이터에 맞는다. .NET 의 `IEnumerable<'T>` 와 같은 것이다.
- `Array` 는 즉시 평가(eager evaluation)되는 고정 길이 연속 메모리다. 인덱스 접근이 빠르고 수치 계산에 유리하다. 2·3·4차원 배열용 모듈이 따로 있다.
- `List` 는 즉시 평가되는 연결 리스트(linked list)다. 구조와 데이터가 모두 불변이다. 맨 앞에 원소를 붙이는 일이 싸고, 인덱스로 중간을 찾는 일은 비싸다.
- 세 타입 모두 같은 이름의 지원 모듈(`Seq`, `Array`, `List`)이 있고, 서로 변환하는 함수도 들어 있다. 그래서 `List` 모듈에서 익힌 함수 이름은 나머지 두 모듈에서도 거의 그대로 통한다.
- 원서는 이 챕터에서 `List` 타입과 `List` 모듈만 다루겠다고 못 박는다(원서 p.63). 이 노트도 그 범위를 따른다.

이름이 헷갈리는 지점이 하나 있다. C# 에서 쓰던 `List<'T>` 는 F# 의 `List` 가 아니다. F# 에서 그것은 `ResizeArray<'T>` 라는 별칭으로 부르고, 이름대로 크기가 변하는 가변 배열이다. F# 의 `List` 는 불변 연결 리스트이므로 성격이 정반대다.

## Core Functionality — 리스트 만들기과 핵심 함수 (원서 pp.63-68)

- 리스트 리터럴은 대괄호에 세미콜론으로 나열한다. `[2; 5; 3]`. 심표를 쓰면 튜플 하나만 담은 리스트가 되므로 주의한다. `[2, 5, 3]` 은 원소가 하나뿐인 `(int * int * int) list` 다.
- 연속된 정수는 범위 식 `[1..5]` 로, 간격을 두려면 `[0..5..20]` 으로 만든다.
- 리스트 컴프리헨션(list comprehension)은 `[ for x in ... do ... ]` 형태다. F# 4.7 부터 대부분의 경우 `yield` 를 생략할 수 있고, 이것을 암시적 `yield` 라 부른다(원서는 F# 5 부터라고 적는다). 같은 문법이 `Seq` 와 `Array` 에도 있지만 이 챕터에서는 쓰지 않는다.
- 빈 리스트 `[]` 는 원소 타입이 정해지지 않아 자동 일반화(auto-generalization)로 `'a list` 가 된다. 특정 타입으로 못 박아야 하면 타입 주석을 붙인다.
- 같은 빈 리스트라도 `let reversed = List.rev []` 처럼 함수를 적용한 결과를 이름에 묶으면 일반화가 막혀 값 제한(value restriction) 오류 FS0030 이 난다. 매개변수 없는 `let` 바인딩은 제네릭이 될 수 없다는 규칙이다. 타입 주석을 붙이거나 쓰이는 문맥에서 타입이 정해지면 풀린다.

```
// 이 단위가 보여주는 것: 리스트를 만드는 여러 방법과 머리/꼬리 구조

// 주석이 없으면 'a list 로 일반화된다. int 로 못 박으려면 타입 주석을 붙인다
let noLaps : int list = []

// 구간별 기록(초)
let laps = [72; 68; 75; 70; 69]

let firstFive = [1..5]           // [1; 2; 3; 4; 5]
let everyFifth = [0..5..20]      // [0; 5; 10; 15; 20]

printfn "noLaps      = %A" noLaps      // []
printfn "laps       = %A" laps        // [72; 68; 75; 70; 69]
printfn "firstFive  = %A" firstFive   // [1; 2; 3; 4; 5]
printfn "everyFifth = %A" everyFifth   // [0; 5; 10; 15; 20]
```

값 제한이 어떤 모양에서 나는지 보면 규칙이 분명해진다. 리스트를 뒤집는 `List.rev` 에 빈 리스트를 넘긴 결과를 이름에 묶는 순간 일반화가 막힌다.

```
// 오류 FS0030: 값 제한. 값 'reversed'에 유추된 제네릭 형식이 있습니다.
//                val reversed: 'a list
let reversed = List.rev []

// let mutable pending = [] 도 같은 FS0030 이다
```

컴프리헨션은 `for` 뒤에 오는 식이 곧 원소가 된다. 조건을 끼우면 걸러 내기도 한 번에 된다.

```
let squares = [ for n in 1..5 do n * n ]           // 각 값을 변환
let longLaps = [ for t in laps do if t > 70 then t ] // 조건을 만족하는 값만

printfn "squares  = %A" squares    // [1; 4; 9; 16; 25]
printfn "longLaps = %A" longLaps   // [72; 75]
```

리스트 맨 앞에 원소를 붙이는 연산자가 `cons` 연산자 `::` 다. 원본은 불변이라 손대지 않고, 새 리스트는 "새 원소 + 원본을 가리키는 포인터"로 만들어진다. 그래서 원소 수가 늘어도 복사 비용이 붙지 않는다.

```
// 앞 블록의 laps 를 그대로 쓴다
let withWarmUp = 80 :: laps

printfn "withWarmUp = %A" withWarmUp // [80; 72; 68; 75; 70; 69]
printfn "laps = %A" laps // [72; 68; 75; 70; 69] (원본 그대로)
```

- 비어 있지 않은 리스트는 원소 하나인 머리(head)와 나머지 리스트인 꼬리(tail)로 이루어진다. 꼬리는 빈 리스트일 수 있다.
- 이 구조가 그대로 패턴이 된다. `[]`, `[x]`, `h :: t` 세 가지로 리스트를 분해할 수 있다.
- `h`, `t` 라는 이름 자체에 의미는 없다. 그냥 바인딩 이름이므로 `first :: rest` 로 적어도 똑같이 동작한다.

```
// describe: route: int list -> string
let describe route =
    match route with
    | [] -> "구간 없음"
    | [only] -> $"구간 하나: {only}"
    | first :: rest -> sprintf "머리: %d, 꼬리: %A" first rest

printfn "%s" (describe []) // 구간 없음
printfn "%s" (describe [72]) // 구간 하나: 72
printfn "%s" (describe laps) // 머리: 72, 꼬리: [68; 75; 70; 69]
```

원소 하나인 케이스 `[only]` 를 지워도 코드는 여전히 빠짐없는 패턴 매칭(exhaustive pattern matching)이 된다. `[72]` 는 `first :: rest` 로도 매칭되고 그때 `rest` 가 `[]` 이기 때문이다. 케이스를 따로 둘 이유는 원소 하나일 때 다른 문장을 내보내야 할 때뿐이다.

위 코드에는 문자열을 만드는 방식이 두 가지 섞여 있다. `$"..."` 문자열 보간과 `sprintf` 다. 보간에서는 서식 지정자 없이 `{only}` 만 써도 되고, 필요하면 `%A{rest}` 처럼 서식을 붙일 수도 있다. `%A` 는 F# 의 구조 출력기를 쓴다. 리스트나 레코드를 사람이 읽을 수 있게 펼쳐 주고, 구조를 모르는 .NET 타입에서는 그 타입의 `ToString()` 결과를 그대로 쓴다. 원서 p.65 가 `%A` 를 `ToString()` 을 쓰는 것으로 적은 것은 이 뒷부분만 말한 것이다.

두 리스트를 이어 붙일 때는 `@` 연산자를 쓴다. 여러 개를 한 번에 이으려면 `List.concat` 을 쓴다.

```
let morning = [72; 68]
let evening = [75; 70]

// (@) : 'a list -> 'a list -> 'a list
printfn "%A" (morning @ evening) // [72; 68; 75; 70]
printfn "%A" (morning @ []) // [72; 68]

// List.concat : 'a list seq -> 'a list
printfn "%A" (List.concat [morning; evening; []]) // [72; 68; 75; 70]
```

`List.concat` 의 시그니처가 `'a list list -> 'a list` 가 아니라 `'a list seq -> 'a list` 라는 점이 눈에 띈다. 바깥 컬렉션은 리스트든 배열이든 아무 시퀀스나 받는다라는 뜻이다. 그리고 불변이라는 성질 덕분에 `morning` 과 `evening` 은 이어 붙인 뒤에도 그대로 남아 있어 다른 곳에서 마음 놓고 재사용할 수 있다.

이제 모듈 함수 쪽이다. 걸러 내기, 합계, 변환, 반복이 기본 네 가지다.



```
// 이 단위가 보여주는 것: filter / sum / map / iter 와 sumBy

// 택배 상자의 부피(L)
let volumes = [12; 7; 20; 3; 15; 9]

// List.filter : ('a -> bool) -> 'a list -> 'a list
let bulky = volumes |> List.filter (fun v -> v >= 10)
printfn "bulky = %A" bulky // [12; 20; 15]

// List.sum : ^a list -> ^a
// (when ^a : (static member (+) : ^a * ^a -> ^a) and ^a : (static member Zero : ^a))
printfn "합계 = %d" (volumes |> List.sum) // 66
```

`List.filter` 에 넘기는 함수는 `'a -> bool` 형태의 술어(predicate)다. 술어가 참을 돌려준 원소만 남는다.

값을 바꿔서 새 리스트를 얻으려면 `List.map` 을, 결과를 만들지 않고 원소마다 부수 효과(side effect)만 일으키려면 `List.iter` 를 쓴다. 둘의 차이는 시그니처에 그대로 드러난다. `map` 에 넘기는 함수는 `'a -> 'b`, `iter` 에 넘기는 함수는 `'a -> unit` 이다.

```
// List.map : ('a -> 'b) -> 'a list -> 'b list
let inLitres = volumes |> List.map (fun v -> float v * 0.7)
printfn "inLitres = %A" inLitres
// [8.4; 4.9; 14.0; 2.1; 10.5; 6.3]

// List.iter : ('a -> unit) -> 'a list -> unit
volumes |> List.iter (fun v -> printfn " 부피 %2d L" v)
// 부피 12 L / 부피 7 L / ... 여섯 줄
```

`List.map` 은 고차 함수다. `'a -> 'b` 함수를 받아 `'a list` 를 `'b list` 로 바꾼다. `'a` 와 `'b` 가 같아도 되고 위 예처럼 `int list` 에서 `float list` 로 타입이 바뀌어도 된다. C# 을 써 봤다면 LINQ 의 `Select` 가 가장 가깝지만 `Select` 는 지연 평가된다. 지연 평가가 필요하다면 `Seq.map` 을 쓰면 된다. 다만 시퀀스는 순회할 때마다 다시 계산되므로 `List` 파이프라인을 그대로 `Seq` 로 바꿔 두면 같은 계산을 여러 번 하게 된다. 이 문제는 6챕터에서 다룬다.

구조가 바뀌는 예를 보자. 튜플 `(int * decimal)` 의 리스트가 있고 앞이 개수, 뒤가 개당 무게라고 하자. 총 무게는 `map` 으로 튜플 리스트를 `decimal` 리스트로 바꾼 뒤 합하면 된다.

```
// 캠핑 장비: (개수, 개당 무게 kg)
let gear = [ (2, 0.35M); (1, 1.80M); (4, 0.12M); (1, 2.40M) ]

// totalWeight: items: (int * decimal) list -> decimal
let totalWeight items =
    items
    |> List.map (fun (count, kg) -> decimal count * kg)
    |> List.sum

printfn "총 무게 = %M kg" (totalWeight gear) // 5.38 kg
```

여기서 두 가지를 짚어야 한다. 첫째, `decimal count` 라는 변환이 명시적으로 들어갔다. F# 은 계산에서 타입에 엄격해서 `int` 값과 `decimal` 값을 곧바로 곱하는 것 같은 암시적 변환(implicit conversion)을 허용하지 않는다. 둘째, 람다의 매개변수 자리에 `(count, kg)` 라고 적어 튜플을 그 자리에서 분해했다. 람다 매개변수도 패턴이므로 이런 분해가 된다.

`map` 다음에 바로 `sum` 이 오는 이 모양은 흔해서 한 함수로 합쳐 놓았다. `List.sumBy` 다.

```
// List.sumBy : ('a -> ^b) -> 'a list -> ^b
// (when ^b : (static member (+) : ^b * ^b -> ^b) and ^b : (static member Zero : ^b))
let totalWeightBy items =
    items
    |> List.sumBy (fun (count, kg) -> decimal count * kg)

printfn "총 무게 = %M kg" (totalWeightBy gear) // 5.38 kg
```

같은 이름 규칙을 따르는 함수가 여럿 있지만 돌려주는 것은 제각각이다. `List.averageBy` 는 `sumBy` 처럼 뽑아낸 값을 집계해 값 하나를 내놓는다. `List.maxBy` 와 `List.minBy` 는 뽑아낸 값을 기준으로 고른 원소를 그대로 돌려주고, `List.countBy` 는 (키 \* 개수) 튜플의 리스트를 돌려준다. 뒤의 셋은 `map` 뒤에 집계 함수를 붙인 것과 결과 타입이 다르므로 쓰기 전에 시그니처를 확인한다. 그리고 `List.average` 와 `List.averageBy` 에는 함정이 하나 더 있다.

```
// List.averageBy : ('a -> ^b) -> 'a list -> ^b
// (when ^b : (static member (+) : ^b * ^b -> ^b)
// and ^b : (static member DivideByInt : ^b * int -> ^b)
// and ^b : (static member Zero : ^b))
let avgKg = gear |> List.averageBy (fun (_, kg) -> kg)
printfn "평균 무게 = %M kg" avgKg // 1.1675 kg
```

제약에 `DivideByInt` 가 걸려 있는데 `int` 에는 이 멤버가 없다. 그래서 정수 리스트의 평균을 `List.average` 나 `List.averageBy` 로 바로 구할 수 없고, 아래 코드는 오류 FS0001 로 컴파일에 실패한다.

```
// 오류 FS0001: 'int' 형식에는 필수(실제 또는 기본 제공) 멤버 'DivideByInt'이(가) 없기 때문에
// 'List.average'이(가) 이 형식을 지원하지 않습니다.
[1; 2; 3] |> List.average
```

`float`, `decimal`, `float32` 에는 컴파일러가 `DivideByInt` 를 기본 제공하므로, 정수 리스트의 평균이 필요하면 먼저 이 세 타입 중 하나로 올려야 한다. `[1; 2; 3] |> List.averageBy float` 로 쓰면 `2.0` 이 나온다. 나눗셈이 들어가는 함수라서 정수를 조용히 잘라 버리는 사고를 타입 수준에서 막아 둔 셈이다.

## Folding — 누적값을 직접 굴리기 (원서 pp.68-69)

- `List.fold` 는 초기값을 상태에 넣고 원소를 하나씩 훑으면서 상태를 갱신하다가, 다 훑으면 마지막 상태를 돌려준다. LINQ 의 `Aggregate` 에 해당한다.
- 앞 절에서 본 `sum`, `sumBy`, `average` 는 전부 `fold` 로 표현할 수 있는 특수한 경우다. 반대로 `fold` 로는 그 함수들이 못 하는 일까지 할 수 있다.
- 그렇다고 `fold` 를 먼저 꺼내지는 말아야 한다. 원서도 목적이 뚜렷한 함수( `sumBy` 같은 것)를 먼저 찾아보라고 권한다. `fold` 는 그런 함수가 없을 때 쓴다.

```
// 이 단위가 보여주는 것: fold / foldBack / reduce 의 시그니처와 차이

// List.fold : ('a -> 'b -> 'a) -> 'a -> 'b list -> 'a
// 인자 순서는 folder(상태, 원소) → 초기값 → 입력이고, 결과는 마지막 상태다
printfn "합 = %d" ([1..10] |> List.fold (fun acc v -> acc + v) 0) // 55

// folder 가 연산자 하나로 끝나면 연산자의 함수 형태로 줄여 쓴다
printfn "합 = %d" ([1..10] |> List.fold (+) 0) // 55
printfn "곱 = %d" ([1..10] |> List.fold ( * ) 1) // 3628800
```

시그니처를 뜯어보는 것이 중요하다. `fold` 의 타입이 `'a -> 'b -> 'a` 다. 앞 상태 `'a`, 뒤가 원소 `'b` 이고 결과가 다시 상태 `'a` 다. 그래서 람다를 `fun acc v -> ...` 로 적을 때 첫 매개변수가 누적값 (accumulator), 둘째가 원소다. 상태와 원소의 타입이 달라도 되는 것도 여기서 보인다. 초기값은 곱셈이면 `1`, 덧셈이면 `0` 처럼 그 연산의 항등원을 쓰는 것이 보통이다.

방향이 반대인 `List.foldBack` 이 따로 있다. 리스트 끝에서 앞으로 훑는다. 매개변수 순서까지 뒤집혀 있어서 처음 보면 헷갈리기 쉽다.

```
// List.foldBack : ('a -> 'b -> 'b) -> 'a list -> 'b -> 'b
// 인자 순서는 folder(원소, 상태) -> 입력 -> 초기값이고, 결과는 마지막 상태다
// folder 의 첫 매개변수가 원소, 둘째가 상태다. fold 와 정반대다.
let steps = [1; 2; 3]

printfn "fold      = %A" (steps |> List.fold (fun acc v -> v :: acc) [])
// [3; 2; 1] - 앞에서 뒤로 훑으며 앞에 붙였으므로 뒤집힌다

printfn "foldBack = %A" (List.foldBack (fun v acc -> v :: acc) steps [])
// [1; 2; 3] - 뒤에서 앞으로 훑으며 앞에 붙였으므로 순서가 유지된다
```

세 가지 차이를 한 번에 정리하면 이렇다. 훑는 방향이 다르고, `fold` 의 매개변수 순서가 다르고, 인자 순서가 다르다. `fold` 는 `folder`, 초기값, 리스트 순이라 `리스트 |> List.fold f 초기값` 으로 파이프에 잘 맞는다. `foldBack` 은 `folder`, 리스트, 초기값 순이라 파이프 끝에 두기 어렵고 보통 `List.foldBack f 리스트 초기값` 으로 직접 적는다.

앞 절의 총 무게 계산도 `fold` 로 쓸 수 있다. 누적값에 튜플에서 계산한 값을 더해 나가면 된다.

```
let gear = [ (2, 0.35M); (1, 1.80M); (4, 0.12M); (1, 2.40M) ]

// totalWeight: items: (int * decimal) list -> decimal
let totalWeight items =
    items
    |> List.fold (fun acc (count, kg) -> acc + decimal count * kg) 0M

printfn "총 무게 = %M kg" (totalWeight gear) // 5.38 kg
```

초기값 `0M` 이 누적값의 시작이다. 곱셈으로 누적한다면 `1M` 이 되어야 한다. 파이프 연산자에는 튜플 하나를 두 인자로 풀어 주는 `||>` 도 있어서 초기값과 리스트를 한 쌍으로 넘길 수도 있다.

```
// (||>) : 'a * 'b -> ('a -> 'b -> 'c) -> 'c
let totalWeightPiped items =
    (0M, items) ||> List.fold (fun acc (count, kg) -> acc + decimal count * kg)

printfn "총 무게 = %M kg" (totalWeightPiped gear) // 5.38 kg
```

이런 연산자가 요긴한 자리가 분명히 있지만, 이 정도로 단순한 계산에서는 앞의 형태가 읽기 쉽다. 원서 저자도 같은 판단을 적어 두었다.

초기값을 아예 주지 않는 `List.reduce` 도 있다. 첫 원소를 초기값으로 삼는다.

```
// List.reduce : ('a -> 'a -> 'a) -> 'a list -> 'a
// 상태와 원소의 타입이 같아야 한다. fold 의 'a 와 'b 가 하나로 묶인 꼴이다.
println "reduce 합 = %d" ([1..10] |> List.reduce (+)) // 55

// longest: words: string list -> string
let longest words =
  words |> List.reduce (fun a b -> if String.length b > String.length a then b else a)

println "가장 긴 낱말 = %s" (longest ["장마"; "소나기"; "돌풍"]) // 소나기
```

`reduce` 는 부분 함수(partial function)다. 가능한 입력 전부에서 값을 돌려주지는 못한다. 빈 리스트에는 초기값으로 쓸 첫 원소가 없으므로 예외를 던진다.

```
try
  [] |> List.reduce (+) |> println "%d"
with :? System.ArgumentException as ex ->
  println "reduce [] 는 %s 예외를 던진다" (ex.GetType().Name)
// reduce [] 는 ArgumentException 예외를 던진다
```

`List` 모듈의 부분 함수 상당수에는 `Option` 을 돌려주는 `try` 짝이 있다. `List.head` 에는 `List.tryHead`, `List.find` 에는 `List.tryFind` 가 있다. 그런데 `reduce` 에는 없다. `List.tryReduce` 를 적으면 오류 FS0039 로 그런 이름이 없다는 말이 나온다. 그래서 `reduce` 를 쓸 때는 호출하는 쪽에서 빈 리스트를 직접 걸러야 한다. 반면 `fold` 는 초기값이 있으므로 빈 리스트에서도 그 초기값을 그대로 돌려준다. 빈 리스트 때문에 실패하는 일은 없다( `fold` 가 예외를 던지면 `fold` 도 함께 실패한다). 둘 중에 무엇을 쓸지 고민되면 `fold` 가 안전한 선택이다.

## Grouping Data and Uniqueness — 묶기와 중복 제거 (원서 pp.69-70)

- `List.groupBy` 는 키를 뽑는 함수를 받아 (키 \* 그 키로 묶인 원소들의 리스트) 튜플의 리스트를 돌려준다.
- 여기서 키만 꺼내면 중복 없는 값 목록이 된다. 하지만 같은 결과를 훨씬 짧게 얻는 `List.distinct` 가 이미 있다.
- 중복 제거만이 목적이라면 `Set` 으로 변환하는 방법도 있다. 다만 정렬 순서가 바뀐다.

```
// 이 단위가 보여주는 것: groupBy / distinct / distinctBy / Set 변환

let tiers = ["일반"; "학생"; "일반"; "경로"; "학생"; "일반"]

// List.groupBy : ('a -> 'b) -> 'a list -> ('b * 'a list) list (when 'b : equality)
let grouped = tiers |> List.groupBy id
println "grouped = %A" grouped
// [{"일반", [{"일반"; "일반"; "일반"}]; ("학생", [{"학생"; "학생"}]; ("경로", [{"경로"}])]
```

`fun x -> x` 를 그대로 넘길 자리에는 항등 함수(identity function) `id` 를 쓸 수 있다. `id : 'a -> 'a` 이고, "키를 원소 자체로 삼는다"는 뜻이 이름으로 드러나 읽기에 낫다.

묶인 결과에서 키만 뽑아도 같은 목록이 나온다. 하지만 이 일에는 전용 함수가 있다.

```
// 묶은 뒤 키만 꺼내는 방법
// uniq: items: 'a list -> 'a list (when 'a : equality)
let uniq items =
  items
  |> List.groupBy id
  |> List.map (fun (key, _) -> key)

printfn "uniq      = %A" (uniq tiers)    // ["일반"; "학생"; "경로"]

// List.distinct: 'a list -> 'a list (when 'a : equality)
printfn "distinct = %A" (tiers |> List.distinct)    // ["일반"; "학생"; "경로"]

// Set.ofList: 'a list -> Set<'a> (when 'a : comparison)
printfn "set      = %A" (tiers |> Set.ofList)
// set ["경로"; "일반"; "학생"]
```

세 결과가 담은 값은 같지만 순서가 다르다. `List.groupBy` 와 `List.distinct` 는 처음 나타난 순서를 지키는 반면 `Set` 은 정렬된 컬렉션이라 값의 비교 순서로 재배치한다. 원서 pp.69-70 의 결과 주석은 정렬된 순서로 적혀 있으나 실제 결과는 입력에서 처음 나타난 순서다. 그래서 순서가 의미 있는 데이터라면 `Set` 으로 우회하지 않는 편이 안전하다. 제약도 다르다. `distinct` 는 `equality` 제약만 요구하지만 `Set` 은 `comparison` 제약을 요구한다. 대부분의 컬렉션 타입 사이에는 이런 변환 함수가 양방향으로 준비되어 있다.

키를 따로 뽑아야 하는 경우가 실제로는 더 흔하다. 레코드 리스트를 특정 필드로 묶거나 그 필드 기준으로 중복을 제거하는 일이다.

```
type Ticket = { Code: string; Tier: string }

let tickets =
  [ { Code = "T-01"; Tier = "일반" }
    { Code = "T-02"; Tier = "학생" }
    { Code = "T-03"; Tier = "일반" }
    { Code = "T-04"; Tier = "경로" } ]

// 등급별 장수 세기
let counts =
  tickets
  |> List.groupBy (fun t -> t.Tier)
  |> List.map (fun (tier, ts) -> tier, List.length ts)

printfn "counts = %A" counts
// [("일반", 2); ("학생", 1); ("경로", 1)]
```

`groupBy` 다음에 `List.length` 로 개수를 세는 모양에도 전용 함수가 있다. `List.countBy (fun t -> t.Tier)` 가 같은 결과를 낸다.

`List.distinctBy` 는 키가 중복되면 먼저 나온 원소를 남기고 나머지를 버린다. 돌려주는 것은 키가 아니라 원래 원소라는 점을 시그니처에서 확인해 두는 것이 좋다.

```
// List.distinctBy: ('a -> 'b) -> 'a list -> 'a list (when 'b : equality)
// 결과 타입이 'b list 가 아니라 'a list 다. 원소를 남긴다.
let samples = tickets |> List.distinctBy (fun t -> t.Tier)

samples |> List.iter (fun t -> printfn " %s / %s" t.Code t.Tier)
// T-01 / 일반
// T-02 / 학생
// T-04 / 경로
```

## Solving a Problem in Many Ways — 같은 문제, 여러 풀이 (원서 pp.70-71)

- `List` 모듈에 함수가 넉넉하게 있으니 대부분의 문제는 여러 방식으로 풀린다.
- 여기서 풀 문제는 이렇다. 회선별 데이터 사용량 목록이 있고 무료 한도가 100 GB 다. 한도를 넘긴 회선의 초과분만 모아 합계를 구한다.
- 풀이마다 중간에 만들어지는 리스트의 개수와 실패 가능성이 다르다. 그 차이가 코드에 드러나게 나눠 본다.

```
// 이 단위가 보여주는 것: 같은 집계를 여섯 가지 방식으로 푸는 비교

// 회선별 데이터 사용량(GB), 무료 한도는 100
let usage = [40; 120; 95; 260; 100; 175]
```

첫째, 단계별로 나눠 쓰는 방법이다. 걸러 내고, 바꾸고, 합한다. 의도가 가장 명확하지만 중간 리스트가 두 개 만들어진다.

```
let stepByStep =
  usage
  |> List.filter (fun gb -> gb > 100)
  |> List.map (fun gb -> gb - 100)
  |> List.sum

printfn "단계별      = %d" stepByStep    // 255
```

둘째, `List.choose` 로 걸러 내기와 변환을 한 번에 하는 방법이다. `'a -> 'b option` 함수를 받아 `Some` 인 것만 남기고 꺾뎀기를 벗겨 준다. 중간 리스트가 하나로 줄고, "골라내면서 바꾼다"는 뜻이 한 줄에 들어온다.

```
// List.choose : ('a -> 'b option) -> 'a list -> 'b list
let withChoose =
  usage
  |> List.choose (fun gb -> if gb > 100 then Some (gb - 100) else None)
  |> List.sum

printfn "choose      = %d" withChoose    // 255
```

셋째, `List.collect` 를 쓰는 방법이다. `'a -> 'b list` 함수를 받아 결과 리스트들을 하나로 이어 붙인다. 원소 하나가 결과 0개나 여러 개로 늘어나는 경우에 맞는 함수이고, 이 문제에서는 `choose` 로 충분하므로 굳이 쓸 이유가 없다.

```
// List.collect : ('a -> 'b list) -> 'a list -> 'b list
let withCollect =
  usage
  |> List.collect (fun gb -> if gb > 100 then [gb - 100] else [])
  |> List.sum

printfn "collect     = %d" withCollect    // 255
```

넷째, `fold` 로 한 번만 훑는 방법이다. 중간 리스트가 아예 없다. 대신 누적값과 조건 판단이 한 람다에 섞여 읽기가 무거워진다.

```
let withFold =
    usage
    |> List.fold (fun acc gb -> acc + (if gb > 100 then gb - 100 else 0)) 0

printfn "fold          = %d" withFold    // 255
```

다섯째는 쓰면 안 되는 방법이다. `fold` 를 `reduce` 로 바꾸고 싶어지지만 이 문제에서는 틀린 답이 나온다. 이유가 두 가지다. `reduce` 는 부분 함수라 빈 리스트를 따로 처리해야 하고, 더 중요한 것은 첫 원소가 `reduce` 에 넘긴 함수(FSharp.Core 의 매개변수 이름은 `reduction` 이다)를 거치지 않고 그대로 초기 상태가 된다는 점이다. 즉 첫 회선의 사용량 40 이 초과분 계산 없이 합계에 얹힌다.

```
let withReduce =
    match usage with
    | [] -> 0
    | items ->
        items
        |> List.reduce (fun acc gb -> acc + (if gb > 100 then gb - 100 else 0))

printfn "reduce      = %d" withReduce    // 295 - 틀렸다. 첫 원소 40 이 그대로 더해졌다
```

여섯째, 권장하는 형태다. `sumBy` 하나로 끝난다. 넘지 않은 회선은 0 을 내놓게 하면 걸러 내기가 필요 없다.

```
let recommended =
    usage
    |> List.sumBy (fun gb -> if gb > 100 then gb - 100 else 0)

printfn "sumBy      = %d" recommended    // 255
```

정리하면 고르는 순서는 이렇다. 목적이 뚜렷한 전용 함수( `sumBy` , `countBy` , `maxBy` )를 먼저 찾고, 없으면 `filter` / `map` / `choose` 를 조합하고, 그래도 표현이 안 되면 `fold` 로 내려간다. `reduce` 는 상태와 원소의 타입이 같고 빈 리스트가 들어올 수 없다고 확신할 때만 쓴다.

## Working Through a Practical Example — 배운 것을 묶어 쓰기 (원서 pp.71-79)

- 여기서 만드는 것은 불변 리스트를 품은 도메인 타입과 그 리스트를 갱신하는 함수다. 원서는 주문과 주문 항목으로 예를 들었다. 이 노트는 커피 배합표로 같은 구조를 만든다.
- 필요한 기능은 다섯 가지다. 원두 추가, 이미 있는 원두의 양 늘리기, 원두 제거, 양 줄이기, 전부 비우기.
- 갱신 함수는 모두 새 `Blend` 를 돌려준다. 원본은 손대지 않는다. 리스트도 레코드도 불변이므로 이것이 자연스러운 형태다.

```
// 이 단위가 보여주는 것: 불변 리스트를 품은 도메인에 갱신 기능을 붙이는 과정

type Portion = { BeanId: int; Grams: int }
type Blend = { BlendId: int; Portions: Portion list }
```

원서는 의사 코드로 순서를 먼저 적고 단계별로 채워 나간다. 같은 방식으로 생각하면 원두를 추가하는 함수는 네 단계다. 새 항목을 리스트 앞에 붙이고, 같은 원두를 하나로 합치고, 원두 번호로 정렬하고, 배합표를 새 리스트로 갱신한다.

첫 시도는 1단계와 4단계만 하는 것이다.

```
// 아직 미완성이다. 같은 원두를 추가하면 항목이 둘로 남는다.
let addPortion portion blend =
  { blend with Portions = portion :: blend.Portions }
```

이 상태로는 이미 있는 원두를 추가했을 때 `[{1; 300}; {1; 250}]` 처럼 같은 번호가 두 줄로 남는다. 원두별로 합치려면 `List.groupBy` 가 필요하다. 그런데 `groupBy` 는 `(int * Portion list) list` 를 돌려주므로 `Portions` 필드가 기대하는 `Portion list` 와 타입이 맞지 않아 컴파일되지 않는다. 묶은 결과를 `List.map` 으로 다시 `Portion` 으로 만들어야 한다. 마지막으로 정렬까지 넣으면 완성이다. 정렬이 있어야 리스트의 구조적 동등성(structural equality) 비교가 항목 순서에 흔들리지 않는다.

세 함수가 이 "합치고 정렬하기"를 공유하므로 처음부터 따로 뽑아 둔다.

```
// recalculate: portions: Portion list -> Portion list
let recalculate portions =
  portions
  |> List.groupBy (fun p -> p.BeanId)           // (int * Portion list) list
  |> List.map (fun (beanId, ps) ->              // 다시 Portion list 로
    { BeanId = beanId; Grams = ps |> List.sumBy (fun p -> p.Grams) })
  |> List.sortBy (fun p -> p.BeanId)           // 동등성 비교를 쉽게 하려고 정렬

// addPortion: portion: Portion -> blend: Blend -> Blend
let addPortion portion blend =
  let portions =
    portion :: blend.Portions
  |> recalculate
  { blend with Portions = portions }
```

`portion :: blend.Portions |> recalculate` 가 의도대로 읽히는 것은 `::` 가 `|>` 보다 강하게 묶기 때문이다. `cons` 가 먼저 일어나고 그 결과가 파이프로 넘어간다.

원두를 여러 개 한 번에 넣으려면 `cons` 연산자만 `@` 로 바꾸면 된다. `::` 는 원소 하나를 앞에 붙이고 `@` 는 리스트 둘을 잇는다. 나머지 로직은 그대로다.

```
// addPortions: newPortions: Portion list -> blend: Blend -> Blend
let addPortions newPortions blend =
  let portions =
    newPortions @ blend.Portions
  |> recalculate
  { blend with Portions = portions }
```

제거는 `List.filter` 로 원두 번호가 다른 것만 남기면 된다.

```
// removeBean: beanId: int -> blend: Blend -> Blend
let removeBean beanId blend =
  let portions =
    blend.Portions
    |> List.filter (fun p -> p.BeanId <> beanId)
    |> List.sortBy (fun p -> p.BeanId)
  { blend with Portions = portions }
```

양을 줄이는 함수가 가장 재미있다. 줄일 양을 음수로 만들어 리스트에 넣고 `recalculate` 에 맡기면 합산만으로 차감이 된다. 그다음 `Grams` 가 0 이하인 항목을 버리면 "전량을 줄이면 항목이 사라진다"와 "없는 원두를 줄이려 하면 아무 일도 없다"가 동시에 처리된다. 없는 원두의 음수 항목은 합산 결과가 음수로 남아 필터에서 버려진다.



```
// reduceBean: beanId: int -> grams: int -> blend: Blend -> Blend
let reduceBean beanId grams blend =
    let portions =
        { BeanId = beanId; Grams = -grams } :: blend.Portions
    |> recalculate
    |> List.filter (fun p -> p.Grams > 0)
    { blend with Portions = portions }

// clearPortions: blend: Blend -> Blend
let clearPortions blend = { blend with Portions = [] }
```

원서는 함수를 하나 만들 때마다 xUnit 테스트를 붙이고 `dotnet test` 로 돌린다. 4챕터에서 만든 구조를 쓰고 이중 백틱 이름으로 상황을 문장처럼 적는다. 이 노트에서는 아래 형태가 그 테스트에 해당한다. 원서의 `Orders.fs / module Domain` 에 해당하는 것이 여기서는 `Blends.fs / module Recipe` 다. 테스트 파일의 네임스페이스는 원서 p.74 를 따랐다. 원서 p.71 은 같은 파일에 `namespace MyProject.Orders` 를 적으라고 했다가 p.74 에서 `namespace OrderTests` 로 바꾸는데, 앞의 것은 코드 쪽 파일의 네임스페이스이므로 그대로 따르면 `open` 이 자기 네임스페이스를 여는 모양이 된다.

```
namespace BlendTests

open MyProject.Blends
open MyProject.Blends.Recipe
open Xunit
open FsUnit

module ``원두를 배합표에 추가할 때`` =

    [<Fact>]
    let ``빈 배합표에 없던 원두를 넣으면 항목이 하나 생긴다`` () =
        let blend = { BlendId = 1; Portions = [] }
        let expected = { BlendId = 1; Portions = [ { BeanId = 1; Grams = 250 } ] }
        let actual = blend |> addPortion { BeanId = 1; Grams = 250 }
        actual |> should equal expected
```

FSI 로만 확인할 때는 기댓값과 실제값을 `=` 로 비교해 참인지 보면 된다. 레코드와 리스트 모두 구조적 동등성이 있어서 필드 하나씩 꺼내 볼 필요가 없다.

```
let check label expected actual =
    printfn "%s %s" (if actual = expected then "OK " else "FAIL") label

let house = { BlendId = 1; Portions = [ { BeanId = 1; Grams = 300 } ] }
let blank = { BlendId = 9; Portions = [] }

check "빈 배합표에 새 원두"
    { BlendId = 9; Portions = [ { BeanId = 1; Grams = 250 } ] }
    (blank |> addPortion { BeanId = 1; Grams = 250 })

check "없던 원두를 추가"
    { BlendId = 1; Portions = [ { BeanId = 1; Grams = 300 }; { BeanId = 2; Grams = 120 } ] }
    (house |> addPortion { BeanId = 2; Grams = 120 })

check "있던 원두를 추가하면 합산"
    { BlendId = 1; Portions = [ { BeanId = 1; Grams = 420 } ] }
    (house |> addPortion { BeanId = 1; Grams = 120 })
// OK 빈 배합표에 새 원두
// OK 없던 원두를 추가
// OK 있던 원두를 추가하면 합산
```

나머지 함수도 같은 방식으로 확인한다. 경계 조건을 빠뜨리지 않는 것이 요령이다. 빈 배합표, 없는 원두, 전량 차감이 그것이다.

```

check "여러 개를 한 번에"
  { BlendId = 1; Portions = [ { BeanId = 1; Grams = 400 }; { BeanId = 3; Grams = 50 } ] }
  (house |> addPortions [ { BeanId = 1; Grams = 100 }; { BeanId = 3; Grams = 50 } ])

check "원두 제거"
  { BlendId = 1; Portions = [] }
  (house |> removeBean 1)

check "없는 원두 제거는 무변화"
  house
  (house |> removeBean 7)

check "양 줄이기"
  { BlendId = 1; Portions = [ { BeanId = 1; Grams = 200 } ] }
  (house |> reduceBean 1 100)

check "전량을 줄이면 항목이 사라진다"
  { BlendId = 1; Portions = [] }
  (house |> reduceBean 1 300)

check "없는 원두를 줄여도 무변화"
  house
  (house |> reduceBean 5 50)

check "빈 배합표에서 줄여도 무변화"
  blank
  (blank |> reduceBean 5 50)

check "전부 비우기"
  { BlendId = 1; Portions = [] }
  (house |> clearPortions)

// 모든 갱신을 거친 뒤에도 원본은 그대로다
check "원본 불변"
  { BlendId = 1; Portions = [ { BeanId = 1; Grams = 300 } ] }
  house
  // OK 여러 개를 한 번에
  // OK 원두 제거
  // OK 없는 원두 제거는 무변화
  // OK 양 줄이기
  // OK 전량을 줄이면 항목이 사라진다
  // OK 없는 원두를 줄여도 무변화
  // OK 빈 배합표에서 줄여도 무변화
  // OK 전부 비우기
  // OK 원본 불변

```

마지막 검사가 이 절의 요점이다. 갱신 함수를 아무리 많이 호출해도 `house` 는 처음 값 그대로다. 상태를 제자리에 서 바꾸지 않고 새 값을 만들어 돌려주기 때문에, 테스트가 서로 간섭하지 않고 순서에 상관없이 돌아간다.

## Summary — 원서의 챕터 요약 (원서 p.79)

- 원서는 이 챕터에서 `List` 모듈의 유용한 함수들을 살펴봤다고 정리한다. `Seq` 와 `Array` 모듈에도 비슷한 함수들이 있고 이어지는 챕터에서 나온다.
- 불변 데이터 구조만으로도 짧고 견고한 업무 기능을 만들 수 있다는 것을 실전 예제에서 확인했다는 말도 덧붙인다.
- 컬렉션으로 할 수 있는 일의 표면만 훑었으므로 `List` 모듈 전체는 F# 공식 문서를 보라고 권한다. 커뮤니티 기여로 모듈 함수마다 예제 코드가 붙어 있다.
- 다음 챕터에서는 CSV 파일에서 데이터 스트림을 읽어 처리하는 방법을 다룬다.

## 정리 — 이 노트의 요약

- F#의 주요 컬렉션은 `Seq` (지연), `Array` (즉시·연속 메모리), `List` (즉시·불변 연결 리스트) 셋이다. C#의 `List<'T>`는 F#에서 `ResizeArray<'T>`이며 F#의 `List`와 다른 것이다.
- 리스트는 머리와 꼬리로 이루어지고, 그 구조가 `[]` / `[x]` / `h :: t` 패턴으로 그대로 쓰인다. `[x]` 케이스는 없어도 빠짐없는 패턴 매칭이 되며, 원소 하나일 때 다른 결과를 내야 할 때만 따로 둔다.
- 빈 리스트 `[]`는 자동 일반화로 `'a list`가 되지만, `List.rev []`처럼 함수 적용 결과를 매개변수 없는 `let`에 묶으면 값 제한 오류 FS0030이 난다.
- `::`는 원소 하나를 앞에 붙이고 `@`는 리스트 둘을 잇는다. `::`가 `|>`보다 강하게 묶이므로 `x :: xs |> f`는 `(x :: xs) |> f`로 읽힌다.
- `List.map`은 새 리스트를 만들고(`'a -> 'b`), `List.iter`는 부수 효과만 낸다(`'a -> unit`). 시그니처의 반환 타입이 둘의 용도를 가른다.
- `map` 뒤에 `sum`을 붙이는 모양은 `sumBy` 하나로 줄어들고 `averageBy`도 같다. 다만 `maxBy` / `minBy`는 뽑아낸 값이 아니라 원소를 돌려주고 `countBy`는 (키 \* 개수) 튜플의 리스트를 돌려주므로 결과 타입이 다르다.
- `List.average`와 `List.averageBy`는 `DivideByInt` 제약 때문에 `int`에 쓸 수 없다(오류 FS0001). `float`나 `decimal`로 먼저 올려야 한다.
- `List.fold`의 `folder`는 `'a(상태) -> 'b(원소) -> 'a`다. `List.foldBack`의 `folder`는 `'a(원소) -> 'b(상태) -> 'b`로 매개변수 순서가 뒤집히고 인자 순서도 다르다.
- `List.reduce`는 상태와 원소의 타입이 같아야 하고 첫 원소를 초기값으로 쓴다. 그래서 빈 리스트에서 `ArgumentException`을 던지고, 첫 원소는 `reduction`을 거치지 않는다. `List.tryReduce`는 존재하지 않는다.
- `List.groupBy`는 `('b * 'a list) list`를 돌려준다. 중복 제거만 필요하다면 `List.distinct`가 짧고, `Set.ofList`는 순서를 정렬 순서로 바꾸며 `comparison` 제약을 요구한다.
- `List.distinctBy`가 돌려주는 것은 키가 아니라 원소다(`'a list`). 키가 겹치면 먼저 나온 원소가 남는다.
- 함수를 고르는 순서는 전용 집계 함수 → `filter` / `map` / `choose` 조합 → `fold`다. `reduce`는 조건이 맞을 때만 쓴다.
- 불변 컬렉션을 품은 레코드는 갱신 함수가 새 값을 돌려주게 만든다. 정렬을 한 번 끼워 두면 구조적 동등성 비교 한 줄로 전체를 검증할 수 있다.

## 원서 대조 표

| 절                                                 | 원서 페이지   | 실행 단위                    |
|---------------------------------------------------|----------|--------------------------|
| Before We Start — 준비                              | p.63     | —                        |
| The Basics — 세 가지 컬렉션과 그 차이                       | p.63     | —                        |
| Core Functionality — 리스트 만들기과 핵심 함수               | pp.63-68 | 05-list-basics , 05-core |
| Folding — 누적값을 직접 굴리기                             | pp.68-69 | 05-fold                  |
| Grouping Data and Uniqueness — 묶기와 중복 제거          | pp.69-70 | 05-group                 |
| Solving a Problem in Many Ways — 같은 문제, 여러 풀이     | pp.70-71 | 05-many-ways             |
| Working Through a Practical Example — 배운 것을 묶어 쓰기 | pp.71-79 | 05-blend                 |
| Summary — 원서의 챕터 요약                               | p.79     | —                        |

## 06 - 파일에서 데이터 읽기 (원서 pp.80-89)

이 챕터에서는 3챕터의 `Result` 와 5챕터의 컬렉션을 합쳐 바깥 세계의 데이터를 프로그램 안으로 들여온다. 소재는 구분자로 나뉜 텍스트 파일 하나지만, 얻어 가야 할 것은 두 가지다. 하나는 지연 평가되는 시퀀스를 `try/with` 로 감싸면 예외를 놓친다는 사실이고, 다른 하나는 데이터 출처를 함수 매개변수로 빼면 파일 없이도 테스트할 수 있다는 설계다. 파싱한 결과를 문자열 그대로 두는 원서의 선택도 의도된 것이며, 실패 이유를 남기는 검증은 8챕터의 몫이다.

시작하기 전에 이 챕터가 계속 쓰는 용어 셋을 정리해 둔다.

- 시퀀스 식(sequence expression)은 `seq { ... }` 안에서 값을 차례로 내놓아 `seq<'T>` 를 만드는 문법이다. `seq<'T>` 는 .NET 의 `IEnumerable<'T>` 와 같은 것이다.
- 지연 평가(lazy evaluation)는 값을 만드는 시점을 실제로 필요할 때까지 미루는 것이다. 시퀀스는 순회할 때 비로소 원소를 만든다.
- 부분 함수(partial function)는 입력 중 일부에서 값을 돌려주지 못하고 예외를 던지는 함수다. 이름이 비슷한 부분 적용(partial application)과는 아무 관계가 없다.

## Setting Up — 예제 데이터를 어디에 둘까 (원서 p.80)

- 원서는 콘솔 프로젝트를 만들고 프로젝트 폴더 아래 `resources` 에 데이터 파일을 둔 뒤, 내장 상수 `__SOURCE_DIRECTORY__` 로 소스 폴더 경로를 얻어 파일 경로를 조립한다.
- `main` 이 마지막에 돌려주는 `0` 은 프로세스 종료 코드다. F# 6 부터는 `[<EntryPoint>]` 를 붙이지 않아도 되고, 마지막 코드 파일의 최상위 코드가 그대로 진입점이 된다.
- 이 노트는 프로젝트 대신 스크립트로 검증하므로 저장소 안에 데이터 파일을 만들지 않는다. 각 실행 단위가 시스템 임시 경로에 자기 입력 파일을 만들고, 끝에서 지운다.
- 소재는 도서관 대출 기록이다. 열은 `LoanId|MemberId|Title|DueDate|Renewed|Fine` 여섯 개이고 구분자는 `|` 다. 마지막 줄은 값이 거의 빈 기록으로, 뒤에서 파싱이 어디까지 버텨 주는지 보는 데 쓴다.

```
// 원서와 같은 프로젝트 구성이라면 진입점이 이런 모양이 된다. `__SOURCE_DIRECTORY__` 는
// 이 코드가 적힌 파일의 폴더를 가리키므로 실행 폴더가 어디든 같은 파일을 찾는다.
// 여기 쓰인 importFromFile 은 이 노트 마지막 절에서 만든다
[<EntryPoint>]
let main _ =
    Path.Combine(__SOURCE_DIRECTORY__, "resources", "loans.csv")
    |> importFromFile
    0
```

아래부터는 이 노트의 방식이다. 같은 구성을 프로젝트 없이 재현하려고 데이터 파일을 임시 경로에 만든다.

```
// 이 단위가 보여주는 것: 입력 파일을 임시 경로에 만들어 시퀀스로 읽어 들이는 방법
open System
open System.IO

// 저장소를 더럽히지 않는다. 임시 경로에 만들고 이 단위 끝에서 지운다
let loansPath =
    Path.Combine(Path.GetTempPath(), "loans-" + Guid.NewGuid().ToString("N") + ".csv")

let rows =
    [ "LoanId|MemberId|Title|DueDate|Renewed|Fine"
      "L-1001|M-07|자료구조 첫걸음|2024-05-02|1|0.00"
      "L-1002|M-11|정보 검색 개론|2024-05-11|0|1.50"
      "L-1003|M-04|근현대사 강의|2024-04-28|1|3.20"
      "L-1004|||||" ]

File.WriteAllLines(loansPath, rows)
printfn "입력 파일 준비: %b" (File.Exists loansPath) // 기대: 입력 파일 준비: true
```

## Loading Data — 파일을 문자열 시퀀스로 (원서 pp.80-82)

- 파일을 읽는 함수의 목표 시그니처는 `string -> string seq` 다. 경로를 받아 줄들의 시퀀스를 돌려준다.
- 리스트나 배열로 만들 수도 있지만 `File.ReadLines` 나 `Directory.EnumerateFiles` 처럼 `IEnumerable<'T>` 를 돌려주는 `System.IO` 메서드가 있어 시퀀스가 그대로 맞물린다.
- `StreamReader` 는 `IDisposable` 을 구현한다. `use` 로 바인딩하면 스코프가 끝날 때 `Dispose()` 가 호출된다. C# 의 `using` 문에 대응한다. 여기서 스코프는 들여쓰기가 정한 시퀀스 식 안쪽이고, 시퀀스 순회가 끝나는 시점에 해제된다. 순회를 중간에 끊어도 끊는 자리에서 해제된다. 반대로 한 번도 순회하지 않으면 이 줄 자체가 실행되지 않아 파일이 열리지도 않는데, 그 성질이 다음 절의 함정을 만든다.
- `IDisposable` 타입의 인스턴스를 만들 때는 `new` 를 붙인다. 붙이지 않으면 컴파일러가 경고 FS0760 으로 알려 준다. 원서가 이 인터페이스를 `IDisposable<'T>` 로 적은 것은 오키이며, 실제 `IDisposable` 은 제네릭이 아니다.

```
// string -> string seq
// while ... do 안에서 값을 그대로 두면 시퀀스의 원소가 된다(암시적 yield)
let readWithReader path =
    seq {
        use reader = new StreamReader(File.OpenRead path)
        while not reader.EndOfStream do
            reader.ReadLine()
    }

// Seq.iter : ('a -> unit) -> 'a seq -> unit
readWithReader loansPath |> Seq.iter (printfn "%s")
// 기대:
// LoanId|MemberId|Title|DueDate|Renewed|Fine
// L-1001|M-07|자료구조 첫걸음|2024-05-02|1|0.00
// L-1002|M-11|정보 검색 개론|2024-05-11|0|1.50
// L-1003|M-04|근현대사 강의|2024-04-28|1|3.20
// L-1004|||||
```

`Seq.iter` 는 원소마다 넘겨받은 함수를 실행하고 `unit` 을 돌려준다. `List` 와 `Array` 모듈에 있는 함수는 대부분 같은 이름으로 `Seq` 모듈에도 있다.

직접 `StreamReader` 를 다루지 않아도 되는 지름길이 둘 있다. 시그니처를 F# Interactive(FSI) 로 실측하면 이렇게 갈린다.

```
File.ReadLines : string -> string seq // .NET 원형은 IEnumerable<string> 을 돌려준다
File.ReadAllLines : string -> string array
```

- `File.ReadAllLines` 는 호출한 자리에서 파일 전체를 읽어 배열로 고정한다. 그 뒤 파일이 바뀌어도 손에 든 배열은 바뀌지 않는다.
- `File.ReadLines` 는 시퀀스를 돌려주고 내용 읽기를 미룬다. 다만 파일을 여는 일은 호출 시점에 한다. 순회를 두 번 하면 두 번째 순회부터 파일을 다시 열어 읽는다.
- 큰 파일을 한 번만 훑을 때는 `File.ReadLines` 가 메모리에 유리하다. 여러 번 훑을 값이라면 `List.ofSeq` 나 `Seq.cache` 로 한 번만 읽어 고정하는 편이 낫다.

아래 블록은 지연 쪽을 `File.ReadLines` 대신 앞에서 만든 `readWithReader` 로 대표하게 두었다. 순회할 때까지 파일을 열지 않으므로, 값을 만든 뒤에 같은 파일로 쓰기를 해도 막히지 않는다.

```
// ReadAllLines 는 이 줄에서 파일을 다 읽고 닫는다
let eagerLines = File.ReadAllLines loansPath // string array

// readWithReader 가 돌려준 시퀀스는 아직 파일을 열지도 않았다
let lazyLines = readWithReader loansPath // string seq

// 두 값을 만든 뒤에 파일에 한 줄을 덧붙인다
File.AppendAllLines(loansPath, [ "L-1005|M-02|서지학 개론|2024-06-01|0|0.00" ])

printfn "지연 시퀀스 : %d" (Seq.length lazyLines) // 기대: 지연 시퀀스 : 6
printfn "즉시 배열 : %d" (Array.length eagerLines) // 기대: 즉시 배열 : 5
```

덧붙인 줄이 `lazyLines` 쪽에만 보인다. 시퀀스는 순회할 때 비로소 파일을 읽기 때문이다. 값을 만든 순서가 아니라 순회한 순서가 결과를 정한다는 점이 다음 절의 함정으로 이어진다.

```
File.Delete loansPath
printfn "임시 파일 정리: %b" (not (File.Exists loansPath)) // 기대: 임시 파일 정리: true
```

## 읽기 실패를 `Result` 로 옮기기 (원서 pp.82-83)

- 파일 읽기는 실패할 수 있다. 경로가 없거나 권한이 없으면 예외가 날아온다. 3챕터에서 본 대로 `try/with` 로 받아 `Result` 로 돌려주면 시그니처가 실패 가능성을 말해 준다.
- 목표 시그니처는 `string -> Result<string seq, exn>` 다.
- 여기서 원서가 짚는 함정이 나온다. `try` 블록 안에서 시퀀스 식을 만들어 `ok` 로 감싸면, `try` 가 끝나는 시점에는 파일을 아직 열지 않았다. 예외는 나중에 순회할 때 터지므로 `with` 절이 잡지 못한다.

```
// 이 단위가 보여주는 것: 자연 평가되는 시퀀스를 try/with 로 감쌀 때 생기는 함정과 그 고침
open System
open System.IO

let tempCsv prefix =
    Path.Combine(Path.GetTempPath(), prefix + "-" + Guid.NewGuid().ToString("N") + ".csv")

let rows =
    [ "LoanId|MemberId|Title|DueDate|Renewed|Fine"
      "L-1001|M-07|자료구조 첫걸음|2024-05-02|1|0.00" ]

let loansPath = tempCsv "loans"
File.WriteAllLines(loansPath, rows)

// 경로 문자열만 만들고 파일은 쓰지 않았다. 읽으려 하면 실패한다
let missingPath = tempCsv "missing"
```

준비가 끝났으니 함정이 있는 판을 먼저 만들어 본다.

```
// string -> Result<string seq, exn>
let readLazily path =
    try
        seq {
            use reader = new StreamReader(File.OpenRead path)
            while not reader.EndOfStream do
                reader.ReadLine()
        }
    |> Ok
    with ex -> Error ex

// 예외 메시지에겐 실행마다 달라지는 경로가 들어가므로 타입 이름만 찍는다
let label result =
    match result with
    | Ok _ -> "Ok"
    | Error (ex: exn) -> "Error " + ex.GetType().Name

printfn "있는 파일: %s" (label (readLazily loansPath)) // 기대: 있는 파일: Ok
printfn "없는 파일: %s" (label (readLazily missingPath)) // 기대: 없는 파일: Ok
```

없는 파일인데도 `Ok` 다. `Ok` 안에 든 시퀀스는 아직 아무것도 하지 않은 약속일 뿐이고, 예외는 그 약속을 실제로 실행할 때 나온다.

```
match readLazily missingPath with
| Ok data ->
    try
        printfn "줄 수 %d" (Seq.length data)
    with ex ->
        printfn "순회 시점에 터진다: %s" (ex.GetType().Name)
        // 기대: 순회 시점에 터진다: FileNotFoundException
| Error _ -> printfn "여기로는 오지 않는다"
```

고치는 방법은 간단하다. `try` 안에서 하는 일이 즉시 실패를 드러내게 만들면 된다. `File.ReadLines` 는 내용 읽기는 미루지만 경로 검사와 파일 열기는 호출 시점에 하므로, 없는 경로는 그 자리에서 예외가 된다. 대신 호출만

해 놓고 순회하지 않으면 열린 파일 핸들(file handle)이 남으므로, 확인만 하는 자리에서도 시퀀스를 한 번 비워 줘야 한다.

```
// string -> Result<string seq, exn>
let readFile path =
    try
        File.ReadLines path |> Ok
    with ex -> Error ex

// File.ReadLines 는 호출 시점에 파일을 열어 두므로, Ok 안의 시퀀스를 순회하지 않고
// 버리면 열린 파일 핸들이 남는다. 확인만 하는 자리에서도 한 번 순회해 닫아 준다
let labelDrained result =
    match result with
    | Ok (data: string seq) ->
        data |> Seq.iter ignore
        "Ok"
    | Error (ex: exn) -> "Error " + ex.GetType().Name

printfn "있는 파일: %s" (labelDrained (readFile loansPath))    // 기대: 있는 파일: Ok
printfn "없는 파일: %s" (labelDrained (readFile missingPath)) // 기대: 없는 파일: Error FileNotFoundException
```

`Result` 를 돌려주게 되었으니 호출하는 쪽은 두 갈래를 모두 처리해야 한다. 성공이면 데이터를 쓰고, 실패면 이유를 알린다. 이 갈림길을 한 함수로 떼어 두면 호출부가 단순해진다.

```
let show path =
    match readFile path with
    | Ok data -> data |> Seq.iter (printfn " %s")
    | Error ex -> printfn " 읽기 실패: %s" (ex.GetType().Name)

show loansPath
show missingPath
// 기대:
//   LoanId|MemberId|Title|DueDate|Renewed|Fine
//   L-1001|M-07|자료구조 첫걸음|2024-05-02|1|0.00
//   읽기 실패: FileNotFoundException

File.Delete loansPath
printfn "임시 파일 정리: %b" (not (File.Exists loansPath)) // 기대: 임시 파일 정리: true
```

## Parsing Data — 줄을 레코드로 (원서 pp.83-85)

- 다음 할 일은 줄 하나를 레코드 하나로 바꾸는 것이다. 이 단계에서는 모든 필드를 `string` 으로 둔다. 값의 의미를 따지는 일은 뒤로 미루고, 모양만 맞추는 데 집중한다.
- 타입 선언은 `let` 바인딩보다 위에 모아 두는 것이 관례다. F# 은 위에서 아래로만 이름이 보이므로 순서가 곧 규칙이다.
- 파일에서 읽은 데이터는 순수하지 않은 경계에서 온 값이다. 칸 수가 맞지 않을 수 있으니 파싱 결과는 `RawLoan` 이 아니라 `RawLoan option` 이어야 한다.



```
// 이 단위가 보여주는 것: 배열 패턴으로 줄을 쪼개 레코드로 바꾸고 실패한 줄을 걸러 내기
type RawLoan =
    { LoanId: string
      MemberId: string
      Title: string
      DueDate: string
      Renewed: string
      Fine: string }

// 마지막 줄은 칸이 셋뿐인 깨진 줄이다
let sample =
    seq {
        "LoanId|MemberId|Title|DueDate|Renewed|Fine"
        "L-1001|M-07|자료구조 첫걸음|2024-05-02|1|0.00"
        "L-1002|M-11|정보 검색 개론|2024-05-11|0|1.50"
        "L-1003|M-04|근현대사 강의|2024-04-28|1|3.20"
        "L-1004|||||"
        "L-1005|M-02|서지학 개론"
    }
}
```

문자열을 구분자로 쪼개는 것은 .NET 의 `Split` 메서드다. `char` 하나를 넘기는 오버로드는 `string array` 를 돌려준다.

```
// .NET 메서드 원형(F# 이 보여 주는 표기)
String.Split(separator: char, ?options: StringSplitOptions) : string array
```

돌려받은 배열을 배열 패턴(array pattern)으로 매칭하면 각 칸을 이름에 바로 묶을 수 있다. 이때 패턴에 적은 이름의 개수와 배열 길이가 정확히 같아야 그 케이스가 성립한다. 쓰지 않을 칸은 와일드카드로 둘 수 있다.

```
// string -> RawLoan option
let parseLine (line: string) : RawLoan option =
    match line.Split('|') with
    | [| loanId; memberId; title; dueDate; renewed; fine |] ->
        Some
            { LoanId = loanId
              MemberId = memberId
              Title = title
              DueDate = dueDate
              Renewed = renewed
              Fine = fine }
    | _ -> None

printfn "%A" (parseLine "L-1005|M-02|서지학 개론") // 기대: None
printfn "%b" (parseLine "L-1004|||||" |> Option.isSome) // 기대: true
```

칸 개수만 맞으면 값이 비어 있어도 `Some` 이다. `L-1004` 줄이 통과하는 것은 이 단계가 모양만 본다는 뜻이다. 값이 비었다는 사실을 문제로 삼으려면 판단 기준이 필요하고, 그것이 8챕터의 검증이다.

이제 시퀀스 전체에 적용한다. 첫 줄은 헤더 줄이므로 버려야 한다.

```
// string seq -> RawLoan seq
// Seq.skip : int -> 'a seq -> 'a seq
// Seq.choose: ('a -> 'b option) -> 'a seq -> 'b seq
let parse (data: string seq) =
    data
    |> Seq.skip 1          // 헤더 줄 버리기
    |> Seq.map parseLine
    |> Seq.choose id       // None 은 버리고 Some 은 벗긴다

sample
|> parse
|> Seq.iter (fun loan -> printfn "%-7s %-6s %A" loan.LoanId loan.MemberId loan.Title)
// 기대:
// L-1001 M-07 "자료구조 첫걸음"
// L-1002 M-11 "정보 검색 개론"
// L-1003 M-04 "근현대사 강의"
// L-1004 ""

printfn "읽은 줄 %d, 만들어진 레코드 %d" (Seq.length sample) (sample |> parse |> Seq.length)
// 기대: 읽은 줄 6, 만들어진 레코드 4
```

- `Seq.map parseLine` 다음에 오는 `Seq.choose id` 가 `RawLoan option seq` 를 `RawLoan seq` 로 좁힌다. `id` 는 `'a -> 'a` 인 항등 함수이고 `fun x -> x` 와 같다.
- `Seq.map f >> Seq.choose id` 는 `Seq.choose f` 한 번과 결과가 같다. 원서는 두 단계로 보여 주지만 실무에서는 `Seq.choose parseLine` 으로 줄여 쓰는 쪽이 흔하다.
- 여기서 세 컬렉션 타입 표기가 모두 등장했다. 리스트는 `[ ... ]`, 배열은 `[| ... |]`, 시퀀스는 `seq { ... }` 다.
- `Seq.map` 에 넘긴 `parseLine` 처럼, 람다로 감싸지 않고 함수 이름만 적는 것이 관용적이다. `fun x -> parseLine x` 는 같은 뜻의 더 긴 표기다.

`Seq.skip` 에는 함정이 하나 있다. 원소가 모자라면 예외를 던지는 부분 함수다. 헤더 줄만 있고 데이터가 없는 파일은 흔하고, 아예 빈 파일도 있을 수 있다. 예외가 나는 시점은 앞 절과 같은 규칙을 따른다. `Seq.skip 1` 을 부르는 순간이 아니라 그 결과를 처음 순회하는 순간이다. 그래서 아래 `try` 는 순회까지 감싸야 한다.

```
try
    Seq.empty<string> |> parse |> Seq.length |> printfn "%d"
with ex ->
    printfn "빈 입력: %s" (ex.GetType().Name) // 기대: 빈 입력: InvalidOperationException
```

원서도 이 문제를 짚어 두지만 챕터 뒤에서 고치겠다고만 적고, 최종 코드에는 `Seq.skip 1` 이 그대로 남아 있다. 지금 고치려면 개수를 세지 말고 위치로 걸러 내면 된다. `Seq.indexed` 로 번호를 붙이고 0번만 버리는 방식은 입력이 비어도 조용히 빈 시퀀스를 돌려준다.

```
// string seq -> string seq
// Seq.indexed: 'a seq -> (int * 'a) seq
let dropHeader (data: string seq) =
    data
    |> Seq.indexed
    |> Seq.filter (fun (index, _) -> index > 0)
    |> Seq.map snd

let parseSafely (data: string seq) =
    data |> dropHeader |> Seq.choose parseLine

printfn "빈 입력(고친 뒤): %d" (Seq.empty<string> |> parseSafely |> Seq.length) // 기대: 빈 입력(고친 뒤): 0
printfn "결과는 동일: %b" (List.ofSeq (parse sample) = List.ofSeq (parseSafely sample)) // 기대: 결과는 동일: true
```

마지막 비교가 참인 것은 레코드의 구조적 동등성 덕분이다. 리스트로 굳히면 `=` 한 번으로 전체를 비교할 수 있다.

## 타입 있는 필드로 한 걸음 더 (원서 pp.83-85 확장)

원서는 모든 필드를 `string` 으로 남겨 둔 채 챕터를 마치지만, 실무에서는 날짜와 금액을 알맞은 타입으로 옮겨 담게 된다. 그 자리에서 만나는 것이 .NET 의 `TryParse` 계열이다.

- .NET 의 `TryParse` 가 F# 에서 `bool * 'a` 튜플로 넘어오는 까닭은 3챕터에서 다뤘다. F# 에는 `out` 매개 변수를 선언하는 문법이 없어 컴파일러가 그 값을 반환값 쪽으로 옮겨 주기 때문이다. 이 튜플을 패턴 매칭해 `Option` 으로 바꾸는 것이 관용적인 처리다.
- 문화권을 지정하는 오버로드를 쓰는 편이 안전하다. `CultureInfo.InvariantCulture` 로 고정하면 실행 환경의 지역 설정에 따라 소수점이나 날짜 해석이 달라지는 일을 막을 수 있다. 허용할 표기도 함께 지정하는데, 숫자 쪽은 `NumberStyles` , 날짜 쪽은 `DateTimeStyles` 다.

두 표기를 나란히 놓으면 `out` 매개변수가 어디로 옮겨 가는지 보인다.

```
// .NET 메서드 원형(F# 이 보여 주는 표기)
Decimal.TryParse(s: string, result: byref<decimal>) : bool

// F# 에서 호출하면 out 매개변수가 반환값에 붙는다
Decimal.TryParse : string -> bool * decimal
```

튜플을 매칭하는 `match` 를 `TryParse` 호출마다 되풀이할 이유는 없다. 한 자리에 모아 두고 재사용한다.

```
// 이 단위가 보여주는 것: TryParse 를 Option 으로 감싸는 어댑터와 타입 있는 레코드 만들기
open System
open System.Globalization

// (string -> bool * 'a) -> string -> 'a option
let tryParseWith (parser: string -> bool * 'a) (text: string) =
    match parser text with
    | true, value -> Some value
    | false, _ -> None
```

`tryParseWith` 는 "성공 여부와 값을 튜플로 돌려주는 함수"를 받아 `Option` 을 돌려주는 함수로 바꿔 준다. `TryParse` 를 쓰는 자리마다 같은 `match` 를 반복하지 않게 하는 어댑터다.

```
let invariant = CultureInfo.InvariantCulture

// 부분 적용으로 필드별 파서를 만든다
// FSI 실측: val tryDecimal: (string -> decimal option)
let tryDecimal = tryParseWith (fun text -> Decimal.TryParse(text, NumberStyles.Number, invariant))

// FSI 실측: val tryDate: (string -> System.DateTime option)
let tryDate = tryParseWith (fun text -> DateTime.TryParse(text, invariant, DateTimeStyles.None))

// string -> bool option
let tryFlag text =
    match text with
    | "1" -> Some true
    | "0" -> Some false
    | _ -> None

printfn "%A" (tryDecimal "3.20") // 기대: Some 3.20M
printfn "%A" (tryDecimal "") // 기대: None
printfn "%A" (tryDate "2024-05-02" |> Option.map (fun date -> date.ToString("yyyy-MM-dd")))
// 기대: Some "2024-05-02"
printfn "%A" (tryFlag "1", tryFlag "예") // 기대: (Some true, None)
```

`tryDecimal` 과 `tryDate` 의 시그니처가 괄호에 싸여 나오는 것은 이 둘이 함수를 담은 값 바인딩이기 때문이다. FSI 는 매개변수를 적어 정의한 함수 바인딩과 이렇게 구분해 보여 준다.

이제 레코드의 필드 타입을 실제 의미에 맞춘다. 배열 패턴으로 칸을 나눈 뒤, 세 파서의 결과를 튜플로 묶어 한 번에 매칭하면 전부 성공한 경우만 골라낼 수 있다.

```
type Loan =
{ LoanId: string
  MemberId: string
  Title: string
  DueDate: DateTime
  Renewed: bool
  Fine: decimal }

// string -> Loan option
let parseLine (line: string) : Loan option =
    match line.Split('|') with
    | [| loanId; memberId; title; dueDate; renewed; fine |] ->
        match tryDate dueDate, tryFlag renewed, tryDecimal fine with
        | Some due, Some isRenewed, Some amount ->
            Some
                { LoanId = loanId
                  MemberId = memberId
                  Title = title
                  DueDate = due
                  Renewed = isRenewed
                  Fine = amount }
        | _ -> None
    | _ -> None
```

세 파서가 모두 `Some` 을 내놓은 줄만 `Loan` 이 된다. 앞 절과 같은 입력에 이 `parseLine` 을 걸어 본다.

```
let sample =
    seq {
        "LoanId|MemberId|Title|DueDate|Renewed|Fine"
        "L-1001|M-07|자료구조 첫걸음|2024-05-02|1|0.00"
        "L-1002|M-11|정보 검색 개론|2024-05-11|0|1.50"
        "L-1003|M-04|근현대사 강의|2024-04-28|1|3.20"
        "L-1004|||||"
        "L-1005|M-02|서지학 개론"
    }

let parse (data: string seq) =
    data
    |> Seq.indexed
    |> Seq.filter (fun (index, _) -> index > 0)
    |> Seq.map snd
    |> Seq.choose parseLine

let loans = parse sample |> List.ofSeq

loans
|> List.iter (fun loan ->
    printfn "%-7s %s %-5b %.2f" loan.LoanId (loan.DueDate.ToString("yyyy-MM-dd")) loan.Renewed loan.Fine)
// 기대:
// L-1001 2024-05-02 true 0.00
// L-1002 2024-05-11 false 1.50
// L-1003 2024-04-28 true 3.20

printfn "레코드 %d건" (List.length loans) // 기대: 레코드 3건
printfn "연체료 합계 %.2f" (loans |> List.sumBy (fun loan -> loan.Fine)) // 기대: 연체료 합계 4.70
```

필드에 타입이 붙자 `L-1004` 줄이 탈락했다. 날짜와 금액 칸이 비어 있어 파싱에 실패했기 때문이다. 얻은 것은 확실하다. `Loan` 값을 손에 넣은 뒤에는 날짜 계산이나 금액 합계를 별도 검사 없이 할 수 있다.

않은 것도 있다. `None` 은 어느 칸이 왜 틀렸는지 말해 주지 않는다. 칸 개수가 안 맞은 줄과 낱자가 깨진 줄이 결과에서 구별되지 않는다. 이유를 남기려면 `Option` 이 아니라 `Result` 가 필요하고, 여러 필드의 실패를 모아 보고 하는 방법이 8챕터의 주제다.

## Testing the Code — 데이터 출처를 매개변수로 (원서 pp.85-87)

- 지금까지 만든 코드는 잘 동작하지만 테스트하기가 번거롭다. 확인하려면 매번 실제 파일이 있어야 한다.
- 열쇠는 읽기 함수의 시그니처다. `string -> Result<string seq, exn>` 는 "문자열을 주면 줄들을 돌려주거나 실패한다"는 뜻일 뿐, 디스크를 언급하지 않는다. 웹 서비스나 테스트용 가짜(fake) 데이터도 이 시그니처를 만족할 수 있다.
- 그러니 읽기 함수를 `import` 안에서 직접 부르지 말고 매개변수로 받는다. 함수를 매개변수로 받으므로 `import` 는 고차 함수가 된다. 매개변수 이름을 `dataReader` 로 두면 출처가 파일에 한정되지 않는다는 의도가 드러난다.
- 긴 함수 타입을 매개변수 자리에 계속 적으면 읽기에 좋지 않다. 타입 약어(type abbreviation)로 이름을 붙이면 시그니처가 문서 구실을 한다. 새 타입을 만드는 것이 아니라 별명을 붙이는 것이므로, 같은 시그니처의 함수는 무엇이든 그 자리에 들어간다.
- 함수 하나만 있는 인터페이스를 넘기는 것과 쓰임새가 같다. 다만 타입을 새로 정의하거나 클래스를 만들 필요가 없다.

```
// 이 단위가 보여주는 것: 타입 약어로 데이터 출처를 추상화하고 가짜 리더로 테스트하기
open System
open System.IO

type RawLoan =
{ LoanId: string
  MemberId: string
  Title: string
  DueDate: string
  Renewed: string
  Fine: string }

// 이 시그니처만 맞으면 파일이든 웹이든 테스트용 가짜 데이터든 상관없다
type DataReader = string -> Result<string seq, exn>
```

파싱은 앞 절의 `parseSafely` 와 같은 방식이고, 출력은 리스트로 한 번 굳혀 놓고 두 번 훑는다.

```

let parseLine (line: string) : RawLoan option =
    match line.Split('|') with
    | [| loanId; memberId; title; dueDate; renewed; fine |] ->
        Some
            { LoanId = loanId
              MemberId = memberId
              Title = title
              DueDate = dueDate
              Renewed = renewed
              Fine = fine }
    | _ -> None

let parse (data: string seq) =
    data
    |> Seq.indexed
    |> Seq.filter (fun (index, _) -> index > 0)
    |> Seq.map snd
    |> Seq.choose parseLine

// 시퀀스를 두 번 순회하면 파일을 다시 열어 두 번 읽는다. 리스트로 한 번 굳혀 놓고 쓴다
let report (loans: RawLoan seq) =
    let items = List.ofSeq loans
    items |> List.iter (fun loan -> printfn " %7s %-6s %A" loan.LoanId loan.MemberId loan.Title)
    printfn " 총 %d건" (List.length items)

```

`import` 는 리더를 받아 결과의 두 갈래를 처리한다. 첫 매개변수의 타입만 약어로 적어 두면 무엇을 넘겨야 하는지 시그니처가 말해 준다.

```

// DataReader -> string -> unit
let import (dataReader: DataReader) path =
    match path |> dataReader with
    | Ok data -> data |> parse |> report
    | Error ex -> printfn " 가져오기 실패: %s" (ex.GetType().Name)

```

타입 약어를 반환 타입 자리가 아니라 바인딩 전체의 타입으로 쓰려면 함수 스타일을 바꿔야 한다. 매개변수를 이름 옆에 적는 대신 람다를 값으로 바인딩한다. 두 표기가 만드는 함수는 같다.

```

// FSI 실측: val readFile: path: string -> Result<string seq, exn>
// 타입 약어는 새 타입이 아니라 별명이므로 펼친 시그니처와 같은 것이다
let readFile: DataReader =
    fun path ->
        try
            File.ReadLines path |> Ok
        with ex ->
            Error ex

// 테스트용 리더. 경로를 받고도 무시한다
let fakeReader: DataReader =
    fun _ ->
        seq {
            "LoanId|MemberId|Title|DueDate|Renewed|Fine"
            "L-2001|M-31|필사본 연구|2024-07-03|0|0.00"
            "L-2002|M-08|서양 서지학|2024-07-19|1|2.40"
        }
    |> Ok

printfn "가짜 리더:"
import fakeReader "(이 경로는 쓰이지 않는다)"
// 기대:
// 가짜 리더:
//   L-2001   M-31   "필사본 연구"
//   L-2002   M-08   "서양 서지학"
//   총 2건

```

파일에서 읽는 조합을 자주 쓴다면 부분 적용으로 이름을 붙여 둔다. 인자 하나만 적용한 새 함수가 되고, 남은 매개변수는 경로 하나다.

```
// FSI 실측: val importFromFile: (string -> unit)
let importFromFile = import readFile

let loansPath =
    Path.Combine(Path.GetTempPath(), "loans-" + Guid.NewGuid().ToString("N") + ".csv")

File.WriteAllLines(
    loansPath,
    [ "LoanId|MemberId|Title|DueDate|Renewed|Fine"
      "L-3001|M-19|도서관 경영론|2024-08-02|1|0.00"
      "L-3002|M-19|장서 개발|2024-08-02|0|0.75" ]
)

printfn "파일 리더:"
importFromFile loansPath
// 기대:
// 파일 리더:
//   L-3001  M-19   "도서관 경영론"
//   L-3002  M-19   "장서 개발"
//   총 2건

printfn "없는 파일:"
importFromFile (Path.Combine(Path.GetTempPath(), "loans-" + Guid.NewGuid().ToString("N") + ".csv"))
// 기대:
// 없는 파일:
// 가져오기 실패: FileNotFoundException

File.Delete loansPath
printfn "임시 파일 정리: %b" (not (File.Exists loansPath)) // 기대: 임시 파일 정리: true
```

같은 `import` 가 세 상황을 모두 받아냈다. 파일도, 가짜 데이터도, 실패도 호출부를 고치지 않고 통과한다. 단위 테스트에서는 `fakeReader` 같은 함수를 넘기고 결과를 어서션으로 확인하면 된다. 이때 파일 시스템은 등장하지 않는다. 4챕터에서 본 "테스트하기 쉬운 코드는 대상이 순수한 코드"라는 기준을 파일 입출력이 섞인 코드에 적용한 것이 이 절이다.

## Final Code — 최종 형태 (원서 pp.87-89)

- 원서는 마지막에 코드 전체를 한 파일로 모아 보여 준다. 위 `06-datareader` 단위가 그 형태에 대응한다.
- 파일 안의 순서에 규칙이 있다. 타입 선언(레코드, 타입 약어)이 먼저, 그다음이 리더, 파싱, 출력, 마지막이 이들을 엮는 `import` 다. F# 은 위에서 아래로만 이름이 보이므로 의존 방향이 그대로 순서가 된다.
- 원서의 최종 코드는 파일 시스템을 건드리는 함수가 `readFile` 하나뿐이다. 나머지는 모두 순수 함수이므로 입력만 주면 테스트할 수 있다. 경계를 한 지점으로 몰아 두는 것이 이 챕터의 설계 요점이다.

## Summary — 원서의 챕터 요약 (원서 p.89)

- 원서는 이 챕터에서 `Seq` 모듈의 자주 쓰는 함수와 시퀀스 식, 그리고 함수 타입을 써서 외부 데이터를 가져오는 방법을 다뤘다.
- 원서는 고차 함수 덕분에 실행 시점에 다른 함수를 끼워 넣을 수 있고 테스트도 쉬워진다는 점을 강조한다.
- 다음 챕터에서는 패턴 매칭을 더 읽기 좋게 만드는 액티브 패턴을 살펴본다.

## 정리 — 이 노트의 요약

- 파일 읽기 함수의 목표 시그니처는 `string -> Result<string seq, exn>` 다. 경로를 받아 줄들을 돌려주거나 실패를 값으로 알린다.
- 시퀀스는 지연 평가된다. `try/with` 안에서 시퀀스를 만들어 `Ok` 로 감싸면 예외는 순회 시점에 터지므로 `with` 절이 잡지 못한다. `try` 블록 안의 일이 즉시 실패를 드러내야 `Result` 가 정직해진다.
- `File.ReadAllLines` 는 그 자리에서 배열로 고정하고, `File.ReadLines` 는 내용 읽기를 미루되 경로 검사와 파일 열기는 호출 시점에 한다. 여러 번 순회할 값이라면 `List.ofSeq` 로 굳혀 두는 편이 낫다.
- `use` 는 스코프가 끝날 때 `Dispose()` 를 부른다. 시퀀스 식 안에서는 순회가 끝나거나 끊기는 시점이 그 지점이며, `IDisposable` 인스턴스는 `new` 로 만든다.
- `Split` 이 돌려준 배열을 배열 패턴으로 매칭하면 칸을 이름에 바로 묶을 수 있다. 개수가 다르면 그 케이스는 성립하지 않으므로 `_ -> None` 이 깨진 줄을 받아낸다.
- `Seq.choose` 는 `None` 을 버리고 `Some` 을 벗긴다. `Seq.map f >> Seq.choose id` 는 `Seq.choose f` 와 같다.
- `Seq.skip` 은 부분 함수다. 원소가 모자라면 첫 순회에서 `InvalidOperationException` 이 나므로, 빈 입력이 있을 수 있는 자리에서는 `Seq.indexed` 와 `Seq.filter` 처럼 개수를 요구하지 않는 방법을 쓴다.
- .NET 의 `TryParse` 는 F# 에서 `bool * 'a` 튜플로 넘어온다. 튜플을 매칭해 `Option` 으로 바꾸는 어댑터 함수 하나( `(string -> bool * 'a) -> string -> 'a option` )를 두면 필드별 파서를 부분 적용으로 찍어 낼 수 있다.
- 파싱 결과를 `Option` 으로 두면 실패 이유가 사라진다. 어느 칸이 왜 틀렸는지 알려야 한다면 `Result` 로 옮겨야 하며, 그 작업이 8첩터다.
- 데이터 출처를 매개변수로 받으면 `import` 가 고차 함수가 되어, 실제 파일 없이도 테스트할 수 있다. 긴 함수 타입에는 타입 약어로 이름을 붙인다. 약어는 별명이므로 시그니처가 같은 함수는 무엇이든 들어간다.
- 파일 시스템에 닿는 함수를 하나로 몰아 두면 나머지는 순수 함수로 남는다. 이 경계 설계가 이 챕터의 결론이다.

## 원서 대조 표

| 절                                | 원서 페이지      | 실행 단위            |
|----------------------------------|-------------|------------------|
| Setting Up — 예제 데이터를 어디에 둘까      | p.80        | 06-load          |
| Loading Data — 파일을 문자열 시퀀스로      | pp.80-82    | 06-load          |
| 읽기 실패를 <code>Result</code> 로 옮기기 | pp.82-83    | 06-reader-result |
| Parsing Data — 줄을 레코드로           | pp.83-85    | 06-parse         |
| 타입 있는 필드로 한 걸음 더                 | pp.83-85 확장 | 06-typed         |
| Testing the Code — 데이터 출처를 매개변수로 | pp.85-87    | 06-datareader    |
| Final Code — 최종 형태               | pp.87-89    | 06-datareader    |
| Summary — 원서의 챕터 요약              | p.89        | —                |



## 07 - 액티브 패턴 (원서 pp.90-102)

지금까지 패턴 자리에 쓸 수 있는 것은 언어가 미리 정해 둔 패턴뿐이었다. 판별 유니온의 케이스, 튜플, 리터럴, 리스트 모양, 와일드카드 정도다. 이 챕터는 그 목록에 직접 만든 패턴을 추가하는 방법을 다룬다. 액티브 패턴(active pattern)은 함수를 패턴 자리에서 쓸 수 있는 이름으로 바꿔 주는 장치다. “문자열이 연도로 읽히는가”, “이 요청이 느린가” 같은 판정을 `match` 식의 케이스 식별자처럼 보이게 만들 수 있고, 판정 결과로 얻은 값을 그 자리에서 바로 바인딩할 수 있다. 종류가 네 가지이고 각각 반환 타입 규칙이 다르므로, 이 챕터의 목표는 네 종류를 구분해 두는 것이다. 다음 챕터의 검증 코드가 여기서 익힌 부분 액티브 패턴 위에 그대로 올라간다.

### Setting Up — 준비 (원서 p.90)

- 이 챕터의 코드는 전부 스크립트 파일( `.fsx` ) 하나와 FSI 로 끝난다. 프로젝트를 만들 필요가 없다.
- 1챕터 노트 끝에서 `(|OnActivePlan|_)` 을 한 번 맛보기로 썼다. 그때는 "필터를 케이스 식별자처럼 쓸 수 있다"는 감각만 얻고 넘어갔다. 여기서는 그 문법이 왜 그렇게 생겼는지와 나머지 세 종류를 다룬다. 맛보기 예제는 다시 쓰지 않는다.

### 네 종류를 먼저 구분한다 (원서 pp.90-98)

원서는 종류를 하나씩 순서대로 소개하지만, 먼저 전체 지도를 보아 두면 덜 헷갈린다. 액티브 패턴의 이름은 항상 `(|` 와 `|)` 사이에 들어간다. 이 괄호 짝을 바나나 클립(banana clips)이라 부른다.

| 종류                                    | 이름 형태                                      | 반환 타입                                   | 매개변수 추가           | 실패할 수 있는가 |
|---------------------------------------|--------------------------------------------|-----------------------------------------|-------------------|-----------|
| 부분 패턴(partial)                        | <code>( Name _)</code>                     | <code>'a option (F# 9 부터 bool 도)</code> | 가능(붙이면 아래 종류가 된다) | 그렇다       |
| 매개변수 있는 부분 패턴 (parameterized partial) | <code>( Name _)</code><br><code>arg</code> | <code>'a option (F# 9 부터 bool 도)</code> | 이미 붙어 있다          | 그렇다       |
| 다중 케이스 패턴(multi-case)                 | <code>( A B C )</code>                     | <code>Choice&lt;...&gt;</code>          | 불가                | 아니다       |
| 단일 케이스 패턴(single-case)                | <code>( Name )</code>                      | 값 그대로                                   | 가능(별도 이름 없음)      | 아니다       |

- 이름 끝에 `_|` 가 붙으면 부분 액티브 패턴이다. 이름 목록의 마지막 자리에 와일드카드가 있다는 뜻이고, "입력 중 일부만 이 패턴에 걸린다"는 선언이다. 그래서 반환 타입이 `option` 이다(F# 9 부터는 `bool` 도 되고, `[<return: Struct>]` 를 붙이면 `voption` 이다. 아래에서 짚는다). 이름이 비슷한 부분 적용(partial application)과는 관계가 없다.
- `_|` 가 없으면 모든 입력이 적어 둔 케이스 중 하나로 반드시 떨어진다. 그래서 `option` 으로 감싸지 않고 값을 그대로 반환한다.
- 케이스 식별자는 대문자로 시작해야 한다. 소문자로 쓰면 오류 FS0623( 활성 패턴 케이스 식별자는 대문자로 시작해야 합니다 )이 난다.
- 네 종류라는 이름은 원서와 공식 문서가 쓰는 관용 분류다. 실제 축은 두 개다. 하나는 실패할 수 있는가(이름에 `_|` 가 있는가)이고, 다른 하나는 케이스가 하나인가 여럿인가. 매개변수는 두 축 위에 얹는 선택지이고 부분 패턴과 단일 케이스 패턴 양쪽에 붙는다.
- 두 축에 매개변수까지 격자로 놓으면 막힌 칸이 두 개다. 부분 패턴과 다중 케이스 패턴을 섞은 `(|Red|Black|_|)` 는 문법 자체가 없어 오류 FS3872 로 거부되고, 매개변수를 붙인 다중 케이스 패턴은 정의는 통과하지만 쓰는 순간 오류 FS0722 가 난다. 그래서 골라 쓸 조합이 네 이름에 거의 다 들어오고, 따로 이름이 붙지 않은 것은 매개변수 있는 단일 케이스 패턴뿐이다.

```
// 오류 FS3872: Multi-case partial active patterns are not supported.
// 오류는 이름 자리에서 난다. 본문을 어떻게 쓰든 이 이름 형태는 성립하지 않는다.
let (|Red|Black|_|) (n: int) =
    if n % 2 = 0 then Some Red else None
```

## Partial Active Patterns — 부분 액티브 패턴 (원서 pp.90-92)

- 입력 중 일부만 성공하는 판정에 쓴다. 반환 타입은 `option` 이고, 성공 케이스에서 값을 담아 보내면 패턴을 쓰는 쪽에서 그 값을 바인딩할 수 있다.
- 파싱과 검증이 대표적인 용도다. `TryParse` 계열 메서드의 튜플 반환을 `option` 으로 바꿔 패턴 자리로 옮기는 일이 가장 잦다.
- 값이 필요 없고 성공/실패만 알면 될 때는 `Some ()` 을 반환한다. 이때 시그니처는 `unit option` 이 된다.

먼저 액티브 패턴 없이 평범한 함수로 써 본다. 도서관 서지 데이터에서 출간 연도 칸을 읽는 상황이다.

```
// 이 단위가 보여주는 것: 부분 액티브 패턴으로 파싱·검증을 패턴 자리로 옮기기
open System

// string -> int option
let tryPublishedYear (input: string) =
    match Int32.TryParse input with
    | true, year when year >= 1450 && year <= 2026 -> Some year
    | _ -> None

printfn "%A" (tryPublishedYear "1998") // Some 1998
printfn "%A" (tryPublishedYear "간행연도미상") // None
printfn "%A" (tryPublishedYear "1200") // None
```

함수로도 잘 돌아간다. 다만 이 판정을 `match` 식의 케이스로는 쓸 수 없다. 가드 절에서 호출하는 것이 전부이고, 성공했을 때 얻은 연도 값을 케이스 안으로 끌어오려면 한 번 더 벗겨내야 한다. 이름만 바나나 클립으로 감싸면 사정이 달라진다.

```
// string -> int option - 본문은 위 함수와 같고 이름만 (| ... |_) 로 감쌌다
let (|PublishedYear|_) (input: string) =
    match Int32.TryParse input with
    | true, year when year >= 1450 && year <= 2026 -> Some year
    | _ -> None
```

- 시그니처는 `tryPublishedYear` 와 완전히 같다. 액티브 패턴은 특별한 종류의 값이 아니라 이름 형태가 특별한 함수다.
- 달라지는 것은 이름을 쓸 수 있는 자리다. `PublishedYear y` 를 패턴으로 적을 수 있고, `Some` 안에 담아 보낸 값이 `y` 에 바인딩된다.

```
// string -> string
let shelve (raw: string) =
    match raw with
    | PublishedYear y when y >= 2000 -> $"%d{y}년 - 개가 열람실"
    | PublishedYear y -> $"%d{y}년 - 보존 서고"
    | _ -> $"'%s{raw}' - 연도 확인 필요"

[ "2014"; "1998"; "삼국사기" ] |> List.iter (shelve >> printfn "%s")
// 2014년 - 개가 열람실
// 1998년 - 보존 서고
// '삼국사기' - 연도 확인 필요
```

- 같은 액티브 패턴을 두 케이스에서 쓰면서 한쪽에만 가드 절을 걸었다. 이렇게 판정과 분기를 나눠 적을 수 있는 것이 함수판과의 실질적 차이이다.
- 부분 액티브 패턴만으로 `match` 식을 구성하면 마지막 `| _ ->` 를 반드시 적어야 한다. 컴파일러는 이 패턴이 실패할 수 있다는 것을 이름의 `|_` 로 알고 있으므로, 빼면 경고 FS0025 가 난다.

값이 필요 없는 경우도 있다. 그때는 `unit` 을 `Some` 에 담는다.

```
// string -> unit option - 성공/실패만 알려 준다
let (|Blank|_) (input: string) =
    if String.IsNullOrEmpty input then Some () else None

// string -> bool
let hasYear (raw: string) =
    match raw with
    | Blank -> false
    | PublishedYear _ -> true
    | _ -> false

printfn "%b %b %b" (hasYear " ") (hasYear "1998") (hasYear "미상") // false true false
```

- `Some ()` 을 반환하는 패턴은 담아 보낼 값이 없으므로 패턴 자리에 이름만 적는다. 실리는 값이 `()` 하나여서 `Blank ()` 나 `Blank _` 도 통과하지만, 이름만 적는 쪽이 관용적이다.
- F# 9 부터는 `unit option` 대신 `bool` 을 반환해도 된다. `if ... then Some () else None` 을 그대로 조건식으로 줄일 수 있다. 원서는 `Some ()` 판으로 설명한다. 두 형태 모두 유효하다.
- `[<return: Struct>]` 를 붙여 `option` 대신 `voption` 을 반환하는 판도 F# 6 부터 쓸 수 있다. 힙 할당을 피하는 최적화이고 종류가 늘어나는 것은 아니어서 이 노트는 다루지 않는다.

```
// string -> bool - bool 을 반환하는 부분 액티브 패턴. 값은 바인딩할 수 없다
let (|Numeric|_) (input: string) =
    input.Length > 0 && input |> Seq.forall Char.IsDigit

printfn "%b %b" (match "1998" with Numeric -> true | _ -> false)
                (match "199a" with Numeric -> true | _ -> false) // true false
```

- `bool` 을 반환하면 담아 보낼 값이 아예 없으므로 패턴 자리에 인자를 적을 수 없다. `Numeric x` 로 적으면 오류 FS3868( 이 활성 패턴에는 인수가 필요하지 않습니다 )이 난다. `unit option` 판과 달리 `Numeric _` 도 받아 주지 않는다.

액티브 패턴이 그냥 함수라는 사실은 실제로 확인할 수 있다. 이름을 바나나 클립재로 적으면 일반 함수처럼 호출하거나 고차 함수에 넘길 수 있다.

```
// 패턴 자리 밖에서 함수로 쓴다
printfn "%A" ((|PublishedYear|_) "1998") // Some 1998
printfn "%A" ([ "1998"; "미상"; "2014" ] |> List.choose (|PublishedYear|_))
// [1998; 2014]
```

- `List.choose` 는 'a -> 'b option 을 받는다. 부분 액티브 패턴의 시그니처가 정확히 그 모양이므로 그 대로 들어맞는다.
- 반대로 말하면, 부분 액티브 패턴 하나를 정의해 두면 패턴 자리와 파이프라인 양쪽에서 쓸 수 있다. 함수판을 따로 두는 대신 액티브 패턴 하나로 통일할 수 있다는 뜻이다.

## Parameterized Partial Active Patterns — 매개변수 있는 부분 액티브 패턴 (원서 pp.92-96)

- 판정 기준을 패턴을 쓰는 쪽에서 정하고 싶을 때 매개변수를 추가한다. 문법적으로 특별한 것은 없다. 매개변수를 더 받는 부분 액티브 패턴이다.
- 규칙은 하나뿐이다. 검사할 값이 항상 마지막 매개변수여야 한다. 앞쪽 매개변수는 패턴 자리에서 이름 뒤에 인자로 적는다.
- 패턴끼리 조합하는 연산자가 있다. `&` 는 둘 다 만족, `|` 는 하나라도 만족이다. 부정 연산자는 없다.

등산로 데이터에 난이도를 매기는 예다. 기준값을 패턴 쪽에서 지정한다.

```
// 이 단위가 보여주는 것: 매개변수 있는 부분 액티브 패턴과 패턴 조합 연산자
type Trail = { Name: string; DistanceKm: float; AscentM: int }

// float -> Trail -> unit option
let (|LongerThan|_) limit (trail: Trail) =
    if trail.DistanceKm > limit then Some () else None

// int -> Trail -> unit option
let (|SteeperThan|_) limit (trail: Trail) =
    if trail.AscentM > limit then Some () else None
```

- 시그니처를 보면 기준값이 앞, 검사 대상이 뒤다. 패턴 자리에 적은 `LongerThan 12.0` 은 컴파일러가 `(|LongerThan|_) 12.0 <검사값>` 호출로 풀어낸다. `match` 식이 검사하는 값이 마지막 인자로 붙기 때문에 검사할 값을 마지막 매개변수에 두어야 한다.
- 매개변수 타입은 자동 일반화되지 않았다. `trail.DistanceKm` 과 비교하므로 `limit` 이 `float` 로, `trail.AscentM` 과 비교하므로 `int` 로 각각 확정되었다.

부정 연산자가 없다는 제약이 여기서 드러난다. "경사가 완만하다"는 조건이 필요하다면 반대 판정을 하나 더 정의해야 한다.

```
// int -> Trail -> unit option - & 와 | 는 있지만 not 은 없어서 반대 패턴을 따로 만든다
let (|GentlerThan|_|) limit (trail: Trail) =
    if trail.AscentM <= limit then Some () else None

// Trail -> string
let grade trail =
    match trail with
    | LongerThan 12.0 & SteeperThan 900 -> "상급"
    | LongerThan 12.0 | SteeperThan 500 -> "중급"
    | GentlerThan 300 -> "가족 코스"
    | _ -> "초급"

let trails =
    [ { Name = "지리산 종주"; DistanceKm = 25.4; AscentM = 1650 }
      { Name = "관악산 사당길"; DistanceKm = 4.6; AscentM = 540 }
      { Name = "북한산 둘레길"; DistanceKm = 8.2; AscentM = 210 }
      { Name = "청계산 매봉"; DistanceKm = 5.3; AscentM = 380 } ]

trails |> List.iter (fun t -> printfn "%s: %s" t.Name (grade t))
// 지리산 종주: 상급
// 관악산 사당길: 중급
// 북한산 둘레길: 가족 코스
// 청계산 매봉: 초급
```

- `&` 와 `|` 는 패턴 전용 연산자다. `&&`, `||`, `not` 같은 일반 논리 연산자는 식에서 쓰는 것이고 패턴 자리에서는 쓸 수 없다.
- 가드 절에는 이 제약이 없다. `| t when t.AscentM <= 300 -> ...` 처럼 쓰면 일반 논리 연산자를 그대로 쓸 수 있다. 원서도 이 점을 짚으면서 액티브 패턴 판과 가드 절 판을 나란히 보여 준다.
- 원서는 FizzBuzz 와 윤년 판정을 예로 들어 조건 조합이 늘어날 때를 실험한다. 조건이 셋이 되면 `&` 조합을 일곱 줄 적어야 하고, 그중 하나를 빠뜨렸는지 눈으로 확인하기 어려워진다. 액티브 패턴이 항상 최선은 아니라는 결론이 이 실험의 요점이다.
- 원서는 이 실험 뒤에 액티브 패턴을 떠나 `List.map / List.reduce` 로 FizzBuzz 를 다시 쓰고 짝 목록을 매개변수로 빼는 데까지 간다. 액티브 패턴 이야기가 아니고 `List.reduce` 는 5챕터에서 이미 다뤘으므로 이 노트는 옮기지 않는다.

경우의 수가 늘어날 때 원서가 제시하는 우회로 중 하나는 매개변수를 리스트로 받는 것이다. 조합마다 패턴을 나열하는 대신 목록 하나로 표현한다.

```
// string list -> Trail -> unit option - 매개변수는 리스트여도 된다
let (|OneOf|_|) (names: string list) (trail: Trail) =
    if names |> List.contains trail.Name then Some () else None

// Trail -> string
let permit trail =
    match trail with
    | OneOf [ "지리산 종주"; "설악산 공룡능선" ] -> "입산 신고 필요"
    | _ -> "자유 입산"

trails |> List.iter (fun t -> printfn "%s: %s" t.Name (permit t))
// 지리산 종주: 입산 신고 필요
// 관악산 사당길: 자유 입산
// 북한산 둘레길: 자유 입산
// 청계산 매봉: 자유 입산
```

매개변수 있는 부분 액티브 패턴도 값을 반환할 수 있다. `unit option` 만 쓰는 것이 아니다. 문자열에서 단위를 떼어내는 예를 보면 매개변수와 반환값을 함께 쓰는 모양이 드러난다.

```
// string -> string -> string option - 접미사가 붙어 있으면 떼어낸 앞부분을 돌려준다
let (|EndingWith|_|) (suffix: string) (input: string) =
    if input.EndsWith suffix
    then Some (input.Substring(0, input.Length - suffix.Length))
    else None

// string -> string
let readPace input =
    match input with
    | EndingWith "km" stem -> $"거리 %s{stem}킬로미터"
    | EndingWith "m" stem -> $"고도 %s{stem}미터"
    | _ -> "단위를 읽을 수 없다"

printfn "%s / %s / %s" (readPace "25km") (readPace "1650m") (readPace "빠름")
// 거리 25킬로미터 / 고도 1650미터 / 단위를 읽을 수 없다
```

- 패턴 자리에 이름, 인자, 바인딩할 변수가 차례로 온다. `EndingWith "km" stem` 에서 `"km"` 은 인자이고 `stem` 은 결과를 받는 이름이다.
- 케이스 순서가 결과를 바꾼다. `"25km"` 는 `"m"` 으로도 끝나므로, 두 케이스를 뒤집으면 `"25k"` 가 고도로 읽힌다. 첫 매칭이 이긴다는 패턴 매칭의 기본 규칙은 액티브 패턴에도 그대로 통한다.
- `|` 로 묶은 두 패턴은 같은 변수 집합을 바인딩해야 한다. `EndingWith "km" x | EndingWith "m" x` 는 되지만 양쪽이 서로 다른 이름을 바인딩하면 오류 FS0018 이 난다.

매개변수는 부분 액티브 패턴과 단일 케이스 액티브 패턴에만 붙일 수 있다. 다중 케이스 패턴에 붙이면 정의는 통과하지만 쓰는 순간 오류 FS0722 가 난다. 원서도 이 조합이 "컴파일은 되는데 실제로는 쓸 수 없다"고 적어 두었다.

```
// 정의는 통과한다
let (|Above|Below|) threshold value =
    if value >= threshold then Above else Below

// 쓰는 순간 오류 FS0722: 하나의 결과를 반환하는 활성 패턴만 인수를 사용할 수 있습니다
let check limit n =
    match n with
    | Above limit -> "위"
    | Below limit -> "아래"
```

## Multi-Case Active Patterns — 다중 케이스 액티브 패턴 (원서 pp.96-97)

- 입력을 정해진 몇 가지 중 하나로 반드시 분류할 때 쓴다. 실패가 없으므로 `option` 을 쓰지 않고, 이름에 `|_|` 도 붙지 않는다.
- 케이스를 전부 적으면 `match` 식이 빠짐없는 패턴 매칭이 되어 `| _ ->` 가 필요 없다. 하나라도 빠뜨리면 경고 FS0025 가 난다.
- 케이스는 최대 일곱 개다. 여덟 개를 적으면 오류 FS0265( 활성 패턴은 7개가 넘는 가능성을 반환할 수 없습니다 )가 난다.
- 실제 반환 타입은 `Choice` 다. 이 점은 시그니처를 실측해 보면 바로 보인다.
- 상한이 7개인 것은 `FSharp.Core` 에 `Choice<'T1, 'T2>` 부터 `Choice<'T1, ..., 'T7>` 까지만 있기 때문이다. 담을 그릇이 없어서 생긴 한계다.

라디오 편성표를 짜면서 음원을 길이로 분류하는 예다.

```
// 이 단위가 보여주는 것: 다중 케이스 액티브 패턴과 Choice 반환
type Track = { Title: string; Seconds: int }

// Track -> Choice<unit,unit,unit>
let (|Short|Standard|Extended|) (track: Track) =
    if track.Seconds < 150 then Short
    elif track.Seconds <= 420 then Standard
    else Extended

// Track -> string - 세 케이스를 다 적었으므로 와일드카드가 필요 없다
let slot track =
    match track with
    | Short -> "간주 구간"
    | Standard -> "정규 편성"
    | Extended -> "심야 편성"
```

- 시그니처가 `Choice<unit,unit,unit>` 이다. 케이스 세 개가 각각 값을 실지 않으므로 세 자리 모두 `unit` 이다. `Short`, `Standard`, `Extended` 는 판별 유니온의 케이스가 아니라 이 액티브 패턴이 정의한 이름이다.
- 이 케이스 식별자는 패턴 자리에서만 쓸 수 있다. 정의 밖에서 `let x = Short` 처럼 값으로 쓰려 하면 오류 FS0039 가 난다. 바나나 클립재로 적어 `(|Short|Standard|Extended|) t` 로 호출하는 것은 되지만, 케이스 식별자 하나만 값으로 꺼내 쓸 수는 없다.
- 다만 그 패턴 자리 안에서는 판별 유니온 케이스와 이름 공간을 함께 쓴다. `Short` 케이스가 있는 판별 유니온이 이미 열려 있으면 뒤에 정의한 액티브 패턴이 그것을 가려서, 원래 판별 유니온을 매칭하던 곳에서 타입 불일치 오류 FS0001 이 난다. 정의 순서를 뒤집어도 마찬가지다. 이름을 겹치지 않게 짓는 편이 안전하다.
- 본문의 마지막 `else Extended` 가 남은 입력 전부를 받는다. 어느 케이스도 반환하지 않는 경로를 남기면 정의 본문의 `match` 식에서 경고 FS0025 가 나고, 실제로 그 경로를 타는 입력이 들어오면 실행 시점에 `MatchFailureException` 을 던진다. 컴파일이 막아 주지 않으므로 정의 안에서 모든 입력을 처리해 줘야 한다.
- 출력으로 쓰지 않는 케이스를 이름에 넣으면 그 케이스의 타입을 추론할 근거가 없어 정의 자리에서 오류 FS1210 이 난다. `: Choice<unit,unit,unit>` 처럼 반환 타입 주석을 달면 통과하지만, 그때는 쓰는 쪽에서 영원히 나오지 않는 케이스까지 적어야 한다. 실제로 반환하는 케이스만 이름에 적는 것이 맞다.

케이스가 값을 실을 수도 있다. 원서에는 없는 형태이지만 쓸 곳이 있다. 같은 입력을 서로 다른 단위로 꺼내 주는 식이다.

```
// Track -> Choice<int,float> - 케이스마다 다른 타입의 값을 실을 수 있다
let (|Seconds|Minutes|) (track: Track) =
    if track.Seconds < 60 then Seconds track.Seconds
    else Minutes (float track.Seconds / 60.0)

// Track -> string
let runtime track =
    match track with
    | Seconds s -> $"%d{s}초"
    | Minutes m -> $"%.1f{m}분"

let playlist =
    [ { Title = "새벽 인트로"; Seconds = 48 }
      { Title = "빗소리"; Seconds = 132 }
      { Title = "네 번째 정류장"; Seconds = 305 }
      { Title = "긴 배웅"; Seconds = 610 } ]

playlist |> List.iter (fun t -> printfn "%s: %s / %s" t.Title (runtime t) (slot t))
// 새벽 인트로: 48초 / 간주 구간
// 빗소리: 2.2분 / 간주 구간
// 네 번째 정류장: 5.1분 / 정규 편성
// 긴 배웅: 10.2분 / 심야 편성
```

- 케이스 식별자 `Seconds` 는 `Track.Seconds` 필드와 이름이 같지만 가려지지 않는다. 레코드 필드와 케이스 식별자는 서로 다른 이름 공간이다. 앞에서 말한 주의는 판별 유니온 케이스와 겹칠 때의 이야기다.

케이스 일곱 개까지가 상한이라는 것도 확인해 둔다. 요일은 정확히 일곱 개라 상한에 딱 맞는다.

```
open System

// DayOfWeek -> Choice<unit,unit,unit,unit,unit,unit,unit> - 상한인 7개
let (|Mon|Tue|Wed|Thu|Fri|Sat|Sun|) (d: DayOfWeek) =
    match d with
    | DayOfWeek.Monday -> Mon
    | DayOfWeek.Tuesday -> Tue
    | DayOfWeek.Wednesday -> Wed
    | DayOfWeek.Thursday -> Thu
    | DayOfWeek.Friday -> Fri
    | DayOfWeek.Saturday -> Sat
    | _ -> Sun

// DayOfWeek -> string - 액티브 패턴의 케이스도 | 로 묶을 수 있다
let studioDay (d: DayOfWeek) =
    match d with
    | Sat | Sun -> "휴무"
    | Wed -> "생방송"
    | _ -> "녹음"

[ DayOfWeek.Wednesday; DayOfWeek.Sunday; DayOfWeek.Monday ]
|> List.iter (studioDay >> printfn "%s")
// 생방송
// 휴무
// 녹음
```

- 마지막 `| _ -> Sun` 은 `DayOfWeek.Sunday` 로 바꿔 적어도 된다. 다만 그렇게 하면 경고 FS0104 가 난다. 열거형은 선언된 이름 밖의 값도 담을 수 있어서, 일곱 이름을 다 적어도 컴파일러는 값 범위가 닫혔다고 보지 않는다. 판별 유니온이라면 이 문제가 없다.
- 쓰는 쪽에서는 반대로 케이스 일부만 적고 나머지를 `| _ ->` 로 묶어도 된다. 위의 `studioDay` 가 그런 예다.

```
// 오류 FS0265: 활성 패턴은 7개가 넘는 가능성을 반환할 수 없습니다
let (|A|B|C|D|E|F|G|H|) n =
    match n with
    | 1 -> A | 2 -> B | 3 -> C | 4 -> D
    | 5 -> E | 6 -> F | 7 -> G | _ -> H
```

## Single-Case Active Patterns — 단일 케이스 액티브 패턴 (원서 pp.97-98)

- 케이스가 하나뿐이고 실패하지 않는다. 입력을 다른 형태로 바꿔 보여 주는 용도다. 판정이 아니라 변환에 가깝다.
- 반환 타입은 `option` 도 `Choice` 도 아니고 변환 결과 타입 그대로다. 항상 성공하므로 감쌀 것이 없다.
- 같은 입력을 여러 각도에서 분해해 두면, 규칙을 요구사항 문장에 가깝게 적을 수 있다.

검색어 문자열을 다루는 예다. 규칙은 두 개다. 공백을 정리한 뒤 두 글자 이상이어야 하고, 낱말은 네 개까지만 받는다.



```
// 이 단위가 보여주는 것: 단일 케이스 액티브 패턴으로 입력을 여러 각도로 분해하기
open System

// string -> string
let (|Normalized|) (input: string) = input.Trim().ToLowerInvariant()

// string -> string list
let (|Tokens|) (input: string) =
    input.Split(' ', StringSplitOptions.RemoveEmptyEntries) |> List.ofArray

printfn "%s" (match " Active Pattern " with Normalized q -> q)
// 'active pattern'
printfn "%A" (match " Active Pattern " with Tokens ts -> ts)
// ["Active"; "Pattern"]
```

- 같은 문자열 하나를 두 방향으로 열어 보았다. 하나는 정리된 문자열, 하나는 낱말 리스트다. 이것이 단일 케이스 패턴의 쓸모다.
- 단일 케이스 패턴은 항상 성공하므로 케이스 하나만으로 `match` 식이 완결된다. `| _ -> 없이 |` `Normalized q -> ...` 한 줄로 끝난다.

두 규칙을 판정 하나로 묶는다. 결과를 튜플로 돌려줄 수도 있지만, 판별 유니온을 쓰면 실패 이유가 없는 경우에 빈 문자열을 채우는 군더더기가 사라진다. 원서도 실제 코드라면 판별 유니온이 낫다고 적어 두었다.

```
type Verdict =
    | Accepted of query: string
    | Rejected of reason: string

// string -> Verdict - 단일 케이스 패턴 안에서 다른 단일 케이스 패턴을 쓴다
let (|Checked|) (input: string) =
    match input with
    | Normalized q when q.Length < 2 -> Rejected "절의는 두 글자 이상이어야 한다"
    | Tokens ts when ts.Length > 4 -> Rejected "낱말은 네 개까지만 받는다"
    | Normalized q -> Accepted q
```

- 앞의 두 케이스는 가드 절이 붙어 실패할 수 있으므로, 가드 절이 없는 `| Normalized q ->` 케이스가 마지막에 있어야 빠짐없는 패턴 매칭이 된다.
- 규칙 판정 로직이 액티브 패턴 안에 들어가느냐 밖에 있느냐는 취향이다. `Normalized` 는 변환만 하고 길이 판정은 밖의 가드 절에서 한다. `Tokens` 도 같은 구조다. 판정까지 안에 넣으면 `(|ShortQuery|_|)` 같은 부분 액티브 패턴이 된다.

`Checked` 를 쓰는 쪽에서는 액티브 패턴이 만들어 준 값을 다시 패턴으로 분해한다.

```
// string -> Result<string, string>
let search input =
    match input with
    | Checked (Accepted q) -> Ok q
    | Checked (Rejected why) -> Error why

[ " Active Pattern "; " F "; "액티브 패턴 이 다섯 낱말 이다" ]
|> List.iter (fun q ->
    match search q with
    | Ok query -> printfn "검색 실행: '%s'" query
    | Error why -> printfn "거절: %s" why)
// 검색 실행: 'active pattern'
// 거절: 절의는 두 글자 이상이어야 한다
// 거절: 낱말은 네 개까지만 받는다
```

- `Checked (Accepted q)` 처럼 액티브 패턴 안에 패턴을 중첩할 수 있다. `Checked` 가 만들어 낸 `Verdict` 값에 다시 판별 유니온 패턴을 적용한 것이다.
- `Accepted / Rejected` 두 케이스가 `Verdict` 를 다 덮으므로 여기도 `| _ ->` 가 필요 없다.
- 단일 케이스 패턴이 돌려준 값에는 이름 바인딩만 걸 수 있는 것이 아니다. 리터럴이든 판별 유니온 케이스든 어떤 패턴이라도 그 자리에 적을 수 있다.
- 매개변수는 단일 케이스 패턴에도 붙일 수 있다. `let (|RoundedTo|) (digits: int) (x: float) = Math.Round(x, digits)` 는 `int -> float -> float` 이고, 패턴 자리에서 `RoundedTo 2 v` 로 쓴다.

## Using Active Patterns in a Practical Example — 실전 예제 (원서 pp.99-102)

- 원서는 축구 스코어 예측 게임의 점수 계산을 예로 삼아 네 종류를 한자리에 모은다. 이 노트는 같은 구성을 웹 서버 접근 로그의 위험도 채점으로 옮긴다.
- 요구사항은 세 가지다. 로그 한 줄을 레코드로 파싱하고, 상태 코드를 부류로 나누고, 응답 시간과 경로에 따라 위험 점수를 매긴다.
- 파싱은 부분 패턴, 상태 코드 분류는 다중 케이스 패턴, 임계값 비교는 매개변수 있는 부분 패턴, 경로 분해는 단일 케이스 패턴이 각각 맡는다.

먼저 로그 한 줄을 레코드로 바꾸는 부분 액티브 패턴이다. 형식이 맞지 않는 줄이 섞여 들어오므로 실패할 수 있고, 따라서 `option` 을 반환한다.

```
// 이 단위가 보여주는 것: 네 종류를 한 문제에 함께 쓰기
open System
open System.Text.RegularExpressions

type Request = { Verb: string; Path: string; Status: int; Ms: int }

// string -> Request option
let (|AccessLine|_|) (line: string) =
    let m = Regex.Match(line, @"^(GET|POST|PUT|DELETE) (\S+) (\d{3}) (\d+)ms$")
    if m.Success then
        Some { Verb = m.Groups[1].Value
              Path = m.Groups[2].Value
              Status = int m.Groups[3].Value
              Ms = int m.Groups[4].Value }
    else None

printfn "%A" ((|AccessLine|_|) "GET /api/tracks 200 84ms")
// Some { Verb = "GET"
//       Path = "/api/tracks"
//       Status = 200
//       Ms = 84 }
printfn "%A" ((|AccessLine|_|) "PATCH /api/tracks 200 30ms") // None
```

- 정규식으로 판정과 추출을 한 번에 하고, 성공했을 때만 값을 담아 보낸다. 다음 챕터에서 검증을 다룰 때 이 형태를 정규식 패턴 자체를 매개변수로 받는 꼴로 일반화해 다시 쓴다.
- 실패 가능성이 있는 파싱을 부분 액티브 패턴으로 감싸 두면, 쓰는 쪽에서 "파싱에 성공한 경우"와 "형식이 틀린 경우"를 `match` 식의 두 케이스로 나란히 적을 수 있다.

상태 코드는 반드시 넷 중 하나로 떨어진다. 실패가 없으므로 다중 케이스 패턴이다.

```
// Request -> Choice<unit,unit,unit,unit>
let (|Succeeded|Redirected|ClientError|ServerError|) req =
    if req.Status < 300 then Succeeded
    elif req.Status < 400 then Redirected
    elif req.Status < 500 then ClientError
    else ServerError

// Request -> int
let statusScore req =
    match req with
    | Succeeded -> 0
    | Redirected -> 1
    | ClientError -> 5
    | ServerError -> 40
```

임계값 비교는 기준을 쓰는 쪽에서 정하는 편이 낫다. 매개변수 있는 부분 액티브 패턴이다.

```
// int -> Request -> unit option
let (|SlowerThan|_|) limit req = if req.Ms > limit then Some () else None

// string -> Request -> unit option
let (|Under|_|) (prefix: string) req = if req.Path.StartsWith prefix then Some () else None

// Request -> int - 위에서 아래로 검사하므로 큰 임계값을 먼저 적는다
let latencyScore req =
    match req with
    | SlowerThan 1000 -> 20
    | SlowerThan 300 -> 5
    | _ -> 0

// Request -> int
let scopeScore req =
    match req with
    | Under "/admin" -> 10
    | Under "/api" -> 2
    | _ -> 0
```

- `SlowerThan 300` 을 먼저 적으면 1200밀리초짜리 요청도 5점을 받고 끝난다. 임계값이 겹치는 패턴을 나열할 때는 좁은 조건이 위에 와야 한다.
- 같은 판정을 기준값만 달리해 두 번 쓰는 것이 매개변수의 값어치다. `(|Slow|_|)` 와 `(|VerySlow|_|)` 를 따로 정의하지 않아도 된다.

경로를 세그먼트 리스트로 열어 주는 단일 케이스 액티브 패턴을 더하면 영역 이름을 뽑을 수 있다.

```
// Request -> string list
let (|Segments|) req =
    req.Path.Split('/', StringSplitOptions.RemoveEmptyEntries) |> List.ofArray

// Request -> string
let area req =
    match req with
    | Segments [] -> "root"
    | Segments (top :: _) -> top
```

- 단일 케이스 패턴이 리스트를 돌려주므로 리스트 패턴을 그대로 이어 쓸 수 있다. 빈 리스트와 `머리 :: 꼬리` 가 리스트 전부를 덮으므로 와일드카드가 필요 없다.

점수 규칙 세 개는 시그니처가 모두 `Request -> int` 로 같다. 같은 시그니처의 함수들은 리스트에 담아 한 번에 합산할 수 있다. 원서가 마지막에 쓰는 정리 기법이 이것이다.

```
// Request -> int - 함수도 리스트에 담을 수 있다. List.sumBy 는 List.map 뒤 List.sum 과 같다
let risk req =
  [ statusScore; latencyScore; scopeScore ]
  |> List.sumBy (fun rule -> rule req)

// int -> string
let alarm score =
  match score with
  | s when s >= 40 -> "긴급"
  | s when s >= 10 -> "주의"
  | _ -> "정상"
```

- 규칙을 추가할 때 `risk` 를 고치는 대신 리스트에 함수 이름 하나를 더 적으면 된다. 규칙 목록을 매개변수로 빼면 채점 정책을 호출하는 쪽에서 알아 끼울 수도 있다.
- `alarm` 은 가드 절만 쓴다. 액티브 패턴으로 만들 수도 있지만, 한 곳에서만 쓰는 구간 판정은 가드 절이 더 짧다. 액티브 패턴은 재사용할 판정에 쓸 때 값어치가 나온다.

전체를 이어 붙여 로그를 훑는다.

```
let lines =
  [ "GET /api/tracks 200 84ms"
    "POST /admin/users 500 1240ms"
    "GET /admin/sessions 403 120ms"
    "GET / 200 12ms"
    "PATCH /api/tracks 200 30ms" ]

lines
|> List.iter (fun line ->
  match line with
  | AccessLine req ->
    let score = risk req
    printfn "%-6s %-4d %-16s %s" (area req) score req.Path (alarm score)
  | _ -> printfn "형식 불일치: %s" line)
// api    2    /api/tracks    정상
// admin  70   /admin/users   긴급
// admin  15   /admin/sessions 주의
// root   0    /              정상
// 형식 불일치: PATCH /api/tracks 200 30ms
```

- 점수 계산은 이렇게 된다. `/admin/users 500 1240ms` 는 서버 오류 40 + 1초 초과 20 + 관리 영역 10 으로 70점, `/admin/sessions 403 120ms` 는 5 + 0 + 10 으로 15점이다.
- 네 종류가 각자 맡은 자리가 분명하다. 실패할 수 있는 변환은 부분 패턴, 기준값이 필요한 판정은 매개변수 있는 부분 패턴, 반드시 하나로 떨어지는 분류는 다중 케이스 패턴, 모양만 바꾸는 분해는 단일 케이스 패턴이다. 이 대응만 잡아 두면 어떤 종류를 쓸지 고민할 일이 없다.

## Summary — 원서의 챕터 요약 (원서 p.102)

- 원서는 액티브 패턴이 유용하고 강력하지만 늘 최선의 선택은 아니라는 말로 챕터를 맺는다. 잘 쓰면 가독성이 올라간다는 조건부 권장이다.
- 이 챕터에서 다루지 않은 종류가 더 있고, 공식 문서를 찾아보라고 안내한다.
- 다음 챕터에서는 여기서 익힌 기능을 6챕터 코드에 얹어 검증을 붙인다.

## 정리 — 이 노트의 요약

- 액티브 패턴은 이름을 `(| |)` 로 감싼 함수다. 시그니처는 같은 본문의 일반 함수와 똑같고, 달라지는 것은 그 이름을 패턴 자리에서 쓸 수 있다는 점뿐이다. 그래서 `List.choose` 같은 고차 함수에 그대로 넘길 수도 있다.
- 이름에 `_|` 가 있으면 실패할 수 있다는 뜻이고 반환 타입이 `option` 이다(F# 9 부터는 `bool` , `[<return: Struct>]` 를 붙이면 `voption` 이다). 없으면 반드시 성공하며 값이나 `Choice` 를 그대로 반환한다. 이 한 가지 규칙이 네 종류의 반환 타입을 모두 설명한다.
- 종류별 실측 시그니처는 다음과 같다. 부분 패턴은 `string -> int option` , 매개변수 있는 부분 패턴은 `float -> Trail -> unit option` , 다중 케이스 패턴은 `Track -> Choice<unit,unit,unit>` , 단일 케이스 패턴은 `string -> string list` 다.
- 매개변수는 부분 패턴과 단일 케이스 패턴에만 붙는다. 다중 케이스 패턴에 붙이면 정의는 통과하지만 쓰는 순간 오류 FS0722 가 난다.
- 다중 케이스 패턴은 케이스 일곱 개가 상한(오류 FS0265)이고, 부분 패턴과 다중 케이스 패턴을 섞은 `(|A|B|_|)` 형태는 존재하지 않는다(오류 FS3872).
- 케이스 식별자는 대문자로 시작해야 한다(오류 FS0623). 이 식별자는 패턴 자리에서만 쓸 수 있고, 그 자리 안에서는 판별 유니온 케이스와 이름 공간을 함께 쓴다. 같은 이름의 판별 유니온 케이스가 이미 스코프에 있으면 나중 정의가 앞의 것을 가려 타입 불일치 오류 FS0001 이 난다.
- 패턴 조합 연산자는 `&` 와 `|` 둘뿐이고 부정이 없다. 부정 조건이 필요하면 반대 판정을 하나 더 정의하거나 가드 절로 옮긴다. 가드 절에서는 일반 논리 연산자를 그대로 쓴다.
- 부분 액티브 패턴은 F# 9 부터 `bool` 을 반환해도 된다. `[<return: Struct>]` 로 `voption` 을 반환해 힙 할당을 피하는 것은 F# 6 부터다. 둘 다 반환 표현만 바뀌는 것이고 종류가 늘지는 않는다. 원서 시점의 `Some` `()` 형태도 그대로 유효하다.
- 액티브 패턴을 쓸지 판단하는 기준은 재사용이다. 여러 `match` 식에서 되풀이되는 판정이면 액티브 패턴으로 뽑고, 한 곳에서만 쓰는 구간 판정이면 가드 절이 짧다.

## 원서 대조 표

| 절                                                         | 원서 페이지    | 실행 단위        |
|-----------------------------------------------------------|-----------|--------------|
| Setting Up — 준비                                           | p.90      | —            |
| 네 종류를 먼저 구분한다                                             | pp.90-98  | —            |
| Partial Active Patterns — 부분 액티브 패턴                       | pp.90-92  | 07-catalog   |
| Parameterized Partial Active Patterns — 매개변수 있는 부분 액티브 패턴 | pp.92-96  | 07-trail     |
| Multi-Case Active Patterns — 다중 케이스 액티브 패턴                | pp.96-97  | 07-playlist  |
| Single-Case Active Patterns — 단일 케이스 액티브 패턴               | pp.97-98  | 07-query     |
| Using Active Patterns in a Practical Example — 실전 예제      | pp.99-102 | 07-accesslog |
| Summary — 원서의 챕터 요약                                       | p.102     | —            |

## 08 - 함수형 검증 (원서 pp.103-117)

6챕터가 만든 파이프라인은 파일을 읽어 레코드로 바꾸는 데까지 갔지만 모든 칸이 `string` 인 상태로 멈췄다. 낱자 칸에 무엇이 적혀 있어도 통과하고, 필수 칸이 비어도 알 수 없다. 이 챕터는 그 파이프라인에 검증(validation)을 끼워 넣는다. 새로 배울 도구는 거의 없다. 7챕터의 부분 액티브 패턴(partial active pattern)으로 문자열을 해석하고, 3챕터의 `Result` 로 실패를 값으로 돌려주고, 5챕터의 `List` 함수로 오류를 모은다. 요점은 마지막에 나온다. 검증은 한 칸이 틀려도 나머지 칸을 계속 검사해야 하는 작업인데, `Result.bind` 로 이으면 첫 오류에서 멈춘다. 첫 오류에서 멈추는 방식과 오류를 모으는 방식의 차이, 그리고 그 차이를 문법으로 감싼 F# 5 의 `and!` 를 이해하는 것이 이 챕터의 목표다.

시작하기 전에 이름이 짧아 헛갈리는 낱말 둘을 갈라 둔다.

- 부분 액티브 패턴은 이름 마지막 자리에 와일드카드가 붙은 `(|Name|_)` 꼴이고 `option` 을 반환한다. 실패할 수 있는 판정과 파싱에 쓴다. 이 노트는 이후 "부분 패턴"으로 줄여 쓴다.
- 부분 함수(partial function)는 가능한 입력 전부에서 값을 돌려주지 못하고 그런 입력에 예외를 던지는 함수다. 이름이 짧은 부분 적용(partial application)과는 관계가 없다. 이 챕터에서도 부분 함수가 하나 등장하는데, 그것을 쓰지 않아도 되게 만드는 과정이 챕터 후반이다.
- 두 낱말을 가르는 것은 뒤에 붙는 "패턴"과 "함수"다. 앞의 낱말만 떼어 놓으면 어느 쪽을 말하는지 알 수 없으므로 이 노트는 그렇게 줄이지 않는다.

소재는 실험실 시료 접수 명세다. 칸은 다섯 개이고 구분자는 `|` 다. `SampleId` 는 반드시 있어야 하고, `ContactEmail` 은 비어도 되지만 적혀 있으면 주소 모양이어야 한다. `Chilled` 는 참거짓이며 `Y` 나 `N` 중 하나로 적어야 하고 빈 칸을 허용하지 않는다. `CollectedOn` 은 날짜, `VolumeMl` 은 수량이고 이 두 칸은 비어 있어도 된다.

### Setting Up — 검증을 붙일 자리 (원서 pp.103-105)

- 원서는 콘솔 프로젝트를 만들고 `resources/customers.csv` 를 둔 뒤 6챕터 마지막 코드를 그대로 붙여 넣는 것으로 시작한다. 이 챕터에서 새로 만드는 것은 없고, 이미 있는 파이프라인에 단계 하나를 끼우는 것이 전부다.
- 이 노트는 파일을 만들지 않는다. 6챕터가 데이터 출처를 `DataReader` 라는 함수 타입 약어로 빼 두었으므로, 같은 시그니처의 함수를 하나 만들면 파일 없이 같은 파이프라인을 돌릴 수 있다. 그 설계가 여기서 제값을 한다.
- 출발점의 특징은 하나다. `RawSample` 의 모든 필드가 `string` 이다. 파싱은 칸 개수만 확인하고 내용은 손대지 않는다.

```
// 이 단위가 보여주는 것: 검증이 없는 6행터 파이프라인의 출발 상태
open System

// 모든 칸이 string 이다. 이 챕터가 고칠 지점이 여기다
type RawSample = {
    SampleId: string
    ContactEmail: string
    Chilled: string
    CollectedOn: string
    VolumeMl: string
}

// 6행터가 정한 함수 타입 약어
type DataReader = string -> Result<string seq, exn>
```

데이터 출처는 파일이 아니어도 된다. `DataReader` 는 타입 약어일 뿐이므로 시그니처가 같은 함수는 무엇이든 그 자리에 들어간다.

```
let rows =
    [ "SampleId|ContactEmail|Chilled|CollectedOn|VolumeMl"
      "S-1041|choi@lab.example|Y|2024-03-02|12.5"
      "S-1042||N|2024-03-04|8"
      "S-1043|park.at.lab.example|Y|2024-03-05|4.25"
      "S-1044|yun@lab.example|maybe|2024-03-06|"
      "S-1045|seo@lab.example|N||3.0"
      "||Y|2024-13-45|- " ]

// FSI 실측: source: string -> Result<string seq,exn>
// : DataReader 로 적었지만 FSI 는 함수 바인딩의 타입을 화살표 꼴로 풀어 보여 준다
// 아래 import 의 reader 처럼 매개변수에 붙인 타입 약어는 이름이 그대로 남는다
let memoryReader : DataReader =
    fun source ->
        if source = "intake-2024-03" then rows |> Seq.ofList |> Ok
        else Error (exn $"알 수 없는 데이터 출처: %{source}")
```

- 검증이 붙으면 걸릴 행은 셋이다. 3행은 주소에 `@` 가 없고, 4행은 `chilled` 칸이 `maybe` 이고, 6행은 `SampleId` 가 비어 있으면서 날짜와 수량도 읽을 수 없다. 검증이 없는 지금은 여섯 행 전부가 통과한다.
- 오류 하나만 들어 있는 행과 셋이 들어 있는 행을 함께 둔 것은 뒤에서 오류를 모으는 방식과 첫 오류에서 멈추는 방식을 구별하려는 것이다.

```
// FSI 실측: row: string -> RawSample option
let parseRow (row: string) : RawSample option =
    match row.Split('|') with
    | [| sampleId; email; chilled; collectedOn; volume |] ->
        Some { SampleId = sampleId
                ContactEmail = email
                Chilled = chilled
                CollectedOn = collectedOn
                VolumeMl = volume }
    | _ -> None

// FSI 실측: data: string seq -> RawSample seq
let parse (data: string seq) =
    data
    |> Seq.skip 1
    |> Seq.map parseRow
    |> Seq.choose id

// FSI 실측: data: RawSample seq -> unit
let output data =
    data
    |> Seq.iter (fun r ->
        printfn "%-7s %-20s %-6s %-11s %s"
            r.SampleId r.ContactEmail r.Chilled r.CollectectedOn r.VolumeMl)

// FSI 실측: reader: DataReader -> source: string -> unit
let import (reader: DataReader) source =
    match source |> reader with
    | Ok data -> data |> parse |> output
    | Error ex -> printfn "읽기 실패: %s" ex.Message

import memoryReader "intake-2024-03"
// S-1041 choi@lab.example Y 2024-03-02 12.5
// S-1042 N 2024-03-04 8
// S-1043 park.at.lab.example Y 2024-03-05 4.25
// S-1044 yun@lab.example maybe 2024-03-06
// S-1045 seo@lab.example N 3.0
// Y 2024-13-45 -
```

- 출력을 보면 문제가 눈에 보인다. maybe 도, 2024-13-45 도, 빈 SampleId 도 아무 저항 없이 지나간다. 파싱은 칸이 다섯 개인지만 확인했다.
- parse 의 시그니처가 string seq -> RawSample seq 라는 점을 기억해 두면 좋다. 이 챕터가 끝날 때 이 시그니처가 어떻게 바뀌는지가 작업의 결과다.

## Solving the Problem — 검증된 값을 담은 타입 (원서 p.105)

- 검증은 "확인"이 아니라 "변환"으로 생각하는 편이 낫다. 확인만 하고 원래 값을 그대로 쓰면 확인했다는 사실 이 타입에 남지 않는다.
- 그래서 원서는 검증을 통과한 값만 담은 레코드를 따로 만든다. 문자열 칸이 각자의 타입으로 바뀌고, 비어도 되는 칸은 option 이 된다.
- 비어도 되는 칸을 빈 문자열로 두지 않고 option 으로 옮기는 것이 요점이다. "값이 없음"과 "빈 문자열"을 구별할 수 있게 된다.



```
// 이 단위가 보여주는 것: 원서가 이 챕터에서 완성하는 검증 파이프라인 전체
open System
open System.Globalization
open System.Text.RegularExpressions

type RawSample = {
    SampleId: string
    ContactEmail: string
    Chilled: string
    CollectedOn: string
    VolumeMl: string
}

// 검증을 통과한 값만 들어온다. 칸마다 제 타입이 있고, 없어도 되는 칸은 Option 이다
type ValidatedSample = {
    SampleId: string
    ContactEmail: string option
    Chilled: bool
    CollectedOn: DateTime option
    VolumeMl: decimal option
}
```

- `SampleId` 만 `string` 으로 남았다. 반드시 있어야 하는 칸이므로 `Option` 이 필요 없다. 9챕터에서는 이 `string` 마저 도메인 타입으로 감싸는 방법을 다룬다.
- `RawSample` 과 `ValidatedSample` 을 나란히 두면 검증이 무엇을 하는 일인지가 타입만 봐도 읽힌다. 왼쪽은 파일에서 방금 읽은 모양, 오른쪽은 프로그램이 믿고 쓸 수 있는 모양이다.

레코드를 만드는 함수를 따로 둔다. 뒤에서 이 함수를 부분 적용해 가며 조립하기 때문이다.

```
// FSI 실측: sampleId: string -> email: string option -> chilled: bool
//          -> collectedOn: System.DateTime option -> volume: decimal option -> ValidatedSample
let create sampleId email chilled collectedOn volume =
    { SampleId = sampleId
      ContactEmail = email
      Chilled = chilled
      CollectedOn = collectedOn
      VolumeMl = volume }
```

- 레코드 식으로 직접 만들 수도 있는데 함수를 따로 두는 이유는 시그니처다. 매개변수를 하나씩 받는 커링된 형태여야 뒤에서 인자를 하나씩 먹여 가며 조립할 수 있다.
- 필드 순서와 매개변수 순서를 맞춰 두어야 한다. 같은 타입의 칸이 이웃해 있으면 순서를 바꿔 넣어도 컴파일 되므로, 이 함수를 쓰는 자리에서 실수하기 쉽다. 9챕터가 이 위험을 줄이는 방법을 다룬다.

## 오류를 판별 유니온으로 (원서 p.106)

- 실패 이유를 문자열로 두면 쓰는 쪽에서 문자열을 파싱해야 한다. 판별 유니온으로 두면 종류가 타입에 적히고 `match` 식에서 빠뜨린 케이스를 컴파일러가 잡아 준다. 3챕터에서 확립한 방식이다.
- 이 챕터에서 예상되는 실패는 두 가지다. 값이 없는 것과 값을 읽을 수 없는 것이다.
- 값이 없을 때는 어느 칸인지만 알면 되고, 읽을 수 없을 때는 어느 칸에 무엇이 적혀 있었는지 함께 알려 주는 편이 낫다. 그래서 케이스 데이터의 모양이 다르다.

```
type ValidationError =
    | MissingField of name: string
    | BadFormat of name: string * value: string
```

- 케이스 데이터에 이름( `name` , `value` )을 붙여 두면 읽는 쪽에서 무엇이 어느 자리인지 헷갈리지 않는다. 두 케이스 모두 첫 자리가 칸 이름이라는 규칙을 지켜 두면 오류를 사람이 읽을 문장으로 바꾸기도 쉽다.
- 실패 타입을 하나로 통일해 두는 것이 중요하다. 3챕터에서 봤듯이 `Result.bind` 로 이으려면 실패 타입이 같아야 하고, 다르면 `Result.mapError` 로 맞춰야 한다. 검증 함수를 처음부터 같은 실패 타입으로 만들어 두면 그 수고가 없어진다.

## 파싱을 부분 패턴으로 (원서 p.106)

- 문자열을 날짜나 수량으로 읽는 일은 실패할 수 있는 변환이다. 7챕터의 분류대로 부분 패턴이 맡을 자리다.
- `TryParse` 계열 메서드는 `bool * 'a` 튜플을 돌려준다. 그 튜플을 `option` 으로 바꾸고 이름을 바나나 클립으로 감싸면 그 판정을 `match` 식의 케이스 자리에서 쓸 수 있다.
- 정규식은 매개변수 있는 부분 액티브 패턴(parameterized partial active pattern)으로 한 번만 감싸 두고 패턴 문자열만 갈아 끼우는 편이 낫다. 7챕터에서 로그 한 줄을 파싱하던 패턴을 정규식 자체를 매개변수로 받는 꼴로 일반화한 것이다.

```
// 이 단위가 보여주는 것: 문자열 해석을 부분 패턴으로 옮기기
open System
open System.Globalization
open System.Text.RegularExpressions

// FSI 실측: pattern: string -> input: string -> string list option
// 검사할 값이 마지막 매개변수여야 한다는 규칙을 지켰다
let (|Captures|_|) (pattern: string) (input: string) =
    let m = Regex.Match(input, pattern)
    if m.Success then
        m.Groups |> Seq.skip 1 |> Seq.map (fun g -> g.Value) |> List.ofSeq |> Some
    else None

printfn "%A" ("S-1041" |> (|Captures|_|) @"^S-(\d+)$") // Some ["1041"]
printfn "%A" ("X-1041" |> (|Captures|_|) @"^S-(\d+)$") // None
```

- `m.Groups` 의 0번은 매칭된 전체 문자열이고 1번부터가 캡처 그룹(capture group)이다. `Seq.skip 1` 로 0번을 버려 캡처 그룹만 리스트로 돌려준다.
- 바나나 클립재로 적으면 그냥 함수라서 패턴 자리 밖에서도 호출할 수 있다. 7챕터에서 확인한 성질이다.
- 6챕터에서 봤듯이 `Seq.skip` 은 원소가 모자라면 예외를 던진다. 여기서는 매칭이 성공했을 때만 이 줄에 닿고 그때는 0번 그룹이 반드시 있으므로 안전하다.

캡처 그룹 개수를 패턴 자리에 조건으로 적어 둘 수 있다는 것이 이 형태의 이점이다. 주소 판정은 캡처 그룹이 정확히 하나 잡힐 때만 성립하게 적는다.

```
// FSI 실측: input: string -> string option
// 그룹 하나만 잡히는 경우로 한정하고, 잡힌 도메인을 돌려준다
let (|EmailLike|_|) input =
    match input with
    | Captures @"^([^\s]+)@([^\s]+\.[^\s]+)$" [ domain ] -> Some domain
    | _ -> None

// FSI 실측: input: string -> unit option
let (|NoValue|_|) (input: string) =
    if input.Trim() = "" then Some () else None

// FSI 실측: input: string -> bool option
let (|Flag|_|) (input: string) =
    match input.Trim().ToUpperInvariant() with
    | "Y" | "YES" -> Some true
    | "N" | "NO" -> Some false
    | _ -> None
```

- `[ domain ]` 은 리스트가 원소 하나인 경우만 받는 리스트 패턴이다. 부분 패턴이 돌려준 값에 다시 패턴을 적용한 것이며, 캡처 그룹이 정확히 하나일 때만 성립한다는 조건이 패턴에 적혀 있는 셈이다. 다만 정규식만 고쳐 그룹 개수를 바꾸면 컴파일러는 오류도 경고도 내지 않는다. 그때는 이 케이스가 성립하지 않아 `(|EmailLike|_|)` 가 언제나 `None` 을 돌려주므로, 정규식과 리스트 패턴은 함께 고쳐야 한다.
- `(|NoValue|_|)` 는 담아 보낼 값이 없어 `Some ()` 을 쓴다. 시그니처가 `unit option` 이 되고 패턴 자리에는 이름만 적는다.
- `(|Flag|_|)` 는 원서의 `1 / 0` 대신 `Y / N` 을 받는다. 어떤 표기를 참거짓으로 받아들일지는 도메인이 정하는 것이고, 그 규칙이 패턴 하나 안에 모여 있다는 점이 중요하다.

숫자와 날짜는 `TryParse` 를 감싼다. 문화권을 명시해 두면 실행 환경이 달라도 결과가 같다.

```
// FSI 실측: input: string -> decimal option
let (|AsDecimal|_|) (input: string) =
    match Decimal.TryParse(input, NumberStyles.Number, CultureInfo.InvariantCulture) with
    | true, value -> Some value
    | _ -> None

// FSI 실측: input: string -> System.DateTime option
// TryParseExact 로 형식을 고정하면 2024-13-45 같은 값이 확실히 걸러진다
let (|AsDate|_|) (input: string) =
    match DateTime.TryParseExact(input, "yyyy-MM-dd", CultureInfo.InvariantCulture, DateTimeStyles.None) with
    | true, value -> Some value
    | _ -> None
```

- 원서는 `Decimal.TryParse input` 과 `DateTime.TryParse` 를 그대로 쓴다. 현재 스레드의 문화권을 따르므로 판정만 갈리는 것이 아니라 값이 조용히 어긋난다. `"12.5"` 는 고정 문화권 (`CultureInfo.InvariantCulture`)에서 12.5로 읽히지만 `de-DE`에서는 `.` 이 천 단위 구분 기호라 125로 통과한다. `ko-KR` 은 고정 문화권과 결과가 같아 이 차이가 눈에 띄지 않고, 그래서 더 위험하다. 6챕터도 같은 이유로 문화권을 명시했다.
- `TryParse` 와 `TryParseExact` 의 차이도 크다. `TryParse` 는 여러 형식을 관대하게 받아들인다. `"03/04/2024"` 는 `en-US` 에서 3월 4일, `de-DE` 에서 4월 3일로 통과한다. 형식이 정해진 입력이라면 `TryParseExact` 로 못 박는 편이 검증에 맞다.

만들어 둔 패턴을 한 `match` 식에 넣어놓으면 각자 무엇을 잡아내는지 한눈에 보인다.

```
// FSI 실측: input: string -> string
let describe (input: string) =
    match input with
    | NoValue -> "빈 칸"
    | Flag value -> $"참거짓 %{value}"
    | AsDecimal value -> $"수량 %M{value}"
    | AsDate value -> $"날짜 %s{value.ToString("yyyy-MM-dd", CultureInfo.InvariantCulture)}"
    | EmailLike domain -> $"주소 도메인 %s{domain}"
    | other -> $"해석 불가 '%s{other}'"

[ " "; "Y"; "no"; "4.25"; "2024-03-05"; "choi@lab.example"; "2024-13-45"; "-" ]
|> List.iter (describe >> printfn "%s")
// 빈 칸
// 참거짓 true
// 참거짓 false
// 수량 4.25
// 날짜 2024-03-05
// 주소 도메인 lab.example
// 해석 불가 '2024-13-45'
// 해석 불가 '-'
```

- 이 다섯 패턴은 서로 겹치지 않으므로 여기서는 케이스 순서를 바꿔도 결과가 같다. 다만 겹치는 패턴이 있으면 위에 적은 케이스가 먼저 걸린다. 앞에서 말한 원서의 규칙처럼 `Flag` 가 `1 / 0` 을 참거짓으로 받으면 `AsDecimal` 과 겹치고, 그때는 둘의 순서가 판정을 가른다. 겹칠 수 있는 자리에서는 좁은 판정을 위에 둔다.
- 마지막 `| other ->` 가 필요하다. 케이스에 쓴 것이 모두 부분 패턴이므로 컴파일러는 이 `match` 식이 빠짐 없다고 판단하지 않는다. 빼면 경고 FS0025 가 난다.

## 필드별 검증 함수 (원서 p.107)

- 검증 함수의 목표 시그니처는 `string -> Result<'a, ValidationError>` 다. `'a` 자리에는 칸마다 다른 검증된 타입이 온다. 문자열을 받아 제 타입의 값을 돌려주거나 왜 안 되는지 알려 준다.
- 함수마다 실패 타입이 `ValidationError` 하나로 같다. 이렇게 맞춰 두면 뒤에서 오류를 한 리스트에 모을 수 있다.
- 비어도 되는 칸과 그렇지 않은 칸의 차이가 반환 타입에 나타난다. 앞의 것은 `Result<'a option, _>`, 뒤의 것은 `Result<'a, _>` 다.

```
// 앞 단위에서 만든 패턴 여섯 개를 그대로 다시 둔다
let (|Captures|_|) (pattern: string) (input: string) =
    let m = Regex.Match(input, pattern)
    if m.Success then
        m.Groups |> Seq.skip 1 |> Seq.map (fun g -> g.Value) |> List.ofSeq |> Some
    else None

let (|EmailLike|_|) input =
    match input with
    | Captures @"^([^\s]+)@([^\s]+\.[^\s]+)$" [ domain ] -> Some domain
    | _ -> None

let (|NoValue|_|) (input: string) = if input.Trim() = "" then Some () else None

let (|Flag|_|) (input: string) =
    match input.Trim().ToUpperInvariant() with
    | "Y" | "YES" -> Some true
    | "N" | "NO" -> Some false
    | _ -> None

let (|AsDecimal|_|) (input: string) =
    match Decimal.TryParse(input, NumberStyles.Number, CultureInfo.InvariantCulture) with
    | true, value -> Some value
    | _ -> None

let (|AsDate|_|) (input: string) =
    match DateTime.TryParseExact(input, "yyyy-MM-dd", CultureInfo.InvariantCulture, DateTimeStyles.None) with
    | true, value -> Some value
    | _ -> None
```

패턴이 준비되었으니 검증 함수는 짧게 적힌다. 판정 규칙이 패턴 이름에 들어 있어 함수 본문은 어느 오류를 낼지만 정한다.

```
// FSI 실측: sampleId: string -> Result<string,ValidationError>
let validateSampleId sampleId =
    if sampleId <> "" then Ok sampleId else Error (MissingField "SampleId")

// FSI 실측: email: string -> Result<string option,ValidationError>
// 비어 있으면 Ok None, 모양이 맞으면 Ok (Some ...), 그 밖은 오류다
let validateEmail email =
    match email with
    | NoValue -> Ok None
    | EmailLike _ -> Ok (Some email)
    | _ -> Error (BadFormat ("ContactEmail", email))

// FSI 실측: chilled: string -> Result<bool,ValidationError>
let validateChilled chilled =
    match chilled with
    | Flag value -> Ok value
    | _ -> Error (BadFormat ("Chilled", chilled))

// FSI 실측: collectedOn: string -> Result<System.DateTime option,ValidationError>
let validateCollectedOn collectedOn =
    match collectedOn with
    | NoValue -> Ok None
    | AsDate value -> Ok (Some value)
    | _ -> Error (BadFormat ("CollectedOn", collectedOn))

// FSI 실측: volume: string -> Result<decimal option,ValidationError>
// 가드 절을 붙여 형식과 값 범위를 함께 검사한다
let validateVolume volume =
    match volume with
    | NoValue -> Ok None
    | AsDecimal value when value > 0m -> Ok (Some value)
    | _ -> Error (BadFormat ("VolumeML", volume))
```

- `validateEmail` 의 `EmailLike` \_ 는 패턴이 돌려준 도메인을 버린다. 여기서서는 모양이 맞지만 필요하다. 도메인을 쓸 곳이 생기면 \_ 자리에 이름을 넣으면 된다.
- `validateVolume` 은 형식 검사와 값 범위 검사를 한 케이스에 붙였다. `AsDecimal` 로 읽히더라도 0 이하면 이 케이스가 성립하지 않아 마지막 줄로 떨어진다. 부분 패턴과 가드 절을 함께 쓰면 이런 이중 조건이 한 줄에 들어간다.
- 다섯 함수의 반환 타입이 저마다 다르지만 실패 타입은 전부 `ValidationError` 다. 이 통일이 다음 절의 조립을 가능하게 한다.

## create 가 Result 를 받지 못한다 (원서 p.108)

- 검증 함수와 `create` 를 바로 이으면 컴파일되지 않는다. `create` 는 `string`, `bool`, `decimal option` 을 기다리는데 검증 함수가 주는 것은 그것들을 `Result` 로 감싼 값이다.
- 원서는 아래 `validateSample` 에 반환 타입 주석을 먼저 달아 둔다. 목표 타입을 못 박아 두면 반환 타입을 바꿔 컴파일만 통과시키는 길이 막히고, 어긋난 자리를 실제로 고쳐야 한다.
- 3챕터에서 본 구조와 같다. `Result` 를 내는 함수와 `Result` 를 모르는 함수를 이으려면 사이에 무언가를 끼워야 한다.

```
// 컴파일되지 않는다
let validateSample (raw: RawSample) : Result<ValidatedSample, ValidationError list> =
  let sampleId = raw.SampleId |> validateSampleId
  let email = raw.ContactEmail |> validateEmail
  let chilled = raw.Chilled |> validateChilled
  let collectedOn = raw.CollectectedOn |> validateCollectedOn
  let volume = raw.VolumeMl |> validateVolume
  create sampleId email chilled collectedOn volume
// error FS0001: 이 식에는
// 'Result<ValidatedSample,ValidationError list>' 형식이 필요하지만
// 여기에서는
// 'ValidatedSample' 형식이 지정되었습니다.
```

- 반환 타입에 `ValidationError list` 를 적은 것도 의도된 것이다. 검증은 오류 하나만 알려 주면 충분하지 않다. 다섯 칸이 다 틀렸으면 다섯 개를 다 알려 줘야 접수 창구에서 한 번에 고칠 수 있다.
- 반환 타입 주석을 지우면 오류 메시지가 달라진다. 컴파일러가 `create` 의 첫 인자 자리를 짚어 `'string'` 형식이 필요하지만 `'Result<string,ValidationError>'` 형식이 지정되었습니다 라고 말한다. 주석이 있으면 식 전체를 짚고, 없으면 인자 자리를 짚는다.
- 3챕터의 `Result.bind` 로 다섯 함수를 이으면 컴파일은 되지만 실패 타입이 `ValidationError` 하나로 남는다. 첫 오류에서 `Error` 선로로 갈아타고 나머지 검증은 아예 실행되지 않는다. 이 방식과의 차이는 뒤에서 코드로 비교한다.

## 오류를 모아 한 번에 돌려주기 (원서 p.109)

- 원서는 먼저 지금까지 배운 것만으로 해법을 만든다. 검증 결과에서 오류만 뽑는 함수와 값만 뽑는 함수를 두고, 오류를 전부 모아 비어 있는지 확인한다.
- 오류가 없다고 확인한 뒤에 값을 꺼내므로 순서가 안전을 보장한다. 대신 그 보장이 타입에 적혀 있지 않다는 것이 이 해법의 약점이다.
- 5챕터의 `List.concat` 이 리스트의 리스트를 한 겹 벗겨 준다. 오류가 없는 검증은 빈 리스트를 내므로 자연히 사라진다.

```
// FSI 실측: result: Result<'a','b> -> 'b list
// 오류가 없으면 빈 리스트다. List.concat 에서 저절로 없어진다
let errorsOf result =
    match result with
    | Ok _ -> []
    | Error e -> [ e ]

// FSI 실측: result: Result<'a','b> -> 'a
// 부분 함수다. Error 를 주면 예외를 던진다
let valueOf result =
    match result with
    | Ok value -> value
    | Error _ -> failwith "오류가 없다고 확인한 뒤에만 부를 수 있다"
```

- `valueOf` 가 이 챕터의 부분 함수다. 시그니처는 `Result<'a,'b> -> 'a` 라고 적혀 있어 어떤 `Result` 든 값을 준다고 약속하지만, `Error` 를 받으면 지키지 못하고 예외를 던진다. 6챕터의 `Seq.skip` 과 같은 성격이다.
- 시그니처가 거짓말을 하는 함수를 손으로 만들었다는 것이 곧 이 해법을 고쳐야 하는 이유다. 챕터 후반의 두 방식은 이 함수를 아예 없앤다.
- 두 함수 모두 자동 일반화되어 `Result<'a,'b>` 를 받는다. `ValidationError` 에 묶이지 않으므로 다른 검증에도 그대로 쓸 수 있다.

```
// FSI 실측: raw: RawSample -> Result<ValidatedSample,ValidationError list>
let validateSample (raw: RawSample) : Result<ValidatedSample, ValidationError list> =
    let sampleId = raw.SampleId |> validateSampleId
    let email = raw.ContactEmail |> validateEmail
    let chilled = raw.Chilled |> validateChilled
    let collectedOn = raw.CollectectedOn |> validateCollectedOn
    let volume = raw.VolumeMl |> validateVolume
    let errors =
        [ sampleId |> errorsOf
          email |> errorsOf
          chilled |> errorsOf
          collectedOn |> errorsOf
          volume |> errorsOf ]
    |> List.concat
    match errors with
    | [] ->
        Ok (create (valueOf sampleId) (valueOf email) (valueOf chilled)
                  (valueOf collectedOn) (valueOf volume))
    | _ -> Error errors
```

- 다섯 검증이 모두 실행된다. `let` 바인딩 다섯 줄이 서로를 기다리지 않으므로 한 칸이 틀려도 나머지 넷은 그대로 검사된다. 이것이 검증에 필요한 동작이다.
- `[ ... ] |> List.concat` 자리는 `List.collect errorsOf [ sampleId; ... ]` 로 줄일 수 없다. 다섯 검증의 성공 타입이 서로 달라 한 리스트에 담기지 않기 때문이다. 리스트에 담는 것은 `errorsOf` 를 지난 결과이며 그 타입은 모두 `ValidationError list` 로 같다.
- `match errors with | [] -> ... | _ -> ...` 는 빈 리스트와 그 밖으로 나뉘어 빠짐없다. `| _ -> Error errors` 자리에서 `errors` 가 비어 있지 않다는 것은 사람만 아는 사실이고 타입은 모른다.

검증을 파이프라인에 끼운다. `parse` 마지막에 `Seq.map validateSample` 한 줄을 더하는 것이 전부다.

```

let rows =
  [ "SampleId|ContactEmail|Chilled|CollectedOn|VolumeMl"
    "S-1041|choi@lab.example|Y|2024-03-02|12.5"
    "S-1042|N|2024-03-04|8"
    "S-1043|park.at.lab.example|Y|2024-03-05|4.25"
    "S-1044|yun@lab.example|maybe|2024-03-06|"
    "S-1045|seo@lab.example|N||3.0"
    "||Y|2024-13-45|- " ]

let parseRow (row: string) : RawSample option =
  match row.Split('|') with
  | [| sampleId; email; chilled; collectedOn; volume |] ->
    Some { SampleId = sampleId
          ContactEmail = email
          Chilled = chilled
          CollectedOn = collectedOn
          VolumeMl = volume }
  | _ -> None

// FSI 실측: data: string seq -> Result<ValidatedSample, ValidationError list> seq
// 출발점의 string seq -> RawSample seq 와 비교해 보면 이 챕터가 한 일이 시그니처에 드러난다
let parse (data: string seq) =
  data
  |> Seq.skip 1
  |> Seq.map parseRow
  |> Seq.choose id
  |> Seq.map validateSample

```

- 시그니처가 `RawSample seq` 에서 `Result<ValidatedSample, ValidationError list> seq` 로 바뀌었다. 쓰는 쪽은 이제 성공과 실패를 모두 다루지 않으면 컴파일되지 않는다.
- 파이프라인에 단계를 끼우는 비용이 한 줄이라는 점이 이 설계의 값어치다. 앞뒤 함수는 하나도 고치지 않았다.

오류를 사람이 읽을 문장으로 바꾸는 함수를 두고 결과를 출력한다.

```

// FSI 실측: error: ValidationError -> string
let describeError error =
  match error with
  | MissingField name -> $"%s{name} 칸이 비었다"
  | BadFormat (name, value) -> $"%s{name} 칸: '%s{value}' 형식 오류"

// FSI 실측: result: Result<ValidatedSample, ValidationError list> -> unit
let show result =
  match result with
  | Ok s ->
    let email = s.ContactEmail |> Option.defaultValue "-"
    let date =
      s.CollectedException
      |> Option.map (fun d -> d.ToString("yyyy-MM-dd", CultureInfo.InvariantCulture))
      |> Option.defaultValue "-"
    let volume = s.VolumeMl |> Option.map (fun v -> $"%M{v}") |> Option.defaultValue "-"
    printfn "통과 %7s %-20s %-6b %-11s %s" s.SampleId email s.Chilled date volume
  | Error errors ->
    printfn "반송 %s" (errors |> List.map describeError |> String.concat "; ")

rows |> Seq.ofList |> parse |> Seq.iter show
// 통과 S-1041 choi@lab.example true 2024-03-02 12.5
// 통과 S-1042 - false 2024-03-04 8
// 반송 ContactEmail 칸: 'park.at.lab.example' 형식 오류
// 반송 Chilled 칸: 'maybe' 형식 오류
// 통과 S-1045 seo@lab.example false - 3.0
// 반송 SampleId 칸이 비었다; CollectedOn 칸: '2024-13-45' 형식 오류; VolumeMl 칸: '-' 형식 오류

```

- 마지막 행에서 오류 세 개가 함께 나온 것이 이 해법의 성과다. 첫 오류에서 멈추는 방식이라면 `SampleId` 하  
나만 보고했을 것이다.
- 빈 칸이 `-` 로 나온 것은 `Option.defaultValue` 가 `None` 을 대신 채운 것이다. 값이 없다는 사실이 타입에  
남아 있으므로 출력 단계에서 어떻게 보일지 고를 수 있다. 빈 문자열로 뭉개 두면 이 선택지가 없다.



## Where are we now? — 원서가 정리하는 지점 (원서 pp.110-114)

- 원서는 여기서 지금까지의 코드를 한 파일로 모아 다섯 페이지에 걸쳐 보여 준다. 위 `08-manual` 단위가 그 전체에 대응한다.
- 파일 안의 순서에 규칙이 있다. 타입 선언, 액티브 패턴, 검증 함수, 도우미 함수, 조립 함수, 파이프라인 순이다. F# 은 위에서 아래로만 이름이 보이므로 의존 방향이 그대로 순서가 된다.
- 원서는 이 코드가 잘 돌아가지만 F# 과 대부분의 함수형 언어에서 더 관용적인 방법이 있다고 말하며 다음 절로 넘어간다. 그 이름이 애플리케티브(applicative)다.
- 원서를 옆에 두고 읽을 때 걸리는 대목이 셋 있다. p.109 의 `ConversionError list` 는 오기이고 pp.110-113 은 다시 `ValidationError list` 다. p.104 의 `DataReader` 가 p.110 에서 `FileReader` 로 이름이 바뀌는데 이 노트는 6챕터를 따라 `DataReader` 로 둔다. pp.110-112 전체 코드에는 p.106 의 `(IsValidDate|_|)` 가 빠져 있어 그대로 붙여 넣으면 컴파일되지 않는다.
- 이 지점에서 남은 문제를 정리해 두면 다음 절이 무엇을 고치는지 분명해진다. 첫째, `valueOf` 라는 부분 함수를 손으로 만들었다. 둘째, 칸이 하나 늘 때마다 `let` 한 줄, `errorsOf` 한 줄, `valueOf` 한 개를 세 곳에 나눠 적어야 한다. 셋째, 그 셋 중 하나를 빼뜨려도 컴파일은 통과한다.

## 오류 타입의 모양을 정하기 (원서 pp.114-116)

- 오류를 모으려면 실패 자리가 리스트여야 한다. 검증 함수 하나는 오류를 하나만 내므로 어딘가에서 리스트로 감싸는 일이 필요하다.
- 감싸는 자리는 두 곳 중 하나다. 검증 함수가 오류 하나를 내고 쓰는 쪽에서 `Result.mapError` 로 감싸든가, 검증 함수가 처음부터 오류 리스트를 내든가다.
- 원서는 둘 중 무엇을 고르든 상관없지만 하나로 통일하라고 말한다. 이 노트는 앞의 것을 고른다. 검증 함수를 다른 곳에서도 쓸 수 있게 남겨 두는 편이 낫다는 판단이다.

```
// 이 단위가 보여주는 것: 첫 오류에서 멈추는 방식과 오류를 모으는 방식의 차이
open System
open System.Globalization

type ValidationError =
    | MissingField of name: string
    | BadFormat of name: string * value: string

// 합성 방식만 비교하려고 칸을 세 개로 줄인 축소판이다. 검증 함수의 모양은 앞 단위와 같다
type Slip = { SampleId: string; Chilled: bool; VolumeMl: decimal }

// FSI 실측: sampleId: string -> chilled: bool -> volume: decimal -> Slip
let makeSlip sampleId chilled volume =
    { SampleId = sampleId; Chilled = chilled; VolumeMl = volume }

// FSI 실측: sampleId: string -> Result<string,ValidationError>
let validateSampleId sampleId =
    if sampleId <> "" then Ok sampleId else Error (MissingField "SampleId")

// FSI 실측: chilled: string -> Result<bool,ValidationError>
let validateChilled (chilled: string) =
    match chilled.Trim().ToUpperInvariant() with
    | "Y" -> Ok true
    | "N" -> Ok false
    | _ -> Error (BadFormat ("Chilled", chilled))

// FSI 실측: volume: string -> Result<decimal,ValidationError>
let validateVolume (volume: string) =
    match Decimal.TryParse(volume, NumberStyles.Number, CultureInfo.InvariantCulture) with
    | true, value when value > 0m -> Ok value
    | _ -> Error (BadFormat ("VolumeMl", volume))
```

오류 하나를 리스트로 감싸는 함수에는 이름을 붙여 둔다. 람다 식 `(fun e -> [ e ])` 를 쓸 수도 있지만 `List` 모듈에 있는 `List.singleton` 이 정확히 그 일을 한다.

```
// FSI 실측: result: Result<'a,'b> -> Result<'a,'b list>
// List.singleton 의 시그니처는 'a -> 'a list 이고 (fun e -> [ e ]) 와 같다
let asList result = result |> Result.mapError List.singleton
```

- `Result.mapError` 는 성공값은 그대로 두고 실패값에만 함수를 적용한다. 3챕터에서 실패 타입을 맞추는 어댑터로 쓴 것과 같은 함수이며, 여기서는 `ValidationError` 를 `ValidationError list` 로 넓히는 데 쓴다.
- 검증 함수가 처음부터 오류 리스트를 내게 만들면 이 감싸기가 사라져 조립 코드가 짧아진다. 대신 오류 하나만 필요한 다른 자리에서 쓰기 불편해진다. 원서가 말한 취향 문제가 이 맞바꿈이다.

## 첫 오류에서 멈추는 방식 (원서 p.117 확장)

- 3챕터의 `Result.bind` 로 검증 함수 셋을 이으면 첫 오류에서 멈춘다. 원서는 이 방식을 모나드 방식 (monadic)이라 부른다.
- 시그니처가 증거다. 실패 타입이 `ValidationError` 하나로 남고 리스트가 되지 않는다. 오류를 둘 이상 담을 자리 자체가 없으니 이 조립으로는 모을 수 없다. 거꾸로 실패 타입이 리스트라고 해서 오류를 모으는 방식인 것은 아니다. 그 경우는 뒤에서 다시 본다.
- 이 방식이 나쁜 것이 아니다. 뒤 단계가 앞 단계의 결과에 의존할 때는 이것이 유일한 방법이다. 검증은 그런 작업이 아니라서 맞지 않는 것이다.

```
// FSI 실측: sampleId: string -> chilled: string -> volume: string
//           -> Result<Slip,ValidationError>
// 실패 자리가 리스트가 아니다. 오류를 하나만 담는다
let validateFirstError sampleId chilled volume =
    validateSampleId sampleId
    |> Result.bind (fun id ->
        validateChilled chilled
        |> Result.bind (fun cold ->
            validateVolume volume
            |> Result.map (fun ml -> makeSlip id cold ml)))
```

- `Result.bind` 는 앞 단계가 `Error` 면 넘긴 함수를 부르지 않는다. `validateSampleId` 가 실패하면 `validateChilled` 와 `validateVolume` 은 실행되지 않는다.
- 원인은 `Result.bind` 의 시그니처에 있다. 첫 매개변수가 `'a -> Result<'b,'c>` 이므로 뒤 계산은 앞 단계의 값을 받아야 비로소 만들어진다. 앞이 실패하면 그 값이 없어 뒤 계산을 만들 수조차 없다. `apply` 의 두 매개변수는 둘 다 이미 만들어진 `Result` 라서 이 의존이 없다.
- 마지막만 `Result.map` 인 것은 `makeSlip` 이 `Result` 를 내지 않기 때문이다. 3챕터에서 정리한 대로 넘기는 함수가 `Result` 를 내면 `bind` , 내지 않으면 `map` 이다.
- 중첩이 깊어지는 것도 눈에 걸린다. 칸이 다섯 개면 다섯 겹이 된다. 12챕터의 계산 식 (computation expression)이 이 중첩을 평평하게 펴 준다.

## 오류를 모으는 방식 — 애플리커티브 (원서 p.114 확장)

- 오류를 모으려면 `Result` 두 개를 받아 둘 다 `Ok` 면 값을 적용하고 둘 다 `Error` 면 오류를 이어 붙이는 함수가 필요하다. 이 함수를 `apply` 라 부르고, 이 방식을 애플리커티브라 부른다.
- `Result.bind` 와의 차이는 함수를 부르는 시점이다. `bind` 는 앞 결과를 보고 다음 함수를 부를지 정하지만, `apply` 는 두 결과를 이미 손에 들고 합칠지 오류를 이어 붙일지만 정한다. 그래서 모든 검증이 실행된다.
- F# 5 의 `and!` 문법이 나오기 전에는 이 함수를 직접 만들어 썼다. 원서도 계산 식 이전의 관용적인 방식을 다룬 글을 소개하며 그 원리를 알아 둘 값어치가 있다고 말한다.

```
// FSI 실측: fResult: Result<'a -> 'b>, 'c list> -> xResult: Result<'a, 'c list>
//          -> Result<'b, 'c list>
// 성공 자리에 함수가 들어 있는 Result 를 받는다는 점이 map/bind 와 다르다
let apply fResult xResult =
    match fResult, xResult with
    | Ok f, Ok x -> Ok (f x)
    | Error left, Error right -> Error (left @ right)
    | Error left, Ok _ -> Error left
    | Ok _, Error right -> Error right
```

- 실패 타입이 `'c list` 로 유추되었다. `left @ right` 로 이어 붙이므로 컴파일러가 리스트라고 판단한 것이다. 오류를 모으는 방식이라는 사실이 시그니처에 저절로 적혔다.
- 네 케이스 중 두 번째가 이 함수의 핵심이다. 양쪽이 다 실패했을 때 하나를 버리지 않고 둘을 이어 붙인다. `Result.bind` 에는 이 자리가 없다.
- 첫 매개변수가 `Result<('a -> 'b), 'c list>` 라는 것이 이 방식의 요령이다. 성공 자리에 함수를 담아 두면 인자를 하나씩 먹여 가며 조립할 수 있다. 2챕터의 부분 적용이 `Result` 안에서 일어나는 셈이다.

관용적으로 쓰이는 연산자 이름을 붙이면 조립이 한 줄로 읽힌다.

```
// FSI 실측: f: ('a -> 'b) -> result: Result<'a, 'c> -> Result<'b, 'c>
let (<|>) f result = Result.map f result

// FSI 실측: fResult: Result<'a -> 'b>, 'c list> -> xResult: Result<'a, 'c list>
//          -> Result<'b, 'c list>
let (<*>) fResult xResult = apply fResult xResult

// FSI 실측: sampleId: string -> chilled: string -> volume: string
//          -> Result<Slip, ValidationErrors list>
// 실패 자리가 리스트다. 오류를 몇 개든 담는다
let validateAll sampleId chilled volume =
    makeSlip
    <|> asList (validateSampleId sampleId)
    <*> asList (validateChilled chilled)
    <*> asList (validateVolume volume)
```

- `<!>` 는 `Result.map` 의 다른 이름이다. `makeSlip <!> asList (validateSampleId sampleId)` 까지의 타입을 실측하면 `Result<(bool -> decimal -> Slip), ValidationError list>` 다. 세 인자 중 하나를 먹은 함수가 `Result` 안에 남았다.
- 이후 `<*>` 가 남은 인자를 하나씩 먹인다. 마지막 `<*>` 를 지나면 함수가 다 채워져 `Result<Slip, ValidationError list>` 가 된다.
- 두 연산자 모두 `<` 로 시작하므로 우선순위가 같고 왼쪽부터 묶인다. 그래서 괄호 없이 위에서 아래로 읽으면 된다.
- 매개변수를 생략하고 `let (<!>) = Result.map` 으로 적어도 통과한다. 다만 FSI 실측 시그니처가 `(('a -> 'b) -> Result<'a, 'c> -> Result<'b, 'c>)` 처럼 바깥 괄호가 붙은 함수 타입 값이 된다. 매개변수를 적어 함수로 정의하는 쪽이 시그니처가 깔끔하고, 뒤에서 부분 적용해 쓸 때 헛갈리지 않는다.

두 방식을 같은 입력에 걸어 보면 차이가 그대로 드러난다.

```
// FSI 실측: errors: ValidationError list -> string
let fieldNames errors =
    errors
    |> List.map (fun e -> match e with MissingField name -> name | BadFormat (name, _) -> name)
    |> String.concat ", "

// 실패 자리의 모양이 달라 출력 함수도 따로 만들어야 한다. 그 점이 곧 시그니처 차이다
let showFirst label result =
    match result with
    | Ok slip -> printfn "%-10s 통과 %s %b %M" label slip.SampleId slip.Chilled slip.VolumeMl
    | Error error -> printfn "%-10s 반송 1건 - %s" label (fieldNames [ error ])

let showAll label result =
    match result with
    | Ok slip -> printfn "%-10s 통과 %s %b %M" label slip.SampleId slip.Chilled slip.VolumeMl
    | Error errors -> printfn "%-10s 반송 %d건 - %s" label (List.length errors) (fieldNames errors)

showFirst "bind" (validateFirstError "" "maybe" "-")
showAll "apply" (validateAll "" "maybe" "-")
showFirst "bind" (validateFirstError "S-1041" "Y" "12.5")
showAll "apply" (validateAll "S-1041" "Y" "12.5")
// bind      반송 1건 - SampleId
// apply      반송 3건 - SampleId, Chilled, VolumeMl
// bind      통과 S-1041 true 12.5
// apply      통과 S-1041 true 12.5
```

- 성공하는 입력에서는 두 방식의 결과가 같다. 갈라지는 것은 실패할 때다.
- 출력 함수를 두 개 만들어야 했던 것이 시그니처 차이의 실제 비용이다. `Error` 안에 든 것이 `ValidationError` 나 `ValidationError list` 냐가 다르므로 `match` 식의 두 번째 케이스가 서로 호환되지 않는다.
- 접수 창구에서 원하는 것은 `apply` 쪽이다. 접수자가 한 번에 세 칸을 고칠 수 있다.

## Functional Validation the F# Way — validation 계산 식 (원서 pp.116-117)

- `apply` 를 손으로 만드는 대신 F# 5 부터는 계산 식의 `and!` 문법으로 같은 일을 한다. 원서가 관용적이라고 말하는 방식이 이것이다.
- `let!` 은 `Result` 안의 값을 꺼내 이름에 묶는다. `let!` 을 연달아 쓰면 뒷줄이 앞줄의 값을 쓸 수 있다. 그 값이 없으면 뒷줄을 만들 수 없으므로 앞줄이 실패하는 순간 멈춘다. `and!` 는 뒷줄이 앞줄의 값을 쓰지 않겠다는 선언이어서 모든 줄이 평가되고 오류가 모인다.
- 계산 식 자체는 12챕터에서 다룬다. 여기서는 `let!` 과 `and!` 의 차이만 보아 두면 된다. `let!` 은 계산 식 빌더의 `Bind` 멤버를, `and!` 는 `MergeSources` 멤버를 부른다. 12챕터는 `Bind · Return · ReturnFrom` 세 멤버로 빌더를 직접 만들어 `let!` 쪽이 어떻게 도는지 확인하고, 그 자리에서 이 챕터의 `and!` 를 다시 짚는다. 원서가 12챕터를 마치면 이 챕터의 검증 예제로 돌아오라고 권하는 것도 그래서다.
- 검증용 계산 식은 언어에 들어 있지 않다. `FsToolkit.ErrorHandling` 패키지가 `validation` 이라는 이름으로 제공한다. 그래서 아래 블록은 이 챕터에서 유일하게 외부 패키지가 필요하다. 처음 한 번은 네트워크가 있어야 패키지를 받아 오고, 그 뒤에는 로컬 NuGet 캐시에 있는 것을 쓴다. 받아 오지 못하면 `error FS0999` 로 실패한다.

```
// 이 단위가 보여주는 것: validation 계산 식의 let! 과 and!
// 프로젝트에서는 dotnet add package FsToolkit.ErrorHandling 로 넣는다
#r "nuget: FsToolkit.ErrorHandling, 5.2.0"

open System
open System.Globalization
open FsToolkit.ErrorHandling.ValidationCE

type ValidationError =
    | MissingField of name: string
    | BadFormat of name: string * value: string

type Slip = { SampleId: string; Chilled: bool; VolumeMl: decimal }

let makeSlip sampleId chilled volume =
    { SampleId = sampleId; Chilled = chilled; VolumeMl = volume }

let validateSampleId sampleId =
    if sampleId <> "" then Ok sampleId else Error (MissingField "SampleId")

let validateChilled (chilled: string) =
    match chilled.Trim().ToUpperInvariant() with
    | "Y" -> Ok true
    | "N" -> Ok false
    | _ -> Error (BadFormat ("Chilled", chilled))

let validateVolume (volume: string) =
    match Decimal.TryParse(volume, NumberStyles.Number, CultureInfo.InvariantCulture) with
    | true, value when value > 0m -> Ok value
    | _ -> Error (BadFormat ("VolumeMl", volume))

let asList result = result |> Result.mapError List.singleton
```

- 스크립트에서는 `#r "nuget: ..."` 로 패키지를 끌어온다. 버전을 못 박아 둔 것은 새 버전이 올라올 때 `open` 경로나 실측 시그니처 주석이 예고 없이 어긋나는 것을 막기 위한 것이다. 프로젝트라면 원서대로 `dotnet add package` 를 쓴다.
- `open FsToolkit.ErrorHandling.ValidationCE` 는 `validation` 계산 식만 들어온다. 그래서 `Validation<'a,'e>` 라는 타입 약어 이름은 아직 보이지 않고, 그 이름을 코드에 적으면 `error FS0039` 가 난다. 약어까지 쓰려면 `open FsToolkit.ErrorHandling` 이 필요하다. 아래에서 반환 타입을 `Result<Slip, ValidationError list>` 로 적은 것도 그래서다.
- 준비 코드는 앞 단위와 같다. 달라지는 것은 조립 방식뿐이다.
- `asList` 는 장식이 아니라 필수 단계다. 이 계산 식은 실패 자리가 리스트인 값을 요구하므로 `Result<string, ValidationError>` 를 그대로 넘기면 `error FS0001` 이 난다. 패키지가 같은 일을 하는 `Validation.ofResult` 를 제공하지만, 그 이름 역시 `open FsToolkit.ErrorHandling` 이 있어야 보인다.

```
// FSI 실측(반환 타입 주석을 지우면): sampleId: string -> chilled: string -> volume: string
//      -> FsToolkit.ErrorHandling.Validation<Slip,ValidationError>
// Validation<'a,'e> 는 Result<'a,'e list> 의 타입 약어다. 그래서 아래 주석이 그대로 통과한다
let validateSlip sampleId chilled volume : Result<Slip, ValidationError list> =
    validation {
        let! id = validateSampleId sampleId |> asList
        and! cold = validateChilled chilled |> asList
        and! ml = validateVolume volume |> asList
        return makeSlip id cold ml
    }
```

- `let!` 의 느낌표는 `Result` 라는 껍데기를 벗기라는 표시다. `id` 의 타입은 `string` 이다. 느낌표를 떼면 `id` 가 `Result<string, ValidationError list>` 가 되고 `makeSlip` 에 넣을 수 없게 된다.
- 첫 줄만 `let!` 이고 나머지가 `and!` 다. `and!` 는 앞줄의 결과에 기대지 않겠다는 선언이며, 그 덕에 세 검증이 모두 실행된다. 병렬로 돈다는 뜻은 아니다. 세 오른쪽 식은 같은 스레드에서 위에서 아래로 차례로 평가된다. 진짜 병렬이 필요하다면 같은 패키지의 `parallelAsyncValidation` 계산 식이 따로 있다.
- `return` 은 계산 식 안에서만 쓰는 낱말이고, F# 에서 `return` 이 필요한 자리는 계산 식뿐이다.
- 반환 타입을 `Result<Slip, ValidationError list>` 로 적어도 통과한다. 패키지가 정한 `Validation<'a,'e>` 는 `Result<'a,'e list>` 에 붙인 타입 약어이므로 같은 타입이다.

`and!` 를 `let!` 로 바꾸면 방금 만든 것이 모나드 방식으로 되돌아간다. 한 낱말이 동작을 가른다.

```
// 세 줄 모두 let! 이다. 앞줄이 실패하면 뒷줄은 실행되지 않는다
let validateSlipStopping sampleId chilled volume : Result<Slip, ValidationError list> =
  validation {
    let! id = validateSampleId sampleId |> asList
    let! cold = validateChilled chilled |> asList
    let! ml = validateVolume volume |> asList
    return makeSlip id cold ml
  }

let fieldNames errors =
  errors
  |> List.map (fun e -> match e with MissingField name -> name | BadFormat (name, _) -> name)
  |> String.concat ", "

let show label result =
  match result with
  | Ok slip -> printfn "%-12s 통과 %s %b %M" label slip.SampleId slip.Chilled slip.VolumeMl
  | Error errors -> printfn "%-12s 반송 %d건 - %s" label (List.length errors) (fieldNames errors)

show "and! 수집" (validateSlip "" "maybe" "-")
show "let! 멈춤" (validateSlipStopping "" "maybe" "-")
show "and! 통과" (validateSlip "S-1041" "Y" "12.5")
// and! 수집      반송 3건 - SampleId, Chilled, VolumeMl
// let! 멈춤      반송 1건 - SampleId
// and! 통과      통과 S-1041 true 12.5
```

- 두 함수의 시그니처가 같다는 점을 눈여겨볼 만하다. 둘 다 `Result<Slip, ValidationError list>` 를 낸다. 계산 식 안이 `Validation` 으로 통일되어 있어 실패 자리가 이미 리스트이기 때문이다.
- 그래서 `let!` 판은 리스트를 낼 수 있는데도 원소를 하나만 담는다. 앞 절의 `Result.bind` 판은 애초에 리스트를 담을 자리가 없었다. 같은 "첫 오류에서 멈춘다"가 타입에 드러나는 정도가 다르다.
- 이 코드가 하는 일은 08-manual 단위의 `validateSample` 과 정확히 같다. `errorsOf`, `valueOf`, `List.concat`, 빈 리스트 검사가 전부 사라졌고 부분 함수도 없어졌다.

## 검증 결과 여러 개를 모으기 (원서 p.109 확장)

- 파이프라인의 결과는 `Result` 의 시퀀스다. 시료 하나하나의 성패는 알 수 있지만 "몇 건 통과했나", "전부 통과했나" 같은 질문에는 한 번 더 모아야 답할 수 있다.
- 5챕터의 `List.choose` 와 `List.collect` 로 통과분과 반송분을 갈라낼 수 있다. 오류 쪽은 리스트의 리스트이므로 `List.collect` 가 한 겹 벗겨 준다.
- 한 건이라도 틀리면 전량을 반송해야 하는 요구라면 `apply` 와 같은 논리를 리스트 수준으로 올린 함수를 쓴다. `List.fold` 한 번으로 만들 수 있다.

```
// 이 단위가 보여주는 것: Result 의 컬렉션을 List 함수로 모으는 두 방식
// 앞 파이프라인이 낸 결과를 오류 타입만 string 으로 줄여 적은 값이다. 6행은 오류가 세 개다
let results : Result<string, string list> list =
    [ OK "S-1041"
      OK "S-1042"
      Error [ "3행 ContactEmail" ]
      Error [ "4행 Chilled" ]
      OK "S-1045"
      Error [ "6행 SampleId"; "6행 CollectedOn"; "6행 VolumeMl" ] ]

// FSI 실측: items: Result<'a,'b list> list -> 'a list * 'b list
let partitionResults items =
    let accepted = items |> List.choose (fun r -> match r with Ok v -> Some v | Error _ -> None)
    let rejected = items |> List.collect (fun r -> match r with Ok _ -> [] | Error e -> e)
    accepted, rejected

let accepted, rejected = partitionResults results
printfn "통과 %d건: %s" (List.length accepted) (String.concat " ", " accepted)
printfn "반송 %d건: %s" (List.length rejected) (String.concat " / " rejected)
// 통과 3건: S-1041, S-1042, S-1045
// 반송 5건: 3행 ContactEmail / 4행 Chilled / 6행 SampleId / 6행 CollectedOn / 6행 VolumeMl
```

- 통과분은 3건인데 오류는 5건이다. 반송된 시료 수와 오류 수가 다르다는 것이 오류를 모으는 방식의 성질이다. 첫 오류에서 멈추는 방식이었다면 두 수가 같았을 것이다.
- 반환값을 튜플로 두면 두 결과를 한 번에 받을 수 있다. `let accepted, rejected = ...` 로 바로 풀어 쓴다.

전량 판정은 `apply` 의 네 케이스를 그대로 `List.fold` 안에 옮긴 모양이 된다.

```
// FSI 실측: items: Result<'a,'b list> list -> Result<'a list,'b list>
// 상태와 원소가 모두 Result 이고, 둘 다 실패면 오류를 이어 붙인다
let sequence items =
    let folder state item =
        match state, item with
        | Ok values, Ok value -> Ok (values @ [ value ])
        | Ok _, Error errors -> Error errors
        | Error errors, Ok _ -> Error errors
        | Error left, Error right -> Error (left @ right)
    items |> List.fold folder (Ok [])

printfn "%A" (sequence [ Ok "S-1041"; Ok "S-1042" ])
// Ok ["S-1041"; "S-1042"]

match sequence results with
| Ok values -> printfn "전량 통과: %d건" (List.length values)
| Error errors -> printfn "전량 반송: 오류 %d건" (List.length errors)
// 전량 반송: 오류 5건
```

- `Result<'a,'b list> list -> Result<'a list,'b list>` 라는 시그니처를 잘 보면 `Result` 와 `list` 의 안팎이 뒤집혔다. 함수형 언어에서는 이렇게 뒤집는 함수를 `sequence` 라 부르고, 원소마다 함수를 적용한 뒤 뒤집는 것을 `traverse` 라 부른다. 컬렉션 타입 `seq` 와는 상관이 없는 이름이다.
- 방금 만든 것은 오류를 모으는 판이다. 첫 오류에서 멈추는 판도 같은 이름으로 불리므로, `FsToolkit.ErrorHandling` 은 `List.sequenceResultA` (모으는 판)와 `List.sequenceResultM` (멈추는 판)처럼 접미사로 둘을 가른다.
- 네 케이스가 `apply` 와 같은 규칙이다. 애플리커티브를 이해해 두면 이런 함수를 필요할 때 만들 수 있다는 것이 원서가 말한 값어치다.
- `values @ [ value ]` 는 리스트 끝에 붙이는 연산이라 원소 수에 비례하는 비용이 든다. 원소가 많으면 앞에 붙인 뒤 마지막에 `List.rev` 하는 편이 낫다. 5챕터에서 짚은 연결 리스트의 성질이다.



## Summary — 원서의 챕터 요약 (원서 p.117)

- 원서는 이 챕터에서 액티브 패턴으로 검증을 붙이는 방법과, 데이터 처리 파이프라인에 기능을 더하는 일이 얼마나 간단한지를 다뤘다고 정리한다.
- 그리고 F# 5 의 계산 식 지원을 쓴 애플리커티브 방식의 해법이 처음 만든 해법보다 우아하다고 덧붙인다.
- 다음 챕터에서는 1챕터의 코드를 도메인 낱말에 가깝게 고치고 원시 타입 사용을 줄이는 방법을 살펴본다.

## 정리 — 이 노트의 요약

- 검증은 확인이 아니라 변환이다. 문자열만 담은 레코드를 받아 칸마다 제 타입이 붙은 레코드로 옮기고, 비어 도 되는 칸은 `Option` 으로 만든다. 검증했다는 사실이 타입에 남는다.
- 검증 함수의 목표 시그니처는 `string -> Result<'a, ValidationError>` 이고 `'a` 는 칸마다 다르다. 실패 이유는 판별 유니온으로 두고 함수 전부가 같은 실패 타입을 쓰게 맞춰 둔다.
- 문자열 해석은 부분 패턴이 맞는다. 정규식은 매개변수 있는 부분 패턴으로 한 번만 감싸 두고 패턴 문자열을 같이 끼운다. `TryParse` 는 문화권을 명시하고, 형식이 정해진 입력이라면 `TryParseExact` 가 검증에 맞다.
- 검증 함수와 레코드 생성 함수는 그대로 이어지지 않는다(오류 FS0001). 생성 함수는 `Result` 를 모르고 검증 함수는 `Result` 를 낸다.
- 원서가 먼저 내놓는 해법은 오류만 뽑는 함수와 값만 뽑는 함수를 두고 `List.concat` 으로 오류를 모으는 것이다. 돌아가지만 값을 뽑는 함수가 부분 함수이며, 칸이 늘 때마다 세 곳을 고쳐야 하고 하나를 빠뜨려도 컴파일이 통과한다.
- `Result.bind` 로 이으면 첫 오류에서 멈춘다. 실측 시그니처가 `... -> Result<Slip, ValidationError>` 로 실패 자리에 리스트가 없다. 오류를 둘 이상 담을 자리 자체가 없다는 뜻이다.
- 오류를 모으려면 `apply` 가 필요하다. 실측 시그니처는 `Result<('a -> 'b), 'c list> -> Result<'a, 'c list> -> Result<'b, 'c list>` 이고, 양쪽이 다 실패했을 때 오류를 이어 붙이는 케이스가 이 함수의 핵심이다. `<!>` ( `Result.map` )와 `<*>` ( `apply` )를 이어 쓰면 생성 함수에 인자를 하나씩 먹여 조립할 수 있다.
- 오류 하나를 리스트로 넓히는 데는 `Result.mapError List.singleton` 을 쓴다. 검증 함수가 처음부터 리스트를 내게 만드는 선택도 있고, 어느 쪽이든 하나로 통일하는 것이 중요하다.
- F# 5 의 `and!` 는 `apply` 를 문법으로 감싼 것이다. `validation` 계산 식 안에서 `let!` 을 연달아 쓰면 뒷줄이 앞줄의 값에 의존해 첫 오류에서 멈추고, `and!` 로 이으면 의존이 끊겨 모든 줄이 평가되고 오류가 모인다. 병렬로 도는 것이 아니라 의존이 없어지는 것이다. 이 `validation` 계산 식은 `FsToolkit.ErrorHandling` 패키지가 제공하며, 그 `Validation<'a, 'e>` 는 `Result<'a, 'e list>` 의 타입 약어다.
- `return` 이 필요한 자리는 F# 에서 계산 식뿐이다.
- 파이프라인에 검증을 끼우는 비용은 `Seq.map validateSample` 한 줄이다. 대신 `parse` 의 시그니처가 `string seq -> RawSample seq` 에서 `string seq -> Result<ValidatedSample, ValidationError list>` `seq` 로 바뀌어 쓰는 쪽이 실패를 다루지 않을 수 없게 된다.
- `Result` 의 컬렉션은 `List.choose` 와 `List.collect` 로 통과분과 오류로 갈라낸다. 전량 판정이 필요하다면 `Result<'a, 'b list> list -> Result<'a list, 'b list>` 로 안팎을 뒤집는 함수를 `List.fold` 로 만든다.

## 원서 대조 표

| 절                                                               | 원서 페이지     | 실행 단위            |
|-----------------------------------------------------------------|------------|------------------|
| Setting Up — 검증을 붙일 자리                                          | pp.103-105 | 08-intake        |
| Solving the Problem — 검증된 값을 담을 타입                              | p.105      | 08-manual        |
| 오류를 판별 유니온으로                                                    | p.106      | 08-manual        |
| 파싱을 부분 패턴으로                                                     | p.106      | 08-patterns      |
| 필드별 검증 함수                                                       | p.107      | 08-manual        |
| <code>create</code> 가 <code>Result</code> 를 받지 못한다              | p.108      | —                |
| 오류를 모아 한 번에 돌려주기                                                | p.109      | 08-manual        |
| Where are we now? — 원서가 정리하는 지점                                 | pp.110-114 | 08-manual        |
| 오류 타입의 모양을 정하기                                                  | pp.114-116 | 08-compose       |
| 첫 오류에서 멈추는 방식                                                   | p.117 확장   | 08-compose       |
| 오류를 모으는 방식 — 애플리커티브                                             | p.114 확장   | 08-compose       |
| Functional Validation the F# Way — <code>validation</code> 계산 식 | pp.116-117 | 08-validation-ce |
| 검증 결과 여러 개를 모으기                                                 | p.109 확장   | 08-collect       |
| Summary — 원서의 챕터 요약                                             | p.117      | —                |

## 09 - 단일 케이스 판별 유니온 (원서 pp.118-129)

코드에 `string` 과 `decimal` 이 널려 있으면 컴파일러가 도와줄 여지가 없다. `decimal` 두 개를 받는 함수는 두 인자를 뒤바꿔 넘겨도 통과하고, 음수 금액이나 범위를 벗어난 좌표도 통과한다. 타입은 맞았지만 뜻이 틀린 값이 그대로 흐른다. 이런 상태를 원시 타입 강박(primitive obsession)이라 부른다. 이 챕터는 원시 타입을 도메인 이름으로 감싸 컴파일러가 뜻까지 검사하게 만드는 방법을 다룬다. 핵심 도구는 케이스가 하나뿐인 판별 유니온(discriminated union)이다. 여기에 `private` 접근 지정자와 스마트 생성자(smart constructor)를 얹으면, 만들어진 값이 언제나 유효하다는 사실을 타입 하나가 보증한다. 1챕터 노트가 "`DriverId` 가 아직 그냥 `string` 이다"라고 남겨 둔 숙제가 바로 이 챕터의 주제다.

## Setting Up — 준비 (원서 p.118)

- 원서는 1챕터에서 만든 코드를 이어서 손본다. 새 폴더에 `code.fsx` 하나를 만들고 FSI 로 돌리는 것이 전부다.
- 이 노트는 1챕터 코드를 다시 꺼내지 않고 새 도메인으로 같은 길을 걷는다. 반려동물 호텔의 숙박 요금 계산이다. 처음에는 숙박 일수와 마리 수가 그냥 `int` 이고, 챕터가 끝날 때는 둘 다 검증을 통과한 값만 담을 수 있는 타입이 된다.
- 원서는 이 챕터의 코드가 8챕터 코드에 어떻게 적용될지 생각해 보라는 숙제를 낸다. 여러 검증 결과를 하나로 모으는 방법은 8챕터의 주제이므로 이 노트에서는 다루지 않는다.

## Solving the Problem — 원시 타입을 도메인 이름으로 (원서 pp.118-125)

원서는 이 절 하나에서 세 단계를 밟는다. 타입 약어 → 단일 케이스 판별 유니온 → `private` 케이스와 스마트 생성자다. 단계마다 무엇이 새로 막히는지가 다르므로 나눠 본다.

### 1단계: 타입 약어로 시그니처에 이름 붙이기 (원서 pp.119-120)

- 시그니처가 `int -> int -> decimal` 이면 읽는 사람이 무엇을 어느 자리에 넣어야 하는지 알 수 없다. 타입 약어(type abbreviation)로 이름을 붙이면 시그니처가 설명 구실을 한다.
- 타입 약어는 기존 타입에 별명을 붙이는 것이고 새 타입을 만들지 않는다. 6챕터에서 함수 타입에 이름을 붙일 때 이미 썼다.
- 매개변수와 반환 타입에 타입 주석으로 약어를 적는 방식과, 함수 타입 전체에 약어 이름을 붙이는 방식이 있다. 결과로 만들어지는 함수는 같다.

```
// 이 단위가 보여주는 것: 타입 약어로 시그니처를 읽기 좋게 만들되, 잘못된 값은 여전히 통과한다
type Nights = int
type PetCount = int
type Fee = decimal

let nightlyRatePerPet = 35000M

// nights: Nights -> petCount: PetCount -> Fee
let estimateFee (nights: Nights) (petCount: PetCount) : Fee =
    decimal nights * decimal petCount * nightlyRatePerPet

printfn "3박 2마리: %M" (estimateFee 3 2) // 3박 2마리: 210000
```

매개변수 자리마다 약어를 적는 대신, 함수 타입 전체에 이름을 붙이고 람다를 그 이름으로 바인딩하는 방식도 있다. 매개변수를 함수 이름 옆에 적을 수 없어 `fun` 으로 받는다는 점만 다르다.

```
type EstimateFee = Nights -> PetCount -> Fee

// nights: Nights -> petCount: PetCount -> Fee
let estimateFee2 : EstimateFee =
    fun nights petCount -> decimal nights * decimal petCount * nightlyRatePerPet

printfn "3박 2마리: %M" (estimateFee2 3 2) // 3박 2마리: 210000
```

두 방식의 실측 시그니처는 같다. FSI 는 둘 다 `nights: Nights -> petCount: PetCount -> Fee` 로 보여 준다. 함수 타입 전체에 붙인 약어는 FSI 가 찍는 `val` 줄에 이름으로 남지 않고 화살표 형태로 펼쳐진다. `Nights`

나 `Fee` 처럼 타입 한 개에 붙인 약어는 그 자리에 이름으로 남는다. 어느 쪽이든 약어는 새 타입이 아니므로 실제 타입은 둘 다 `int -> int -> decimal` 이다.

## 타입 약어가 막지 못하는 것 (원서 p.120)

- 약어는 별명이므로 `Nights` 를 요구하는 자리에 아무 `int` 나 들어간다. `PetCount` 도 마찬가지다. 두 약어의 원래 타입이 같으면 서로 바꿔 넣어도 컴파일러가 아무 말을 하지 않는다.
- 범위도 통제하지 못한다. 숙박 일수가 음수든 마리 수가 400 이든 `int` 이기만 하면 통과한다.
- 원서는 위도와 경도를 뒤바꿔 넣는 예로 이 문제를 보인다. 여기서는 숙박 일수와 마리 수를 뒤집어 본다.

```
type Reservation = { Nights: Nights; PetCount: PetCount }

// 9박 2마리를 적으려다 두 칸을 뒤집었다. 그래도 컴파일된다
let swapped : Reservation = { Nights = 2; PetCount = 9 }
// 있을 수 없는 값도 통과한다
let absurd : Reservation = { Nights = -5; PetCount = 400 }

printfn "뒤집힌 예약: %A" swapped      // 뒤집힌 예약: { Nights = 2
                                     //      PetCount = 9 }
printfn "있을 수 없는 예약: %A" absurd // 있을 수 없는 예약: { Nights = -5
                                     //      PetCount = 400 }
printfn "뒤집힌 요금: %M" (estimateFee 2 9) // 뒤집힌 요금: 630000
```

이 함수는 두 값을 곱하기만 하므로 뒤집어 넣어도 요금이 같게 나왔다. 그러나 할인 규칙이 숙박 일수에만 걸리는 순간(아래에서 7박 이상 10% 할인을 쓴다) 두 결과가 갈린다. 이런 실수는 테스트를 더 써서 막을 일이 아니라 타입 시스템이 막아야 할 일이다.

## 2단계: 단일 케이스 판별 유니온 (원서 pp.120-122)

- 케이스가 하나뿐인 판별 유니온을 만들면 원래 타입이 같아도 서로 다른 타입이 된다. 이것이 단일 케이스 판별 유니온이다.
- 케이스가 하나라는 뜻을 드러내려고 앞의 `|` 를 생략하는 것이 관례다. 나중에 케이스가 늘어날 여지가 있으면 `|` 를 남겨 둔다.
- 타입 이름과 케이스 식별자를 같은 이름으로 쓰는 것이 보통이다. 둘은 다른 이름 공간에 있어 충돌하지 않는다. 케이스 식별자를 다른 이름으로 지으면 값을 만들 때와 분해할 때 그 이름을 써야 한다.

```
// 이 단위가 보여주는 것: 케이스가 하나뿐인 판별 유니온과 값을 꺼내는 방법들
type Nights = Nights of int
type PetCount = PetCount of int
type Fee = decimal

let nightlyRatePerPet = 35000M

let threeNights = Nights 3
printfn "%A" threeNights // Nights 3
```

이제 두 타입은 원래 타입이 똑같이 `int` 이지만 서로 대입되지 않는다. 아래 코드는 컴파일되지 않는다.

```
type Reservation = { Nights: Nights; PetCount: PetCount }

// 오류 FS0001 - 필요한 타입은 Nights 인데 PetCount 가 왔다고 알려 준다
let swapped : Reservation = { Nights = PetCount 2; PetCount = Nights 9 }
```

FSI 가 내는 메시지는 이 식에는 'Nights' 형식이 필요하지만 여기에서는 'PetCount' 형식이 지정되었습니다. 다. 원시 타입과 타입 약어를 단일 케이스 판별 유니온으로 바꿔 두면 이런 뒤바뀜은 컴파일 단계에서 끝난다. 걱정하고 우회하려는 사람을 막아 주지는 않지만, 넘어야 할 문턱이 하나 더 생긴다.

## 값을 꺼내는 방법 (원서 pp.121-122)

- 감쌌으면 꺼내야 계산할 수 있다. 이 절에서 세 가지를 보고, 네 번째 방법인 같은 이름의 모듈에 둔 함수는 뒤의 모듈 절에서 다룬다.
- 첫째는 `let` 바인딩에서 패턴으로 분해하는 것이다. 케이스가 하나뿐이므로 빠짐없는 패턴 매칭이 성립해 경고가 나지 않는다.
- 둘째는 함수 매개변수 자리에서 바로 분해하는 것이다. 함수 본문이 원시 타입을 쓰던 때와 똑같아지므로 코드가 가장 짧다.
- 셋째는 타입에 프로퍼티를 붙여 놓고 `.value` 로 읽는 것이다. 다음 단계에서 쓴다.

```
// let 바인딩에서 분해한다
let (Nights nightCount) = threeNights
printfn "숙박 일수: %d" nightCount // 숙박 일수: 3
```

매개변수 자리에서 분해하면 본문에 `nights` 라는 이름의 `int` 가 그대로 들어온다. 감싸기 전 코드와 본문이 같아지는 것이 이 방식의 장점이다.

```
// Nights -> PetCount -> Fee
let estimateFee (Nights nights) (PetCount petCount) : Fee =
    decimal nights * decimal petCount * nightlyRatePerPet

printfn "3박 2마리: %M" (estimateFee (Nights 3) (PetCount 2)) // 3박 2마리: 210000
```

시그니처는 실측으로 `Nights -> PetCount -> decimal` 이다. 반환 타입에 `Fee` 를 적어 두면 FSI 는 `Nights -> PetCount -> Fee` 로 보여 준다. 약어는 새 타입이 아니지만 표시에는 남으므로 문서 구실을 계속 한다.

## 3단계: private 케이스와 스마트 생성자 (원서 pp.122-124)

- 여기까지는 뒤바뀜만 막았다. 숙박 일수 자리에 `Nights (-3)` 을 넣는 것은 아직 통과한다. 도메인 값은 범위가 있는 것이 보통이므로, 범위 밖의 값으로는 아예 만들어지지 않게 해야 한다.
- 케이스 식별자에 `private` 을 붙이면 그 타입을 담은 모듈 밖에서는 값을 만들 수도 분해할 수도 없다. 남은 생성 경로는 그 모듈 안에 둔 코드뿐이다. 여기서는 타입에 붙인 정적 멤버를 그 경로로 쓴다. 이것이 스마트 생성자다.
- 검증에 실패했을 때 예외를 던지는 대신 `Result` 로 돌려준다. 실패 가능성이 반환 타입에 드러나므로 값을 쓰려면 `Ok` 와 `Error` 를 갈라 처리해야 한다. 예외처럼 시그니처에 안 보이는 채로 흐르지 않는다. `Result` 는 3챕터에서 다뤘다.

```
// 이 단위가 보여주는 것: private 케이스와 스마트 생성자로 검증을 타입 안에 넣기
type ValidationError =
  | OutOfRange of string

module PetHotel =

  type Nights = private Nights of int
  with
    // member Value: int
    member this.Value = match this with Nights n -> n

    // static member Create: input: int -> Result<Nights,ValidationError>
    static member Create input =
      if input >= 1 && input <= 30 then Ok (Nights input)
      else Error (OutOfRange "숙박 일수는 1박 이상 30박 이하여야 한다")

open PetHotel

printfn "%A" (Nights.Create 3)    // Ok Nights 3
printfn "%A" (Nights.Create 31)  // Error (OutOfRange "숙박 일수는 1박 이상 30박 이하여야 한다")
```

- `this` 는 인스턴스를 가리키는 자기 식별자(self identifier)일 뿐이고 F# 에서 특별한 뜻이 없다. `x` 나 `s` 로 지어도 되고, 본문에서 쓰지 않으면 `_` 도 된다.
- `Value` 는 프로퍼티라서 시그니처가 `member Value: int` 로 나온다. 함수가 아니므로 호출 괄호가 없다.
- `Create` 의 반환 타입은 실측으로 `Result<Nights,ValidationError>` 다. 성공 케이스에만 값이 들어 있으니, 이 관문을 통과한 `Nights` 는 언제나 1 이상 30 이하다.

`private` 이 실제로 막는 범위를 확인해 둘 필요가 있다. 아래는 `PetHotel` 모듈 밖이므로 컴파일되지 않는다.

```
// 오류 FS1093 - 케이스 식별자에 접근할 수 없다
let sneaky = Nights 99
// 분해도 같은 오류다
let peek n = match n with Nights v -> v
```

메시지는 '`Nights`' 형식의 공용 구조체 케이스 또는 필드는 이 코드 위치에서 액세스할 수 없습니다. 다. 주의할 점은 "닫은 모듈"의 범위다. 스크립트 최상위에 그냥 선언하면 파일 전체가 그 모듈이므로 `private` 이 아무것도 막지 못한다.

```
// 스크립트 최상위 선언에서는 private 이 같은 파일 안의 코드를 막지 못한다
type Leaky = private Leaky of int
let leaked = Leaky 99
printfn "%A" leaked    // Leaky 99
```

구멍이 하나 더 있다. 감싸는 모듈을 두어도 그 모듈 안의 중첩 모듈은 `private` 케이스를 그대로 본다.

```
// 감싸는 모듈 안의 중첩 모듈은 private 케이스를 그대로 본다
module Inner =

  type Days = private Days of int

  module Days =
    let create input =
      if input >= 1 then Ok (Days input)
      else Error (OutOfRange "일수는 1 이상이어야 한다")

    // 같은 모듈 안이므로 create 를 건너뛸 수 있다
    module Bypass =
      let skipped = Days (-999)

printfn "%A" Inner.Bypass.skipped    // Days -999
```

`Inner.Bypass` 는 `Inner` 안에 있으므로 `Days.create` 를 건너뛰고 값을 만들 수 있다. `private` 이 세우는 경계는 타입을 담은 모듈의 안팎이고, 그 안의 중첩 구조까지 나누지는 않는다.

값을 꺼내 쓸 때는 `.Value` 를 읽는다. 이때 타입 추론이 한계에 부딪힌다. 매개변수에 타입 주석이 없으면 컴파일러는 `nights` 가 어떤 타입인지 모르므로 멤버 조회를 확정할 수 없다.

```
// 오류 FS0072 - 어떤 타입의 .Value 인지 알 수 없다
let estimateFee nights = decimal nights.Value * 35000M
```

메시지는 이 프로그램 지점 전의 정보를 기반으로 하는 확인할 수 없는 형식의 개체를 대상으로 조회를 수행합니다. 로 시작한다. 매개변수에 타입 주석을 달면 해결된다. 타입 추론에 기대는 것이 원칙이지만, 멤버 조회처럼 추론이 닿지 않는 자리에서는 타입 주석을 적는다. 일반 함수는 구체 타입 하나로 컴파일되어야 하므로 컴파일러가 멤버 이름만 보고 타입을 거꾸로 찾아 주지 않는다. 타입 매개변수에 멤버 제약을 걸 수 있는 것은 호출 지점마다 코드를 새로 만드는 `inline` 함수뿐이다.

```
// nights: PetHotel.Nights -> decimal
let estimateFee (nights: Nights) =
    decimal nights.Value * 35000M

Nights.Create 4 |> Result.map estimateFee |> printfn "%A" // Ok 140000M
```

## 보충: 타입 주석 없이 `.Value` 를 쓰는 방법 (노트 보충)

타입 주석을 달지 않고도 멤버 조회를 컴파일하는 길이 하나 있다. 함수를 `inline` 으로 선언하고 타입 매개변수에 멤버 제약(member constraint)을 직접 적는 것이다. 코드에는 `'a` 로 적지만 `inline` 함수의 타입 매개변수는 호출 지점마다 확정되므로, FSI 시그니처에는 정적으로 확인되는 타입 매개변수(statically resolved type parameter, SRTP)를 뜻하는 `^a` 로 나타난다. `Value: int` 프로퍼티가 있는 타입이면 무엇이든 받는다.

```
// nights: ^a -> decimal when ^a: (member Value: int)
let inline nightlyFee (nights: 'a when 'a: (member Value: int)) =
    decimal nights.Value * 35000M

Nights.Create 2 |> Result.map nightlyFee |> printfn "%A" // Ok 70000M
```

`inline` 만 붙이고 `nights.Value` 라고 쓰면 여전히 오류 FS0072 다. 제약을 손으로 적어야 추론이 성립한다. 도메인 코드에서 이렇게까지 할 이유는 거의 없다. 여기서 얻을 교훈은 프로퍼티 방식이 타입 추론과 잘 맞물리지 않는다는 점이고, 그래서 다음 절의 모듈 방식이 더 편하다.

## 타입에 동작을 붙이면 (원서 pp.124-125)

- 판별 유니온이나 레코드에도 멤버를 붙일 수 있다. 할인율 계산 같은 규칙을 데이터 정의 옆에 두면 재사용하기 좋다.
- 다만 멤버로 옮기는 과정에서 규칙이 쪼개지기 쉽다. 원서는 할인율을 고객 타입의 멤버로 뽑았다가 "고객 등급과 결제 금액이 함께 걸리는 규칙"이 끊어지는 것을 보이고, 두 값을 함께 받는 멤버로 다시 고친다.
- 결론은 멤버를 쓰지 말자는 쪽이다. 데이터와 동작을 섞지 않는 편이 낫다는 것이 원서의 권고다. 규칙이 아니라 강한 안내라고 못을 박는다.

```
// 데이터 정의에 동작을 붙인 형태. 7박 이상이면 10% 할인이다
type Reservation =
  { Nights: Nights
    Pets: int }
  with
    // member DiscountRate: decimal
    member this.DiscountRate =
      if this.Nights.Value >= 7 then 0.1M else 0.0M

match Nights.Create 8 with
| Ok n -> printfn "할인율: %M" { Nights = n; Pets = 2 }.DiscountRate // 할인율: 0.1
| Error _ -> ()
```

이 형태 자체는 잘 돌아간다. 문제는 요금 규칙이 늘어날수록 레코드 정의가 계산 코드로 뒤덮인다는 점이다. 다음 절이 대안이다.

## Using Modules — 모듈로 옮기기 (원서 pp.125-127)

- 타입과 같은 이름의 모듈을 만들어 함수를 그 안에 둔다. `List`, `Option`, `Result` 가 모두 이 구조다. F# 코드 전반의 관례이므로 읽는 사람에게 설명이 필요 없다.
- 타입 이름과 모듈 이름을 같게 두어도 컴파일 오류가 나지 않는다. 컴파일된 이름이 겹치지 않도록 컴파일러가 모듈 쪽에 `Module` 접미사를 붙이기 때문이다(실측한 컴파일 이름은 `PetHotel+Nights` 와 `PetHotel+NightsModule` 이다). F# 코드에서는 둘을 같은 이름으로 쓰고, 컴파일러가 `Nights.value` 는 모듈 함수로, `Nights.Create` 는 타입의 정적 멤버로 갈라 찾아 준다.
- 모듈 함수는 정의 타입의 인스턴스를 마지막 매개변수로 받는 것이 관례다. `List.map f list` 처럼 파이프 라인 끝에 값이 흘러 들어오게 하려는 것이다.
- `private` 케이스는 그 타입을 담은 모듈 안에서만 보인다. 감싸는 모듈 하나를 두고 그 안에 타입과 같은 이름의 모듈을 같이 넣으면, 바깥에서는 `create` 를 거치지 않고 값을 만들 수 없다.
- 원서 p.126 의 `Spend.Value spend` 는 오기다. 그 시점 `Spend` 에는 인스턴스 프로퍼티만 있고 같은 이름의 모듈이 아직 없으므로 `spend.Value` 여야 한다(그대로 적으면 오류 FS0806). 모듈로 옮긴 p.127 부터는 소문자 `Spend.value spend` 다.

```
// 이 단위가 보여주는 것: 타입과 같은 이름의 모듈로 생성 경로와 값 접근을 모으기
type ValidationError =
  | OutOfRange of string

module PetHotel =

  type Nights = private Nights of int

  module Nights =
    // Nights -> int
    let value (Nights n) = n

    // input: int -> Result<Nights,ValidationError>
    let create input =
      if input >= 1 && input <= 30 then Ok (Nights input)
      else Error (OutOfRange "숙박 일수는 1박 이상 30박 이하여야 한다")
```

`Nights.value` 는 매개변수 자리에서 값을 바로 분해한다. 프로퍼티 방식과 달리 이쪽은 시그니처가 `Nights -> int` 로 확정되므로, 이 함수를 쓰는 코드에는 타입 주석이 필요 없다. 앞 절에서 본 오류 FS0072 를 만나지 않는다.

예약 레코드는 새 모듈에 두고, 그 레코드와 같은 이름의 모듈을 함께 넣는다. 마리 수는 아직 `int` 로 남겨 둔다.



```

module Booking =

  open PetHotel

  type Reservation =
    { Nights: Nights
      Pets: int }

  module Reservation =
    let nightlyRatePerPet = 35000M

    // reservation: Reservation -> decimal
    let discountRate reservation =
      if Nights.value reservation.Nights >= 7 then 0.1M else 0.0M

    // reservation: Reservation -> decimal
    let fee reservation =
      let nights = decimal (Nights.value reservation.Nights)
      let pets = decimal reservation.Pets
      nights * pets * nightlyRatePerPet * (1.0M - discountRate reservation)

```

- 모듈 이름을 `Booking` 으로 새로 뒀다. 한 스크립트 안에서 `PetHotel` 을 한 번 더 선언해 내용을 이어 붙일 수는 없다. 같은 이름의 모듈을 두 번 적으면 오류 FS0037( 형식, 예외 또는 모듈입니다. 'PetHotel'의 정의가 중복되었습니다. )이 난다. 앞 절의 `PetHotel` 은 따로 실행되는 코드이므로 이 절과 부딪히지 않는다.
- 이렇게 나누면 경계가 오히려 분명해진다. `Booking` 은 `PetHotel` 밖이므로 `private` 케이스가 보이지 않고, `Nights.value` 와 `Nights.create` 만 쓸 수 있다. `Booking` 을 `PetHotel` 안에 중첩 모듈로 넣으면 `private` 케이스가 다시 보이므로 `create` 를 건너뛴 값을 만들 수 있다. 검증을 통제하려면 쓰는 쪽이 그 모듈 밖에 있어야 한다.
- `discountRate` 와 `fee` 는 둘 다 `Reservation` 을 마지막(이자 유일한) 매개변수로 받는다. 그래서 파이프 라인에 그대로 얹힌다.

```

open PetHotel
open Booking

// nights: int -> pets: int -> Result<decimal, Validation Error>
let quote nights pets =
  Nights.create nights
  |> Result.map (fun n -> { Nights = n; Pets = pets })
  |> Result.map Reservation.fee

printfn "%A" (quote 3 2) // Ok 210000.0M
printfn "%A" (quote 7 1) // Ok 220500.0M
printfn "%A" (quote 0 1) // Error (OutOfRange "숙박 일수는 1박 이상 30박 이하여야 한다")

```

7박 1마리는 245000 에서 10% 를 뺀 220500 이다. `(1.0M - discountRate reservation)` 을 곱하므로 소수 자릿수가 하나 늘어 `220500.0M` 으로 표시된다.

## A Few Minor Improvements — 남은 원시 타입 정리 (원서 pp.127-129)

- 원서는 마지막으로 남은 원시 타입을 정리한다. 고객 식별자로 쓰던 `string` 을 단일 케이스 판별 유니온으로 감싸고, 계산 결과에 타입 약어를 붙인다.
- 이 노트에서 남은 것은 두 개다. 마리 수가 아직 `int` 이고, 손님 식별자( `GuestId` )가 아직 없다. 마리 수는 검증이 필요하니 `private` 케이스로, 손님 식별자는 범위 제약이 없으니 그냥 감싸기만 한다.
- 검증이 두 군데로 늘어나면 `Result` 두 개를 하나로 합쳐야 한다. 여기서는 튜플 패턴으로 직접 처리한다. 오류를 모아서 한꺼번에 보고하는 방법은 8챕터의 주제다.
- 코드량은 시작할 때보다 분명히 늘었다. 얻는 것은 잘못된 값이 도메인 안으로 들어올 수 없다는 보증이다.

```
// 이 단위가 보여주는 것: 남은 원시 타입까지 감싼 완성 형태
type ValidationError =
  | OutOfRange of string

module PetHotel =

  type GuestId = GuestId of string
  type Fee = decimal
  type DiscountRate = decimal

  type Nights = private Nights of int

  module Nights =
    // Nights -> int
    let value (Nights n) = n

    // input: int -> Result<Nights,ValidationError>
    let create input =
      if input >= 1 && input <= 30 then Ok (Nights input)
      else Error (OutOfRange "숙박 일수는 1박 이상 30박 이하여야 한다")

  type PetCount = private PetCount of int

  module PetCount =
    // PetCount -> int
    let value (PetCount n) = n

    // input: int -> Result<PetCount,ValidationError>
    let create input =
      if input >= 1 && input <= 4 then Ok (PetCount input)
      else Error (OutOfRange "한 예약에 맡길 수 있는 마리 수는 1에서 4까지다")
```

`GuestId` 는 범위 검증이 없으니 케이스를 공개해 둔다. `private` 은 검증을 강제할 필요가 있을 때만 쓰는 장치다. `Fee` 와 `DiscountRate` 는 타입 약어이므로 새 타입이 아니고, 시그니처를 읽기 좋게 만드는 역할만 한다.

```

module Booking =

    open PetHotel

    type Reservation =
        { GuestId: GuestId
          Nights: Nights
          PetCount: PetCount }

    module Reservation =
        let nightlyRatePerPet = 35000M

        // reservation: Reservation -> PetHotel.DiscountRate
        let discountRate reservation : DiscountRate =
            if Nights.value reservation.Nights >= 7 then 0.1M else 0.0M

        // reservation: Reservation -> PetHotel.Fee
        let fee reservation : Fee =
            let nights = decimal (Nights.value reservation.Nights)
            let pets = decimal (PetCount.value reservation.PetCount)
            nights * pets * nightlyRatePerPet * (1.0M - discountRate reservation)

        // guestId: string -> nights: int -> petCount: int -> Result<Reservation,ValidationError>
        let create guestId nights petCount =
            match Nights.create nights, PetCount.create petCount with
            | Ok n, Ok p -> Ok { GuestId = GuestId guestId; Nights = n; PetCount = p }
            | Error e, _ -> Error e
            | _, Error e -> Error e

```

`create` 는 두 스마트 생성자를 호출하고 결과를 튜플로 묶어 한 번에 분해한다. 성공 케이스가 하나뿐이므로 나머지 두 줄이 실패를 받아 낸다. 반환 타입은 실측으로 `Result<Reservation,ValidationError>` 다.

```

open PetHotel
open Booking

// guestId: string -> nights: int -> petCount: int -> Result<PetHotel.Fee,ValidationError>
let quote guestId nights petCount =
    Reservation.create guestId nights petCount
    |> Result.map Reservation.fee

printfn "%A" (quote "G-001" 3 2) // Ok 210000.0M
printfn "%A" (quote "G-002" 7 1) // Ok 220500.0M
printfn "%A" (quote "G-003" 0 1) // Error (OutOfRange "숙박 일수는 1박 이상 30박 이하여야 한다")
printfn "%A" (quote "G-004" 5 9) // Error (OutOfRange "한 예약에 맡길 수 있는 마리 수는 1에서 4까지다")

```

호출하는 쪽이 다루는 값은 `int` 와 `string` 이고, 도메인 안으로 들어가는 순간 검증된 타입으로 바뀐다. 검증 지점이 `create` 한 군데로 모이는 것이 이 구조의 이득이다.

## Using a Record Type — 레코드로도 된다 (원서 p.129)

- 같은 일을 레코드로도 할 수 있다. 필드를 하나만 둔 레코드에 `private` 을 붙이면 단일 케이스 판별 유니온과 역할이 같아진다.
- 값을 꺼낼 때 패턴 분해가 아니라 필드 접근을 쓴다는 점만 다르다. 필드 이름을 타입 이름과 같게 두는 것이 원서 표기다. 필드 이름을 `Value` 로 두어 `input.Value` 로 읽는 형태도 흔하다. 여기서는 원서 표기를 따랐다.
- 어느 쪽을 쓸지는 취향이다. F# 커뮤니티에서 더 자주 보이는 것은 단일 케이스 판별 유니온이다.

```
// 이 단위가 보여주는 것: 레코드로 만든 같은 구조, 그리고 [<Struct>] 를 붙였을 때의 차이
type ValidationError =
    | OutOfRange of string

module Kennel =

    type Nights = private { Nights: int }

    module Nights =
        // input: Nights -> int
        let value input = input.Nights

        // input: int -> Result<Nights,ValidationError>
        let create input =
            if input >= 1 && input <= 30 then Ok { Nights = input }
            else Error (OutOfRange "숙박 일수는 1박 이상 30박 이하여야 한다")

    open Kennel
    printfn "%A" (Nights.create 5 |> Result.map Nights.value) // Ok 5
```

## 보충: [<Struct>] 를 붙이면 (노트 보충)

- 단일 케이스 판별 유니온은 기본적으로 참조 타입이다. `int` 하나를 감싸기 위해 힙에 객체가 하나 생긴다. 값을 많이 만드는 코드에서는 이 비용이 눈에 보일 수 있다.
- [<Struct>] 특성을 붙이면 값 타입이 된다. 패턴 분해, 구조적 동등성, `private` 케이스가 모두 그대로 작동한다.

```
[<Struct>]
type Weight = Weight of decimal
type Boxed = Boxed of decimal

printfn "Weight 값 타입인가: %b" (typeof<Weight>.IsValueType) // Weight 값 타입인가: true
printfn "Boxed 값 타입인가: %b" (typeof<Boxed>.IsValueType) // Boxed 값 타입인가: false

let (Weight kg) = Weight 4.5M
printfn "분해: %M" kg // 분해: 4.5
printfn "동등성: %b" (Weight 4.5M = Weight 4.5M) // 동등성: true
```

대가가 하나 있다. 값 타입에는 항상 기본값이 있으므로, 검증을 건너뛴 값이 만들어질 틈이 생긴다. 참조 타입 쪽은 같은 자리에서 `null` 이 나오고, 값 타입 쪽은 겉보기에 정상인 값이 나온다.

```
[<Struct>]
type StructNights = StructNights of int
type RefNights = RefNights of int

let structSlots : StructNights[] = Array.zeroCreate 1
printfn "struct 기본값: %A" structSlots[0] // struct 기본값: StructNights 0

let refSlots : RefNights[] = Array.zeroCreate 1
printfn "ref 기본값이 null 인가: %b" (isNull (box refSlots[0])) // ref 기본값이 null 인가: true
```

`StructNights 0` 은 `create` 를 거치지 않고 나온 값이고 검증 범위 밖이다. 두 경우 모두 정상적인 F# 코드에서는 만나지 않는 우회로이지만, [<Struct>] 쪽은 실패가 눈에 덜 띈다는 점을 알아 둘 만하다.

## Summary — 원서의 챕터 요약 (원서 p.129)

- 원시 타입을 줄이고 도메인 이름이 붙은 타입을 늘리면 코드가 견고해지고 읽기 좋아진다는 것이 이 챕터의 결론이다.
- 단일 케이스 판별 유니온을 쓰면 원시 타입을 그대로 쓸 때보다 담을 수 있는 값의 범위를 좁힐 수 있다.
- 원서는 타입을 확장하는 방법이 둘이라는 것도 정리한다. 멤버를 붙이는 방법과 같은 이름의 모듈에 함수를 두는 방법이다.
- 다음 챕터에서는 F# 의 객체 프로그래밍을 다룬다.

## 정리 — 이 노트의 요약

- 타입 약어는 시그니처를 읽기 좋게 만들지만 새 타입이 아니다. 원래 타입이 같으면 서로 바꿔 넣어도 통과하고 범위 검증도 없다. 이름만 필요할 때 쓰는 도구다.
- 단일 케이스 판별 유니온은 진짜 새 타입이다. `type Nights = Nights of int` 형태로 적고, 케이스가 하나라는 뜻에서 앞의 `|` 를 생략하는 것이 관례다. 원래 타입이 같은 두 값을 뒤바꿔 넣으면 오류 FS0001 이 난다.
- 값을 꺼내는 방법은 네 가지다. `let (Nights n) = nights` 로 분해, 매개변수 자리에서 `(Nights nights)` 로 분해, `member this.Value` 를 붙여 `.Value` 로 읽기, 그리고 같은 이름의 모듈에 `let value (Nights n) = n` 을 두고 `Nights.value` 로 부르기다. 매개변수 자리 분해와 모듈 함수는 시그니처가 `Nights -> int` 로 확정되어 추론이 잘 붙고, `.Value` 만 타입 주석을 요구한다.
- `.Value` 프로퍼티는 타입 추론과 잘 맞물리지 않는다. 타입 주석 없는 매개변수에 `.Value` 를 쓰면 오류 FS0072 다. 매개변수에 타입 주석을 달거나, `inline` 함수의 타입 매개변수에 `'a when 'a: (member Value: int)` 제약을 손으로 적어야 한다. 실측 시그니처는 `nights: ^a -> decimal when ^a: (member Value: int)` 다.
- 케이스 식별자에 `private` 을 붙이면 그 타입을 담은 모듈 밖에서는 값 생성과 패턴 분해가 모두 막히고, 위반하면 오류 FS1093 이다. 막히는 기준은 "모듈 밖"이므로 스크립트 최상위에 선언하면 파일 전체가 그 모듈이라 아무것도 막히지 않고, 감싸는 모듈 안의 중첩 모듈도 케이스를 그대로 본다. 타입과 스마트 생성자만 담은 모듈을 두고 쓰는 코드는 그 밖에 두어야 경계가 선다.
- 스마트 생성자는 `int -> Result<Nights, ValidationError>` 처럼 실패 가능성을 시그니처에 드러낸다. 이 관문을 지나온 값은 언제나 유효하므로, 값을 쓰는 쪽에서 범위를 다시 검사할 이유가 없다.
- 함수를 어디에 둘지는 두 갈래다. 타입 멤버로 붙이면 데이터 정의 옆에 규칙이 모이지만 데이터와 동작이 섞인다. 타입과 같은 이름의 모듈에 두면 `List · Option` 과 같은 구조가 되고, 정의 타입을 마지막 매개변수로 받으면 파이프라인에 그대로 얹힌다. 원서는 후자를 권한다.
- 같은 구조를 필드 하나짜리 `private` 레코드로도 만들 수 있다. 값 접근이 필드 읽기로 바뀌는 것뿐이다.
- `[<Struct>]` 를 붙이면 값 타입이 되어 힙 할당이 사라진다. 패턴 분해, 구조적 동등성, `private` 케이스는 그대로 작동한다. 대신 기본값이 존재하므로 `Array.zeroCreate` 같은 경로로 검증을 거치지 않은 값이 나올 수 있다.
- 얻는 것과 치르는 것을 견줘 보면, 코드량이 늘어나는 대신 검증 지점이 한곳으로 모이고 잘못된 값이 도메인에 들어오지 못한다. 도메인 값에 범위가 있고 그 값이 여러 곳으로 흐른다면 해 볼 만한 거래다.

## 원서 대조 표

| 절                                                                  | 원서 페이지     | 실행 단위         |
|--------------------------------------------------------------------|------------|---------------|
| Setting Up — 준비                                                    | p.118      | —             |
| Solving the Problem / 1단계: 타입 약어로 시그니처에 이름 붙이기                     | pp.119-120 | 09-abbrev     |
| Solving the Problem / 타입 약어가 막지 못하는 것                              | p.120      | 09-abbrev     |
| Solving the Problem / 2단계: 단일 케이스 판별 유니온                           | pp.120-122 | 09-singlecase |
| Solving the Problem / 값을 꺼내는 방법                                    | pp.121-122 | 09-singlecase |
| Solving the Problem / 3단계: private 케이스와 스마트 생성자                    | pp.122-124 | 09-smartctor  |
| Solving the Problem / 타입 주석 없이 <code>.value</code> 를 쓰는 방법 (노트 보충) | —          | 09-smartctor  |
| Solving the Problem / 타입에 동작을 붙이면                                  | pp.124-125 | 09-smartctor  |
| Using Modules — 모듈로 옮기기                                            | pp.125-127 | 09-module     |
| A Few Minor Improvements — 남은 원시 타입 정리                             | pp.127-129 | 09-final      |
| Using a Record Type — 레코드로도 된다                                     | p.129      | 09-record     |
| <code>[&lt;Struct&gt;]</code> 를 붙이면 (노트 보충)                        | —          | 09-record     |
| Summary — 원서의 챕터 요약                                                | p.129      | —             |

## 10 - 객체 프로그래밍 (원서 pp.130-140)

F# 은 함수 우선(functional-first) 언어이지만 객체를 쓸 수 있고, 쓰는 편이 나은 자리도 있다. 원서가 드는 이유는 두 가지다. 함수형 색이 덜한 다른 .NET 언어로 쓴 코드와 맞물려야 할 때, 그리고 내부 자료구조나 가변 상태를 밖에서 못 만지게 감싸고 싶을 때다. 이 챕터는 그 목적에 필요한 만큼만 골라 다룬다. 클래스 타입, 인터페이스, 캡슐화, 동등성 네 가지다. 원서는 F# 이 C#/VB.NET 의 객체 기능을 거의 다 할 수 있다고 적으면서도, 기능 목록을 늘어놓는 대신 "함수만으로는 안 되는 자리에서 무엇을 꺼내 쓰는가"만 본다.

### Setting Up — 준비 (원서 p.130)

- 원서는 새 폴더에 스크립트 세 개( `FizzBuzz.fsx` , `RecentlyUsedList.fsx` , `Coordinate.fsx` )를 만들어 두고 절마다 다른 파일에서 실습하게 한다. FSI 로 조각조각 실행하는 방식이라 프로젝트는 필요하지 않다.
- 이 노트는 원서와 겹치지 않는 도메인 하나를 챕터 전체에 깎다. 설비 점검 일정이다. 점검 주기 규칙표를 받아 며칠째에 어떤 점검이 걸리는지 알려주는 객체를 만들고, 거기에 인터페이스·캡슐화·동등성을 차례로 엮는다.
- 실행 단위가 나뉘어 있으므로 단위끼리는 서로의 타입을 볼 수 없다. 한 단위 안에서 같은 개념을 단계별로 보여줄 때는 단계마다 타입 이름을 달리 둔다.

## 언제 객체를 쓰는가 (원서 p.130)

- 원서의 판단 기준은 상호운용(interop)과 캡슐화다. 4챕터에서 본 대로 함수를 담는 그릇으로는 모듈이 기본이고, 클래스 타입은 상태와 동작을 한 덩어리로 묶어야 할 때 꺼낸다.
- 3챕터가 .NET 라이브러리를 F# 에서 부르는 쪽( TryParse , 예외 처리)을 다뤘다면, 이 챕터는 방향이 반대다. C# 쪽에서 자연스럽게 쓰이는 모양을 F# 으로 내놓는 법이다. 동등성 절의 op\_Equality 가 그 예다.
- 원서는 인터페이스 절 끝에서 한 번 제동을 건다. 함수 하나짜리 인터페이스를 만들고 있다면 그 추가 코드와 복잡도가 정말 필요한지, 그냥 함수로 충분하지 않은지 자문하라는 것이다(원서 p.134). 6챕터에서 함수를 매 개변수로 넘겨 가짜(fake)를 끼워 넣던 방식과 같은 이야기다.
- 즉 원서의 태도는 "객체를 피하라"가 아니라 "객체가 제 몫을 하는 자리인지 확인하고 쓰라"다. 이 챕터의 예제 대부분은 그 확인을 통과하는 자리, 곧 가변 상태를 감싸거나 .NET 계약을 구현하는 자리다.

## Class Types — 클래스 타입 (원서 pp.130-132)

- 클래스 타입(class type) 선언은 type 이름() = 으로 시작한다. 타입 이름 뒤의 괄호는 생략할 수 없다. 이 괄호가 생성자(constructor) 자리이고, 안에 생성자 매개변수를 적는다. 타입 이름 뒤 괄호로 선언하는 이 생성자가 주 생성자(primary constructor)다.
- member 키워드가 밖에서 볼 수 있는 멤버(member)를 정의한다. member 다음의 \_ 는 인스턴스 자신을 받는 자기 식별자(self identifier)다. 이름은 무엇이든 쓸 수 있지만 관례는 \_ 아니면 this 둘 중 하나다.
- 인스턴스를 만들 때 C# 과 달리 new 를 쓰지 않는다. IDisposable 을 구현한 타입만 예외인데, 그쪽은 let 대신 use 로 바인딩하고 new 도 함께 적는다. new 를 빠뜨려도 컴파일은 되고 경고만 뜬다(이 노트 마지막 절).
- 멤버를 호출할 때 인자 괄호는 생략할 수 있지만 적어 두는 편이 낫다. 생성자와 메서드의 인자는 튜플이고, 괄호가 있으면 함수 적용이 아니라 객체 멤버 호출임이 눈에 보인다. 원하면 |> 로 넘겨도 된다.

```
// 이 단위가 보여주는 것: 클래스 타입 선언, 주 생성자, 멤버 호출, 멤버의 평가 시점
type FixedSchedule() =
    member _.Label(day) =
        let hits =
            [(7, "주간"); (30, "월간")]
            |> List.filter (fun (period, _) -> day % period = 0)
            |> List.map snd
        if List.isEmpty hits then $"D{day}" else String.concat "+" hits
```

FSI 가 보여주는 이 타입의 모양은 아래와 같다. 매개변수 없는 생성자(default constructor)가 unit -> FixedSchedule 로 나오고, 멤버는 day: int -> string 으로 추론된다. day % period 와 \$"D{day}" 만으로 int 가 확정된다.

```
type FixedSchedule =
    new: unit -> FixedSchedule
    member Label: day: int -> string
```

```
let schedule = FixedSchedule()

printfn "%s" (schedule.Label(7))      // 주간
printfn "%s" (schedule.Label(30))     // 월간
printfn "%s" (schedule.Label(210))    // 주간+월간
printfn "%s" (schedule.Label(4))      // D4
printfn "%s" (210 |> schedule.Label)  // 주간+월간
```

규칙표가 코드 안에 박혀 있으면 쓸 데가 하나뿐이다. 규칙표를 생성자 매개변수로 받으면 같은 클래스로 여러 일정을 만들 수 있다. 생성자 매개변수는 `let` 바인딩으로 다시 받아 둘 필요 없이 클래스 본문 어디서나 그대로 보인다.

```
type RuleSchedule(rules) =
    member _.Label(day) =
        let hits =
            rules
            |> List.filter (fun (period, _) -> day % period = 0)
            |> List.map snd
        if List.isEmpty hits then $"D{day}" else String.concat "+" hits

// rules: (int * string) list -> days: int list -> string list
let labelAll rules days =
    let schedule = RuleSchedule(rules)
    days |> List.map schedule.Label

labelAll [(3, "여과기"); (5, "윤활")] [1..6]
|> String.concat " / "
|> printfn "%s"
// D1 / D2 / 여과기 / D4 / 윤활 / 여과기
```

`List.map (fun n -> schedule.Label(n))` 대신 `List.map schedule.Label` 로 적었다. 멤버도 함수 값으로 넘길 수 있다. 다만 멤버의 매개변수는 튜플로 묶이므로, 매개변수가 둘인 멤버를 이렇게 넘기면 값의 타입이 `int * int -> int` 처럼 튜플 하나를 받는 함수가 된다. 커링된 함수를 기대하는 자리에는 람다로 감싸 넘겨야 한다.

멤버 본문이 길어지면 계산을 클래스 본문의 내부 함수로 빼고 멤버는 그 함수를 부르게 하는 편이 읽기 좋다. 내부 `let` 은 밖에서 보이지 않으므로 공개하는 이름과 구현이 분리된다.

```
type MaintenanceSchedule(rules) =
    let label day =
        let hits =
            rules
            |> List.filter (fun (period, _) -> day % period = 0)
            |> List.map snd
        if List.isEmpty hits then $"D{day}" else String.concat "+" hits

    member _.Label(day) = label day
    member _.RuleCount = List.length rules

let plan = MaintenanceSchedule([(3, "여과기"); (5, "윤활")])
printfn "규칙 %d개, D15 -> %s" plan.RuleCount (plan.Label 15)
// 규칙 2개, D15 -> 여과기+윤활
```

## 클래스 본문의 평가 시점 (노트 보충)

클래스 본문의 `let` 바인딩과 `do` 블록은 주 생성자의 본문이다. 인스턴스를 만들 때 위에서 아래로 한 번 실행되고, 그 결과가 인스턴스 안에 남는다. 반면 `member` 본문은 접근할 때마다 다시 실행된다. 아래처럼 표시를 찍어 보면 순서가 그대로 보인다.



```

type ScheduleProbe(rules) =
  let ruleCount =
    printfn " [let] 바인딩 평가"
    List.length rules

  do printfn " [do] 블록 실행 (규칙 %d개)" ruleCount

  member _.RuleCount = ruleCount
  member _.CountedNow =
    printfn " [member] 본문 평가"
    List.length rules

printfn "인스턴스를 만들기 전"
let probe = ScheduleProbe([(3, "여과기"); (5, "윤활")])
printfn "인스턴스를 만든 뒤"
printfn "RuleCount 1회: %d" probe.RuleCount
printfn "RuleCount 2회: %d" probe.RuleCount
printfn "CountedNow 1회: %d" probe.CountedNow
printfn "CountedNow 2회: %d" probe.CountedNow
// 인스턴스를 만들기 전
// [let] 바인딩 평가
// [do] 블록 실행 (규칙 2개)
// 인스턴스를 만든 뒤
// RuleCount 1회: 2
// RuleCount 2회: 2
// [member] 본문 평가
// CountedNow 1회: 2
// [member] 본문 평가
// CountedNow 2회: 2

```

`RuleCount` 는 생성 시점에 계산된 값을 읽기만 하므로 표시가 찍히지 않는다. `CountedNow` 는 읽을 때마다 본문을 다시 돈다. 비싼 계산을 멤버 본문에 그대로 두면 접근 횟수만큼 되풀이된다는 뜻이다.

`member val` 은 이 둘 사이에 있다. 오른쪽 식을 생성 시점에 한 번만 평가하고 그 값을 담아 두는 자동 프로퍼티(auto-property)를 만든다. 뒤에 `with get, set` 을 붙이면 밖에서 바꿀 수 있는 프로퍼티(property)가 되고, 붙이지 않으면 읽기 전용이다.

```

let mutable issued = 0

let issueTicket () =
  issued <- issued + 1
  issued

type Gauge() =
  member val Serial = issueTicket ()           // 생성 시 한 번
  member _.NextTicket = issueTicket ()         // 접근마다
  member val Location = "미지정" with get, set

let gauge = Gauge()
printfn "Serial 1회/2회: %d / %d" gauge.Serial gauge.Serial
printfn "NextTicket 1회: %d" gauge.NextTicket
printfn "NextTicket 2회: %d" gauge.NextTicket
printfn "Location 초기값: %s" gauge.Location
gauge.Location <- "3라인 압축기"
printfn "Location 변경 후: %s" gauge.Location
// Serial 1회/2회: 1 / 1
// NextTicket 1회: 2
// NextTicket 2회: 3
// Location 초기값: 미지정
// Location 변경 후: 3라인 압축기

```

FSI 로 재어 본 `Gauge` 의 모양은 아래와 같다. 선언에 `with get, set` 을 붙인 프로퍼티에만 시그니처에도 그 표기가 붙는다.

```
type Gauge =
  new: unit -> Gauge
  member Location: string with get, set
  member NextTicket: int
  member Serial: int
```

## Interfaces — 인터페이스 (원서 pp.132-134)

- 인터페이스(interface)는 구현이 지켜야 할 계약이다. 선언은 `abstract member 이름 : 시그니처` 를 늘어놓는 것으로 끝난다. 이름 앞의 `I` 는 문법이 요구하는 것이 아니라 .NET 관례다.
- 클래스가 인터페이스를 구현할 때는 `interface 이름 with` 블록 안에 멤버를 적는다. F# 에는 암시적 구현이 없다. 같은 이름의 멤버를 클래스 본문에 적어 두는 것으로 인터페이스가 채워지지 않는다. `interface IMaintenanceSchedule` 한 줄만 적고 멤버를 클래스 본문에 두면 `error FS0366` 이 뜬다. 구현되지 않은 인터페이스 멤버를 열거하고, 모든 인터페이스 멤버를 `interface ... with member ...` 선언에 나열하려고 이어 말한다. 구현하지 않으면 FS0366, 구현했지만 변환 없이 부르면 뒤에 나오는 FS0039 다.
- 그래서 구현한 멤버는 클래스 타입의 멤버가 아니다. 인스턴스에서 바로 부르려고 하면 컴파일되지 않는다. 인터페이스 타입으로 상향 변환(upcast, `:>`)해야 보인다.
- 원서는 이것을 F# 이 암시적 변환을 지원하지 않는 탓으로 설명하고, 컴파일러의 마법에 기대지 않으니 코드가 무엇을 하는지 확실해진다는 이점을 든다(원서 p.134). 원서의 이 서술은 F# 5 시점 기준이다. F# 6 이 몇 자리에 암시적 변환을 넣었으므로 그 범위는 뒤에서 갈라 본다. 멤버 조회 자리에는 여전히 암시적 변환이 없으니 이 절의 결론은 그대로다.

```
// 이 단위가 보여주는 것: 인터페이스 선언과 명시적 구현, 상향 변환이 필요한 이유
type IMaintenanceSchedule =
  abstract member Label : int -> string
  abstract member RuleCount : int

type RuleTable(rules: (int * string) list) =
  let label day =
    let hits =
      rules
      |> List.filter (fun (period, _) -> day % period = 0)
      |> List.map snd
    if List.isEmpty hits then $"D{day}" else String.concat "+" hits

  interface IMaintenanceSchedule with
    member _.Label(day) = label day
    member _.RuleCount = List.length rules
```

FSI 가 보여주는 `RuleTable` 의 모양이 상황을 그대로 설명한다. 생성자와 `interface IMaintenanceSchedule` 한 줄뿐이고 멤버 목록이 비어 있다.

```
type RuleTable =
  interface IMaintenanceSchedule
  new: rules: (int * string) list -> RuleTable
```

그러니 인스턴스에서 `.Label` 을 바로 부르면 오류다. 실측한 메시지는 `error FS0039: 'RuleTable' 형식은 'Label' 필드, 생성자 또는 멤버를 정의하지 않습니다.` 다.

```
let table = RuleTable([(3, "여과기")])
table.Label 3 // error FS0039
```

고치는 방법은 두 갈래다. 바인딩할 때 한 번 상향 변환해 두거나, 쓰는 자리에서 변환한다. 전자가 멤버를 여러 번 부를 때 깔끔하다.

```
// 바인딩 시점에 변환해 두기
let table = RuleTable([(3, "여과기"); (5, "윤활")]) :> IMaintenanceSchedule
println "규칙 %d개, D15 -> %s" table.RuleCount (table.Label 15)
// 규칙 2개, D15 -> 여과기+윤활

// 쓰는 자리에서 변환하기
let row = RuleTable([(2, "육안")])
println "D4 -> %s" ((row :> IMaintenanceSchedule).Label 4)
// D4 -> 육안
```

변환을 손으로 적지 않아도 되는 자리가 둘 있다. 인터페이스 타입으로 주석을 단 `let` 바인딩과, 인터페이스 타입 매개변수를 받는 함수의 인자 자리다. 뒤쪽은 오래된 동작이지만 앞쪽은 `--langversion:5.0` 에서 `error FS0001: 이 식에는 'IMaintenanceSchedule' 형식이 필요하지만 여기에서는 'RuleTable' 형식이 지정되었습니다.` 로 막힌다. 실측해 보면 F# 6 부터 통한다.

```
// 타입 주석이 붙은 바인딩 - F# 6 부터 암시적 상향 변환이 붙는다
let annotated : IMaintenanceSchedule = RuleTable([(2, "육안")])
println "D3 -> %s" (annotated.Label 3)
// D3 -> D3

// 인터페이스 타입 매개변수 자리 - 변환을 적지 않아도 된다
let report (s: IMaintenanceSchedule) days =
    days |> List.map s.Label |> String.concat " / "

println "%s" (report (RuleTable([(3, "여과기")])) [1..6])
// D1 / D2 / 여과기 / D4 / D5 / 여과기
```

함수를 매개변수로 받는 것과 인터페이스를 받는 것을 견줘 보면, 계약에 멤버가 둘 이상이고 그 묶음이 함께 움직일 때 인터페이스가 제 몫을 한다. `Label` 하나만 필요했다면 `int -> string` 함수를 받는 편이 코드가 짧다. 원서가 경계하는 지점이 정확히 그것이다.

## 추상 클래스와 상속 (원서 p.133 확장)

원서는 인터페이스에 `[<AbstractClass>]` 특성을 붙이면 추상 클래스(abstract class)가 된다고 한 줄로만 언급한다. 실제로 추상 클래스는 인터페이스와 달리 구현과 상태를 함께 담을 수 있다. `abstract member` 는 하위 타입(derived type)이 반드시 채워야 하고, `default` 를 붙여 기본 구현을 주면 하위 타입이 선택적으로 재정의(override)한다. F# 인터페이스에는 기본 구현도 상태도 담을 수 없다. 인터페이스 선언에 `default` 를 붙이면 그 타입이 클래스가 되어 다른 타입에서 `interface ... with` 로 구현할 때 `error FS0887: 'IGreeter' 형식은 인터페이스 형식이 아닙니다.` 가 나고, 인터페이스 본문에 `let` 을 적으면 `error FS0963` 이다. 둘 중 하나라도 필요하면 추상 클래스로 간다.

```
[<AbstractClass>]
type ScheduleBase(rules: (int * string) list) =
  member _.Rules = rules
  abstract member Label : int -> string
  abstract member Owner : string
  default _.Owner = "미배정"
  member this.Describe(day) = $"D{day} [{this.Owner}] -> {this.Label day}"

type LineSchedule(rules, owner) =
  inherit ScheduleBase(rules)

  override this.Label(day) =
    let hits = this.Rules |> List.filter (fun (period, _) -> day % period = 0) |> List.map snd
    if List.isEmpty hits then "없음" else String.concat "+" hits

  override _.Owner = owner

type SparseSchedule(rules) =
  inherit ScheduleBase(rules)
  override _.Label(_) = "확인 필요"

printfn "%s" (LineSchedule([(3, "여과기"); (5, "윤활")], "2조").Describe 15)
// D15 [2조] -> 여과기+윤활
printfn "%s" (SparseSchedule([]).Describe 1)
// D1 [미배정] -> 확인 필요
```

- 상속(inheritance)은 `inherit` 기반타입(인자) 한 줄로 선언한다. 기반 타입(base type)의 생성자를 여기서 부른다.
- 인터페이스와 달리 상속받은 멤버(`Rules`, `Describe`)는 하위 타입의 멤버로 그대로 보인다. 변환이 필요하지 않다.
- 생성자 매개변수가 튜플로 묶인다는 점이 시그니처에 드러난다. FSI 는 `LineSchedule` 의 생성자를 `new: rules: (int * string) list * owner: string -> LineSchedule` 로 보여준다. 커링된 매개변수가 아니라 `*` 로 이어진 튜플이다.

## Object Expressions — 객체 식 (원서 pp.134-136)

- 객체 식(object expression)은 클래스 타입을 선언하지 않고 인터페이스 구현 하나를 그 자리에서 만드는 문법이다. `{ new 인터페이스 with 멤버들 }` 형태다.
- 만들어지는 것은 이름 없는 타입이다. 그래서 그 타입 이름으로 무언가를 더 할 수는 없지만, 대신 바인딩의 정적 타입이 곧바로 인터페이스가 되어 상향 변환을 적을 일이 없다.
- 원서가 드는 용도는 두 가지다. 한 번만 쓰고 버릴 서비스, 그리고 테스트다. 6챕터에서 함수를 넘겨 가짜를 끼워 넣던 방식의 인터페이스판이다.

```
// 이 단위가 보여주는 것: 객체 식으로 인터페이스 구현을 그 자리에서 만들기
open System
open System.Globalization

type IClock =
    abstract member Now : DateTime
    abstract member Zone : string

// 클래스로 구현하면 쓸 때 상향 변환이 필요하다
type SystemClock() =
    interface IClock with
        member _.Now = DateTime.UtcNow
        member _.Zone = "UTC"

// 객체 식은 그 단계를 건너뛴다
let fixedClock =
    { new IClock with
        member _.Now = DateTime(2026, 3, 1, 9, 30, 0)
        member _.Zone = "UTC" }

printfn "IClock 구현인가: %b" (typeof<IClock>.IsAssignableFrom(fixedClock.GetType()))
printfn "고정 시각: %s" (fixedClock.Now.ToString("yyyy-MM-dd HH:mm", CultureInfo.InvariantCulture))
// IClock 구현인가: true
// 고정 시각: 2026-03-01 09:30
```

FSI로 재어 보면 `val fixedClock: IClock` 이다. 런타임 타입은 컴파일러가 붙인 이름 없는 타입이지만, 정적 타입은 인터페이스 그 자체다. `SystemClock() :> IClock` 처럼 변환을 적을 필요가 없는 까닭이 여기 있다.

인터페이스를 생성자 매개변수로 받는 클래스를 두면, 실제 구현과 가짜를 같은 자리에 끼울 수 있다. 아래 클래스는 내부에 가변 카운터를 감춰 둔다. 클래스 안의 `let mutable` 은 밖에서 보이지 않고, 읽기 전용 프로퍼티로만 드러난다.

```
type BatchStamper(clock: IClock) =
    let mutable stamped = 0

    member _.Stamp(label) =
        stamped <- stamped + 1
        sprintf "[%s %s] %s" (clock.Now.ToString("yyyy-MM-dd HH:mm", CultureInfo.InvariantCulture)) clock.Zone label

    member _.Stamped = stamped

let stamper = BatchStamper(fixedClock)
printfn "%s" (stamper.Stamp "압축기 점검")
printfn "%s" (stamper.Stamp "냉각수 보충")
printfn "찍은 횟수: %d" stamper.Stamped
// [2026-03-01 09:30 UTC] 압축기 점검
// [2026-03-01 09:30 UTC] 냉각수 보충
// 찍은 횟수: 2
```

시각이 고정되어 있으니 출력이 결정적이다. 다만 시각을 고정한 것만으로는 문자열까지 고정되지 않는다.

`DateTime` 을 문자열로 바꿀 때 문화권을 넘기지 않으면 실행 환경의 문화권이 쓰이고, 달력과 시간 구분 기호가 함께 갈린다. `IClock` 이 내놓는 것은 `DateTime` 이고 그것을 어떻게 적을지는 별개의 결정이므로, 가짜 시계를 끼워도 서식 쪽을 `CultureInfo.InvariantCulture` 로 못 박아야 출력이 어느 환경에서나 같다. 실제 시계를 넣으면 같은 코드가 현재 시각을 찍는다.

```
let live = SystemClock() :> IClock
printfn "시스템 시계가 2020년 이후인가: %b" (live.Now.Year >= 2020)
// 시스템 시계가 2020년 이후인가: true
```

객체 식은 만들어지는 자리의 지역 값을 붙잡아 둘 수 있다. 2챕터에서 본 클로저와 같은 성질이다. `let mutable` 도 붙잡히므로, 호출할 때마다 값이 바뀌는 가짜를 함수 하나로 찍어낼 수 있다. 테스트에서 시간이 흐르는 상황을

재현할 때 쓴다.

```
// start: System.DateTime -> stepMinutes: float -> IClock
let steppingClock (start: DateTime) (stepMinutes: float) =
    let mutable current = start
    { new IClock with
        member _.Now =
            current <- current.AddMinutes stepMinutes
            current
        member _.Zone = "UTC" }

let stepping = steppingClock (DateTime(2026, 3, 1, 9, 0, 0)) 15.0
printfn "1회: %s" (stepping.Now.ToString("HH:mm", CultureInfo.InvariantCulture))
printfn "2회: %s" (stepping.Now.ToString("HH:mm", CultureInfo.InvariantCulture))
printfn "3회: %s" (stepping.Now.ToString("HH:mm", CultureInfo.InvariantCulture))
// 1회: 09:15
// 2회: 09:30
// 3회: 09:45
```

이 함수의 반환 타입이 `IClock` 이라는 점을 눈여겨볼 만하다. 함수 하나가 인터페이스 구현을 값으로 돌려준다. 클래스 타입을 하나 늘리지 않고도 구현을 갈아 끼울 수 있다.

## Encapsulation — 캡슐화 (원서 pp.136-138)

- 캡슐화(encapsulation)는 객체를 쓸 이유 중 원서가 가장 무게를 두는 쪽이다. 가변 컬렉션을 클래스 안에 감추고, 정해 둔 멤버로만 만지게 한다.
- 예제로 쓰는 자료구조는 정원(capacity)이 정해진 최근 이력이다. 이 노트에서 정원은 도메인이 정한 최대 크기이고, `ResizeArray` 의 `Capacity` 프로퍼티와는 다른 것이다. 최신 항목이 앞에 오고, 같은 항목을 다시 올리면 앞으로 옮겨 오며, 정원을 넘으면 가장 오래된 항목이 밀려 나간다. 원서는 "최근 사용 목록"으로, 이 노트는 편집기의 되돌리기 이력으로 만든다.
- 내부 저장소로 쓰는 `ResizeArray<'T>` 는 .NET 의 가변 `List<'T>` 에 붙은 F# 타입 약어(type abbreviation)다. 이름이 `List` 와 겹쳐 혼란을 부르므로 F# 에서는 이 이름을 쓴다.
- 클래스 본문의 `let` 은 밖에서 보이지 않는다. 저장소에 직접 손댈 길이 없으니 불변식(invariant), 곧 중복 없음과 최신 순, 그리고 뒤에서 더할 정원 상한을 깨뜨릴 수 있는 코드가 이 타입 안으로 한정된다. 가변 상태를 두고도 마음을 놓을 수 있는 근거가 그것이다.

```
// 이 단위가 보여주는 것: 가변 컬렉션을 클래스 안에 감추고 멤버로만 열어 주기
type StepLog() =
    let steps = ResizeArray<string>()

    let push step =
        steps.Remove step |> ignore
        steps.Add step

    let peek index =
        if index >= 0 && index < steps.Count then Some steps[steps.Count - index - 1]
        else None

    member _.IsEmpty = steps.Count = 0
    member _.Depth = steps.Count
    member _.Clear() = steps.Clear()
    member _.Push(step) = push step
    member _.TryPeek(index) = peek index
```

`push` 는 같은 항목을 먼저 지우고 다시 맨 뒤에 넣는다. 그래서 중복이 생기지 않고 최근에 올린 것이 항상 맨 뒤다. `peek` 은 인덱스를 뒤에서부터 세어 0 이 가장 최근이 되게 뒤집고, 범위를 벗어나면 예외 대신 `None` 을 준다.

FSI 가 보여주는 시그니처를 보면 어느 멤버가 프로퍼티이고 어느 것이 메서드인지 구분된다. `unit -> unit` 이 붙은 `Clear` 는 메서드이고, 타입만 적힌 `Depth` , `IsEmpty` 는 읽기 전용 프로퍼티다.

```
type StepLog =  
  new: unit -> StepLog  
  member Clear: unit -> unit  
  member Push: step: string -> unit  
  member TryPeek: index: int -> string option  
  member Depth: int  
  member IsEmpty: bool
```

```
let log = StepLog()  
printfn "처음에 비었나: %b" log.IsEmpty  
log.Push "글꼴 변경"  
log.Push "표 삽입"  
log.Push "글꼴 변경"           // 이미 있는 항목 -> 앞으로 옮겨진다  
printfn "깊이: %d" log.Depth  
printfn "가장 최근: %A" (log.TryPeek 0)  
printfn "그 다음: %A" (log.TryPeek 1)  
printfn "범위 밖: %A" (log.TryPeek 5)  
log.Clear()  
printfn "비운 뒤: %b / 깊이 %d" log.IsEmpty log.Depth  
// 처음에 비었나: true  
// 깊이: 2  
// 가장 최근: Some "글꼴 변경"  
// 그 다음: Some "표 삽입"  
// 범위 밖: None  
// 비운 뒤: true / 깊이 0
```

여기에 정원을 붙이고 밖으로 내놓는 멤버 목록을 인터페이스로 옮긴다. 정원은 생성자 매개변수로 받는다.

```

type IUndoHistory =
    abstract member IsEmpty : bool
    abstract member Depth : int
    abstract member Capacity : int
    abstract member Clear : unit -> unit
    abstract member Push : string -> unit
    abstract member TryPeek : int -> string option

type UndoHistory(capacity: int) =
    let steps = ResizeArray<string>(capacity)

    let push step =
        steps.Remove step |> ignore
        if steps.Count = capacity then steps.RemoveAt 0
        steps.Add step

    let peek index =
        if index >= 0 && index < steps.Count then Some steps[steps.Count - index - 1]
        else None

    interface IUndoHistory with
        member _.IsEmpty = steps.Count = 0
        member _.Depth = steps.Count
        member _.Capacity = capacity
        member _.Clear() = steps.Clear()
        member _.Push(step) = push step
        member _.TryPeek(index) = peek index

let history = UndoHistory(3) :> IUndoHistory
printfn "정원: %d" history.Capacity
["글꼴 변경", "표 삽입", "이미지 삽입", "여백 조정"] |> List.iter history.Push
printfn "깊이: %d" history.Depth
printfn "0번: %A" (history.TryPeek 0)
printfn "2번: %A" (history.TryPeek 2)
printfn "3번: %A" (history.TryPeek 3)
history.Push "표 삽입"
printfn "다시 올린 뒤 0번: %A / 깊이 %d" (history.TryPeek 0) history.Depth
// 정원: 3
// 깊이: 3
// 0번: Some "여백 조정"
// 2번: Some "표 삽입"
// 3번: None
// 다시 올린 뒤 0번: Some "표 삽입" / 깊이 3

```

네 항목을 올렸지만 깊이가 3에서 멈춘다. 정원이 찬 상태에서 새 항목이 들어오면 `RemoveAt 0` 이 가장 오래된 것을 밀어낸다.

한 가지 짚어 둘 것이 있다. 원서는 정원 판정에 `items.Capacity` 를 쓴다(원서 p.137). `ResizeArray` 의 `Capacity` 는 정원이 아니라 미리 확보해 둔 자리 수이고, 자리가 모자라면 늘어난다. 실측하면 이렇다.

```

let probe = ResizeArray<string>(0)
printfn "0으로 만든 Capacity: %d" probe.Capacity
probe.Add "항목"
printfn "하나 넣은 뒤 Capacity: %d / Count: %d" probe.Capacity probe.Count
printfn "3으로 만든 Capacity: %d" (ResizeArray<string>(3).Capacity)
// 0으로 만든 Capacity: 0
// 하나 넣은 뒤 Capacity: 4 / Count: 1
// 3으로 만든 Capacity: 3

```

`Capacity` 를 0으로 잡아 만든 뒤 항목 하나를 넣으면 그 값이 4로 된다. 생성자에 넘긴 값과 이렇게 갈라지므로 `Capacity` 는 정원의 이름이 될 수 없다. 위 코드가 생성자 인자를 그대로 붙잡아 두고 그것으로 판정하는 이유다.

원서 코드도 정원 1 이상에서는 실제로 돈다. `ResizeArray<'T>(n)` 이 딱 `n` 칸을 잡아 두고 판정이 `Count` 가 그 수를 넘는 것을 막기 때문에 `Capacity` 가 늘어날 일이 오지 않는다. 다만 그것은 `List<'T>` 구현이 그렇다는 사정일 뿐 계약이 아니다. 정원 0은 두 판 모두 빈 `ResizeArray` 에 `RemoveAt 0` 을 불러 `ArgumentOutOfRangeException` 이 난다. 정원에 하한이 필요하다면 생성자에서 검사하는 편이 낫다.



멤버 단위로 노출을 조절하고 싶으면 접근 지정자(access modifier)를 붙여 `member private this.X` 나 `member internal this.X` 로 좁힐 수 있다. 다만 캡슐화의 큰 몫은 이미 클래스 본문의 `let` 이 해 준다.

## Equality — 동등성 (원서 pp.138-140)

- F# 의 대다수 타입(레코드, 튜플, 판별 유니온)은 구조적 동등성(structural equality)을 기본으로 준다. 담긴 값이 같으면 `=` 가 참이다.
- 클래스 타입은 그렇지 않다. .NET 의 다른 대다수와 마찬가지로 참조 동등성(reference equality)을 쓴다. 같은 인스턴스를 가리킬 때만 참이다.
- 값처럼 견주고 싶으면 손으로 붙여야 한다. `GetHashCode` 와 `Equals` 를 재정의하고 `IEquatable<'T>` 를 구현한다. 다른 .NET 언어에서 `==` 로 쓰이게 하려면 `op_Equality` 정적 멤버를 더한다.  
`[<AllowNullLiteral>]` 은 목적이 다르다. 그 특성이 있어야 F# 코드가 이 타입 자리에 `null` 을 쓸 수 있고, 아래 예제의 `isNull` 도 그것 없이는 컴파일되지 않는다.

```
// 이 단위가 보여주는 것: 클래스는 참조 동등성이 기본이고, 구조적 동등성은 손으로 붙인다
open System

type Swatch(red: int, green: int, blue: int) =
    member _.Red = red
    member _.Green = green
    member _.Blue = blue

let s1 = Swatch(200, 40, 40)
let s2 = Swatch(200, 40, 40)
let s3 = s1

printfn "값이 같은 두 인스턴스: %b" (s1 = s2) // false
printfn "같은 인스턴스: %b" (s1 = s3)        // true
```

같은 세 숫자로 만들었는데 `s1 = s2` 가 거짓이다. 여기서 `=` 는 참조를 견준다. 아래는 같은 값을 담았으면 같다고 보게 만든 판이다.

```
// [<AllowNullLiteral>] 이 있어야 아래 isNull 이 컴파일된다
[<AllowNullLiteral>]
type Colour(red: int, green: int, blue: int) =
    let equals (other: Colour) =
        if isNull other then false
        else red = other.Red && green = other.Green && blue = other.Blue

    member _.Red = red
    member _.Green = green
    member _.Blue = blue

    override this.GetHashCode() = hash (this.Red, this.Green, this.Blue)

    override _.Equals(candidate) =
        match candidate with
        | :? Colour as other -> equals other
        | _ -> false

    interface IEquatable<Colour> with
        member _.Equals(other: Colour) = equals other

    static member op_Equality(left: Colour, right: Colour) = left.Equals(right)
```

- 실제 비교는 내부 함수 `equals` 하나에 모아 둔다. `Equals` 재정의와 `IEquatable<Colour>` 구현이 이 함수를 부르고, `op_Equality` 는 `Equals` 를 거쳐 같은 곳에 닿는다. 비교 규칙이 한곳에만 있으니 셋이 갈라질 일이 없다.
- `Equals(candidate)` 는 `obj` 를 받으므로 타입 테스트 패턴(type test pattern) `?: Colour as other` 로 좁힌다. 3챕터에서 예외를 걸러낼 때 쓴 것과 같은 패턴이다. 다른 타입이 들어오면 거짓이다.
- `isNull` 과 `hash` 는 F# 이 기본으로 주는 함수다. `hash` 에 튜플을 넘기면 구성 요소를 엮은 해시를 만들어 준다.
- 둘 중 하나만 재정의하면 컴파일러가 경고한다. `Equals` 만 재정의하고 `GetHashCode` 를 빼면 warning FS0346: 구조체, 레코드 또는 공용 구조체 형식 'HalfColour'에 'Object.Equals'의 명시적 구현이 있습니다. 로 시작하는 경고가 뜨고, `Object.GetHashCode()` 쪽 재정의도 맞춰 두라고 이어 말한다. 반대로 `GetHashCode` 만 재정의하면 warning FS0345: 구조체, 레코드 또는 공용 구조체 형식 'HalfColour2'에 'Object.GetHashCode'의 명시적 구현이 있습니다. 로 시작하는 경고가 뜨고, `Object.Equals(obj)` 쪽 재정의도 맞춰 두라고 이어 말한다. 두 메시지가 "구조체, 레코드 또는 공용 구조체 형식"이라고 말하지만 여기서 걸린 대상은 클래스 타입이다. 재정의 짝을 맞추지 않으면 해시가 어긋난 값을 디버거 키로 쓸 때 넣어 둔 값을 다시 찾지 못한다.
- `op_Equality` 는 F# 코드에서 쓰이지 않는다. F# 의 `=` 는 `Equals` 로 간다. C# 이나 VB.NET 에서 `==` 로 견줄 수 있게 하려고 내놓는 계약이다.
- `[<AllowNullLiteral>]` 은 상호운용 특성이 아니다. 다른 .NET 언어는 이 특성이 없어도 F# 클래스 타입 자리에 `null` 을 넘긴다. 이 특성이 여는 것은 F# 쪽이다. F# 은 자기가 선언한 클래스 타입에 `null` 리터럴을 허용하지 않으므로, 특성을 떼면 `isNull other` 가 error FS0001: 'Colour' 형식은 적절한 값으로 'null'을 가지지 않습니다. 로 막힌다. 원서 p.139 가 `op_Equality` 와 이 특성을 한 문장에 묶어 둔 것은 정확하지 않다.

두 경고를 손으로 확인해 볼 코드다. 경고가 나므로 검증 대상에 넣지 않는다.

```
// GetHashCode 를 빼면 warning FS0346 이 뜬다
type HalfColour(red: int) =
    member _.Red = red
    override _.Equals(candidate) =
        match candidate with
        | :? HalfColour as other -> red = other.Red
        | _ -> false
```

```
// 반대로 Equals 를 빼면 warning FS0345 가 뜬다
type HalfColour2(red: int) =
    member _.Red = red
    override this.GetHashCode() = hash this.Red
```

```
let c1 = Colour(200, 40, 40)
let c2 = Colour(200, 40, 40)

printfn "값이 같은 두 인스턴스: %b" (c1 = c2) // true
printfn "참조가 같은가: %b" (obj.ReferenceEquals(c1, c2)) // false
printfn "해시가 같은가: %b" (c1.GetHashCode() = c2.GetHashCode()) // true
printfn "null 과 비교: %b" (c1.Equals(null)) // false
printfn "List.distinct 후 개수: %d" (List.distinct [c1; c2] |> List.length) // 1
printfn "재정의 없는 Swatch: %d" (List.distinct [s1; s2] |> List.length) // 2
```

`=` 가 참이 되었을 뿐 아니라 `List.distinct` 의 결과까지 달라진다. `equality` 제약은 컴파일 시점 조건이라 재정의가 없어도 컴파일은 된다. 달라지는 것은 결과다. 참조는 여전히 다르다는 것도 함께 확인해 둘 만하다.

여기서 건줘 볼 것이 있다. 같은 일을 레코드로 하면 타입 선언 한 줄로 끝난다.

```
type ColourRecord = { Red: int; Green: int; Blue: int }

let r1 = { Red = 200; Green = 40; Blue = 40 }
let r2 = { Red = 200; Green = 40; Blue = 40 }
printfn "레코드 동등성: %b / 해시: %b" (r1 = r2) (r1.GetHashCode() = r2.GetHashCode())
// 레코드 동등성: true / 해시: true
```

그러니 위의 긴 코드는 "F# 에서 값처럼 건주는 타입을 만드는 법"이 아니다. 클래스 타입이라야 하는 사정이 있을 때, 곧 다른 .NET 언어에 내놓는 타입이거나 상태를 감춰야 할 때 그 타입에 동등성을 붙이는 법이다. F# 안에서만 쓸 값이라면 레코드가 먼저다.

## 연산자 오버로딩 (노트 보충)

앞 절에서는 `op_Equality` 라는 .NET 내부 이름을 그대로 멤버 이름으로 적었다. F# 에서 연산자를 정의하는 보통 문법은 연산자 기호를 괄호에 담은 정적 멤버이고, 매개변수는 튜플로 받는다.

```
type Pigment(red: int, green: int, blue: int) =
    member _.Red = red
    member _.Green = green
    member _.Blue = blue

    static member (+) (left: Pigment, right: Pigment) =
        Pigment(
            min 255 (left.Red + right.Red),
            min 255 (left.Green + right.Green),
            min 255 (left.Blue + right.Blue))

    static member (*) (p: Pigment, factor: float) =
        Pigment(
            min 255 (int (float p.Red * factor)),
            min 255 (int (float p.Green * factor)),
            min 255 (int (float p.Blue * factor)))

let mixed = Pigment(100, 0, 0) + Pigment(200, 30, 0)
printfn "섞은 값: (%d, %d, %d)" mixed.Red mixed.Green mixed.Blue
// 섞은 값: (255, 30, 0)

let dimmed = Pigment(200, 100, 50) * 0.5
printfn "절반 값: (%d, %d, %d)" dimmed.Red dimmed.Green dimmed.Blue
// 절반 값: (100, 50, 25)
```

- 컴파일러는 후보를 찾을 때 양쪽 피연산자의 타입을 모두 뒤진다. 왼쪽 피연산자 타입에 정의가 없어도 오른쪽 타입에 있으면 뽑힌다.
- 다만 선언한 매개변수 순서와 쓰는 순서는 맞아야 한다. 위 `*` 는 `(Pigment, float)` 순서로만 선언했으므로 `Pigment(200, 100, 50) * 0.5` 는 통하고 `0.5 * Pigment(200, 100, 50)` 은 `error FS0193: 형식 제약 조건이 일치하지 않습니다.` 다. 양쪽 순서를 다 쓰려면 `static member (*) (factor: float, p: Pigment)` 를 정적 멤버로 하나 더 적는다.
- 2챕터에서 이름이 나온 사용자 정의 연산자(custom operator)는 모듈 수준 `let (+.) a b = ...` 형태다. 그쪽은 커링된 매개변수이고, 이쪽은 튜플이다. 타입에 딸린 연산자를 만들 때는 정적 멤버 형태를 쓴다.

## IDisposable 과 use (원서 p.131 확장)

원서는 클래스 타입 절에서 `new` 를 쓰지 않는다고 하면서 예외 하나를 단다. `IDisposable` 을 구현한 타입이면 `let` 이 아니라 `use` 로 스코프 블록을 만든다는 것이다. 6챕터에서 `StreamReader` 를 쓰는 쪽만 봤다면 이번에

는 `IDisposable` 을 구현하는 쪽을 본다. (원서가 이 인터페이스를 `IDisposable<'T>` 로 적은 것은 옳다. 실제 `IDisposable` 은 제네릭이 아니고 이름 철자도 다르다. 원서 p.81 도 같은 대목에서 어긋나지만 그쪽은 철자는 맞고 제네릭 표기만 틀렸다( `IDisposable<'T>` ).)

```
// 이 단위가 보여주는 것: IDisposable 구현과 use 의 맞물림
open System

type MachineLock(machineId: string, trail: ResizeArray<string>) =
    let mutable released = false
    do trail.Add $"{machineId} 점유"

    member _.Inspect(part) = trail.Add $"{machineId}/{part}"

    interface IDisposable with
        member _.Dispose() =
            if not released then
                released <- true
                trail.Add $"{machineId} 반납"
```

- 점유와 반납을 `trail` 에 적어 두어 순서를 눈으로 확인한다. 점유는 `do` 블록이므로 생성 시점에 한 번 실행 된다.
- `released` 로 두 번 해제되는 것을 막았다. `Dispose()` 가 여러 번 불려도 안전해야 한다는 것이 .NET 의 규약이다.

```
let trail = ResizeArray<string>()

let inspectAll () =
    use held = new MachineLock("CMP-01", trail)
    held.Inspect "여과기"
    held.Inspect "윤활"

inspectAll ()
trail |> String.concat " -> " |> printfn "%s"
// CMP-01 점유 -> CMP-01/여과기 -> CMP-01/윤활 -> CMP-01 반납
```

`use` 로 바인딩했으므로 함수 스코프가 끝나는 자리에서 `Dispose()` 가 불렸다. `let` 으로 바꾸면 그 호출이 사라진다.

```
let trail2 = ResizeArray<string>()

let leaky () =
    let held = new MachineLock("CMP-02", trail2)
    held.Inspect "여과기"

leaky ()
trail2 |> String.concat " -> " |> printfn "%s"
// CMP-02 점유 -> CMP-02/여과기
```

반납 기록이 없다. 스코프를 벗어난 뒤에도 설비가 점유된 채 남았다는 뜻이다. 6챕터에서 파일 핸들이 열린 채 남는 사고를 본 것과 같은 구조다. 반대로 예외가 나는 경로에서도 `use` 는 해제를 보장한다.

```
let trail3 = ResizeArray<string>()

let failing () =
    use held = new MachineLock("CMP-03", trail3)
    held.Inspect "여과기"
    failwith "압력 이상"

try failing () with ex -> trail3.Add $"예외: {ex.Message}"
trail3 |> String.concat " -> " |> printfn "%s"
// CMP-03 점유 -> CMP-03/여과기 -> CMP-03 반납 -> 예외: 압력 이상
```

반납이 예외 기록보다 앞에 온다. 스코프를 벗어나는 시점에 해제가 먼저 일어나고 그 뒤에 예외가 호출 사슬을 올라간다.

두 가지를 더 실측해 둔다. 하나는 `new` 를 빠뜨렸을 때다. `use held = MachineLock(...)` 로 적으면 `warning FS0760: IDisposable 인터페이스를 지원하는 개체는 생성 값이 리소스를 소유할 수도 있다는 것을 표시하기 위해 생성자를 나타내는 함수 값으로 'Type(args)' 또는 'Type'이 아니라 'new Type(args)' 구문을 사용하여 만드는 것이 좋습니다.` 가 뜬다. 다른 하나는 `Dispose()` 를 손으로 부를 때다. 이 멤버도 인터페이스 구현이므로 인스턴스에서 바로 부르면 `error FS0039: 'MachineLock' 형식은 'Dispose' 필드, 생성자 또는 멤버를 정의하지 않습니다.` 다. 상향 변환이 필요하다. 반면 `use` 는 변환을 적지 않아도 알아서 찾아 부른다.

```
let trail4 = ResizeArray<string>()
let manual = new MachineLock("CMP-04", trail4)
(manual :> IDisposable).Dispose()
trail4 |> String.concat " -> " |> printfn "%s"
// CMP-04 점유 -> CMP-04 반납
```

## Summary — 원서의 챕터 요약 (원서 p.140)

- 원서는 이 챕터를 F# 객체 프로그래밍의 입문으로 정리한다. 다룬 것은 스코프와 가시성, 캡슐화, 인터페이스와 변환, 동등성이다.
- .NET 생태계의 나머지와 맞물릴 일이 늘어날수록 객체 프로그래밍이 필요해질 여지도 커진다는 것이 원서의 결론이다.
- 여기서 다룬 것은 F# 객체 프로그래밍의 표면이라고 밝히고, 더 파고들 독자에게 Kit Eason 의 Stylish F# 을 권한다.
- 다음 챕터에서는 F# 의 재귀를 다룬다.

## 정리 — 이 노트의 요약

- 클래스 타입은 `type 이름 (생성자 매개변수) =` 으로 선언한다. 타입 이름 뒤 괄호는 생략할 수 없고, 생성자·메서드의 매개변수는 커링되지 않은 튜플로 묶인다. 인스턴스를 만들 때 `new` 는 쓰지 않는다. `IDisposable` 구현체만 예외이고 그때는 `use` 와 `new` 를 함께 쓴다. `new` 를 빠뜨리면 오류가 아니라 경고 `FS0760` 이다.
- 클래스 본문의 `let · do` 는 주 생성자의 본문이라서 인스턴스를 만들 때 한 번 실행된다. `member` 본문은 접근할 때마다 다시 실행된다. `member val` 은 생성 시점에 한 번 평가한 값을 담아 두는 자동 프로퍼티이고, `with get, set` 을 붙이면 밖에서 바꿀 수 있다.
- 내부 `let` 은 밖에서 보이지 않는다. 이 성질이 캡슐화의 핵심이다. 가변 `ResizeArray` 를 안에 감추고 멤버로만 열어 주면, 불변식을 깨뜨릴 수 있는 코드가 그 타입 안으로 한정된다.
- 인터페이스 구현은 F# 에서 명시적이다. `interface I with` 블록에 적은 멤버는 클래스 타입의 멤버가 아니므로, 인스턴스에서 바로 부르면 `error FS0039` 다. `:>` 로 상향 변환해야 보인다. FSI 시그니처에도 `interface I` 한 줄만 찍히고 멤버 목록이 비어 있다.
- 변환을 적지 않아도 되는 자리는 인터페이스 타입 매개변수를 받는 함수의 인자 자리, 그리고 인터페이스 타입으로 주석을 단 `let` 바인딩이다. 뒤쪽은 실측하면 F# 6 부터 통하고 `--langversion:5.0` 에서는 `error FS0001` 이다.
- 객체 식은 `{ new I with ... }` 로 이름 없는 구현을 그 자리에서 만든다. 정적 타입이 곧 인터페이스라 변환이 필요 없고, 지역 값과 `let mutable` 을 붙잡을 수 있다. 한 번만 쓰는 서비스와 테스트용 가짜에 알맞다.
- 추상 클래스는 `[<AbstractClass>]` 로 만든다. `abstract member` 는 하위 타입이 반드시 채우고, `default` 를 붙이면 기본 구현이 된다. 상속은 `inherit` 한 줄이고, 상속받은 멤버는 인터페이스와 달리 변환 없이 보인다.
- 클래스 타입의 `=` 는 참조를 견준다. 값으로 견주게 하려면 `GetHashCode` 와 `Equals` 를 함께 재정의하고 `IEquatable<'T>` 를 구현한다. `Equals` 만 재정의하면 `warning FS0346` , `GetHashCode` 만 재정의하면 `warning FS0345` 다. 다른 .NET 언어의 `==` 까지 맞추려면 `op_Equality` 정적 멤버를 더한다. `[<AllowNullLiteral>]` 은 상호운용을 위한 특성이 아니라 F# 코드가 그 타입에 `null` 리터럴을 쓸 수 있게 하는 특성이고, `isNull` 로 그 타입의 값을 검사하려면 반드시 붙여야 한다.
- 그 모든 코드가 레코드에서는 선언 한 줄로 따라온다. 클래스에 동등성을 붙이는 일은 클래스라야 하는 사정이 있을 때 하는 작업이고, F# 안에서만 쓰는 값이라면 레코드가 먼저다.
- 타입에 딸린 연산자는 `static member (+) (a, b) = ...` 형태로 정의한다. 매개변수는 튜플이고, 왼쪽과 오른쪽 타입을 다르게 잡아도 된다. 다만 선언한 매개변수 순서와 쓰는 순서가 맞아야 하고, 뒤집어 쓰려면 순서를 바꾼 정적 멤버를 하나 더 적는다. 모듈 수준의 사용자 정의 연산자가 커링된 매개변수를 쓰는 것과 대비된다.
- 원서의 태도를 한 줄로 옮기면, 객체는 F# 의 기본값이 아니지만 상호운용과 캡슐화라는 두 자리에서는 함수보다 나은 도구다. 함수 하나짜리 인터페이스를 만들고 있다면 그 자리는 아마 아니다.

## 원서 대조 표

| 절                               | 원서 페이지     | 실행 단위            |
|---------------------------------|------------|------------------|
| Setting Up — 준비                 | p.130      | —                |
| 언제 객체를 쓰는가                      | p.130      | —                |
| Class Types — 클래스 타입            | pp.130-132 | 10-class         |
| 클래스 본문의 평가 시점 (노트 보충)           | —          | 10-class         |
| Interfaces — 인터페이스              | pp.132-134 | 10-interface     |
| 추상 클래스와 상속 (원서 p.133 확장)        | p.133      | 10-interface     |
| Object Expressions — 객체 식       | pp.134-136 | 10-objexpr       |
| Encapsulation — 캡슐화             | pp.136-138 | 10-encapsulation |
| Equality — 동등성                  | pp.138-140 | 10-equality      |
| 연산자 오버로딩 (노트 보충)                | —          | 10-equality      |
| IDisposable 과 use (원서 p.131 확장) | p.131      | 10-dispose       |
| Summary — 원서의 챕터 요약             | p.140      | —                |

## 11 - 재귀 (원서 pp.141-152)

자기 자신을 부르는 함수로 반복을 표현하는 것이 재귀다. 명령형 언어의 `for / while` 이 하는 일을 함수 호출 하나로 바꿔 놓는 셈이고, 상태를 변경하지 않고도 반복을 쓸 수 있다는 점에서 함수형 코드와 잘 맞는다. 다만 소박하게 적은 재귀는 호출이 돌아올 때까지 할 일을 스택에 쌓아 두므로 입력이 커지면 메모리를 먹고 결국 스택을 넘긴다. 이 챕터는 그 문제를 누적값(accumulator)과 꼬리 재귀(tail recursion)로 푸는 방법, 같은 일을 5챕터의 `List.fold` 로 다시 쓰는 방법, 그리고 재귀가 가장 자연스러운 자리인 계층 데이터 처리를 다룬다. 5챕터에서 `fold` 의 `folder` 가 (상태, 원소) 순이었던 것을 기억해 두면 이 챕터의 절반은 이미 아는 이야기가 된다.

### Setting Up — 준비 (원서 p.141)

- 원서는 새 폴더를 만들고 `.fsx` 파일을 FSI 로 돌리는 것이 전부다. 패키지도 프로젝트도 필요 없다.
- 이 노트의 예제는 원서와 도메인을 달리 잡았다. 공연 편성 가짓수, 주간조·야간조 교대, 계단 오르는 경로 수, 공연 부대 행사표, 응답 시간 정렬, 창고 구역 트리, 섬 항로망 최단 경로다. 마지막 항로망 문제만 원서와 구조가 같고(계층 데이터에서 최단 경로 찾기) 데이터와 자료구조 정의는 새로 잡았다.
- 원서는 항로 데이터를 `resources/data.csv` 파일에서 읽는다. 파일 읽기는 6챕터의 주제이므로 이 노트는 같은 CSV 를 문자열 리터럴로 코드 안에 두고 파싱만 보인다.

## Solving The Problem — 재귀 함수의 두 갈래 (원서 pp.141-142)

- 재귀 함수를 만들려면 `let rec` 로 선언한다. `rec` 이 있어야 함수 본문에서 자기 이름을 볼 수 있다.
- 재귀 함수는 반드시 두 갈래로 갈린다. 재귀를 멈추고 값을 내놓는 기저 경우(base case)와, 자기를 다시 부르는 재귀 경우다.
- 기저 경우가 없거나 입력이 그 지점에 닿지 못하면 함수는 끝나지 않는다. 갈래를 패턴 매칭으로 적는 것이 관례인데, 와일드카드 없이 갈래를 늘어놓으면 빠뜨린 갈래를 컴파일러가 경고 FS0025 로 알려 주기 때문이다.
- 원서는 팩토리얼로 이 구조를 보인다. 여기서는 공연 `n` 편을 한 무대에 올릴 때 순서를 짜는 방법의 수로 같은 점화식을 쓴다.

```
// 이 단위가 보여주는 것: rec 키워드, 기저 경우와 재귀 경우, 상호 재귀
// 공연 n 편의 편성 순서 가짓수는 n! 이다
// orderCount: n: int -> int64
let rec orderCount n =
    match n with
    | 0 | 1 -> 1L
    | n -> int64 n * orderCount (n - 1)

printfn "5편 편성 = %d 가지" (orderCount 5)      // 5편 편성 = 120 가지
printfn "10편 편성 = %d 가지" (orderCount 10)    // 10편 편성 = 3628800 가지
```

기저 경우는 `0` 과 `1` 이고, 나머지가 재귀 경우다. 재귀 호출이 어떤 순서로 풀리는지 손으로 펼쳐 보면 이 구현의 성질이 드러난다.

```
orderCount 4
→ 4 * orderCount 3
→ 4 * (3 * orderCount 2)
→ 4 * (3 * (2 * orderCount 1))
→ 4 * (3 * (2 * 1))
→ 4 * (3 * 2)
→ 4 * 6
→ 24
```

곱셈은 가장 안쪽 호출이 값을 내놓은 뒤에야 시작된다. 그때까지 `4 *`, `3 *`, `2 *` 라는 "돌아오면 할 일"이 전부 살아 있어야 하고, 그것을 담아 두는 자리가 스택 프레임(stack frame)이다. `n` 이 커지면 프레임 수가 그만큼 늘어난다. 이것이 꼬리 호출 절에서 고칠 문제다.

`rec` 을 빼면 이름을 아직 모르는 상태에서 자신을 부르는 꼴이 되어 컴파일되지 않는다.

```
// rec 이 없는 버전 - 오류 FS0039
let orderCount n =
    match n with
    | 0 | 1 -> 1L
    | n -> int64 n * orderCount (n - 1)
// error FS0039: 'orderCount' 값 또는 생성자가 정의되지 않았습니다.
```

기저 경우를 적었어도 입력이 그리로 가지 않으면 소용이 없다. 위 `orderCount` 에 음수를 넣으면 `n` 이 계속 작아지면서 `0` 을 지나쳐 버린다.

```
// 기저 경우에 닿지 못하는 호출 - 실행하면 프로세스가 죽는다
orderCount (-1)
```



인자를 (-1) 로 괄호에 넣은 것은 관례일 뿐이다. F# 은 앞에 공백이 있고 뒤에 공백이 없는 -1 을 음수 리터럴로 읽으므로 `orderCount -1` 도 `orderCount (-1)` 과 똑같이 파싱돼 그대로 실행된다. 적용으로 읽힌다는 것은 함수가 아닌 값에 붙여 보면 드러난다 — `let g = 10` 뒤에 `g -1` 을 적으면 `error FS0003: 이 값은 함수가 아니며 적용할 수 없습니다.` 가 난다. 뻔셈이 되는 것은 `orderCount - 1` 처럼 - 양쪽에 공백을 둘 때이고, 그때는 타입이 맞지 않아 오류 FS0001 이 난다. 이 호출은 예외로 잡히지 않는다. .NET 의 `StackOverflowException` 은 `try ... with` 로 잡을 수 없고 프로세스를 그대로 끝낸다. 실행하면 표준 오류에 `Stack overflow.` 한 줄과 같은 함수 이름이 반복되는 스택 목록이 찍히고 종료 코드 134 로 죽는다. 그래서 이 블록에는 `id` 를 붙이지 않았다. 같은 현상을 다른 함수로 실측한 값은 뒤의 꼬리 호출 절에 적어 뒀다. 재귀를 적을 때 기저 경우가 실제로 닿는지 확인하는 것은 문법 문제가 아니라 논리 문제다. 원서의 팩토리얼도 기저 경우가 1 하나뿐이어서 0 을 넣으면 같은 일이 벌어진다.

## 상호 재귀 — and 로 잇는다 (노트 보충)

원서는 다루지 않지만 재귀 문법에는 갈래가 하나 더 있다. 두 함수가 서로를 부르는 상호 재귀(mutual recursion)다. `let rec` 로 첫 함수를 열고 둘째부터 `and` 로 잇는다.

```
// 상호 재귀: 주간조와 야간조가 하루씩 번갈아 근무한다
// dayShiftOn: n: int -> bool
let rec dayShiftOn n =
    if n = 0 then true else nightShiftOn (n - 1)
// nightShiftOn: n: int -> bool
and nightShiftOn n =
    if n = 0 then false else dayShiftOn (n - 1)

printfn "7일 뒤 주간조 근무? %b" (dayShiftOn 7) // 7일 뒤 주간조 근무? false
printfn "7일 뒤 야간조 근무? %b" (nightShiftOn 7) // 7일 뒤 야간조 근무? true
printfn "10000000일 뒤 주간조 근무? %b" (dayShiftOn 10_000_000) // 10000000일 뒤 주간조 근무? true
```

F# 은 파일 위에서 아래로 이름을 확인하므로, `and` 없이 두 함수를 따로 적으면 먼저 나온 쪽이 아직 없는 이름을 부르게 되어 실패한다. `rec` 을 빼고 `and` 만 쓰는 것도 막혀 있다.

```
// rec 없이 and 만 쓴 경우 - 오류 FS0576
let dayShiftOn n = if n = 0 then true else nightShiftOn (n - 1)
and nightShiftOn n = if n = 0 then false else dayShiftOn (n - 1)
// error FS0576: 비재귀적 바인딩을 위한 선언 형식 'let ... and ...'는 F# 코드에서 사용되지 않습니다. 대신 'let' 바인딩 시퀀스를 사용하세요.
```

두 함수 모두 재귀 호출의 결과가 곧 자기 결과이므로 프레임이 쌓이지 않는다. 그래서 1000만 일도 즉시 끝난다. 다만 다른 함수로 넘어가는 재귀에서 이 성질이 성립하는 데는 조건이 하나 붙는데, 그 조건은 꼬리 호출을 정의한 다음 절에서 적는다. 물론 이 계산 자체는  $n \% 2 = 0$  한 줄로 끝나므로 상호 재귀를 쓸 자리가 아니다. 상호 재귀가 제 몫을 하는 자리는 서로를 참조하는 타입 두 개를 함께 훑을 때다. 트리의 가지와 말단을 각각 다른 타입으로 정의했다면 두 순회 함수가 서로를 부르는 모양이 된다.

## Tail Call Optimisation — 꼬리 호출과 누적값 (원서 pp.142-143)

- 함수가 마지막으로 하는 일이 어떤 호출이고 그 호출의 결과가 곧 그 함수의 반환값이면, 즉 호출이 돌아온 뒤에 할 일이 남지 않으면 그 호출을 꼬리 호출(tail call)이라고 한다. 재귀 호출일 필요는 없다.
- 꼬리 호출은 돌아올 자리를 기억할 필요가 없으므로 컴파일러와 런타임이 프레임을 새로 쌓지 않고 재사용한다. 결과적으로 반복문과 같은 메모리를 쓴다. 이것이 꼬리 호출 최적화(tail call optimisation)다.
- 이 최적화에는 조건이 하나 붙는다. 자기 자신을 직접 부르는 꼬리 호출은 컴파일러가 반복문으로 바꿔 주므로 언제나 성립하지만, 앞 절의 상호 재귀처럼 다른 함수로 넘어가는 꼬리 호출은 컴파일러 옵션 `--tailcalls+`가 켜져 있어야 한다. `dotnet fsi` 와 Release 빌드는 켜져 있고 Debug 프로젝트 빌드는 `--tailcalls-`이므로, 앞 절의 `dayShiftOn` 을 프로젝트에 옮겨 Debug 로 빌드하면 1000만에서 스택을 넘긴다.
- 소박한 재귀를 꼬리 재귀로 바꾸는 표준 수법이 누적값이다. 지금까지 계산한 결과를 매개변수로 함께 넘겨서, 재귀 호출을 마지막 동작으로 만든다.
- 누적값을 공개 시그니처에 노출하지 않으려면 안쪽에 `loop` 같은 지역 함수를 두고 바깥 함수가 초기값을 넣어 첫 호출을 한다. 원서도 이 배치를 쓴다.
- 누적값을 한 연산으로 이어 붙여 나가는 꼴이면 초기값은 그 연산의 항등원이다. 곱셈이면 `1`, 덧셈이면 `0` 이다. 5장에서 `List.fold` 의 초기값을 정할 때와 똑같은 기준이다. 점화식의 시작 값을 나르는 누적값은 이 기준에서 벗어난다. 뒤에 나오는 `stepWays` 가 그런 경우다.

```
// 이 단위가 보여주는 것: 누적값을 나르는 꼬리 재귀
// 앞 절의 소박한 버전과 공개 시그니처가 같다
// orderCount: n: int -> int64
let orderCount n =
    let rec loop remaining acc =
        match remaining with
        | 0 | 1 -> acc
        | n -> loop (n - 1) (acc * int64 n)
    loop n 1L

printfn "5편 편성 = %d 가지" (orderCount 5)      // 5편 편성 = 120 가지
printfn "10편 편성 = %d 가지" (orderCount 10)    // 10편 편성 = 3628800 가지
```

바뀐 것은 세 가지다. 재귀를 지역 함수 `loop` 로 옮겼고, `acc` 매개변수를 더했고, 바깥 함수 끝에 `loop n 1L` 로 첫 호출을 하는 줄을 뒀다. 기저 경우가 돌려주는 값이 `1L` 이 아니라 `acc` 라는 점이 핵심이다. 계산이 이미 끝나 있으므로 그것을 그대로 내놓기만 한다.

```
orderCount 4
→ loop 4 1
→ loop 3 4
→ loop 2 12
→ loop 1 24
→ 24
```

앞 절의 전개와 견주면 괄호가 사라졌다. 매 단계에서 살아 있어야 하는 값은 `remaining` 과 `acc` 두 개뿐이고, 돌아와서 할 일이 없으므로 프레임이 쌓이지 않는다.

```
// 덧셈으로 누적하면 초기값이 0 이다
// sumHours: n: int -> int64
let sumHours n =
    let rec loop remaining acc =
        match remaining with
        | 0 -> acc
        | n -> loop (n - 1) (acc + int64 n)
    loop n 0L

printfn "1..1000000 누적 = %d" (sumHours 1_000_000) // 1..1000000 누적 = 500000500000
```

100만 번을 돌아도 즉시 끝난다. 같은 계산을 누적값 없이 적으면 어떻게 되는지가 이 절의 요점이다.

```
// 누적값이 없는 버전 - 재귀 호출 뒤에 덧셈이 남아 있어 꼬리 호출이 아니다
let rec sumHoursNaive n =
    match n with
    | 0 -> 0L
    | n -> int64 n + sumHoursNaive (n - 1)

printfn "%d" (sumHoursNaive 200_000) // 이 저장소에서는 통과했다
printfn "%d" (sumHoursNaive 500_000) // 스택을 넘긴다
```

이 저장소에서 실측한 결과는 이렇다. `sumHoursNaive 200_000` 은 `2000001000000` 을 정상으로 돌려줬고, 이어서 `sumHoursNaive 500_000` 에서 `Stack overflow.` 와 `Repeated ... times:` 로 시작하는 스택 목록을 찍으며 종료 코드 134 로 프로세스가 죽었다. 반복 횟수는 26만 번대이고 실행마다 달라진다. 정확한 한계는 스택 크기와 프레임 크기에 따라 달라지므로 숫자 자체를 외울 것은 아니다. 중요한 것은 소박한 재귀에는 입력 크기에 비례하는 상한이 있고 그 상한을 넘기면 예외가 아니라 프로세스 종료로 나타난다는 점이다. 꼬리 재귀에는 그 상한이 없다.

## [<TailCall>] 로 컴파일러에게 확인받기 (노트 보충)

꼬리 호출인지 아닌지는 눈으로 판단해야 하는데, 함수가 길어지면 놓치기 쉽다. F# 8 부터는 [<TailCall>] 특성을 붙여 컴파일러에게 검사를 맡길 수 있다.

```
// F# 8 부터 쓸 수 있는 [<TailCall>] - 꼬리 호출이 맞으므로 조용히 통과한다
// countDown: remaining: int -> acc: int64 -> int64
[<TailCall>]
let rec countDown remaining acc =
    match remaining with
    | 0 -> acc
    | n -> countDown (n - 1) (acc + int64 n)

printfn "countDown 1000000 = %d" (countDown 1_000_000 0L) // countDown 1000000 = 500000500000
```

특성을 꼬리 호출이 아닌 함수에 붙이면 경고가 나온다. `sumTo` 라는 이름으로 실측한 메시지가 `warning FS3569: 멤버 또는 함수 'sumTo'에 'TailCallAttribute' 특성이 있지만 비상 재귀적인 방식으로 사용되고 있지 않습니다.` 다. 실측에서 확인한 제약이 세 가지 있다. 첫째, 이 검사는 프로젝트 빌드에서만 돌고 `dotnet fsi` 로 스크립트를 실행할 때는 경고가 나오지 않았다. 둘째, `--langversion:7.0` 으로 낮추면 특성은 그대로 붙지만 검사가 돌지 않는다. 셋째, 지역 `let rec` 에는 특성을 붙일 수 없어 오류 FS0010 이 난다. 그래서 안쪽 `loop` 를 검사받고 싶으면 그 함수를 모듈 수준으로 끌어올려야 한다.

## Expanding the Accumulator — 누적값을 튜플로 넓히기 (원서 pp.143-144)

- 누적값은 숫자 하나일 필요가 없다. 튜플이나 레코드로 넓히면 여러 값을 함께 나눌 수 있다.
- 원서는 피보나치 수열로 이것을 보인다. 직전 두 항이 필요한 점화식이므로 누적값이 값 두 개가 되어야 한다.
- 여기서는 같은 점화식을 계단으로 바꿔 쓴다. 한 번에 1칸 또는 2칸을 오를 수 있을 때 `n` 칸을 오르는 경로의 수는 직전 두 칸의 경로 수를 더한 값이다.

```
// 소박한 버전: 재귀 호출이 두 개라 같은 값을 몇 번씩 다시 센다
// stepWaysNaive: steps: int -> int64
let rec stepWaysNaive steps =
    match steps with
    | 0 | 1 -> 1L
    | n -> stepWaysNaive (n - 1) + stepWaysNaive (n - 2)

printfn "5칸 = %d 가지" (stepWaysNaive 5)      // 5칸 = 8 가지
printfn "30칸 = %d 가지" (stepWaysNaive 30)    // 30칸 = 1346269 가지
```

이 구현의 문제는 스택보다 시간이다. `stepWaysNaive 28` 을 계산하려고 `stepWaysNaive 27` 과 `stepWaysNaive 26` 을 부르는데, 앞의 호출도 안에서 `stepWaysNaive 26` 을 다시 계산한다. 겹치는 계산이 지수로 불어난다. 이 저장소에서 40칸은 약 1.0초, 45칸은 약 8.5초가 걸렸다. 원서가 `fib 50L` 로 1분 가까이 걸린다고 적은 것과 같은 현상이다.

```
// 누적값을 튜플로 넓혀 직전 두 값을 함께 나른다
// stepWays: steps: int -> int64
let stepWays steps =
    let rec loop remaining (prev, curr) =
        match remaining with
        | 0 -> prev
        | 1 -> curr
        | n -> loop (n - 1) (curr, prev + curr)
    loop steps (1L, 1L)

printfn "5칸 = %d 가지" (stepWays 5)      // 5칸 = 8 가지
printfn "90칸 = %d 가지" (stepWays 90)    // 90칸 = 4660046610375530309 가지
```

누적값 `(prev, curr)` 는 매 단계에서 한 칸 앞으로 밀린다. `(curr, prev + curr)` 가 그 밀기다. 계산 횟수가 `steps` 에 비례하므로 90칸도 즉시 나온다. 다만 이제는 다른 한계에 부딪힌다. 91칸까지는 `int64` 에 담기지만 92칸은 넘쳐서 `-6246583658587674878` 이 나온다. F# 의 산술은 기본적으로 넘침을 검사하지 않으므로, 값이 커지는 계산에서는 `bigint` 로 올리거나 상한을 코드에서 막아야 한다. 넘침을 조용히 지나치지 않게 하는 길도 있다. `open Microsoft.FSharp.Core.Operators.Checked` 로 검사판 연산자를 켜면 `System.Int64.MaxValue + 1L` 이 `OverflowException` 을 던진다.

기저 경우가 `0` 과 `1` 두 개인 것도 짝여 둘 만하다. 점화식이 직전 두 항을 참조하면 기저 경우도 두 개가 필요하다. 소박한 버전에서 `| 0 | 1 -> 1L` 로 묶어 둔 것을 꼬리 재귀 버전에서는 `| 0 -> prev` 와 `| 1 -> curr` 로 갈랐다. 누적값의 어느 칸을 내놓아야 하는지가 다르기 때문이다.

## Using Recursion to Solve FizzBuzz — 규칙 목록을 누적하기 (원서 pp.144-145)

- 원서는 FizzBuzz 를 (나누는 수, 붙일 말) 목록으로 두고 그 목록을 재귀로 훑는다. 규칙을 데이터로 빼 뒀으므로 규칙을 더하는 일이 목록에 한 줄 더하는 일이 된다.
- 여기서는 같은 구조를 공연 부대 행사표로 바꿔 쓴다. 회차 번호가 주기의 배수가 되면 그 행사 이름을 이어 붙이고, 걸리는 주기가 하나도 없으면 회차 번호를 그대로 적는다.
- 누적값이 문자열이므로 초기값은 빈 문자열이다. 문자열 이어 붙이기의 항등원이 빈 문자열이기 때문이다.
- 이 절에서 리스트를 머리(head)와 꼬리(tail)로 분해한다. 리스트의 꼬리와 꼬리 재귀의 "꼬리"는 글자만 같고 다른 말이다. 앞은 머리를 뺀 나머지 리스트이고, 뒤는 함수 본문에서 재귀 호출이 놓인 자리를 가리킨다. 헛갈리지 않게 이 노트는 분해한 나머지를 `rest` 로 적는다.

```
// 이 단위가 보여주는 것: 규칙 목록을 누적값에 접어 넣는 꼬리 재귀
// 공연 부대 행사표 - 회차 번호가 주기의 배수가 되면 그 행사를 연다
let eventPlan = [ (4, "사인회"); (6, "포토타임") ]

// eventsAt: plan: (int * string) list -> showNo: int -> string
let eventsAt plan showNo =
  let rec loop remaining acc =
    match remaining with
    | [] -> if acc = "" then string showNo else acc
    | (cycle, title) :: rest ->
      let label = if showNo % cycle = 0 then title else ""
      loop rest (acc + label)
  loop plan ""

printfn "%s" ([ 1 .. 14 ] |> List.map (eventsAt eventPlan) |> String.concat " ")
// 1 2 3 사인회 5 포토타임 7 사인회 9 10 11 사인회포토타임 13 14
```

패턴 매칭의 두 갈래가 각각 이렇게 읽힌다. 규칙이 다 떨어졌으면 누적값을 내놓는데, 빈 문자열이면 걸린 규칙이 없다는 뜻이므로 회차 번호를 문자열로 바꿔 돌려준다. 규칙이 남아 있으면 그 규칙의 판정 결과를 누적값에 붙이고 나머지 규칙으로 재귀한다. 12회차에서 `사인회포토타임` 이 나온 것은 4와 6이 동시에 걸려 두 이름이 이어 붙은 결과다.

`List.map (eventsAt eventPlan)` 은 매개변수 두 개짜리 함수에 첫 인자만 넘긴 부분 적용(partial application)이다. 규칙 목록을 먼저 고정해 두면 `int -> string` 함수가 남고 그것을 `List.map` 에 그대로 엮을 수 있다.

규칙을 늘리는 데 함수를 고칠 일이 없다는 것이 이 설계의 이점이다.

```
// 규칙을 하나 더해도 eventsAt 은 그대로다
let widePlan = [ (4, "사인회"); (6, "포토타임"); (10, "앙코르") ]

printfn "%s" ([ 1 .. 20 ] |> List.map (eventsAt widePlan) |> String.concat " ")
// 1 2 3 사인회 5 포토타임 7 사인회 9 앙코르 11 사인회포토타임 13 14 15 사인회 17 포토타임 19 사인회앙코르
```

### List.fold 로 다시 쓰기 (원서 p.145)

원서는 같은 문제를 `List.fold` 로 다시 쓴다. 그럴 수 있는 이유가 분명하다. 위 `loop` 가 하는 일이 정확히 `fold` 의 정의다. 리스트를 앞에서 훑으며 상태를 갱신하고 마지막 상태를 돌려준다.

```
// 같은 일을 List.fold 로 - 재귀 문법이 사라진다
// eventsAtFold: plan: (int * string) list -> showNo: int -> string
let eventsAtFold plan showNo =
    plan
    |> List.fold (fun acc (cycle, title) -> if showNo % cycle = 0 then acc + title else acc) ""
    |> fun labels -> if labels = "" then string showNo else labels

printfn "%s" ([ 1 .. 14 ] |> List.map (eventsAtFold eventPlan) |> String.concat " ")
// 1 2 3 사인회 5 포토타임 7 사인회 9 10 11 사인회포토타임 13 14
```

5챕터에서 본 대로 `fold` 의 첫 매개변수가 누적값이고 둘째가 원소다. 여기서는 원소가 `(int * string)` 튜플이라 람다 매개변수 자리에서 `(cycle, title)` 로 바로 분해했다. 초기값 `""` 는 꼬리 재귀 버전에서 `loop plan ""` 로 넘겼던 그 값이다. 기저 경우에 있던 “빈 문자열이면 회차 번호” 판정은 `fold` 가 끝난 뒤 파이프 한 칸으로 옮겨 갔다. `fold` 는 리스트가 다 떨어졌을 때 무엇을 할지 스스로 알기 때문에 그 갈래를 적을 자리가 없다.

`reduce` 는 여기서 후보가 아니다. 누적값이 `string` 이고 원소가 `(int * string)` 이라 타입이 갈리는데, `reduce` 는 상태와 원소의 타입이 같아야 한다.

두 버전의 실측 시그니처는 완전히 같다.

```
// eventsAt      : plan: (int * string) list -> showNo: int -> string
// eventsAtFold : plan: (int * string) list -> showNo: int -> string
```

결과도 같다.

```
// 240회차까지 두 구현의 결과를 맞춰 본다
printfn "두 구현이 같은가 = %b"
([ 1 .. 240 ] |> List.forall (fun n -> eventsAt eventPlan n = eventsAtFold eventPlan n))
// 두 구현이 같은가 = true
```

원서는 여기서 한 걸음 더 나아가 초기값을 빈 문자열이 아니라 입력을 문자열로 바꾼 값으로 두고, 마지막 판정까지 `fold` 안으로 끌어들인 변형을 보인다. 파이프 한 칸이 줄지만 `fold` 가 "이미 뭔가 붙었는지"를 직접 따져야 해서 읽기가 무거워진다. 5챕터에서 `fold` 한 방에 몰아넣은 코드가 읽기 어려워졌던 것과 같은 저울질이다. 원서가 결과를 출력할 때 `List.map` 대신 `List.iter (eventsAtFold eventPlan >> printfn "%s")` 처럼 합성 연산자를 쓰는 것도 취향 차이이고 결과는 같다.

무엇을 쓸지는 취향 문제가 아니라 형태 문제다. 리스트를 앞에서 한 번 훑으며 상태를 갱신하는 것이 전부라면 `fold` 가 짧고, 재귀 문법을 읽는 부담이 없고, 꼬리 호출 여부를 걱정할 일도 없다(`List.fold` 자체가 반복문으로 구현돼 있다). 재귀를 직접 적어야 하는 자리는 훑는 대상이 리스트가 아닐 때, 갈래마다 다르게 재귀해야 할 때, 중간에 멈춰야 할 때다. 이 챕터 마지막 절의 트리가 그런 경우다.

## Quicksort using recursion — 퀵소트 (원서 pp.145-146)

- 퀵소트는 재귀의 교과서 예제다. 기준값 하나를 고르고 나머지를 그보다 작거나 같은 쪽과 큰 쪽으로 나눈 다음, 두 쪽을 각각 다시 정렬해 이어 붙인다.
- `List` 모듈에 필요한 조각이 이미 있다. `List.partition` 이 술어(`'a -> bool`) 하나로 리스트를 두 개로 갈라 튜플로 돌려준다.
- 기저 경우는 빈 리스트다. 나눌 것이 없으면 그대로 정렬된 상태다.
- 원서는 정수 리스트로 보여 준다. 여기서는 응답 시간 측정값을 정렬한다.

```
// 이 단위가 보여주는 것: List.partition 을 쓴 퀵소트
// quickSort: values: 'a list -> 'a list (when 'a : comparison)
let rec quickSort values =
  match values with
  | [] -> []
  | pivot :: rest ->
    let atMost, above = rest |> List.partition (fun v -> v <= pivot)
    quickSort atMost @ [ pivot ] @ quickSort above

let latency = [ 41; 12; 41; 7; 130; 12; 3; 88; 55; 7 ]
printfn "정렬 = %A" (quickSort latency)
// 정렬 = [3; 7; 7; 12; 12; 41; 41; 55; 88; 130]
```

머리를 기준값으로 쓰고 나머지만 나눈다는 점이 중요하다. 기준값까지 나누는 쪽에 넣으면 리스트가 줄지 않아 재귀가 끝나지 않는다. 술어를 `<=` 로 둘지 `<` 로 둘지는 결과를 바꾸지 않는다. 기준값과 같은 값이 앞쪽 묶음에 들어가느냐 뒤쪽 묶음에 들어가느냐만 달라지고, 어느 쪽이든 원소는 두 묶음 중 정확히 한 곳에 들어가므로 개수와 정렬 결과가 같다.

`List.partition` 이 무엇을 돌려주는지 따로 보면 이렇다.

```
// List.partition : ('a -> bool) -> 'a list -> 'a list * 'a list
// 술어를 만족하는 것과 그렇지 않은 것을 순서를 지킨 채 튜플로 돌려준다
printfn "나누기 = %A" ([ 12; 41; 7; 130; 12; 3; 88; 55; 7 ] |> List.partition (fun v -> v <= 41))
// 나누기 = ([12; 41; 7; 12; 3; 7], [130; 88; 55])
```

`quickSort` 는 비교할 수 있는 아무 타입에나 붙는다. 비교 연산자 `<=` 만 썼으므로 자동 일반화가 `'a list -> 'a list` 에 `when 'a : comparison` 제약을 붙여 줬다.

```
printfn "문자열 = %A" (quickSort [ "라"; "가"; "다"; "나" ])
// 문자열 = ["가"; "나"; "다"; "라"]
printfn "List.sort 와 같은가 = %b" (quickSort latency = List.sort latency)
// List.sort 와 같은가 = true
```

원서는 여기서 멈추지만 두 가지는 덧붙여 둘 만하다. 첫째, 이 `quickSort` 는 꼬리 재귀가 아니다. 재귀 호출이 두 개이고 그 결과를 `@` 로 이어 붙이는 일이 남아 있으므로 프레임이 쌓인다. 재귀 깊이는 분할이 고르게 되면 원소 수의 로그 규모이므로 실무 크기에서는 문제가 되지 않는다. 둘째, 이미 정렬된 입력에서는 분할이 매번 한쪽으로 몰려 깊이가 원소 수만큼 깊어지고 비교 횟수가 제곱 규모로 커진다. 이 저장소에서 실측하면 무작위 순서 2만 개는 0.15초에 끝나지만 이미 정렬된 2만 개는 12.3초가 걸렸고, 정렬된 1만 개가 3.5초였으니 입력이 두 배 될 때 시간이 세 배 반으로 된 셈이다. 이때 걱정되는 것은 스택이지만 실제로 먼저 한계에 닿는 것은 시간이다. 정렬된 10만 개는 재귀 깊이 10만으로 6분 19초를 쓰고도 스택 오버플로 없이 끝났다. 깊이를 더 키우려면 매 단계가 남은 원소를 다 훑어야 하므로, 스택이 터지기 전에 실행 시간이 먼저 감당할 수 없게 커진다. 실제 코드에서 리스트를 정렬할 일이 있으면 `List.sort` 를 쓰면 된다. 퀵소트를 직접 적는 것은 재귀를 익히기 위한 연습이다.

## Recursion with Hierarchical Data — 계층 데이터 (원서 pp.146-150)

- 재귀가 대안 없이 필요한 자리가 계층 데이터다. 트리는 자기 자신을 품는 구조이므로 그것을 훑는 코드도 자기 자신을 부르는 모양이 된다.
- 원서는 재귀형 판별 유니온(discriminated union)으로 트리를 정의한다. 가지 케이스가 같은 타입의 자식을 품는 것이 요령이다.
- 원서는 이 절에서 지역 간 거리 CSV 를 읽어 출발지에서 도착지까지 가능한 경로를 모두 트리로 펼치고 그중 가장 짧은 것을 고른다. 이 노트는 같은 순서를 밟되 섬 사이 항로망으로 데이터를 갈아 끼웠다.
- 작업을 세 토막으로 나눈다. 데이터 적재, 가능한 경로 펼치기, 최단 경로 고르기다. 긴 문제를 작은 함수로 쪼개 하나씩 FSI 로 확인하며 나아가는 것이 원서가 이 절에서 보여 주는 작업 방식이다.

### 트리 정의와 순회 (원서 pp.146-147, 149)

트리를 가지와 말단 두 케이스로 정의하고, 그것을 훑는 함수 두 개를 만들어 본다.

```
// 이 단위가 보여주는 것: 재귀형 판별 유니온과 그것을 훑는 재귀 함수
// 가지는 값과 자식 목록을 품고, 말단은 값만 품는다
type Hierarchy<'T> =
  | Node of 'T * Hierarchy<'T> list
  | Tip of 'T

let zones =
  Node ("A동", [
    Node ("1층", [ Tip "냉장"; Tip "상온" ])
    Node ("2층", [ Tip "위험물" ])
  ])
  ])
```

`Node` 케이스가 `Hierarchy<'T> list` 를 품는 것이 재귀형 정의다. 원서는 자식을 `seq` 로 두는데, 여기서는 `list` 로 잡았다. 지연 평가가 필요 없고 `%A` 로 찍어 보기 편하기 때문이다. 원서처럼 자식을 `seq` 로 두면 훑는 만큼만 만들어지고, `list` 로 두면 트리 전체가 먼저 만들어진다. 마지막 절에서 말하는 완전 탐색의 한계가 `list` 판에서는 더 이르게 드러난다.

트리를 훑는 함수도 케이스마다 한 갈래씩 적으면 자연히 재귀가 된다. 말단이 기저 경우다.

```
// 말단 값만 모아 평평한 리스트로 만든다
// tips: tree: Hierarchy<'a> -> 'a list
let rec tips tree =
  match tree with
  | Tip x -> [ x ]
  | Node (_, children) -> children |> List.collect tips

// 가장 깊은 갈래의 깊이
// depth: tree: Hierarchy<'a> -> int
let rec depth tree =
  match tree with
  | Tip _ -> 1
  | Node (_, children) -> 1 + (children |> List.map depth |> List.max)

printfn "말단 = %A" (tips zones)      // 말단 = ["냉장"; "상온"; "위험물"]
printfn "깊이 = %d" (depth zones)    // 깊이 = 3
```

`List.collect` 가 자식마다 나온 리스트를 하나로 이어 붙인다. 자식을 재귀로 처리하고 그 결과를 합치는 이 모양이 트리 순회의 기본형이다. `depth` 는 합치는 방법만 다르다. 자식들의 결과에서 최댓값을 골라 `1` 을 더한다. 둘 다 꼬리 재귀가 아니고 그렇게 만들기도 쉽지 않지만, 재귀 깊이가 트리 깊이를 넘지 않으므로 실무 크기의 트리



에서는 문제가 되지 않는다. 한 가지 단서가 있다. 자식이 없는 `Node` 를 만들면 `List.max` 가 예외를 던진다. 이 노트의 트리는 그런 값을 만들지 않는다는 전제가 깔려 있다.

## 데이터 적재 (원서 pp.147-148)

원서는 CSV 파일을 읽어 `Map<string, Connection list>` 로 만든다. 왜 리스트가 아니라 `Map` 인지가 이 절의 요점이다. `Map` 은 키-값 쌍을 담은 불변 정렬 컬렉션이고, 이 데이터에 던질 질문이 “이 지점에서 갈 수 있는 곳은?” 이므로 키로 바로 찾는 쪽이 맞는 모양이다.

```
// 항로 한 구간
type Leg = { From: string; To: string; Km: int }

// 원서는 이 데이터를 resources/data.csv 에서 읽는다. 파일 읽기는 6챕터의 주제이므로
// 여기서는 같은 내용을 문자열 리터럴로 둔다
let legCsv = """출항,입항,km
소금항,노을항,62
소금항,물마루항,145
소금항,등대항,260
노을항,물마루항,71
노을항,바람골항,96
물마루항,바람골항,40
물마루항,등대항,120
바람골항,등대항,55
바람골항,자갈항,88
등대항,자갈항,30"""
```

한 줄이 한 구간이고 왕복 거리는 같다고 본다. 그래서 파싱할 때 한 줄에서 구간 두 개를 만든다.

```
// buildLegMap: csv: string -> Map<string,Leg list>
let buildLegMap (csv: string) =
    csv.Split('\n')
    |> Array.toList
    |> List.skip 1
    |> List.collect (fun row ->
        match row.Trim().Split(',') with
        | [| from; dest; km |] ->
            [ { From = from; To = dest; Km = int km }
              { From = dest; To = from; Km = int km } ]
        | _ -> failwithf "행 형식이 잘못됐다: %s" row)
    |> List.groupBy (fun leg -> leg.From)
    |> Map.ofList

let legMap = buildLegMap legCsv

printfn "항구 수 = %d" (legMap |> Map.count) // 항구 수 = 6
printfn "노을항에서 = %A" (legMap["노을항"] |> List.map (fun leg -> leg.To, leg.Km))
// 노을항에서 = [("소금항", 62); ("물마루항", 71); ("바람골항", 96)]
```

헤더 줄을 `List.skip 1` 로 버리고, 각 줄을 배열 패턴 `[| from; dest; km |]` 로 받는다. 칸 수가 셋이 아니면 `failwithf` 로 던진다. 3챕터에서 본 대로 예외는 정말로 복구할 수 없는 경우에 쓰는 것이고, 이 자리에서는 데이터 파일이 깨졌다는 뜻이므로 계속 진행할 이유가 없다. 마지막의 `List.groupBy` 로 출항지별로 묶고 `Map.ofList` 로 `Map` 을 만든다. `List.groupBy` 가 (키, 값 목록) 튜플 리스트를 돌려주므로 `Map.ofList` 에 그대로 들어간다.

`legMap["노을항"]` 은 대괄호 인덱서 문법이다. F# 6 부터 쓸 수 있고, `--langversion:5.0` 으로 낮추면 오류 FS3217 이 나면서 인덱서를 쓰려던 것인지 되묻는다. 이 문법은 값이 없으면 예외를 던지므로 키가 확실할 때만 쓴다.

## 가능한 경로 펼치기 (원서 pp.148-150)

- 경로를 만들려면 지금 어디에 있고, 어디를 거쳐 왔고, 몇 km 를 왔는지를 함께 들고 다녀야 한다. 원서는 이것을 `Waypoint` 레코드로 잡는다. 사실 이 레코드가 이 절의 누적값이다.
- 거쳐 온 항구 목록이 있으면 되돌아가지 않을 수 있다. 이 조건이 재귀를 끝내는 장치 구실도 한다. 항구는 유한하므로 갈 곳이 언젠가 떨어진다.
- 트리로 만드는 이유는 갈림길이 여러 개이기 때문이다. 한 지점에서 갈 수 있는 곳이 셋이면 자식이 셋인 가지가 된다.

```
// 항해 중 한 시점 - 이 레코드 전체가 누적값 구실을 한다
type Voyage = { Port: string; Wake: string list; TotalKm: int }

// 아직 들르지 않은 다음 항구들
// nextHops: legs: Leg list -> current: Voyage -> Voyage list
let nextHops legs current =
  legs
  |> List.filter (fun leg -> current.Wake |> List.contains leg.To |> not)
  |> List.map (fun leg ->
    { Port = leg.To
      Wake = leg.From :: current.Wake
      TotalKm = leg.Km + current.TotalKm })
```

`Wake` 는 지나온 자취다. 지금 있는 항구는 아직 여기 들어 있지 않고, 다음 항구로 넘어갈 때 `leg.From :: current.Wake` 로 앞에 붙는다. 그래서 `Wake` 는 뒤집힌 순서로 쌓인다. `List.filter` 가 자취에 이미 있는 항구를 걸러 내므로 같은 항구를 두 번 들르는 경로는 만들어지지 않는다.

`nextHops legs current` 의 인자 순서는 원서 `getUnvisited connections current` 를 그대로 따른 것이다. 그래서 파이프에 바로 얹히지 않아 `|> fun legs -> nextHops legs current` 한 칸이 필요한데, 원서와 대조하며 읽는 편의를 택해 순서를 바꾸지 않았다.

원서 p.148 의 `getUnvisited` 주석은 첫 인자를 `Connection list` 로, p.151 완성 코드의 주석은 `Map<string, Connection list>` 로 적어 서로 다르다. 실제 인자는 `Map` 에서 꺼낸 리스트이므로 p.148 쪽이 맞다.

```
// 출발지에서 도착지까지 가능한 경로를 전부 트리로 펼친다
// expandRoutes: start: string -> finish: string -> legMap: Map<string,Leg list> -> Hierarchy<Voyage>
let expandRoutes start finish (legMap: Map<string, Leg list>) =
  let rec grow current =
    let hops =
      legMap
      |> Map.tryFind current.Port
      |> Option.defaultValue []
      |> fun legs -> nextHops legs current
    if current.Port = finish || List.isEmpty hops then Tip current
    else Node (current, hops |> List.map grow)
  grow { Port = start; Wake = []; TotalKm = 0 }
```

가지를 더 뺄지 않는 경우가 두 가지다. 도착지에 닿은 경우와 갈 수 있는 곳이 남지 않은 경우다. 둘 다 `Tip` 이 되고 그 차이는 나중에 도착지인지 확인해서 가린다. `grow` 의 초기 인자가 이 재귀의 시작 누적값이다. 앞 절들의 `loop n 1L` 과 같은 자리다.

`Map.tryFind` 를 쓴 것은 의도적이다. 원서는 `routeMap[current.Location]` 으로 바로 읽는데, `Map.find` 와 인덱서는 부분 함수(partial function)다. 가능한 입력 전부에서 값을 돌려주지 못하고 없는 키에는 예외를 던진다. 이름이 비슷한 부분 적용과는 관계가 없다. 여기서는 왕복 구간을 다 만들었으므로 모든 항구에 항로가 하나 이

상 있어서 실제로 실패할 일은 없지만, 데이터가 한쪽 방향만 담고 있을 때 예외 대신 빈 목록으로 흘러가는 편이 다루기 쉽다. `Option.defaultValue []` 가 그 처리다.

트리를 펼쳤으니 말단만 모아 보면 후보 경로가 나온다. 이때 도착지에 닿지 못한 막다른 경로를 걸러야 한다.

```
// candidates: start: string -> finish: string -> legMap: Map<string,Leg list> -> Voyage list
let candidates start finish legMap =
  expandRoutes start finish legMap
  |> tips
  |> List.filter (fun voyage -> voyage.Port = finish)

let routeTree = expandRoutes "소금항" "자갈항" legMap

printfn "말단 전체 = %d" (routeTree |> tips |> List.length) // 말단 전체 = 23
printfn "완주 경로 = %d" (candidates "소금항" "자갈항" legMap |> List.length) // 완주 경로 = 17
printfn "트리 깊이 = %d" (depth routeTree) // 트리 깊이 = 6
```

말단이 23개인데 도착지에 닿은 것은 17개다. 나머지 6개는 되돌아갈 수 없어 막힌 자리다. 앞 절에서 만들어 둔 `tips` 와 `depth` 를 그대로 쓴 것을 눈여겨볼 만하다. `Hierarchy<'T>` 를 제네릭으로 잡아 뒀으므로 참고 구역 트리에 쓴 함수가 항로 트리에도 그대로 붙는다.

## 최단 경로 고르기 (원서 p.150)

후보가 다 모였으니 남은 일은 거리로 하나를 고르는 것이다.

```
// shortest: start: string -> finish: string -> legMap: Map<string,Leg list> -> string list * int
let shortest start finish legMap =
  candidates start finish legMap
  |> List.minBy (fun voyage -> voyage.TotalKm)
  |> fun voyage -> (voyage.Port :: voyage.Wake |> List.rev), voyage.TotalKm

printfn "%A" (shortest "소금항" "자갈항" legMap)
// (["소금항"; "노을항"; "바람골항"; "등대항"; "자갈항"], 243)
```

거리를 이미 `TotalKm` 에 누적해 뒀으므로 고르는 일은 `List.minBy` 한 번이다. 경로를 사람이 읽을 형태로 되돌리려면 지금 항구를 자취 앞에 붙이고 `List.rev` 로 뒤집는다. 자취를 `::` 로 쌓았기 때문에 뒤집는 단계가 필요한 것이고, 이것은 리스트 앞에 붙이는 연산만 빠른 F# 리스트에서 흔히 쓰는 방식이다.

`List.minBy` 도 부분 함수다. 후보가 하나도 없으면 예외를 던진다. 도착지가 항로망에 없는 이름이면 그 일이 실제로 벌어지므로, 진짜 코드로 만들 것이라면 `List.isEmpty` 로 먼저 걸러 `Option` 이나 `Result` 를 돌려주는 것이 맞다.

거리 순으로 몇 개를 늘어놓아 보면 이 문제가 왜 최단 경로 문제인지 보인다.

```
candidates "소금항" "자갈항" legMap
|> List.sortBy (fun voyage -> voyage.TotalKm)
|> List.truncate 4
|> List.iter (fun voyage ->
  printfn "%5d km %s" voyage.TotalKm (voyage.Port :: voyage.Wake |> List.rev |> String.concat " > "))
// 243 km 소금항 > 노을항 > 바람골항 > 등대항 > 자갈항
// 246 km 소금항 > 노을항 > 바람골항 > 자갈항
// 258 km 소금항 > 노을항 > 물마루항 > 바람골항 > 등대항 > 자갈항
// 261 km 소금항 > 노을항 > 물마루항 > 바람골항 > 자갈항
```

기항지를 하나 더 거치는 4구간 경로가 3구간 경로보다 짧다. 구간 수와 거리가 따로 움직이므로 눈으로 고를 수 없고 전부 계산해 봐야 한다는 점이 이 예제의 재미다.

한 가지 한계는 분명히 해 둘 만하다. 이 방식은 가능한 단순 경로를 모두 펼치는 완전 탐색이다. 항구 6개에 후보 17개였지만 항구가 늘면 후보 수가 지수로 불어난다. 최단 경로만 필요하다면 다익스트라 같은 알고리즘이 맞다. 원서가 이 예제로 보이려는 것은 최단 경로 알고리즘이 아니라 계층 구조를 재귀로 만들고 재귀로 허무는 방법이다.

## Finished Code — 완성된 코드 (원서 pp.151-152)

원서는 여기까지 만든 함수를 한 파일로 모아 다시 실행한다. 이 노트에서는 `11-tree` 실행 단위의 블록들이 문서 순서대로 이어 붙어 그 한 파일이 된다. 순서를 확인해 두면 이렇다. 트리 타입과 순회 함수( `tips` , `depth` ) → 구간 타입과 CSV → `buildLegMap` → `Voyage` 와 `nextHops` → `expandRoutes` → `candidates` → `shortest` 다. 각 함수가 앞 함수의 결과 타입을 받는 모양이므로, 원서가 원하는 대로 한 토막씩 FSI 에 올려 결과를 눈으로 보며 쌓아 갈 수 있다.

## Other Uses Of Recursion — 그 밖의 쓸모 (원서 p.152)

- 원서가 꼽는 자리는 파일 시스템이나 XML 같은 계층 데이터 처리, 평평한 데이터와 계층 사이의 변환, 그리고 끝이 정해지지 않은 이벤트 반복이다. 어느 쪽이든 반복 횟수를 미리 알 수 없다는 공통점이 있다.
- 위 `tips` 와 `depth` 처럼 트리를 값 하나로 줄이는 함수는 리스트의 `fold` 와 같은 자리에 있다. 트리용 `fold` 를 한 번 만들어 두고 그것으로 순회 함수들을 적는 방식도 있는데, 원서는 스콧 블라신(Scott Wlaschin)의 글로 넘긴다.
- 반대 방향도 있다. 평평한 목록을 부모 키로 묶어 트리로 세우는 일도 재귀다. 앞의 계층 데이터 절에서 만든 `expandRoutes` 가 바로 그 방향이었다.

## Summary — 원서의 챕터 요약 (원서 p.152)

- 원서는 재귀의 기본, 누적값과 꼬리 호출 최적화, 그리고 계층 데이터 문제 하나를 이 챕터에서 다뤘다고 정리한다.
- 다음 챕터에서는 8챕터의 함수형 검증에서 잠깐 마주쳤던 계산 식(computation expression)을 정면으로 다룬다.

## 정리 — 이 노트의 요약

- 재귀 함수는 `let rec` 로 선언한다. `rec` 이 없으면 오류 FS0039 로 이름을 모른다는 말이 나온다. 상호 재귀는 `let rec` 뒤에 `and` 로 잇고, `rec` 없이 `and` 만 쓰면 오류 FS0576 이다.
- 재귀는 기저 경우와 재귀 경우로 갈린다. 기저 경우를 적는 것만으로는 부족하고 입력이 그 지점에 실제로 닿아야 한다. 닿지 못하면 `StackOverflowException` 이 나는데 이것은 `try ... with` 로 잡히지 않고 프로세스를 끝낸다.
- 함수가 마지막으로 하는 일이 어떤 호출이고 그 결과가 곧 반환값이면 그 호출이 꼬리 호출이다. 재귀 호출이 꼬리 호출이면 스택 프레임이 재사용되어 반복문과 같은 메모리로 돈다. 소박한 재귀를 꼬리 재귀로 바꾸는 표준 수법이 누적값을 매개변수로 나르는 것이다. 자기 자신을 직접 부르는 꼬리 호출은 언제나 최적화되지만, 상호 재귀처럼 다른 함수로 넘어가는 꼬리 호출은 `--tailcalls+` 가 켜져 있어야 한다.
- 누적값을 한 연산으로 이어 붙여 나갈 때 초기값은 그 연산의 항등원이다. 곱셈은 `1`, 덧셈은 `0`, 문자열 이어 붙이기는 `""` 다. `List.fold` 의 초기값을 정하는 기준과 같다. 점화식의 시작 값을 담는 누적값은 이 기준에서 벗어나고, `stepWays` 의 `(1L, 1L)` 이 그런 경우다.
- 누적값을 공개 시그니처에 드러내지 않으려면 안쪽에 지역 `loop` 를 두고 바깥 함수가 초기값을 넣어 첫 호출을 한다. 실측하면 소박한 버전과 꼬리 재귀 버전의 시그니처가 `n: int -> int64` 로 똑같다.
- 누적값은 튜플이나 레코드로 넓힐 수 있다. 직전 두 항이 필요한 점화식은 `(prev, curr)` 를 나르고 매 단계에서 `(curr, prev + curr)` 로 한 칸 민다. 계층 데이터 절의 `Voyage` 레코드도 같은 역할이다.
- 리스트를 앞에서 한 번 훑으며 상태를 갱신하는 재귀는 `List.fold` 로 그대로 옮겨진다. 실측하면 두 구현의 시그니처가 `plan: (int * string) list -> showNo: int -> string` 로 같고 결과도 같다. 리스트 하나를 훑는 일이라면 직접 재귀보다 `fold` 가 먼저다. 전용 집계 함수가 있으면 그쪽이 먼저라는 5챕터의 순서는 그대로다.
- [`<TailCall>`] 특성은 F# 8 부터 꼬리 호출 여부를 컴파일러에게 확인받는 수단이다. 검사는 프로젝트 빌드에서만 돌고 FSI 스크립트에서는 돌지 않으며, 지역 `let rec` 에는 붙일 수 없어 오류 FS0010 이 난다.
- 퀵소트는 `List.partition` 과 `@` 로 여섯 줄에 적힌다. 꼬리 재귀는 아니고 이미 정렬된 입력에서 시간이 제곱으로 늘어난다. 실측하면 무작위 2만 개 0.15초, 정렬된 2만 개 12.3초다. 실무에서는 `List.sort` 를 쓴다.
- 계층 데이터는 재귀형 판별 유니온으로 정의하고 케이스마다 한 갈래씩 적으면 순회 함수가 자연히 재귀가 된다. 자식 결과를 `List.collect` 로 합치는 형태가 기본형이다.
- `Map.find` 와 대괄호 인덱서, `List.minBy` 는 부분 함수다. 실패를 값으로 다루려면 `Map.tryFind` 와 `List.isEmpty` 검사를 앞에 둔다. 대괄호 인덱서 문법은 F# 6 부터이고 F# 5 에서는 오류 FS3217 이다.
- 긴 문제는 데이터 적재, 구조 만들기, 결과 고르기처럼 토막으로 쪼개고 토막마다 FSI 로 확인하며 나아간다. 원서가 이 챕터에서 보여 주는 작업 방식 자체가 배울 거리다.

## 원서 대조 표

| 절                                                     | 원서 페이지          | 실행 단위        |
|-------------------------------------------------------|-----------------|--------------|
| Setting Up — 준비                                       | p.141           | —            |
| Solving The Problem — 재귀 함수의 두 갈래                     | pp.141-142      | 11-basics    |
| 상호 재귀 — <code>and</code> 로 잇는다 (노트 보충)                | —               | 11-basics    |
| Tail Call Optimisation — 꼬리 호출과 누적값                   | pp.142-143      | 11-tailrec   |
| <code>[&lt;TailCall&gt;]</code> 로 컴파일러에게 확인받기 (노트 보충) | —               | 11-tailrec   |
| Expanding the Accumulator — 누적값을 튜플로 넓히기              | pp.143-144      | 11-tailrec   |
| Using Recursion to Solve FizzBuzz — 규칙 목록을 누적하기       | pp.144-145      | 11-fold      |
| <code>List.fold</code> 로 다시 쓰기                        | p.145           | 11-fold      |
| Quicksort using recursion — 퀵소트                       | pp.145-146      | 11-quicksort |
| Recursion with Hierarchical Data / 트리 정의와 순회          | pp.146-147, 149 | 11-tree      |
| Recursion with Hierarchical Data / 데이터 적재             | pp.147-148      | 11-tree      |
| Recursion with Hierarchical Data / 가능한 경로 펼치기         | pp.148-150      | 11-tree      |
| Recursion with Hierarchical Data / 최단 경로 고르기          | p.150           | 11-tree      |
| Finished Code — 완성된 코드                                | pp.151-152      | 11-tree      |
| Other Uses Of Recursion — 그 밖의 쓸모                     | p.152           | —            |
| Summary — 원서의 챕터 요약                                   | p.152           | —            |

## 12 - 계산 식 (원서 pp.153-165)

3챕터가 `Option` 과 `Result` 를 세우고 실패를 값으로 다루는 법을 알려 준 뒤로, 노트는 줄곧 `map` 과 `bind` 로 그 값을 이어 왔다. 이 방식은 정확하지만 함수가 네다섯 개로 늘어나면 파이프라인이 익명 함수와 괄호로 덮인다. 8챕터 후반에서 중첩이 다섯 겹까지 깊어지는 것도 이미 봤다. 계산 식(computation expression)은 그 이음매를 문법 설탕(syntactic sugar)으로 감춰 준다. 감추는 것은 실패 선로뿐이고 하는 일은 조금도 바뀌지 않는다. 이 챕터는 `Option` 용 계산 식 빌더를 직접 만들어 그 안을 들여다보고, 같은 문법이 `Result` 와 `Async` 에도 그대로 통하는 것을 확인한 뒤, 효과 둘을 겹친 `asyncResult` 까지 간다.

읽기 전에 챕터 둘을 전제한다.

- 3챕터의 `Option · Result`, `bind · map · mapError` 의미론, 그리고 여러 함수를 이으려면 실패 타입을 하나로 맞춰야 한다는 사정. 계산 식은 이 셋을 없애 주는 것이 아니라 부르는 자리를 감춰 주는 것이다.
- 8챕터의 `validation` 계산 식. 원서 p.153 이 이 챕터를 마치면 8챕터 검증 예제를 다시 보라고 권한다. 그쪽에서 `let!` 과 `and!` 의 차이는 이미 다뤘고, 여기서는 그 문법이 어느 멤버를 부르는지를 본다.

용어 하나를 먼저 못 박는다. 이 챕터에서 효과(effect)는 `Option · Result · Async` 처럼 값을 한 겹 감싸 성패나 비동기 같은 맥락을 함께 나르는 타입을 가리킨다. 원서 p.153 의 낱말이다. 3챕터부터 써 온 부수 효과(side effect)와는 다른 것이므로 이 노트는 부수 효과를 줄여 쓰지 않는다.

## Setting Up — 준비 (원서 p.153)

- 원서는 콘솔 프로젝트를 만들고 `OptionDemo.fs · ResultDemo.fs · AsyncDemo.fs · AsyncResultDemo.fs` 를 차례로 엮으며 `dotnet run` 으로 확인한다.
- 이 노트는 파일을 만들지 않고 FSI 스크립트로 확인한다. 원서의 네임스페이스와 모듈 구획은 실행 단위 구획으로 대신한다.
- 뒤쪽 두 절은 `FsToolkit.ErrorHandling` 패키지가 필요하다. 원서는 `dotnet add package` 를 쓰고 이 노트는 `#r "nuget: ..."` 로 버전을 못 박아 끌어온다. 처음 한 번은 네트워크가 있어야 패키지를 받아 오고 그 뒤에는 로컬 NuGet 캐시로 돈다. 받아 오지 못하면 `error FS0999` 로 실패한다.

## Introduction — 손으로 이은 파이프라인 (원서 pp.153-155)

- 출발점은 효과를 내는 함수와 내지 않는 함수가 섞인 파이프라인이다. 그대로 이으면 컴파일되지 않는다.
- 원서는 같은 함수를 세 번 다시 쓴다. `match` 로 펼친 것, `Option.map / Option.bind` 로 줄인 것, 계산 식으로 감싼 것이다. 세 번 다 결과가 같다는 점이 이 절의 요지다.
- 소재는 곡물 창고다. 총 중량을 트럭 수로 나눠 한 대에 실을 무게를 구하고, 덮개 무게를 더한 뒤, 그 무게를 자루 수로 다시 나눈다. 나눗셈이 두 번 나오므로 `0` 으로 나누는 경우가 두 번 생기고, 그것이 `Option` 을 쓸 이유가 된다.

```
// 이 단위가 보여주는 것: 효과를 계산 식 없이 있는 두 방법과 그 결과가 같다는 것
// FSI 실측: split: total: int -> parts: int -> int option
let split total parts =
    if parts = 0 then None else Some (total / parts)

// FSI 실측: withCover: load: int -> int
let withCover load = load + 3
```

`split` 은 효과를 만드는 함수이고 `withCover` 는 만들지 않는 함수다. 이 차이가 뒤에 나오는 모든 판단의 기준이 된다. 두 함수를 그냥 이으면 다음처럼 되는데 이것은 컴파일되지 않는다.

```
let perSack total trucks sacks =
    split total trucks
    |> fun load -> withCover load           // error FS0001
    |> fun covered -> split covered sacks
```

`split total trucks` 가 내놓는 것은 `int option` 인데 `withCover` 는 `int` 를 받는다. 오류는 `'int'` 형식이 필요하지만 ... `'int option'` 형식이 지정되었습니다 다. 위 블록의 마지막 줄이 익명 함수를 쓰는 것은 `split` 의 매개변수 순서가 앞으로 파이프하기에 맞지 않아서다. 파이프된 값이 둘째 자리에 들어가야 하므로 이름으로 받아 넘긴다. 이 군더더기도 계산 식이 없애 줄 것 가운데 하나다.

`match` 로 펼치면 컴파일된다. 코드는 길어지지만 무슨 일이 벌어지는지가 남김없이 드러난다.

```
// FSI 실측: perSackMatched: total: int -> trucks: int -> sacks: int -> int option
let perSackMatched total trucks sacks =
    split total trucks
    |> fun r ->
        match r with
        | Some load -> withCover load |> Some
        | None -> None
    |> fun r ->
        match r with
        | Some covered -> split covered sacks
        | None -> None
```

`trucks = 0` 인 경우를 따라가 보면 이 함수의 성격이 보인다. `trucks = 0` 이면 첫 `split` 이 `None` 을 내고, 그 뒤 두 `match` 는 `None` 갈래만 지난다. `withCover` 도 두 번째 `split` 도 아예 호출되지 않는다. 함수 중간에서 빠져나가는 조기 반환(early return)이 일어난 것이 아니다. 조기 반환은 F# 에 없다. 두 `match` 는 끝까지 평가 되고, 다만 `None` 갈래가 아무것도 부르지 않는 것이다.

두 `match` 의 모양은 3챕터에서 본 것과 똑같다. 효과를 만들지 않는 함수를 적용하는 자리는 `Option.map` 이고 만드는 함수를 적용하는 자리는 `Option.bind` 다.

```
// FSI 실측: perSackPiped: total: int -> trucks: int -> sacks: int -> int option
let perSackPiped total trucks sacks =
    split total trucks
    |> Option.map withCover
    |> Option.bind (fun covered -> split covered sacks)

// %A 에는 꼭 지정이 먹지 않으므로 sprintf 로 문자열을 만든 뒤 %-9s 로 폭을 맞춘다
let show label a b = printfn "%-9s %-9s %s" label (sprintf "%A" a) (sprintf "%A" b)
show "trucks=0" (perSackMatched 900 0 5) (perSackPiped 900 0 5)
show "sacks=0" (perSackMatched 900 3 0) (perSackPiped 900 3 0)
show "ok" (perSackMatched 900 3 5) (perSackPiped 900 3 5)
// trucks=0   None      None
// sacks=0    None      None
// ok         Some 60   Some 60
```

세 줄 모두 두 함수의 결과가 같고 시그니처도 같다. `map` / `bind` 로 줄인 것은 짧아졌을 뿐 동작이 달라지지 않았다.

## Introduction — option 계산 식을 직접 만든다 (원서 pp.155-156)

- F# 코어에는 `option` 용 계산 식이 없다. 코어에 들어 있는 것은 `seq` · `async` · `task` · `query` 넷이다(실측 확인).
- 그래서 이 절은 사용자 정의 계산 식(custom computation expression)을 만든다. 계산 식 빌더는 정해진 이름의 멤버를 담은 클래스 타입이고, 계산 식 이름은 그 타입의 인스턴스를 묶은 소문자 값이다.
- 멤버 이름은 컴파일러가 이름으로 찾아 부르는 것이므로 원어 그대로 쓴다. `Bind` 가 `let!` 과 `do!` 를, `Return` 이 `return` 을, `ReturnFrom` 이 `return!` 을 받는다.



```
// 이 단위가 보여주는 것: option 계산 식 빌더를 만들어 앞 절의 파이프라인을 다시 쓰기
let split total parts =
    if parts = 0 then None else Some (total / parts)
let withCover load = load + 3

[<AutoOpen>]
module GrainCe =

    type OptionBuilder() =
        // let! 과 do! 를 받는다
        // FSI 실측: member Bind: x: 'c option * f: ('c -> 'd option) -> 'd option
        member _.Bind(x, f) = Option.bind f x
        // return 을 받는다
        // FSI 실측: member Return: x: 'b -> 'b option
        member _.Return(x) = Some x
        // return! 을 받는다
        // FSI 실측: member ReturnFrom: x: 'a -> 'a
        member _.ReturnFrom(x) = x

    // 계산 식 이름은 이 소문자 값이다. 쓰는 모양은 option { ... }
    let option = OptionBuilder()
```

[<AutoOpen>] 은 4챕터에서 이미 쓴 특성이다. 이 특성을 붙인 모듈은 그 네임스페이스나 어셈블리를 참조하기만 하면 모듈 안의 이름이 `open` 없이 보인다. 스크립트에서도 마찬가지인데, 이때는 암시적 모듈 안에 중첩된 `GrainCe` 가 그 자리에서 열린 상태가 되는 것이다. 특성을 떼고 `module GrainCe` 로만 두면 `option { ... }` 자리에서 `error FS0800: 형식 이름을 잘못 사용했습니다` 가 난다(실측 확인). 빌더가 안 보이면 컴파일러가 `option` 을 `Option<'T>` 의 타입 약어 이름으로 읽기 때문에 이런 낯선 오류가 나온다.

이제 앞 절의 함수를 계산 식으로 다시 쓴다.

```
// FSI 실측: perSackCe: total: int -> trucks: int -> sacks: int -> int option
let perSackCe total trucks sacks =
    option {
        let! load = split total trucks
        let covered = withCover load
        let! perSack = split covered sacks
        return perSack
    }
```

`map / bind` 로 쓴 것과 시그니처가 같고 결과도 같다. 달라진 것은 `None` 선로가 코드에서 사라진 점이다. 남은 네 줄은 실패가 한 번도 나지 않을 때 지나는 길, 곧 해피 패스(happy path)만 적고 있다.

줄마다 무엇이 일어나는지 보면 이렇다.

- `let!` 의 `!` 는 효과를 한 겹 벗기라는 표시다. `split total trucks` 의 타입은 `int option` 인데 `load` 의 타입은 `int` 다. 벗기는 일은 빌더의 `Bind` 가 하고, `Bind` 는 `None` 을 받으면 뒤에 오는 함수를 부르지 않는다. 앞 절 `match` 의 `None` 갈래가 여기로 들어간 것이다.
- `!` 가 없는 `let covered = ...` 는 평범한 `let` 바인딩이다. 원서 p.155 는 이 자리를 `option.map` 을 쓸 뻔한 `let` 바인딩은 계산 식이 알아서 처리한다는 식으로 설명하는데, 그대로 읽으면 오해가 생긴다. 이 `let` 은 `map` 으로 바뀌지 않는다. 아래에서 실측으로 확인한다.
- `return` 은 `!` 의 반대다. 감싸지 않은 값을 받아 효과를 입힌다. 빌더의 `Return` 이 `Some x` 인 것이 그 일이다. F# 에서 `return` 이라는 낱말이 필요한 자리는 계산 식뿐이다.

`perSackCe` 의 네 줄이 각각 어느 멤버로 풀리는지는 빌더를 직접 불러 보면 눈으로 확인된다.

```
// 계산 식이 풀린 모양을 손으로 적어 본 것. perSackCe 와 결과가 같다
// FSI 실측: perSackByHand: total: int -> trucks: int -> sacks: int -> int option
let perSackByHand total trucks sacks =
  option.Bind(split total trucks, fun load ->
    let covered = withCover load
    option.Bind(split covered sacks, fun perSack ->
      option.Return perSack))

printfn "%-8s %A" "byHand" (perSackByHand 900 3 5)
// byHand    Some 60
```

let! 두 줄이 option.Bind 두 번으로, return 이 option.Return 으로 풀린 것이다. ! 없는 let covered 만 빌더를 거치지 않고 그대로 남았다.

let! 에서 ! 를 떼면 어긋나는 지점이 바로 드러난다.

```
let perSackBroken total trucks sacks =
  option {
    let load = split total trucks
    let covered = withCover load // error FS0001: 'int' 형식이 필요하지만 'int option'
    let! perSack = split covered sacks
    return perSack
  }
```

load 가 int option 으로 묶여 withCover 에 들어가지 못한다. 원서는 마우스를 얹어 타입을 확인해 보라고 하는데, 스크립트에서는 타입 주석으로 같은 것을 확인할 수 있다. ! 가 있는 자리에 int 주석을 달아 두면 load 가 한 겹 벗겨져 int 가 된 것이 눈으로 확인되고, 주석이 없어도 컴파일된다.

```
// return! 을 쓰는 형태. 마지막 식이 이미 int option 이므로 return 이 아니라 return! 이다
// FSI 실측: perSackFrom: total: int -> trucks: int -> sacks: int -> int option
let perSackFrom total trucks sacks =
  option {
    let! (load: int) = split total trucks // 주석은 ! 가 한 겹 벗겼음을 보이려고 단 것이고 없어도 된다
    let covered = withCover load
    return! split covered sacks
  }

printfn "%-8s %A" "return" (perSackCe 900 3 5)
printfn "%-8s %A" "return!" (perSackFrom 900 3 5)
printfn "%-8s %A" "fail" (perSackFrom 900 3 0)
// return    Some 60
// return!   Some 60
// fail      None
```

return 과 return! 의 갈림은 오른쪽 식이 효과를 만드는지 하나로 정해진다. split covered sacks 는 이미 int option 이므로 Return 으로 한 겹 더 감싸면 int option option 이 된다. 그래서 ReturnFrom 이 필요하고, 빌더에 적은 member \_.ReturnFrom(x) = x 가 아무 일도 하지 않는 것이 옳다. 실측 시그니처가 x: 'a -> 'a 인 것도 같은 말이다.

! 없는 let 이 map 으로 바뀌지 않는다는 것은 빌더를 하나 더 만들면 확인된다. Return 하나만 있는 빌더로도 let 은 돈다.

```
// Bind 도 ReturnFrom 도 없는 빌더
type ReturnOnlyBuilder() =
    member _.Return(x) = Some x
let onlyReturn = ReturnOnlyBuilder()

// FSI 실측: plain: n: int -> int option
let plain n =
    onlyReturn {
        let doubled = n * 2
        let raised = doubled + 1
        return raised
    }
printfn "%A" (plain 20) // Some 41
```

`map` 에 해당하는 멤버가 빌더에 없는데도 `let` 두 줄이 통과한다. 계산 식 안의 `!` 없는 `let` 은 빌더 멤버를 부르지 않는 보통 바인딩이다. 뒤집어 말하면 `Bind · Return · ReturnFrom` 세 멤버만 적어 두면 `let!` · `let · return · return!` · `do!` 가 전부 돈다(실측 확인). `Zero` 는 `else` 없는 `if` 를 적을 때 필요해지는데, 빌더에 없으면 컴파일러가 `error FS0708` 로 그 멤버 이름을 알려 준다. `Combine` 은 계산 식 값을 내는 식이 둘 이상 이어질 때 필요해진다. 둘 다 원서의 범위 밖이다.

## The Result Computation Expression — 효과가 바뀌어도 문법은 그대로 (원서 pp.156-158)

- 계산 식의 값어치는 효과마다 문법을 새로 배우지 않아도 된다는 점에 있다. `Option` 에서 `Result` 로 갈아타도 `let!` · `let · return!` 의 쓰임은 그대로다.
- `result` 계산 식은 F# 코어에 없다. 빌더를 또 만드는 대신 원서는 `FsToolkit.ErrorHandling` 이 제공하는 것을 쓴다. 이 패키지는 8챕터에서 `validation` 계산 식을 쓸 때 이미 끌어왔다.
- 소재는 농가 정산이다. 수확량을 조회하고, 기준을 넘으면 인증 표시를 붙이고, 인증 여부에 따라 보조금을 더한다. 세 함수 가운데 둘이 `Result` 를 내고 하나는 내지 않는다.

아래 블록은 처음 한 번 네트워크로 패키지를 받아 오고 그 뒤에는 로컬 NuGet 캐시로 돈다. 받아 오지 못하면 `error FS0999` 로 실패한다.

```
// 이 단위가 보여주는 것: 효과가 Result 로 바뀌어도 계산 식 문법이 그대로라는 것
#r "nuget: FsToolkit.ErrorHandling, 5.2.0"
open FsToolkit.ErrorHandling

type Farm = {
    Code: string
    Certified: bool
    Subsidy: decimal
}

// FSI 실측: fetchYield: farm: Farm -> Result<(Farm * decimal),exn>
let fetchYield farm =
    try
        // 실제로는 수확 기록 저장소를 조회하는 자리다
        let tons = if farm.Code.EndsWith "0" then 42M else 17M
        Ok (farm, tons)
    with ex -> Error ex

// FSI 실측: certifyIfAbundant: farm: Farm * tons: decimal -> Farm
let certifyIfAbundant (farm, tons) =
    if tons > 30M then { farm with Certified = true } else farm

// FSI 실측: addSubsidy: farm: Farm -> Result<Farm,exn>
let addSubsidy farm =
    try
        let extra = if farm.Certified then 300M else 120M
        Ok { farm with Subsidy = farm.Subsidy + extra }
    with ex -> Error ex
```

`fetchYield` 와 `addSubsidy` 는 효과를 만들고 `certifyIfAbundant` 는 만들지 않는다. 앞 절의 `split / withCover` 와 같은 구도다. 실패 타입을 둘 다 `exn` 으로 맞춰 둔 것도 우연이 아니다. 3챕터에서 본 대로 실패 타입이 어긋나면 이을 수 없다.

```
// map/bind 로 이은 형태
// FSI 실측: settlePiped: farm: Farm -> Result<Farm,exn>
let settlePiped farm =
    farm
    |> fetchYield
    |> Result.map certifyIfAbundant
    |> Result.bind addSubsidy

// 같은 일을 result 계산 식으로
// FSI 실측: settleCe: farm: Farm -> Result<Farm,exn>
let settleCe farm =
    result {
        let! withYield = fetchYield farm
        let judged = certifyIfAbundant withYield
        return! addSubsidy judged
    }
```

! 가 붙은 줄은 효과를 만드는 함수 두 개뿐이다. 계산 식이 아닌 쪽에서 `Result.map` 을 쓸 자리가 ! 없는 `let` 이고, `Result.bind` 를 쓸 자리가 `let!` 이며, 마지막 `Result.bind` 는 `return!` 이 받는다. `map` 이 따로 필요 없어지는 이유는 `bind` 가 이미 한 겹 벗긴 값을 뒤에 오는 함수에 넘겨 주기 때문이다. `let!` `withYield = fetchYield farm` 이 지나간 뒤 `withYield` 의 타입은 `Result<(Farm * decimal),exn>` 이 아니라 `Farm * decimal` 이고, 그래서 다음 줄은 감싼 값을 열어 주는 `map` 없이 `certifyIfAbundant` 를 그대로 부른다. 어느 쪽이 읽기 좋은지는 취향이 갈리는데, 원서도 두 형태를 나란히 두고 계산 식 쪽을 권하는 정도로 말한다.

```
let show label (r: Result<Farm, exn>) =
    match r with
    | Ok f -> printfn "%-6s %s 인증=%-5b 보조금=%M" label f.Code f.Certified f.Subsidy
    | Error ex -> printfn "%-6s 실패 %s" label ex.Message

let bumper = { Code = "F-100"; Certified = false; Subsidy = 0M }
let lean = { Code = "F-101"; Certified = false; Subsidy = 0M }
show "piped" (settlePiped bumper)
show "ce" (settleCe bumper)
show "piped" (settlePiped lean)
show "ce" (settleCe lean)
// piped F-100 인증=true 보조금=300
// ce F-100 인증=true 보조금=300
// piped F-101 인증=false 보조금=120
// ce F-101 인증=false 보조금=120
```

한 가지 함정이 있다. 계산 식이 만든 값을 모듈 수준에 그냥 묶으면 실패 타입이 정해지지 않아 값 제한에 걸린다.

```
// error FS0030: 값 제한: 값 'staged'에 유추된 제네릭 형식이 있습니다.
// val staged: Result<Farm, 'a>
let staged = result { return bumper }
```

`return bumper` 는 성공 타입만 알려 주고 실패 타입은 아무것도 정하지 않는다. `Result<Farm, 'a>` 의 `'a` 가 그것이다. 타입 주석을 달면 해결된다.

```
// FSI 실측: staged: Result<Farm,exn>
let staged : Result<Farm, exn> = result { return bumper }
printfn "%A" (staged |> Result.map (fun f -> f.Code)) // Ok "F-100"
```

함수 안에서 쓰면 이 문제가 나지 않는다. 값 제한에 걸릴 수 있는 것은 매개변수 없는 `let` 뿐이고, 매개변수가 있으면 정해지지 않은 타입이 자동 일반화된다. `let wrap (x: int) = result { return x }` 의 시그니처가 `x: int -> Result<int, 'a>` 로 나오는 것이 그 증거다(실측 확인). 위의 `settleCe` 는 자동 일반화까지 갈 일도 없다. 몸통에서 `addSubsidy` 를 부르므로 실패 타입이 `exn` 으로 정해진다. 참고로 `result { ... }` 를 패키지 없이 적으면 오류가 `error FS0039: 'result' 값 또는 생성자가 정의되지 않았습니다` 다. 앞 절의 `option` 은 타입 약어 이름과 겹쳐 `error FS0800` 이었는데, `result` 는 겹치는 이름이 없어 오류가 다르게 나온다.

## Introduction to Async — 지연 평가되는 효과 (원서 pp.158-159)

- `async` 는 코어에 들어 있는 계산 식이고 F# 의 비동기 지원이다. C# 의 `async/await` 와 결이 비슷하지만 지연 평가된다는 점이 다르다.
- `Async<'a>` 값을 만드는 것만으로는 본문이 돌지 않는다. `Async.RunSynchronously` 같은 실행 함수를 만나야 돈다. 원서는 이 함수를 애플리케이션 진입점에서만 쓰라고 못 박는다.
- .NET 라이브러리 함수는 `Async` 가 아니라 `Task` 를 내놓는다. `Async.AwaitTask` 로 바꿔야 `let!` 이 받는다.
- 원서는 `resources/customers.csv` 를 만들어 읽지만 이 노트는 스크립트가 임시 파일을 직접 만들어 쓰고 지운다.

```
// 이 단위가 보여주는 것: async 계산 식, Async.AwaitTask, 그리고 지연 평가
open System.IO

type LogFacts = {
    Name: string
    Bytes: int
}

// FSI 실측: readFacts: path: string -> Async<LogFacts>
let readFacts path =
    async {
        printfn " (async 본문 시작)"
        let! bytes = File.ReadAllBytesAsync(path) |> Async.AwaitTask
        let name = Path.GetFileName(path)
        return { Name = name; Bytes = bytes.Length }
    }
```

`File.ReadAllBytesAsync` 는 .NET 함수라서 `Task<byte array>` 를 내놓는다. `Async.AwaitTask` 를 끼우지 않으면 `let!` 이 그것을 벗길 수 없다. 벗긴 뒤 `bytes` 의 타입은 `byte array` 이고, `Path.GetFileName` 은 효과를 만들지 않으므로 ! 없는 `let` 이다. 마지막 `return` 은 레코드를 `Async` 로 감싼다. 앞 두 절과 문법이 한 글자도 다르지 않다.

```
let path = Path.Combine(Path.GetTempPath(), "grain-intake.log")
File.WriteAllText(path, "F-100|42\nF-101|17\n")

let job = readFacts path
printfn "async 값을 만든 뒤"
let facts = job |> Async.RunSynchronously
printfn "%s / %d 바이트" facts.Name facts.Bytes
// async 값을 만든 뒤
// (async 본문 시작)
// grain-intake.log / 18 바이트
```

출력 순서가 지연 평가의 증거다. `readFacts path` 를 부른 시점에는 본문의 `printfn` 이 돌지 않았고, `Async.RunSynchronously` 를 만나서야 돌았다. `Task` 는 이 점이 반대다.

```
let started = task { printfn " (task 본문)"; return 7 }
printfn "task 를 만든 뒤"
printfn "%d" started.Result
File.Delete path
// (task 본문)
// task 를 만든 뒤
// 7
```

`task` 는 만드는 순간 본문이 시작된다. 코어에 들어 있는 계산 식은 `seq · async · task · query` 넷인데 `task` 는 F# 6 부터다. F# 5 로 낮추면 `error FS3350` 으로 6.0 이상을 쓰라고 한다(실측 확인). 13챕터부터 웹 개발로 들어가면 `task` 를 쓰게 된다.

## Compound Computation Expressions — 효과 둘을 겹치기 (원서 pp.159-163)

- 효과가 하나면 여기까지로 충분하지만 실무에서는 둘이 겹친다. 비동기로 조회하면서 실패도 값으로 다루려면 타입이 `Async<Result<'a, 'e>>` 가 된다.
- 이런 자리에 쓰는 것이 복합 계산 식(compound computation expression)이다. `asyncResult` 는 `FsToolkit.ErrorHandling` 이 제공하고 `Async` 가 `Result` 를 감싼 순서를 다룬다. 원서는 이 조합이 F# 로 쓴 업무용 애플리케이션(LOB, Line of Business)에서 아주 흔하다고 말한다.
- 겹친 만큼 실패 타입을 맞추는 일이 늘어난다. 그 손질을 패키지의 도우미 함수가 대신한다. 비동기 쪽 값은 `AsyncResult` 모듈, 동기 쪽 값은 `Result` 모듈에서 찾는다.
- 소재는 공유 자전거 일일 이용권 발급이다. 회원을 조회하고, 비밀번호를 확인하고, 이용 자격을 확인하고, 이용권을 발급한다. 네 단계가 각각 다른 모양의 효과를 낸다.

이 단위도 `FsToolkit.ErrorHandling` 이 필요하다. 처음 한 번은 네트워크로 받아 오고 그 뒤에는 로컬 NuGet 캐시로 돌며, 받아 오지 못하면 `error FS0999` 로 실패한다.

```
// 이 단위가 보여주는 것: Async 와 Result 를 겹친 asyncResult 계산 식
#r "nuget: FsToolkit.ErrorHandling, 5.2.0"
open System
open FsToolkit.ErrorHandling

type StandingError =
    | AccountOnHold

type IssueError =
    | TerminalFault of string

// 네 단계의 실패를 한 타입으로 모은다
type PassError =
    | UnknownRider
    | WrongPin
    | NotInGoodStanding of StandingError
    | IssueFailed of IssueError

type DayPass = DayPass of Guid

type RiderStatus =
    | Good
    | Frozen
    | Barred

type Rider = {
    Name: string
    Pin: string
    Status: RiderStatus
}
```

실패 타입이 셋이라는 점을 눈여겨볼 만하다. 자격 확인은 `StandingError` , 발급은 `IssueError` 로 실패하고, 이용권 발급 함수 전체는 `PassError` 로 실패한다. `PassError` 의 두 케이스가 앞의 두 타입을 감싸고 있다. 3챕터가 세운 방식대로 각 단계가 제 실패 타입을 쓰고, 이어 붙이는 자리에서 `mapError` 로 갈아 끼우는 구조다.

```
[<Literal>]
let GoodPin = "4821"
[<Literal>]
let GoodRider = "rider-ok"
[<Literal>]
let FrozenRider = "rider-frozen"
[<Literal>]
let BarredRider = "rider-barred"
[<Literal>]
let JinxedRider = "rider-jinxed"
[<Literal>]
let FaultMessage = "단말기 카드 리더가 응답하지 않는다"
```

[<Literal>] 은 컴파일 시점 상수를 만든다. 이 특성이 붙은 이름만 `match` 케이스의 상수 패턴으로 쓸 수 있다. 특성을 떼면 `| GoodRider ->` 가 상수 패턴이 아니라 변수 패턴이 되어 모든 입력을 받아 삼킨다. 그때 컴파일러는 대문자 이름을 적은 줄마다 `warning FS0049` 로 대문자 변수 식별자를 쓰지 말라고 알려 주고, 뒤 케이스들에는 `warning FS0026` 으로 이 규칙은 결코 일치하지 않는다고 알려 준다(실측 확인). 경고이지 오류가 아니어서 그대로 돌아가는 만큼 더 위험하다. 원서는 [<Literal>] 을 뒷줄에 두는 형태와 `let [<Literal>] GoodPin = "4821"` 형태를 모두 보여 주는데 둘 다 유효하다.

```
// FSI 실측: tryFindRider: name: string -> Async<Rider option>
let tryFindRider name =
    async {
        let rider = { Name = name; Pin = GoodPin; Status = Good }
        return
            match name with
            | GoodRider -> Some rider
            | FrozenRider -> Some { rider with Status = Frozen }
            | BarredRider -> Some { rider with Status = Barred }
            | JinxedRider -> Some rider
            | _ -> None
    }

// FSI 실측: isPinValid: pin: string -> rider: Rider -> bool
let isPinValid pin rider =
    pin = rider.Pin

// FSI 실측: checkStanding: rider: Rider -> Async<Result<unit,StandingError>>
let checkStanding rider =
    async {
        return
            match rider.Status with
            | Good -> Ok ()
            | _ -> AccountOnHold |> Error
    }

// FSI 실측: issuePass: rider: Rider -> Result<DayPass,IssueError>
let issuePass rider =
    try
        if rider.Name = JinxedRider then failwith FaultMessage
        else Guid.NewGuid() |> DayPass |> Ok
    with ex -> ex.Message |> TerminalFault |> Error
```

네 함수의 반환 타입이 제각각이다. `Async<Rider option>`, `bool`, `Async<Result<unit,StandingError>>`, `Result<DayPass,IssueError>` 다. 안이 어떻게 구현됐는지는 중요하지 않다. 이 네 모양을 한 줄기로 잇는 것이 다음 함수의 일이다.

```
// FSI 실측: requestPass: name: string -> pin: string -> Async<Result<DayPass,PassError>>
let requestPass name pin : Async<Result<DayPass, PassError>> =
    asyncResult {
        let! rider = name |> tryFindRider |> AsyncResult.requireSome UnknownRider
        do! rider |> isPinValid pin |> Result.requireTrue WrongPin
        do! rider |> checkStanding |> AsyncResult.mapError NotInGoodStanding
        return! rider |> issuePass |> Result.mapError IssueFailed
    }
```



네 줄을 하나씩 읽는다.

- 첫 줄의 `AsyncResult.requireSome` 은 `Async<Rider option>` 을 `Async<Result<Rider, PassError>>` 로 바꾼다. `None` 이면 인자로 준 `UnknownRider` 를 실패로 삼는다. `let!` 이 두 겹을 한 번에 벗겨 `rider` 의 타입은 `Rider` 다.
- 둘째 줄의 `Result.requireTrue` 는 `bool` 을 `Result<unit, PassError>` 로 바꾼다. `isPinValid` 는 비동기가 아니므로 도우미 함수도 `Result` 모듈에서 가져온다. 비동기인 셋째 줄은 `AsyncResult` 모듈을 쓴다. 이 갈림이 원서 p.162 가 짚는 규칙이다.
- 셋째 줄의 `AsyncResult.mapError` 는 `StandingError` 를 `PassError` 로 갈아 끼운다. 3챕터에서 실패 타입을 맞추려고 `mapError` 를 어댑터로 쓴 것과 똑같은 쓰임이다.
- 마지막 줄은 `issuePass` 가 이미 `Result` 를 내놓으므로 `return!` 이다. `Result.mapError` 로 실패 타입만 `PassError` 로 넓힌다.

반환 타입 주석은 없어도 컴파일된다. 계산 식 네 줄에 나오는

`UnknownRider · WrongPin · NotInGoodStanding · IssueFailed` 가 모두 `PassError` 의 케이스라 실패 타입이 그 자리에서 정해지고, 주석을 떼도 `Async<Result<DayPass, PassError>>` 로 유추된다(실측 확인). 주석은 네 단계를 겹친 반환 타입을 한눈에 보이게 하려고 원서가 적어 둔 것이다.

```
let run name pin = requestPass name pin |> Async.RunSynchronously

let show label (r: Result<DayPass, PassError>) =
    match r with
    | Ok (DayPass id) -> printfn "%s 발급 (%s)" label (if id = Guid.Empty then "빈 Guid" else "Guid 생성")
    | Error e -> printfn "%s 거절 %A" label e

show "정상발급" (run GoodRider GoodPin)
show "번호오류" (run GoodRider "0000")
show "회원없음" (run "rider-nobody" GoodPin)
show "계정정지" (run FrozenRider GoodPin)
show "이용제재" (run BarredRider GoodPin)
show "단말고장" (run JinxedRider GoodPin)
// 정상발급 발급 (Guid 생성)
// 번호오류 거절 WrongPin
// 회원없음 거절 UnknownRider
// 계정정지 거절 NotInGoodStanding AccountOnHold
// 이용제재 거절 NotInGoodStanding AccountOnHold
// 단말고장 거절 IssueFailed (TerminalFault "단말기 카드 리더가 응답하지 않는다")
```

여섯 경우가 네 단계 가운데 어디서 갈라졌는지를 실패값이 그대로 말해 준다. 원서는 이 확인을

`isOk · matchError` 같은 판정 함수와 `bool` 출력으로 하는데, 실패값을 그대로 찍으면 어느 단계에서 멈췄는지까지 보이므로 이 노트는 실패값을 그대로 찍는 형태를 택했다.

`Async.RunSynchronously` 는 `run` 에서 한 번만 부른다. `requestPass` 안에서 부르면 비동기의 의미가 없어진다. 원서가 이 함수를 진입점에서만 쓰라고 하는 것이 그 말이다.

## do! 가 실제로 받는 것 (원서 p.162 확장)

- 원서 p.162 는 `do!` 가 `unit` 을 반환하는 함수를 지원한다고 설명하는데 그대로 읽으면 틀린다. `do!` 가 받는 것은 평범한 `unit` 이 아니라 효과가 감싼 `unit` 이다. `Result<unit, 'e>` 나 `Async<unit>` 이다.
- `do!` 는 이름에 묶을 값이 없는 `let!` 이다. 부르는 멤버도 `Bind` 로 같다. 성공값이 `unit` 이니 버리고, 실패면 그 자리에서 실패 선로로 갈아탄다.
- 그래서 `do!` 는 검사 단계를 적는 자리다. 값을 얻으려는 것이 아니고 통과 여부만 확인하려는 것이다.

이 단위도 `FsToolkit.ErrorHandling` 이 필요하다. 처음 한 번은 네트워크로 받아 오고 그 뒤에는 로컬 NuGet 캐시로 돌며, 받아 오지 못하면 `error FS0999` 로 실패한다.

```
// 이 단위가 보여주는 것: do! 가 받는 타입과, 실패한 뒤 뒷줄이 호출되지 않는다는 것
#r "nuget: FsToolkit.ErrorHandling, 5.2.0"
open FsToolkit.ErrorHandling

// FSI 실측: ensureWeighed: net: decimal -> Result<unit, string>
let ensureWeighed net =
    if net > 0M then Ok () else Error "계근표가 비어 있다"

// FSI 실측: ensureSealed: seal: string -> Result<unit, string>
let ensureSealed seal =
    if seal <> "" then Ok () else Error "봉인 번호가 없다"

// FSI 실측: acceptLoad: seal: string -> net: decimal -> Result<string, string>
let acceptLoad seal net =
    result {
        do! ensureWeighed net
        do! ensureSealed seal
        return sprintf "%s/%.1fkg" seal net
    }

printfn "%A" (acceptLoad "S-77" 812.5M) // Ok "S-77/812.5kg"
printfn "%A" (acceptLoad "S-77" 0M)    // Error "계근표가 비어 있다"
printfn "%A" (acceptLoad "" 812.5M)    // Error "봉인 번호가 없다"
```

두 검사 함수의 반환 타입은 `unit` 이 아니라 `Result<unit, string>` 이다. 성공 자리가 `unit` 이라서 얻을 값이 없고, 쓸모는 실패 자리에 있다. 여기에 평범한 `unit` 을 반환하는 함수를 넣으면 컴파일되지 않는다.

```
let logIt (net: decimal) : unit = printfn "%M" net

let brokenResult seal net =
    result {
        // error FS0041: 'Source' 메서드와 일치하는 오버로드가 없습니다.
        // 알려진 인수 형식: unit
        do! logIt net
        return seal
    }

let brokenAsync (net: decimal) =
    async {
        // error FS0001: 이 식에는 'Async<'a>' 형식이 필요하지만 'unit' 형식이 지정되었습니다.
        do! logIt net
        return net
    }
```

오류 코드가 계산 식마다 다른 것은 빌더에 적힌 멤버가 다르기 때문이다(실측 확인). `async` 나 앞 절에서 손으로 만든 빌더는 `Bind` 의 첫 매개변수 타입이 그대로 어긋나 `error FS0001` 이 난다. `FsToolkit.ErrorHandling` 의 `result` 는 오른쪽 식을 한 번 걸러 주는 `Source` 오버로드가 따로 있어서 `error FS0041` 로 그 오버로드 목록을 보여 준다. 어느 쪽이든 말하는 것은 하나다. `do!` 에 평범한 `unit` 은 들어가지 않는다.

`Async<unit>` 도 같은 자리에 들어간다. 아래의 `Async.Sleep` 이 그런 함수다.

```
// FSI 실측: dispatch: seal: string -> Async<string>
let dispatch seal =
    async {
        do! Async.Sleep 1
        return sprintf "%s 출차" seal
    }
printfn "%s" (dispatch "S-77" |> Async.RunSynchronously)    // S-77 출차
```

## Debugging Code — `Error` 선로로 갈아탄 뒤 (원서 p.164)

- 원서는 중단점을 걸어 디버깅하는 방법을 소개하며 F# 개발자가 그것을 자주 하지 않는다고 덧붙인다. 순수 함수와 FSI 로 확인하는 편이 빠르기 때문이다.
- 원서가 중단점으로 짚는 사실은 코드로도 확인된다. 한 번 실패 선로로 갈아타면 뒤에 남은 성공 선로의 코드는 아예 호출되지 않는다.
- 이것을 조기 반환이라고 부르지 않는 이유는 앞서 본 것과 같다. 빠져나가는 것이 아니라 `Bind` 가 뒷줄을 담은 함수를 부르지 않는 것이다.

```
// 호출되면 흔적을 남기는 함수
let makeSlip seal net =
    printfn " (전표 발행 호출됨)"
    sprintf "%s/%.1fkg" seal net

let acceptLoadTraced seal net =
    result {
        do! ensureWeighed net
        do! ensureSealed seal
        return makeSlip seal net
    }

printfn "통과 경우"
printfn "%A" (acceptLoadTraced "S-77" 812.5M)
printfn "실패 경우"
printfn "%A" (acceptLoadTraced "S-77" 0M)
// 통과 경우
// (전표 발행 호출됨)
// Ok "S-77/812.5kg"
// 실패 경우
// Error "계근표가 비어 있다"
```

실패 경우에는 흔적이 남지 않았다. 첫 `do!` 에서 `Bind` 가 `Error` 를 받고 뒷줄 전체를 담은 함수를 부르지 않았기 때문이다. 원서가 중단점이 두 번만 걸린다고 말하는 것도 같은 현상이다.

## Further Reading — 더 읽을 것 (원서 p.164)

---

- 계산 식의 쓰임은 효과 처리 하나가 아니다. 도메인 특화 언어(DSL, Domain-Specific Language)를 만드는 데도 쓰인다. 원서는 예로 Saturn 과 Farmer 를 든다. 둘 다 `let!` 로 값을 벗기는 것과는 결이 다르게, 중괄호 안에 선언을 쌓아 설정을 기술하는 형태다.
- 비동기는 이 챕터가 다룬 것보다 넓다. `Async` 와 `Task` 의 관계, 취소, 병행 실행은 MS 공식 문서의 비동기 프로그래밍 문서를 따라가는 것이 좋다.
- 8챕터의 `validation` 계산 식을 다시 볼 자리가 여기다. 그쪽의 `and!` 는 이 챕터에서 만든 `Bind` 가 아니라 `MergeSources` 멤버를 부른다. `let!` 을 연달아 쓰면 뒷줄이 앞줄의 값에 기대므로 첫 실패에서 멈추고, `and!` 로 이르면 그 의존이 끊겨 모든 줄이 평가되고 실패가 모인다. 병렬로 도는 것이 아니라 의존이 없어지는 것이며, `and!` 로 이은 오른쪽 식들은 같은 스레드에서 차례로 평가된다. 진짜 병렬이 필요하다면 같은 패키지의 `parallelAsyncValidation` 계산 식이 따로 있다.

## Summary — 원서의 챕터 요약 (원서 p.165)

---

- 원서는 계산 식이 처음에는 헛갈리지만 효과를 다루는 일반적인 수단이고 코드를 짧고 읽기 좋게 만들어 준다고 정리한다.
- 다음 챕터부터는 Giraffe 라이브러리로 API 와 웹 페이지를 만들며 지금까지의 도구를 실제 애플리케이션에 쓴다.

## 정리 — 이 노트의 요약

- 계산 식은 효과를 다루는 코드에 씌우는 문법 설탕이다. `match` 로 펼친 것, `map / bind` 로 줄인 것, 계산 식으로 감싼 것은 시그니처도 결과도 같다. 감춰지는 것은 실패 선로이고 남는 것은 해피 패스다.
- 계산 식 빌더는 정해진 이름의 멤버를 담은 클래스 타입이고, 계산 식 이름은 그 인스턴스를 묶은 소문자 값이다. `Bind · Return · ReturnFrom` 세 멤버만 있으면 `let!` · `let` · `return` · `return!` · `do!` 가 전부 돈다(실측 확인).
- `let!` 은 효과를 한 겹 벗겨 이름에 묶고 `Bind` 를 부른다. `!` 를 떼면 감싼 값이 그대로 묶여 타입이 어긋난다. `!` 없는 `let` 은 보통 바인딩이며 `map` 으로 바뀌지 않는다. `Return` 하나만 있는 빌더로도 `let` 이 되는 것이 증거다.
- `return` 은 감싸지 않은 값에 효과를 입히고, `return!` 은 이미 감싼 값을 그대로 낸다. 오른쪽 식이 효과를 만드는지로 갈린다.
- `do!` 는 이름에 묶을 값이 없는 `let!` 이다. 받는 것은 평범한 `unit` 이 아니라 효과가 감싼 `unit ( Result<unit, 'e> · Async<unit> )`이다. 원서 p.162 의 서술을 그대로 읽으면 틀린다. 평범한 `unit` 을 주면 `async` 와 손으로 만든 빌더는 `error FS0001`, `FsToolkit.ErrorHandling` 의 `result` 는 `Source` 오버로드 때문에 `error FS0041` 이 난다(실측 확인).
- 코어에 들어 있는 계산 식은 `seq · async · task · query` 넷이고 `task` 는 F# 6 부터다. `option` 과 `result` 는 코어에 없다. 빌더 없이 `option { ... }` 을 적으면 `option` 이 타입 약어 이름으로 읽혀 `error FS0800`, `result { ... }` 는 `error FS0039` 다.
- `async` 는 지연 평가된다. `Async<'a>` 를 만드는 것만으로는 본문이 돌지 않고 `Async.RunSynchronously` 를 만나야 돈다. `task` 는 반대로 만드는 순간 시작한다. `Async.RunSynchronously` 는 진입점에서만 쓴다. .NET 함수가 내놓는 `Task` 는 `Async.AwaitTask` 로 바뀌야 `let!` 이 받는다.
- 효과가 둘 겹치면 복합 계산 식을 쓴다. `asyncResult` 는 `Async<Result<'a, 'e>>` 를 다루고 `FsToolkit.ErrorHandling` 이 제공한다. 도우미 함수는 비동기 쪽 값에 `AsyncResult` 모듈, 동기 쪽 값에 `Result` 모듈을 쓴다. 단계마다 다른 실패 타입은 `mapError` 로 함수 전체의 실패 타입으로 갈아 끼운다.
- 계산 식이 만든 값을 모듈 수준에 그냥 묶으면 실패 타입이 미정이라 값 제한 `error FS0030` 에 걸린다. 타입 주석을 달거나 함수 안에서 쓴다.
- `[<Literal>]` 을 붙인 이름만 `match` 케이스의 상수 패턴이 된다. 떼면 변수 패턴이 되어 모든 입력을 삼키고 `warning FS0049` 와 뒤 케이스의 `warning FS0026` 이 난다. 오류가 아니라 경고이므로 더 조심할 자리다.
- 실패 선로로 갈아탄 뒤 뒷줄이 실행되지 않는 것은 조기 반환이 아니다. `Bind` 가 뒷줄을 담은 함수를 부르지 않는 것이며, 호출 흔적을 남기는 함수를 끼워 보면 그대로 확인된다.

## 원서 대조 표

| 절                                                    | 원서 페이지     | 실행 단위             |
|------------------------------------------------------|------------|-------------------|
| Setting Up — 준비                                      | p.153      | —                 |
| Introduction — 손으로 이은 파이프라인                          | pp.153-155 | 12-option-manual  |
| Introduction — option 계산 식을 직접 만든다                   | pp.155-156 | 12-option-builder |
| The Result Computation Expression — 효과가 바뀌어도 문법은 그대로 | pp.156-158 | 12-result-ce      |
| Introduction to Async — 지연 평가되는 효과                   | pp.158-159 | 12-async          |
| Compound Computation Expressions — 효과 둘을 겹치기         | pp.159-163 | 12-asyncresult    |
| do! 가 실제로 받는 것                                       | p.162 확장   | 12-do-bang        |
| Debugging Code — Error 선로로 갈아탄 뒤                     | p.164      | 12-do-bang        |
| Further Reading — 더 읽을 것                             | p.164      | —                 |
| Summary — 원서의 챕터 요약                                  | p.165      | —                 |

## 13 - Giraffe 로 웹 프로그래밍 시작하기 (원서 pp.166-177)

여기서부터 마지막 세 챕터는 앞에서 배운 것을 웹 애플리케이션 하나에 엮는다. 도구는 Giraffe 다. [ASP.NET Core](#) 위에 얹게 덮인 함수형 껍데기이고, 라우팅을 값으로 적고 요청 처리를 함수로 적는다. 이 챕터에서 새로 나오는 문법은 둘이다. 값을 그 자리에서 만드는 익명 레코드(anonymous record) `{ | ... | }` 와 핸들러를 이어 붙이는 `>=>` 다. 프로퍼티를 설정하는 할당 연산자 `<-` 와 그 짝인 `mutable` 은 1챕터와 10챕터에서 이미 나왔고, 비동기 작업을 적는 `task` 계산 식(computation expression)은 12챕터가 다뤘다. 이 챕터에서는 그 문법이 웹 애플리케이션을 구성하는 코드에 놓인다. 나머지는 2챕터의 함수 합성, 3챕터의 `Option · Result`, 9챕터의 단일 케이스 판별 유니온, 4챕터의 파일 컴파일 순서가 그대로 쓰이는 장면이다. 이 챕터에서 세운 프로젝트를 14챕터가 API 쪽으로, 15챕터가 화면 쪽으로 늘린다.

이 노트가 만드는 것은 옥상 양봉장의 벌통 점검 기록을 보여 주는 `HiveLog` 다. 벌통 코드로 점검 기록을 찾아 JSON 으로 돌려주는 API 와, 소개 문구만 담은 HTML 첫 화면이 이 챕터의 결과물이다.

## Getting Started — 빈 웹 프로젝트 만들기 (원서 pp.166-167)

- 원서는 폴더를 만들고 그 안에서 `dotnet new web -lang F#` 을 실행한다. 이 템플릿은 컨트롤러도 뷰도 없는 최소 웹 애플리케이션이다. 4챕터에서 쓴 `console` 템플릿과의 가장 큰 차이는 프로젝트 SDK 가 `Microsoft.NET.Sdk.Web` 이라는 점이고, 설정 파일 `appsettings.json` 과 실행 프로필 `Properties/launchSettings.json` 이 함께 만들어진다.
- 프로젝트 이름은 폴더 이름에서 온다. 이 노트는 `HiveLog` 로 잡는다. 14·15챕터가 같은 프로젝트를 이어 쓴다.

```
mkdir HiveLog
cd HiveLog
dotnet new web -lang "F#"
```

- `-lang "F#"` 은 따옴표를 빼도 동작하지만, 셀에 따라 `#` 이 주석으로 읽힐 수 있으므로 붙여 두는 편이 안전하다(4챕터에서 같은 이야기를 했다).

템플릿이 만든 `Program.fs` 는 다음과 같다. .NET 10.0.111 SDK 의 출력이 원서(2023년 1월판)의 코드와 한 글자도 다르지 않다.

```
open System
open Microsoft.AspNetCore.Builder
open Microsoft.Extensions.Hosting

[<EntryPoint>]
let main args =
    let builder = WebApplication.CreateBuilder(args)
    let app = builder.Build()

    app.MapGet("/", Func<string>(fun () -> "Hello World!")) |> ignore

    app.Run()

0 // Exit code
```

- `MapGet` 에는 C# 을 염두에 두고 만든 오버로드가 여럿 있다. 하나는 `RequestDelegate` 를 받고 다른 하나는 `System.Delegate` 를 받는다. F# 은 대상 델리게이트(delegate) 타입이 정해진 자리라면 람다를 자동으로 변환해 주지만, `fun () -> "Hello World!"` 를 그냥 넘기면 `RequestDelegate` 오버로드가 잡혀 `HttpContext` 를 요구하는 `error FS0001` 이 난다. `System.Delegate` 쪽은 어느 델리게이트를 만들지 정할 수 없다. 그래서 `Func<string>(...)` 으로 이름을 대 준다. 이 어색함이 Giraffe 를 쓰는 이유 중 하나다.
- `MapGet` 은 결과를 돌려주므로 `|> ignore` 로 버린다. 값을 버리지 않으면 경고 FS0020 이 난다.
- `[<EntryPoint>]` 가 붙은 함수는 프로젝트의 마지막 파일에 있어야 한다. 4챕터에서 본 컴파일 순서 규칙이 여기서도 그대로 적용된다.

프로젝트 파일은 이렇게 생겼다.

```
<Project Sdk="Microsoft.NET.Sdk.Web">

  <PropertyGroup>
    <TargetFramework>net10.0</TargetFramework>
  </PropertyGroup>

  <ItemGroup>
    <Compile Include="Program.fs" />
  </ItemGroup>

</Project>
```

실행은 `dotnet run` 이다. 종료는 그 터미널에서 `CTRL+C` 다.

```
dotnet run
```

```
info: Microsoft.Hosting.Lifetime[14]
      Now listening on: http://localhost:5291
info: Microsoft.Hosting.Lifetime[0]
      Application started. Press Ctrl+C to shut down.
info: Microsoft.Hosting.Lifetime[0]
      Hosting environment: Development
```

- 포트 번호는 템플릿이 프로젝트를 만들 때 무작위로 정하고 `Properties/launchSettings.json` 에 적어 둔다. 그 파일에는 `http` 와 `https` 두 프로필이 들어 있고 `dotnet run` 은 첫 프로필인 `http` 를 쓴다. HTTPS 로 띄우려면 `dotnet run --launch-profile https` 다.
- 원서는 실행 로그에 URL 두 개가 나오는 화면을 보여 준다. 기본 프로필 `http` 의 `applicationUrl` 에는 URL 이 하나뿐이라 한 줄만 나오는 것이고, `https` 프로필로 띄우면 그 프로필의 `applicationUrl` 에 URL 이 둘이라 원서와 같이 두 줄이 나온다. 원서 화면과 다르다고 잘못 만든 것이 아니다.

## Using Giraffe — 패키지과 시작 코드 (원서 pp.168-169)

- Giraffe 는 두 조각으로 나뉘어 있다. 라우팅과 핸들러가 들어 있는 `Giraffe` , HTML 을 만드는 `Giraffe.ViewEngine` 이다. 이 챕터는 둘을 다 쓴다.
- 패키지 버전을 적어 두면 나중에 같은 코드를 다시 받아 돌릴 때 결과가 흔들리지 않는다. 이 노트는 `Giraffe 8.3.0`, `Giraffe.ViewEngine 1.4.0` 으로 확인했다.

```
dotnet add package Giraffe --version 8.3.0
dotnet add package Giraffe.ViewEngine --version 1.4.0
```

```
<ItemGroup>
  <PackageReference Include="Giraffe" Version="8.3.0" />
  <PackageReference Include="Giraffe.ViewEngine" Version="1.4.0" />
</ItemGroup>
```

Giraffe 를 쓰는 `Program.fs` 의 뼈대는 다음 세 조각이다. 각 조각의 속은 뒤 절에서 하나씩 채운다.

```
let configureServices (services: IServiceCollection) = ... // 서비스 등록
let configureApp (appBuilder: IApplicationBuilder) = ... // 요청 처리 순서
[<EntryPoint>]
let main args = ... // 호스트 시작
```



- 최소 템플릿보다 코드가 길어지는 것은 Giraffe 탓이 아니다. 최소 템플릿이 생략한 서비스 등록과 요청 처리 순서를 직접 적기 때문이다. [ASP.NET Core](#) 를 다뤄 본 독자에게는 익숙한 두 함수다.
- `configureServices` 는 의존성 주입(dependency injection) 컨테이너에 무엇을 넣을지 정한다. 라우팅과 Giraffe 를 등록한다.
- `configureApp` 은 요청이 지나갈 미들웨어(middleware)의 순서를 정한다. 이 순서를 미들웨어 파이프라인(middleware pipeline)이라 부른다. 2챕터에서 `|>` 로 값을 흘려보낸 함수 파이프라인과는 다른 것이므로 줄여 쓰지 않고 온낱말로 적는다. 순서가 곧 동작이다. 라우팅을 먼저 켜고, Giraffe 엔드포인트를 등록하고, 어느 엔드포인트에도 걸리지 않은 요청을 받을 핸들러를 마지막에 둔다.
- 원서 코드에는 `if app.Environment.IsDevelopment() then app.UseDeveloperExceptionPage()` 가 들어 있다. 현재 SDK에서는 `WebApplication` 이 개발 환경일 때 이 미들웨어를 자동으로 넣는다. 일부러 예외를 던지는 경로를 만들어 확인해 보면, 그 호출을 지워도 예외 화면이 그대로 나오고 스택 추적에 `DeveloperExceptionPageMiddlewareImpl` 이 찍힌다. 남겨 두어도 해가 없지만 없어도 같다.

## 이 노트의 예제를 돌리는 방법 (노트 보충)

- 앞선 챕터들의 예제는 `dotnet fsi` 로 검증했다. 웹 서버도 같은 방식으로 검증할 수 있다. 스크립트 안에서 서버를 띄우고, 같은 스크립트에서 `HttpClient` 로 요청을 보내 응답을 찍고, 서버를 내리면 된다. 이 노트의 `13-server` 실행 단위가 그것이다.
- 준비 코드가 하나 필요하다. FSI 는 [ASP.NET Core](#) 공유 프레임워크를 자동으로 참조하지 않는다. `#r "nuget: Giraffe, 8.3.0"` 만 적고 `open Microsoft.AspNetCore.Builder` 를 쓰면 `error FS0039: 'AspNetCore' 네임스페이스가 정의되지 않았습니다` 가 난다. 패키지가 프레임워크 참조 ( `FrameworkReference` )로 [ASP.NET Core](#) 를 요구하는데 FSI 의 패키지 해석은 그 요구를 채워 주지 않기 때문이다.
- 그래서 설치된 공유 프레임워크를 스크립트가 직접 찾아 참조한다. 선행 요구사항은 두 가지다. [ASP.NET Core](#) 공유 프레임워크가 포함된 .NET SDK(이 노트는 10.0.111 로 확인했다), 그리고 처음 한 번 패키지를 받아 올 네트워크다. 받아 오지 못하면 `error FS0999` 로 실패한다.
- `#r` 로 적는 어셈블리 10개는 SDK 버전이 정한 목록이 아니라 이 챕터의 코드가 타입 이름을 직접 적는 어셈블리다. 코드가 다른 네임스페이스를 건드리면 그만큼 줄을 더해야 한다. 실행 중에만 필요한 어셈블리는 아래 (2)의 해석기가 채우므로 적지 않아도 된다.
- 프로젝트를 만들어 `dotnet run` 으로 확인하는 것이 실제 개발 방식이다. 아래 준비 코드는 노트의 예제를 자동으로 검증하기 위한 장치일 뿐, `Program.fs` 에는 들어가지 않는다.

```

// 이 단위가 보여주는 것: 스크립트에서 Giraffe 앱을 띄워 응답까지 확인하기
// 준비 코드 - Program.fs 에는 들어가지 않는다
open System
open System.IO
open System.Runtime.Loader

// (1) 설치된 ASP.NET Core 공유 프레임워크에서 실행 중인 런타임과 같은 계열의 가장 높은 버전을 고른다
let sharedFramework =
    let core =
        Runtime.InteropServices.RuntimeEnvironment.GetRuntimeDirectory().TrimEnd(Path.DirectorySeparatorChar)
        |> DirectoryInfo
    // 폴더 이름을 문자열로 비교하면 10.0.9 가 10.0.11 보다 크므로 Version 으로 비교한다
    let version (path: string) =
        match Version.TryParse((Path.GetFileName path).Split('-')[0]) with
        | true, v -> v
        | _ -> Version(0, 0)
    let root = Path.Combine(core.Parent.Parent.FullName, "Microsoft.AspNetCore.App")
    if not (Directory.Exists root) then
        failwithf "ASP.NET Core 공유 프레임워크 폴더가 없다: %s. 이 폴더가 포함된 .NET SDK 가 필요하다." root
    let candidates = Directory.GetDirectories root |> Array.sortBy version
    match candidates |> Array.filter (fun d -> (version d).Major = (version core.FullName).Major) with
    | [||] -> Array.last candidates
    | sameLine -> Array.last sameLine

// (2) 실행 중 찾지 못한 어셈블리는 그 폴더에서 직접 읽게 한다
AssemblyLoadContext.Default.add_Resolving(
    Func<AssemblyLoadContext, Reflection.AssemblyName, Reflection.Assembly>(fun context name ->
        let dll = Path.Combine(sharedFramework, name.Name + ".dll")
        if File.Exists dll then context.LoadFromAssemblyPath dll else null))

// (3) #r 은 문자열 리터럴만 받으므로 참조할 어셈블리를 스크립트 폴더 옆에 링크한다(안 되면 복사)
let referenceDir = Path.Combine(__SOURCE_DIRECTORY__, "aspnetcore")
Directory.CreateDirectory referenceDir |> ignore
for name in [ "Microsoft.AspNetCore"
    "Microsoft.AspNetCore.Hosting"
    "Microsoft.AspNetCore.Hosting.Abstractions"
    "Microsoft.AspNetCore.Http"
    "Microsoft.AspNetCore.Http.Abstractions"
    "Microsoft.AspNetCore.Routing"
    "Microsoft.Extensions.DependencyInjection.Abstractions"
    "Microsoft.Extensions.Hosting.Abstractions"
    "Microsoft.Extensions.Logging"
    "Microsoft.Extensions.Logging.Abstractions" ] do
    let source = Path.Combine(sharedFramework, name + ".dll")
    if not (File.Exists source) then
        failwithf "%s.dll 을 %s 에서 찾지 못했다." name sharedFramework
    let target = Path.Combine(referenceDir, name + ".dll")
    if not (File.Exists target) then
        try File.CreateSymbolicLink(target, source) |> ignore
        with _ -> File.Copy(source, target, true)

printfn "ASP.NET Core %s" (Path.GetFileName sharedFramework) // ASP.NET Core 10.0.11

```

- (1)의 계산은 `Microsoft.NETCore.App/10.0.11` 옆에 `Microsoft.AspNetCore.App/10.0.11` 이 있다는 배치를 이용한다. 두 프레임워크의 버전이 항상 같다고 보장되지 않으므로, 실행 중인 런타임과 메이저 버전이 같은 폴더 가운데 가장 높은 것을 고른다. 폴더 이름을 문자열로 비교하면 `10.0.9` 가 `10.0.11` 보다 크게 나오므로 `Version` 으로 비교해야 한다.
- (2)의 해석기가 없으면 실행 중에 `FileNotFoundException` 이 난다. FSI 의 기본 로드 컨텍스트 (`AssemblyLoadContext.Default`) 는 런타임 폴더와 명시적으로 준 어셈블리 쪽만 뒤지고 `ASP.NET Core` 공유 프레임워크 폴더는 뒤지지 않는다. `#r` 로 적은 10개가 다시 끌어오는 어셈블리는 50개가 넘는데 (`Kestrel`, `Configuration`, `Options`, `Primitives` 같은 것들) 그 어셈블리가 모두 그 폴더에만 있다. 이름만 보고 그 폴더에서 찾아 주면 해결된다. 해석기가 `null` 을 돌려주는 것은 "나는 모른다"는 뜻이고, 그러면 로더가 다음 방법을 시도한다 — `Giraffe` 처럼 NuGet 캐시에서 오는 어셈블리는 그 경로로 해결된다.
- (3)에서 링크를 만드는 것은 `#r` 이 문자열 리터럴만 받기 때문이다. 실행 중에 계산한 경로를 `#r` 에 직접 넘길 수는 없으므로, 스크립트 폴더 옆에 정해진 이름으로 걸어 두고 상대 경로로 참조한다. 원본이 없어도 링크는 조용히 만들어지므로 원본이 있는지 먼저 확인하고, 없으면 그 자리에서 어느 어셈블리가 빠졌는지 알리며 실패한다. 링크가 막힌 환경에서는 복사로 넘어간다. `#r` 줄을 담은 스크립트를 만들어 `#load` 하는 방법도 있지만, 참조하는 어셈블리가 노트에 그대로 보이는 쪽을 택했다.

준비가 끝나면 참조와 `open` 을 적는다. 여기부터는 프로젝트의 `Program.fs` 와 같은 코드다.

```
#r "aspnetcore/Microsoft.AspNetCore.dll"
#r "aspnetcore/Microsoft.AspNetCore.Hosting.dll"
#r "aspnetcore/Microsoft.AspNetCore.Hosting.Abstractions.dll"
#r "aspnetcore/Microsoft.AspNetCore.Http.dll"
#r "aspnetcore/Microsoft.AspNetCore.Http.Abstractions.dll"
#r "aspnetcore/Microsoft.AspNetCore.Routing.dll"
#r "aspnetcore/Microsoft.Extensions.DependencyInjection.Abstractions.dll"
#r "aspnetcore/Microsoft.Extensions.Hosting.Abstractions.dll"
#r "aspnetcore/Microsoft.Extensions.Logging.dll"
#r "aspnetcore/Microsoft.Extensions.Logging.Abstractions.dll"
#r "nuget: Giraffe, 8.3.0"
#r "nuget: Giraffe.ViewEngine, 1.4.0"

open System.Globalization
open System.Text.Encodings.Web
open System.Text.Json
open Microsoft.AspNetCore.Builder
open Microsoft.AspNetCore.Http
open Microsoft.Extensions.DependencyInjection
open Microsoft.Extensions.Logging
open Giraffe
open Giraffe.EndpointRouting
open Giraffe.ViewEngine
```

- `open Giraffe` 는 핸들러(`text`, `json`, `htmlView`, `RequestErrors`)를, `open Giraffe.EndpointRouting` 은 라우팅 함수(`GET`, `route`, `routeF`, `subRoute`)를, `open Giraffe.ViewEngine` 은 HTML 요소 함수를 들여온다. 세 개를 다 열어야 이 챕터의 코드가 컴파일된다.

도메인은 별통과 점검 기록이다. 코드는 9챕터의 단일 케이스 판별 유니온으로 감싸고, 찾기 실패는 3챕터의 `Result` 로 돌려준다.

```

type HiveCode = HiveCode of string

type Hive = {
    Code: HiveCode
    Frames: int
    LastCheckedOn: DateOnly
    QueenSeen: bool
}

let hives =
    [ { Code = HiveCode "H-07"; Frames = 8; LastCheckedOn = DateOnly(2026, 4, 12); QueenSeen = true }
      { Code = HiveCode "H-11"; Frames = 10; LastCheckedOn = DateOnly(2026, 4, 19); QueenSeen = false } ]

// FSI 실행: code: string -> Result<Hive,string>
let findHive (code: string) : Result<Hive, string> =
    hives
    |> List.tryFind (fun hive -> hive.Code = HiveCode(code.ToUpperInvariant()))
    |> Option.map Ok
    |> Option.defaultWith (fun () -> Error $"등록되지 않은 벌통 코드: {code}")

```

- 저장소는 지금 메모리 안의 리스트다. 14챕터가 이 자리를 손볼 것이므로, 조회 경로를 `findHive` 하나로 좁혀 두었다.
- `Option` 을 `Result` 로 바꾸는 두 줄은 3챕터에서 본 형태다. 실패에 이유를 담아야 하는데 `tryFind` 는 이유를 알려 주지 않으므로, 없다는 사실을 메시지로 바꿔 붙인다. 메시지는 값을 끝에 두는 모양으로 적었다. `{code}` 뒤에 `은 / 는` 을 붙이면 코드가 `H-11` 일 때와 `H-12` 일 때 맞는 조사가 달라지는데 문자열 보간은 그것을 가릴 수 없다. `Option.defaultValue` 는 대안 값을 미리 만들어 두고 `Option.defaultWith` 는 없을 때만 만든다. 여기서는 문자열 하나라 차이가 없지만, 대안 값을 만드는 데 비용이 든다면 뒤쪽을 골라야 한다.

## Mutability — 가변 바인딩과 할당 연산자 (원서 p.169)

- [ASP.NET Core](#) 를 설정하려면 .NET 객체의 프로퍼티에 값을 넣어야 한다. 그래서 이 챕터에서 가변 (mutable) 바인딩과 할당 연산자 `<-` 를 다시 짚는다. 문법 자체는 1챕터가 `=` 의 세 가지 얼굴을 가르며 이미 보여 주었고, 10챕터는 클래스 본문과 객체 식이 감춰 두는 상태에 썼다. 달라지는 것은 쓰는 자리다. 도메인 코드가 아니라 애플리케이션을 구성하는 코드에서 필요해진다.
- 원서 p.169 는 여기까지 가변을 직접 쓴 적이 없다고 적었는데, 원서 1챕터의 `Multi-purpose = operator` 사이드바(원서 p.14)가 이미 `let mutable myInt = 0` 과 `<-` 를 보여 준다. 이 노트의 1챕터도 그 자리를 따라갔다.
- F# 의 `let` 바인딩은 기본이 불변이다. 값을 바꿀 수 있게 하려면 `mutable` 키워드를 붙인다.
- 값을 바꾸는 연산자는 `<-` 다. `=` 는 F# 에서 비교 연산자이므로 값이 바뀌지 않고 참거짓만 나온다. 이 실수가 위험한 것은 컴파일러가 실패하지 않는다는 점이다. 프로젝트에서 `frames = 8` 을 문장으로 적으면 경고 FS0020 이 나는데, 현재 컴파일러의 메시지는 같음 식의 결과가 `bool` 이라 버려진다는 것을 알려 주고 `frames <- expression` 을 쓰라고 대안까지 적어 준다. 경고를 지나치면 아무 일도 하지 않는 코드가 남는다.
- 스크립트에서는 사정이 다르다. 같은 한 줄을 `.fsx` 최상위에 적고 `dotnet fsi` 로 돌려 보면 경고가 아예 나오지 않는다. FSI 가 최상위 식의 결과를 `it` 에 묶어 두므로 버려지는 값으로 보지 않는 것이다. 경고에 기대 이 실수를 잡을 생각이면 프로젝트에서 확인해야 한다.

```
// 이 단위가 보여주는 것: 가변 바인딩, 할당 연산자 <-, 프로퍼티 설정
let mutable checkedFrames = 0

// = 는 비교다. 값은 그대로 0 이다
printfn "%b" (checkedFrames = 8) // false
printfn "%d" checkedFrames      // 0
```

<- 를 써야 값이 바뀐다.

```
checkedFrames <- 8
printfn "%b" (checkedFrames = 8) // true
printfn "%d" checkedFrames      // 8
```

- .NET 객체의 프로퍼티도 같은 연산자로 설정한다. 객체 쪽에는 `mutable` 을 붙일 일이 없다. 이미 가변이다.

```
open System.Text.Json

let options = JsonSerializerOptions(JsonSerializerDefaults.Web)
options.WriteIndented <- true
printfn "%b" options.WriteIndented // true
```

- 이 절의 내용은 이 챕터 뒤쪽에서 JSON 직렬화기를 설정할 때 그대로 쓰인다. 설정 코드가 가변에 기대는 것은 F# 의 선택이 아니라 .NET 설정 API 의 모양 때문이다. 애플리케이션 코드는 계속 불변으로 쓴다.

## Endpoints — 라우팅을 값으로 적는다 (원서 p.170)

- 엔드포인트 라우팅(endpoint routing)은 [ASP.NET Core](#) 의 라우팅 기능에 얹혀 동작한다. 패키지를 들여다 보면 5.0.0 에 `Giraffe.EndpointRouting` 이 들어 있고 4.1.0 에는 없다. Giraffe 5 부터 쓸 수 있는 방식이며 이 노트가 쓰는 8.3.0 도 같다.
- 핵심은 라우팅이 값이라는 점이다. 엔드포인트 목록의 타입은 `Endpoint list` 이고, 목록을 만드는 것과 서버에 등록하는 것이 분리되어 있다. 그래서 라우팅 표를 함수로 조립하거나 다른 모듈에서 만들어 넘길 수 있다.
- 이전 방식에서는 라우팅도 `HttpHandler` 였다. `choose [ ... ]` 로 후보를 늘어놓고 요청마다 위에서 아래로 시도해, 처리한 핸들러가 나오면 멈추는 식이었다. 엔드포인트 라우팅은 경로 표를 [ASP.NET Core](#) 라우팅에 넘겨 매칭을 그쪽에 맡기고, Giraffe 는 매칭된 뒤의 처리만 맡는다. 그래서 라우팅이 함수가 아니라 값이 되었다.

```
// 형태만 보이면 이렇다
GET [ route "/path" handler ] // HTTP 메서드로 묶고, 경로에 핸들러를 붙인다
routef "/hives/%s" handlerTakingOne // 경로 조각을 핸들러 인자로 넘긴다
subRoute "/api" nestedEndpoints // 접두 경로를 묶는다
```

- 경로에 붙는 것은 모두 `HttpHandler` 다. Giraffe 가 제공하는 것( `text` , `json` , `htmlView` )을 그대로 쓰거나 직접 만든다.
- `HttpHandler` 는 타입 약어 세 개가 겹쳐 있다. 컴파일러에게 물어보면 이렇게 확인된다.

```
// HttpFuncResult.HttpFunc.HttpHandler 가 무엇의 약어인지 컴파일러로 확인한다
// 타입 약어는 원래 타입과 같은 것이므로, 아래 세 정의가 컴파일된다는 것이 곧 약어가 그 모양이라는 뜻이다
let asHttpFuncResult (t: Threading.Tasks.Task<HttpContext option>) : HttpFuncResult = t
let asHttpFunc (f: HttpContext -> HttpFuncResult) : HttpFunc = f
let asHttpHandler (h: HttpFunc -> HttpContext -> HttpFuncResult) : HttpHandler = h
printfn "세 약어를 컴파일러가 받아들였다"
```

정리하면 세 줄이다.

```
type HttpFuncResult = Task<HttpContext option>
type HttpFunc = HttpContext -> HttpFuncResult
type HttpHandler = HttpFunc -> HttpContext -> HttpFuncResult
```

- 핸들러는 "다음 핸들러"와 "현재 요청 컨텍스트"를 받아 작업을 돌려준다. 첫 매개변수가 다음 핸들러라서 핸들러를 이어 붙일 수 있다. 이것이 2챕터에서 본 함수 합성과 같은 발상이 웹 코드에 나타난 모습이고, Giraffe 는 핸들러를 이어 붙이는 `>=>` 를 준다.
- 결과가 `Task<HttpContext option>` 인 것도 눈여겨볼 만하다. `None` 은 "이 핸들러가 이 요청을 처리하지 않았다"는 뜻이다. 3챕터에서 없음을 값으로 표현했던 것과 같은 방식이며, 그 덕분에 처리 여부를 예외 없이 판단할 수 있다.

## 404 - Not Found Handler — 못 찾은 요청을 받는 핸들러 (원서 p.170)

- 어느 엔드포인트에도 걸리지 않은 요청에 응답할 핸들러를 따로 만든다. HTTP 상태 코드 404 로 응답하는 것이 관례다.
- `RequestErrors` 모듈이 4xx 응답을 만드는 함수를 담고 있다. `RequestErrors.notFound` 는 다른 핸들러를 받아, 상태 코드를 404 로 정한 뒤 그 핸들러를 부르는 핸들러를 만든다.

```
// FSI 실측: notFoundHandler: HttpHandler
let notFoundHandler : HttpHandler =
    "요청한 경로가 없다"
    |> text
    |> RequestErrors.notFound
```

- 파이프 두 개로 읽으면 뜻이 그대로 드러난다. 문자열을 본문으로 쓰는 핸들러를 만들고, 그것을 404 로 감싼다. `RequestErrors.notFound h` 의 실체는 `setStatusCode 404 >=> h` 다. 상태 코드를 404 로 먼저 정하고 그다음에 안쪽 핸들러가 실행된다. `text` 는 본문과 `Content-Type` 만 건드리고 상태 코드를 건드리지 않으므로 404 가 그대로 남는다. 200 을 나중에 404 로 덮는 것이 아니다 — `>=>` 는 왼쪽이 먼저다.
- 이 핸들러는 엔드포인트 목록에 넣지 않는다. 라우팅 뒤에 놓이는 마지막 미들웨어로 등록한다. 등록 코드는 뒤의 `configureApp` 에 나온다.

## Creating an API Route — json 핸들러와 익명 레코드 (원서 pp.170-171)

- JSON 을 돌려주는 데는 내장 핸들러 `json` 을 쓴다. 넘길 값의 타입은 무엇이든 된다. 원서처럼 익명 레코드를 쓰면 응답 전용 타입을 따로 선언하지 않아도 된다.
- 익명 레코드는 `{| ... |}` 로 그 자리에서 만든다. 이름이 없으므로 타입 선언이 없고, 필드 이름과 값만 적는다.

```
// 이 단위가 보여주는 것: 익명 레코드와 그 JSON 표현
open System.Text.Json

let payload = [| Hive = "H-07"; Frames = 8; QueenSeen = true |]

// 필드를 적은 순서와 무관하게 같은 타입이다. 구조적 동등성도 레코드와 같다
printfn "%b" (payload = [| Frames = 8; Hive = "H-07"; QueenSeen = true |]) // true
```

- 직렬화 결과를 보면 두 가지가 눈에 띈다. 키가 알파벳 순으로 나오고, JsonSerializerDefaults.Web 을 주면 첫 글자가 소문자로 바뀐다.

```
// 필드 순서는 F# 이 익명 레코드를 만들 때 알파벳 순으로 정렬한 결과다
printfn "%s" (JsonSerializer.Serialize payload)
// {"Frames":8,"Hive":"H-07","QueenSeen":true}

// Giraffe 의 json 핸들러가 쓰는 기본값도 이쪽이다
printfn "%s" (JsonSerializer.Serialize(payload, JsonSerializerOptions(JsonSerializerDefaults.Web)))
// {"frames":8,"hive":"H-07","queenSeen":true}
```

- 응답 본문을 익명 레코드로 만드는 데는 또 하나의 이유가 있다. System.Text.Json 은 Option 이 아닌 판별 유니온을 직렬화하지 못한다. HiveCode 를 그대로 넘기면 System.NotSupportedException: F# discriminated union serialization is not supported. 가 나고, 웹에서는 응답 대신 500 을 받게 된다. 감싼 타입을 벗겨 평평한 익명 레코드로 만들어 넘기는 것이 이 문제를 피하는 가장 간단한 방법이다.

이제 API 의 첫 두 경로에 붙일 핸들러를 만든다.

```
// FSI 실측: apiSummaryHandler: HttpHandler
let apiSummaryHandler : HttpHandler =
    setHttpHeader "X-APIary" "seongsu-rooftop"
    >=> json [| Apiary = "성수 옥상"; Hives = List.length hives |]
```

- >=> 가 핸들러 두 개를 이어 붙인다. 왼쪽 핸들러가 응답 헤더를 하나 달고 다음 핸들러를 부르며, 오른쪽 핸들러가 본문을 쓴다. 2챕터에서 본 >> 와는 잇는 방식이 다르다. >> 는 앞 함수의 결과를 뒤 함수의 입력으로 넘기지만, >=> 는 오른쪽 핸들러를 왼쪽 핸들러의 next 자리로 넣는다. 그래서 왼쪽이 next 를 부르지 않으면 오른쪽은 아예 실행되지 않는다. 상태 코드를 먼저 정하고 본문을 나중에 쓰는 RequestErrors.notFound 가 이 순서에 기댄다.
- 이어 붙인 결과의 타입도 HttpHandler 다. 그래서 몇 개를 이어도 경로에 붙이는 방법은 달라지지 않는다.

두 번째 핸들러는 벌통 목록을 돌려준다. 여기서는 HttpContext 의 확장 멤버를 쓴다.

```
// FSI 실측: hiveListHandler: HttpFunc -> ctx: HttpContext -> HttpFuncResult
let hiveListHandler : HttpHandler =
    fun _ ctx ->
        hives
        |> List.map (fun hive ->
            let (HiveCode code) = hive.Code
            [| Hive = code; Frames = hive.Frames |])
        |> ctx.WriteJsonAsync
```

- `ctx.WriteJsonAsync` 는 값을 JSON 으로 직렬화해 응답 본문에 쓰고, `Content-Type` 을 `application/json` 으로, `Content-Length` 를 맞게 설정한다. 다음 핸들러를 부르지 않고 여기서 응답을 끝내므로 첫 매개변수를 `_` 로 버렸다.
- `:` `HttpHandler` 라고 적었는데도 FSI 는 약어를 펼쳐 `HttpFunc -> ctx: HttpContext -> HttpFuncResult` 로 보여 준다. 앞 절의 세 줄을 확인하는 셈이다. 반대로 `notFoundHandler` 처럼 함수를 조립해 만든 값은 `HttpHandler` 그대로 나온다.
- `let (HiveCode code) = hive.Code` 는 9챕터의 감싼 값을 꺼내는 패턴이다.

## Creating a Custom HttpHandler — 직접 만드는 핸들러 (원서 pp.171-172)

- 경로 일부를 값으로 받는 핸들러는 `routeof` 로 연결한다. `routeof "/hives/%s"` 는 `%s` 자리의 경로 조각을 핸들러의 첫 인자로 넘긴다. 그래서 핸들러는 `string -> HttpHandler` 모양이 된다.
- 원서 p.171 은 이 값을 쿼리 문자열(query string) 항목이라고 적었는데, 실제로 넘어오는 것은 경로 매개변수다. `/api/hives/H-07` 의 `H-07` 처럼 경로의 한 조각이며 `?code=H-07` 이 아니다. 쿼리 문자열을 읽으려면 `ctx.TryGetQueryStringValue` 처럼 `HttpContext` 에서 직접 꺼내야 한다.

```
// FSI 실측: code: string -> next: HttpFunc -> ctx: HttpContext -> HttpFuncResult
let hiveStatusHandler (code: string) : HttpHandler =
    fun next ctx ->
        task {
            match findHive code with
            | Ok hive ->
                let (HiveCode found) = hive.Code
                let payload =
                    { | Hive = found
                      Frames = hive.Frames
                      LastCheckedOn = hive.LastCheckedOn.ToString("yyyy-MM-dd", CultureInfo.InvariantCulture)
                      QueenSeen = hive.QueenSeen | }
                return! json payload next ctx
            | Error message ->
                return! RequestErrors.notFound (json { | Error = message | }) next ctx
        }
```

- `task { ... }` 는 `System.Threading.Tasks.Task<'T>` 를 만드는 계산 식이고 C# 의 `async / await` 에 해당한다. F# 6 부터 코어에 들어 있어 패키지 없이 쓸 수 있다. F# 에는 `Async` 도 있지만 Giraffe 가 `Task` 를 직접 쓰기로 정했으므로 여기서는 `task` 를 쓴다. 계산 식 자체는 12챕터의 주제다.
- 날짜를 문자열로 바꾸는 자리에는 `CultureInfo.InvariantCulture` 를 함께 넘긴다. 형식 문자열과 문화권은 별개의 인자다. 형식을 `"yyyy-MM-dd"` 로 못박아도 문화권을 넘기지 않으면 `ToString` 이 실행 환경의 문화권을 쓴다. 응답에 실리는 값의 모양은 서버가 어디서 도는지에 따라 달라져서는 안 되므로 문화권까지 코드가 정한다.
- `return!` 은 다른 핸들러의 결과를 그대로 이 핸들러의 결과로 삼는다. `json payload next ctx` 처럼 핸들러에 `next` 와 `ctx` 를 직접 넘겨 부르는 것이 핸들러를 손으로 이어 붙이는 방법이다.
- 3챕터의 `Result` 가 웹에서 하는 일이 이것이다. 성공과 실패를 갈라 응답을 만든다. `Ok` 는 200, `Error` 는 404 로 대응시켰다. 조회 함수는 HTTP 를 모르고, 핸들러만 상태 코드를 안다. 이 경계 덕분에 `findHive` 는 그대로 테스트할 수 있다.
- 안쪽 람다를 밖으로 펼쳐 매개변수를 나란히 적어도 같은 타입이다. Giraffe 코드에서는 위의 형태가 관례다.



```
// 같은 뜻이지만 관례가 아닌 형태
let hiveStatusHandler (code: string) (next: HttpFunc) (ctx: HttpContext) : HttpFuncResult =
    task { ... }
```

- 이 형태의 반환 타입은 `HttpFuncResult` 다. 원서 p.172 는 같은 코드에 `HttpHandler` 라고 적었는데, `next` 와 `ctx` 를 이미 받은 뒤에 남는 것은 결과뿐이므로 그렇게 적으면 `error FS0193: 형식 제약 조건이 일치하지 않습니다` 가 난다.

## Creating a View — HTML 을 F# 로 짜는 DSL (원서 pp.172-173)

- Giraffe View Engine 은 HTML 을 만드는 도메인 특화 언어(DSL, Domain-Specific Language)다. 요소 대부분이 리스트 두 개를 받는다. 앞은 특성 목록, 뒤는 자식 요소나 데이터 목록이다. `link` 나 `meta` 처럼 자식을 담을 수 없는 요소는 특성 목록만 받는다.
- 문자열 템플릿 대신 DSL 을 쓰는 이유는 구조가 깨진 HTML 을 만들 수 없다는 것이다. 태그를 닫는 것을 잊을 수 없고, 자식 자리에 요소가 아닌 것을 넣으면 컴파일되지 않는다.
- 아래 `13-view` 실행 단위는 `Giraffe.ViewEngine` 하나만 쓰므로 준비 코드가 필요 없다. 처음 한 번은 네트워크로 패키지를 받아 오고 그 뒤에는 로컬 NuGet 캐시로 돌며, 받아 오지 못하면 `error FS0999` 로 실패한다.

```
// 이 단위가 보여주는 것: View Engine 의 요소 함수와 문자열로 렌더링하기
#r "nuget: Giraffe.ViewEngine, 1.4.0"

open Giraffe.ViewEngine

// FSI 실측: code: string * frames: int -> XmlNode
let hiveRow (code: string, frames: int) =
    tr [] [
        td [] [ str code ]
        td [] [ str (string frames) ]
    ]
```

- 요소 함수의 결과 타입은 `XmlNode` 하나다. 그래서 조각을 함수로 빼내 목록에 끼워 넣을 수 있다. 15챕터가 쓸 부분 뷰가 이 성질에 기대나.

```
// FSI 실측: rows: (string * int) list -> XmlNode
let hiveTableView rows =
    html [] [
        head [] [
            title [] [ str "HiveLog" ]
            link [ _rel "stylesheet"; _href "/css/hive.css" ]
        ]
        body [] [
            h1 [] [ str "벌통 점검 기록" ]
            table [ _class "hives" ] [
                thead [] [ tr [] [ th [] [ str "벌통" ]; th [] [ str "소비" ] ] ]
                tbody [] (rows |> List.map hiveRow)
            ]
        ]
    ]
```

- 특성 함수 이름에는 밑줄이 붙는다(`_class`, `_id`, `_href`). `class` 같은 이름이 F# 키워드이거나 요소 함수 이름과 겹치는 것을 피하려는 규칙이다.
- 렌더링은 `RenderView` 모듈이 한다. 문서 전체는 `htmlDocument` (앞에 `<!DOCTYPE html>` 이 붙는다), 조각은 `htmlNode` 다. 둘 다 `XmlNode -> string` 이다.

```
println "%s" (RenderView.AsString.htmlDocument (hiveTableView [ ("H-07", 8); ("H-11", 10) ]))
// <!DOCTYPE html>
// <html><head><title>HiveLog</title><link rel="stylesheet" href="/css/hive.css"></head><body><h1>벌통 점검 기록</h1><table cla

println "%s" (RenderView.AsString.htmlNode (hiveRow ("H-07", 8)))
// <tr><td>H-07</td><td>8</td></tr>
```

첫 화면에 쓸 뷰는 간단하게 둔다. 15챕터가 이 자리를 늘린다.

```
// FSI 실측: indexView: XmlNode
let indexView =
  html [] [
    head [] [
      title [] [ str "HiveLog" ]
    ]
    body [] [
      h1 [] [ str "HiveLog" ]
      p [ _class "lede"; _id "intro" ] [ str "옥상 양봉장 점검 기록" ]
    ]
  ]
```

- 뷰를 응답으로 내보내는 핸들러는 `htmlView` 다. `htmlView indexView` 가 `HttpHandler` 이므로 경로에 그대로 붙는다.

## Adding Subroutes — 라우팅 중복 걷어내기 (원서 pp.173-175)

- 경로가 늘면 `/api` 접두가 반복된다. `subRoute` 로 접두를 한 번만 적고 그 아래 목록을 따로 뺀다.
- 뺀 목록도 그냥 `Endpoint list` 다. 다른 파일이나 모듈에서 만들어 넘겨도 된다. 14챕터가 그렇게 나눌 자 리다. 원서 p.174 는 이 목록을 “another `HttpHandler`” 라고 적었는데 `Endpoint list` 다. 원서 코드 주 석에는 `// Endpoint list` 로 맞게 적혀 있다.

```
// FSI 실측: apiEndpoints: Endpoint list
let apiEndpoints =
[
  GET [
    route "" apiSummaryHandler
    route "/hives" hiveListHandler
    routef "/hives/%s" hiveStatusHandler
  ]
]

// FSI 실측: endpoints: Endpoint list
let endpoints =
[
  GET [
    route "/" (htmlView indexView)
  ]
  subRoute "/api" apiEndpoints
]
```

- 접두를 뺐으므로 안쪽 경로는 `""` 와 `"/hives"` 가 된다. `route ""` 가 `/api` 자체다.
- `GET` 을 안쪽 목록에 둔 것에 뜻이 있다. `/api` 아래에 `POST` 나 `PUT` 이 생기면 `apiEndpoints` 안에서 `HTTP` 메서드별로 묶으면 되고, 바깥 `endpoints` 는 손대지 않는다. 14챕터에서 실제로 그렇게 늘어난다.
- 핸들러 이름은 <대상><동작 또는 응답 내용>Handler , 엔드포인트 목록 이름은 <영역>Endpoints , 뷰 이름은 <이름>View 로 맞춰 두었다. 원서에서 온 `notFoundHandler` 는 대상이 없어 규칙 밖이지만 이름이 널리 쓰 이므로 그대로 둔다. 14·15챕터가 이 규칙을 이어 쓴다.

## Reviewing the Code — 완성된 Program.fs (원서 pp.175-177)

서비스 등록과 미들웨어 파이프라인을 정하는 두 함수가 남았다.

```
// FSI 실행: services: IServiceCollection -> unit
let configureServices (services: IServiceCollection) =
    let options = JsonSerializerOptions(JsonSerializerDefaults.Web)
    options.Encoder <- JavaScriptEncoder.UnsafeRelaxedJsonEscaping
    services
        .AddRouting()
        .AddGiraffe()
        .AddSingleton<Json.ISerializer>(Json.Serializer options)
    |> ignore

// FSI 실행: appBuilder: IApplicationBuilder -> unit
let configureApp (appBuilder: IApplicationBuilder) =
    appBuilder
        .UseRouting()
        .UseGiraffe(endpoints)
        .UseGiraffe(notFoundHandler)
```

- `AddRouting` 과 `AddGiraffe` 는 라우팅과 Giraffe 가 쓰는 서비스를 등록한다. 등록 함수는 컨테이너를 돌려주므로 마지막에 `|> ignore` 가 필요하다.
- 세 번째 줄은 JSON 직렬화기를 바꿔 끼운 것이다. 기본 직렬화기는 ASCII 밖의 문자를 이스케이프해서 담으므로 성수 옥상 이 `\uC131\uC218 \uC625\uC0C1` 로 나간다. 한글 응답을 그대로 보려면 인코더를 갈아 준 직렬화기를 `Json.ISerializer` 로 등록한다. 이름에 `Unsafe` 가 붙은 것은 HTML 특수문자까지 이스케이프하지 않는다는 뜻이므로, JSON 을 HTML 안에 그대로 끼워 넣는 코드에서는 쓰지 않는다.  
`options.Encoder <- ...` 가 앞 절의 할당 연산자다.
- `configureApp` 에 적은 순서가 곧 미들웨어 파이프라인이다. `UseRouting` 이 라우팅을 켜고, `UseGiraffe(endpoints)` 가 엔드포인트를 등록하고, `UseGiraffe(notFoundHandler)` 가 어디에도 걸리지 않은 요청을 받는다. 앞의 두 호출은 빌더를 돌려주지만 마지막 오버로드는 `unit` 을 돌려주므로 함수 전체가 `unit` 이 되고 `ignore` 가 필요 없다.

`Program.fs` 의 마지막 조각은 진입점이다. 여기까지가 프로젝트 코드다.

```
[<EntryPoint>]
let main args =
    let builder = WebApplication.CreateBuilder(args)
    configureServices builder.Services
    let app = builder.Build()
    configureApp app
    app.Run()
    0
```

- 순서에 이유가 있다. `configureServices` 는 `builder.Build()` 앞이어야 한다. 컨테이너는 `Build()` 에서 굳으므로 그 뒤에 서비스를 더하려 하면 `InvalidOperationException` 이 난다. `configureApp` 은 만들어진 앱에 미들웨어를 붙이는 것이므로 `Build()` 뒤여야 한다.
- `app.Run()` 은 요청을 받기 시작하고 종료 신호가 올 때까지 돌아오지 않는다. 그 뒤의 `0` 이 종료 코드다.
- 이 챕터가 만든 프로젝트의 구조는 다음과 같다. 파일 하나에 다 들어 있는 상태이고, 14챕터에서 나눌 것이다.

```
HiveLog/
├─ HiveLog.fsproj
├─ Program.fs           도메인 · 뷰 · 핸들러 · 라우팅 · 진입점
├─ appsettings.json
├─ appsettings.Development.json
├─ Properties/
└─ launchSettings.json   프로필과 포트
```

- `Program.fs` 안의 순서도 컴파일 순서다. 도메인, 뷰, 핸들러, 엔드포인트 목록, 설정 함수, 진입점 순으로 두어야 이름이 보인다. 4챕터의 규칙이 파일 안에서도 그대로다.

프로젝트를 띄워 확인하는 명령과 결과는 다음과 같다.

```
dotnet run
# 다른 터미널에서
curl -i http://localhost:5291/api
curl http://localhost:5291/api/hives/h-07
curl -i http://localhost:5291/no-such-path
```

```
# curl -i http://localhost:5291/api
HTTP/1.1 200 OK
Content-Type: application/json; charset=utf-8
X-Apiary: seongsu-rooftop

{"apiary":"성수 옥상","hives":2}

# curl http://localhost:5291/api/hives/h-07
{"frames":8,"hive":"H-07","lastCheckedOn":"2026-04-12","queenSeen":true}

# curl -i http://localhost:5291/no-such-path
HTTP/1.1 404 Not Found
Content-Type: text/plain; charset=utf-8

요청한 경로가 없다
```

- 날짜나 전송 방식 같은 헤더는 위 출력에서 생략했다. 확인할 것은 상태 코드, `Content-Type`, 그리고 `apiSummaryHandler` 가 `>=>` 로 달아 둔 `X-Apiary` 헤더다.
- 브라우저로 `http://localhost:5291/` 을 열면 `indexView` 가 만든 HTML 이 보인다. API 경로는 브라우저로도 되고 `curl` 이나 Postman 같은 도구로도 된다.

## 스크립트에서 서버를 띄워 응답을 확인한다 (노트 보충)

- 앞의 `13-server` 블록들이 프로젝트 코드와 같은 정의를 쌓아 두었다. 남은 것은 진입점 대신 호스트를 직접 띄우고 요청을 보내는 부분이다.
- 포트는 `0` 으로 준다. 운영체제가 빈 포트를 골라 주므로 검증을 반복해도 충돌하지 않는다. 실제 포트는 `app.Url`s 에서 읽는다.

```

let builder = WebApplication.CreateBuilder()
configureServices builder.Services
builder.Logging.ClearProviders() |> ignore // 호스트 로그를 끈다

let app = builder.Build()
configureApp app
app.Uris.Add "http://127.0.0.1:0" // 0 = 빈 포트를 골라 달라는 뜻
app.StartAsync() |> Async.AwaitTask |> Async.RunSynchronously

let baseUrl = app.Uris |> Seq.head
let client = new Net.Http.HttpClient()

let request (path: string) =
    let response = client.GetAsync(baseUrl + path) |> Async.AwaitTask |> Async.RunSynchronously
    let body = response.Content.ReadAsStringAsync() |> Async.AwaitTask |> Async.RunSynchronously
    printfn "%-18s %d %s" path (int response.StatusCode) body

```

- `Async.AwaitTask` 로 `Task` 를 `Async` 로 바꿔 동기로 기다린다. 스크립트에서 결과를 순서대로 찍으려는 것이므로 이렇게 했다. 애플리케이션 코드에서 `Async.RunSynchronously` 를 곳곳에 쓰는 것은 좋은 습관이 아니다.

```

request "/"
// / 200 <!DOCTYPE html>
// <html><head><title>HiveLog</title></head><body><h1>HiveLog</h1><p class="lede" id="intro">옥상 양봉장 점검 기록</p></body></h
request "/api"
// /api 200 {"apiary":"성수 옥상","hives":2}
request "/api/hives"
// /api/hives 200 [{"frames":8,"hive":"H-07"}, {"frames":10,"hive":"H-11"}]
request "/api/hives/h-07"
// /api/hives/h-07 200 {"frames":8,"hive":"H-07","lastCheckedOn":"2026-04-12","queenSeen":true}
request "/api/hives/H-99"
// /api/hives/H-99 404 {"error":"등록되지 않은 벌통 코드: H-99"}
request "/no-such-path"
// /no-such-path 404 요청한 경로가 없다

app.StopAsync() |> Async.AwaitTask |> Async.RunSynchronously
client.Dispose()
printfn "서버를 내렸다"

```

- 서버를 내리고 클라이언트를 해제하는 두 줄이 중요하다. 그래야 스크립트가 끝난다. 서버가 남아 있으면 다음 검증을 막는다.
- 소문자로 보낸 `h-07` 이 `H-07` 로 찾아지는 것은 `findHive` 가 대문자로 바꿔 비교하기 때문이다. 없는 코드로 요청하면 상태 코드 404 와 JSON 본문이 함께 오고, 라우팅에 아예 걸리지 않는 경로는 `notFoundHandler` 가 평문으로 답한다. 같은 404 지만 응답을 만든 자리가 다르다.

## Summary — 원서의 챕터 요약 (원서 p.177)

- 원서는 Giraffe 로 API 와 HTML 페이지를 만드는 방법을 맛보았다는 말로 챕터를 맺는다. 걸만 훑었다는 것도 함께 적어 두었다.
- 라우팅에서 `HttpHandler` 가 중심이라는 점, 내장 핸들러가 여럿 있다는 점, 직접 만들 수 있다는 점이 이 챕터의 요지다.
- 다음 챕터에서는 API 쪽을 늘린다.

## 정리 — 이 노트의 요약

- Giraffe 는 **ASP.NET** Core 위의 얇은 함수형 껍데기다. 라우팅은 `Endpoint list` 라는 값이고, 요청 처리는 `HttpHandler` 라는 함수다. 값과 함수로 되어 있으니 조립하고 나눠 담을 수 있다.
- `HttpHandler` 는 `HttpFunc -> HttpContext -> HttpFuncResult` 이고, `HttpFunc` 는 `HttpContext -> HttpFuncResult` , `HttpFuncResult` 는 `Task<HttpContext option>` 이다. 첫 매개 변수가 다음 핸들러라서 `>=>` 로 이어 붙일 수 있고, 결과의 `option` 이 처리 여부를 나타낸다.
- 설정 코드에서만 가변이 필요하다. `let mutable` 로 선언하고 `<-` 로 값을 넣는다. `=` 는 비교이므로 값을 바꾸지 않으며, 프로젝트에서는 경고 FS0020 이 그 실수를 알려 주지만 스크립트 최상위에서는 그 경고조차 나오지 않는다.
- 응답 본문은 익명 레코드가 편하다. 필드는 알파벳 순으로 정렬되고, 웹 기본값에서는 키가 소문자로 시작한다. 한글을 이스케이프 없이 내보내려면 인코더를 바꾼 직렬화기를 `Json.ISerializer` 로 등록한다.
- 응답에 도메인 타입을 그대로 실어 보내지 않는다. `System.Text.Json` 은 `Option` 이 아닌 판별 유니온을 직렬화하지 못하고 `NotSupportedException` 을 던지므로, 감싼 값을 벗겨 익명 레코드로 만들어 넘긴다.
- `routeof "/hives/%s"` 가 넘기는 것은 경로 매개변수다. 원서 p.171 은 쿼리 문자열이라고 적었지만 실제로는 경로의 한 조각이다.
- 매개변수를 펼쳐 적은 핸들러의 반환 타입은 `HttpFuncResult` 다. 원서 p.172 는 그 자리에 `HttpHandler` 라고 적었고, 그대로 쓰면 `error FS0193` 이 난다.
- 뷰는 요소마다 특성 목록과 자식 목록을 받는 DSL 로 짤다. 모든 요소가 `XmlNode` 라서 조각을 함수로 빼 재 사용할 수 있고, `RenderView.AsString.htmlDocument` 로 문자열이 된다.
- 원서 코드의 `UseDeveloperExceptionPage` 호출은 현재 SDK 에서는 없어도 된다. 개발 환경이면 `WebApplication` 이 자동으로 넣는다.
- 검증은 스크립트에서 서버를 띄워 했다. FSI 가 **ASP.NET** Core 공유 프레임워크를 자동으로 참조하지 않으므로 준비 코드가 필요하지만, 그 대가로 라우팅·핸들러·뷰·상태 코드가 실제 응답으로 확인된다.

## 14·15챕터가 이어받는 것

프로젝트는 `HiveLog` 이고 `dotnet new web -lang "F#"` 로 만든 단일 프로젝트다. 패키지는 `Giraffe 8.3.0` 과 `Giraffe.ViewEngine 1.4.0` 이며 버전을 고정해 쓴다. 도메인은 옥상 양봉장의 벌통 점검 기록이고, 타입은 `HiveCode` (단일 케이스 판별 유니온)와 `Hive` 레코드, 저장소는 메모리 리스트 `hives` , 조회 경로는 `findHive : string -> Result<Hive,string>` 하나다. 라우팅은 바깥 `endpoints` 에 첫 화면과 `subRoute "/api"` `apiEndpoints` 를 두고, HTTP 메서드 묶음은 안쪽 `apiEndpoints` 에 둔다. 그래서 `/api` 아래에 POST 나 PUT 을 더할 때 바깥 목록을 건드릴 일이 없다. 이름 규칙은 핸들러 `<대상><동작 또는 응답 내용>Handler` , 엔드포인트 목록 `<영역>Endpoints` , 뷰 `<이름>View` 다. 원서에서 온 `notFoundHandler` 는 대상이 없어 규칙 밖이다. 서비스 등록은 `configureServices` , 미들웨어 파이프라인은 `configureApp` 이 정하고 진입점은 그 둘을 부르기만 한다. 지금은 `Program.fs` 한 파일이므로 14챕터에서 도메인과 핸들러를 앞쪽 파일로 나눌 때 `.fsproj` 의 `<Compile Include=... />` 순서를 함께 적어야 한다. 검증 방식도 그대로 물려받는다. 13-server 실행 단위의 준비 코드(공유 프레임워크 탐색, 어셈블리 해석기, 참조 링크)를 그대로 쓰고, 포트 0 으로 띄워 `HttpClient` 로 요청한 뒤 `StopAsync` 로 내리면 반복 실행해도 안전하다. 15챕터가 CSS:JS 를 둘 `wwwroot/` 는 아직 없으므로 그 챕터에서 만든다.

## 원서 대조 표

| 절                                          | 원서 페이지     | 실행 단위                  |
|--------------------------------------------|------------|------------------------|
| Getting Started — 빈 웹 프로젝트 만들기             | pp.166-167 | —                      |
| Using Giraffe — 패키지와 시작 코드                 | pp.168-169 | —                      |
| 이 노트의 예제를 돌리는 방법                           | (노트 보충)    | 13-server              |
| Mutability — 가변 바인딩과 할당 연산자                | p.169      | 13-mutability          |
| Endpoints — 라우팅을 값으로 적는다                   | p.170      | 13-server              |
| 404 - Not Found Handler — 못 찾은 요청을 받는 핸들러  | p.170      | 13-server              |
| Creating an API Route — json 핸들러와 익명 레코드   | pp.170-171 | 13-payload , 13-server |
| Creating a Custom HttpHandler — 직접 만드는 핸들러 | pp.171-172 | 13-server              |
| Creating a View — HTML 을 F# 로 짜는 DSL       | pp.172-173 | 13-view , 13-server    |
| Adding Subroutes — 라우팅 중복 걷어내기             | pp.173-175 | 13-server              |
| Reviewing the Code — 완성된 Program.fs        | pp.175-177 | 13-server              |
| 스크립트에서 서버를 띄워 응답을 확인한다                     | (노트 보충)    | 13-server              |
| Summary — 원서의 챕터 요약                        | p.177      | —                      |

## 14 - Giraffe 로 API 만들기 (원서 pp.178-184)

13챕터가 세운 `HiveLog` 에 쓰기 경로를 붙이는 챕터다. 읽기만 하던 API 에 POST·PUT·DELETE 가 생기고, 그러면 세 가지가 새로 필요해진다. 요청이 끝나도 남아 있어야 하는 저장소, 그 저장소를 핸들러에 건네주는 의존성 주입, 요청 본문을 F# 타입으로 되돌리는 모델 바인딩(model binding)이다. 새 문법은 없다. 10챕터의 클래스 타입, 3챕터의 `Option · Result` , 8챕터의 검증, 4챕터의 컴파일 순서가 API 코드에서 어떻게 만나는지 보는 챕터다.

13챕터에서 넘어오는 것은 이렇다. 프로젝트는 `dotnet new web -lang "F#"` 로 만든 단일 프로젝트 `HiveLog` , 패키지는 `Giraffe 8.3.0` 과 `Giraffe.ViewEngine 1.4.0` , 도메인은 옥상 양봉장의 벌통 점검 기록, 라우팅은 바깥 `endpoints` 와 그 안의 `subRoute "/api" apiEndpoints` 두 층이다. 이 챕터는 그 위에 점검 기록이라는 쓰기 대상을 하나 얹는다.

## Getting Started · Our Task — 이 챕터가 더하는 것 (원서 p.178)

- 원서는 Postman 같은 HTTP 도구를 준비하라고 안내하고, 항목을 만들고 고치고 지우는 API 를 목표로 잡는다. 이 노트는 13챕터의 도메인을 이어받아 벌통 점검 기록을 다루는 다섯 경로를 만든다.
- 13챕터의 `/api/hives` 는 등록된 벌통을 읽기만 한다. 새로 붙이는 `/api/inspections` 는 점검 기록을 쌓는 자리이므로 요청이 서버의 상태를 바꾼다. 상태가 바뀌는 순간부터 저장소·검증·상태 코드 선택이 모두 문제가 된다.

|        |                                    |             |
|--------|------------------------------------|-------------|
| GET    | <code>/api/inspections</code>      | 점검 기록 목록    |
| GET    | <code>/api/inspections/{id}</code> | 한 건 조회      |
| POST   | <code>/api/inspections</code>      | 새 점검 기록 만들기 |
| PUT    | <code>/api/inspections/{id}</code> | 한 건 교체      |
| DELETE | <code>/api/inspections/{id}</code> | 한 건 삭제      |

- 원서는 이 라우팅 표를 먼저 적고 핸들러를 나중에 채운다. 이 노트는 스크립트가 위에서 아래로 컴파일되도록 순서를 뒤집어 저장소 → 요청·응답 타입 → 핸들러 → 엔드포인트 목록으로 적는다. 4챕터에서 본 “정의가 사용보다 앞”이라는 규칙이 노트의 서술 순서까지 정한다.

## Handlers — 파일을 넷으로 나눈다 (원서 p.181)

- 13챕터의 `Program.fs` 는 도메인·뷰·핸들러·라우팅·진입점을 한 파일에 담고 있었다. 핸들러가 다섯 개 늘어나면 그 파일이 감당하지 못한다. 원서도 같은 판단으로 저장소와 핸들러를 새 파일로 뺐다.
- F# 에서 파일을 나누는 비용은 `.fsproj` 에 순서를 적는 것이다. 컴파일러가 파일을 적힌 순서대로 읽으므로, 뒤 파일은 앞 파일의 이름만 볼 수 있다.

```
<ItemGroup>
  <Compile Include="Domain.fs" />
  <Compile Include="Store.fs" />
  <Compile Include="Inspections.fs" />
  <Compile Include="Program.fs" />
</ItemGroup>
```

|                   |                             |                           |
|-------------------|-----------------------------|---------------------------|
| HiveLog/          |                             |                           |
| ├─ HiveLog.fsproj |                             |                           |
| ├─ Domain.fs      | module HiveLog.Domain       | 벌통 · 점검 기록 타입과 조회         |
| ├─ Store.fs       | module HiveLog.Store        | 점검 기록 저장소                 |
| ├─ Inspections.fs | module HiveLog.Inspections  | 요청 · 응답 타입, 핸들러, 엔드포인트 목록 |
| └─ Program.fs     | 뷰 · 벌통 핸들러 · 라우팅 · 설정 · 진입점 |                           |



- 순서의 근거는 참조 방향 하나다. `Store.fs` 는 `Domain.fs` 의 타입을 쓰고, `Inspections.fs` 는 그 둘을 쓰고, `Program.fs` 는 셋을 다 쓴다. [`<EntryPoint>`] 가 붙은 함수는 마지막 파일에 있어야 하므로 `Program.fs` 는 자동으로 맨 끝이다.
- 원서는 파일 이름을 `TodoStore.fs` 로 잡고 그 안의 모듈 이름도 `GiraffeExample.TODOStore` , 타입 이름도 `TodoStore` 로 둔다. 컴파일에 문제는 없지만 세 이름이 겹쳐 읽기 어렵다. 이 노트는 파일과 모듈을 `Store` , 타입을 `InspectionStore` 로 갈라 두었다.
- `Program.fs` 에서 다른 파일의 이름을 쓰려면 `open` 을 적거나 `HiveLog.Inspections.inspectionEndpoints` 처럼 정규화된 이름을 쓴다. 모듈 전체를 열려면 `open HiveLog.Domain` , 모듈 이름을 접두로 남겨 두려면 `open HiveLog` 를 적고 `Inspections.inspectionEndpoints` 처럼 쓴다. 이 노트는 엔드포인트 목록만 접두를 남긴다.

## 스크립트에서 앱을 세우는 준비 (노트 보충)

---

- 검증 방식은 13챕터와 같다. `dotnet fsi` 스크립트 안에서 Giraffe 앱을 띄우고, 같은 스크립트의 `HttpClient` 로 요청을 보내 응답을 찍고, 마지막에 서버를 내린다. 준비 코드도 13챕터의 것을 그대로 쓴다.
- 선행 요구사항은 두 가지다. [ASP.NET Core](#) 공유 프레임워크가 포함된 .NET SDK(이 노트는 10.0.111 로 확인했다), 그리고 처음 한 번 `Giraffe 8.3.0` 과 `Giraffe.ViewEngine 1.4.0` 을 받아 올 네트워크다. 받아 오지 못하면 `error FS0999` 로 실패한다.
- 실제 개발은 프로젝트를 만들어 `dotnet run` 으로 확인하는 것이다. 아래 준비 코드는 노트의 예제를 자동으로 검증하기 위한 장치이고 `Program.fs` 에는 들어가지 않는다.

```
// 이 단위가 보여주는 것: 저장소 · 모델 바인딩 · 다섯 메서드를 갖춘 API 를 띄워 응답까지 확인하기
// 준비 코드 - 프로젝트 코드에는 들어가지 않는다(13챕터와 같다)
open System
open System.IO
open System.Runtime.Loader

let sharedFramework =
    let core =
        Runtime.InteropServices.RuntimeEnvironment.GetRuntimeDirectory().TrimEnd(Path.DirectorySeparatorChar)
        |> DirectoryInfo
    // 폴더 이름을 문자열로 비교하면 10.0.9 가 10.0.11 보다 크므로 Version 으로 비교한다
    let version (path: string) =
        match Version.TryParse((Path.GetFileName path).Split('-')[0]) with
        | true, v -> v
        | _ -> Version(0, 0)
    let root = Path.Combine(core.Parent.Parent.FullName, "Microsoft.AspNetCore.App")
    if not (Directory.Exists root) then
        failwithf "ASP.NET Core 공유 프레임워크 폴더가 없다: %s. 이 폴더가 포함된 .NET SDK 가 필요하다." root
    let candidates = Directory.GetDirectories root |> Array.sortBy version
    match candidates |> Array.filter (fun d -> (version d).Major = (version core.FullName).Major) with
    | [||] -> Array.last candidates
    | sameLine -> Array.last sameLine

AssemblyLoadContext.Default.add_Resolving(
    Func<AssemblyLoadContext, Reflection.AssemblyName, Reflection.Assembly>(fun context name ->
        let dll = Path.Combine(sharedFramework, name.Name + ".dll")
        if File.Exists dll then context.LoadFromAssemblyPath dll else null))

let referenceDir = Path.Combine(__SOURCE_DIRECTORY__, "aspnetcore")
Directory.CreateDirectory referenceDir |> ignore
for name in [ "Microsoft.AspNetCore"
    "Microsoft.AspNetCore.Hosting"
    "Microsoft.AspNetCore.Hosting.Abstractions"
    "Microsoft.AspNetCore.Http"
    "Microsoft.AspNetCore.Http.Abstractions"
    "Microsoft.AspNetCore.Routing"
    "Microsoft.Extensions.DependencyInjection.Abstractions"
    "Microsoft.Extensions.Hosting.Abstractions"
    "Microsoft.Extensions.Logging"
    "Microsoft.Extensions.Logging.Abstractions" ] do
    let source = Path.Combine(sharedFramework, name + ".dll")
    if not (File.Exists source) then
        failwithf "%s.dll 을 %s 에서 찾지 못했다." name sharedFramework
    let target = Path.Combine(referenceDir, name + ".dll")
    if not (File.Exists target) then
        try File.CreateSymbolicLink(target, source) |> ignore
        with _ -> File.Copy(source, target, true)

printfn "ASP.NET Core %s" (Path.GetFileName sharedFramework) // ASP.NET Core 10.0.11
```

참조와 `open` 은 13챕터에 `System.Collections.Concurrent` 한 줄만 더한 것이다. 저장소가 그 네임스페이스의 타입을 쓴다.

```
#r "aspnetcore/Microsoft.AspNetCore.dll"
#r "aspnetcore/Microsoft.AspNetCore.Hosting.dll"
#r "aspnetcore/Microsoft.AspNetCore.Hosting.Abstractions.dll"
#r "aspnetcore/Microsoft.AspNetCore.Http.dll"
#r "aspnetcore/Microsoft.AspNetCore.Http.Abstractions.dll"
#r "aspnetcore/Microsoft.AspNetCore.Routing.dll"
#r "aspnetcore/Microsoft.Extensions.DependencyInjection.Abstractions.dll"
#r "aspnetcore/Microsoft.Extensions.Hosting.Abstractions.dll"
#r "aspnetcore/Microsoft.Extensions.Logging.dll"
#r "aspnetcore/Microsoft.Extensions.Logging.Abstractions.dll"
#r "nuget: Giraffe, 8.3.0"
#r "nuget: Giraffe.ViewEngine, 1.4.0"

open System.Collections.Concurrent
open System.Globalization
open System.Text.Encodings.Web
open System.Text.Json
open Microsoft.AspNetCore.Builder
open Microsoft.AspNetCore.Http
open Microsoft.Extensions.DependencyInjection
open Microsoft.Extensions.Logging
open Giraffe
open Giraffe.EndpointRouting
open Giraffe.ViewEngine
```

- 이 프로젝트는 파일마다 최상위 모듈 하나를 선언한다(4챕터에서 본 `module Shop.Learners` 형태다). 스크립트에서는 최상위 `module X.Y` 를 쓸 수 없으므로 그 구조를 중첩 모듈로 본뜬다. 아래 코드의 `module Domain =` 은 프로젝트의 `Domain.fs` 첫 줄 `module HiveLog.Domain` 에 해당하고, 그 아래 들여쓴 부분이 그 파일의 본문이다. 선언 순서가 곧 `.fsproj` 의 컴파일 순서다.
- 대응이 어긋나는 곳이 하나 있다. `open` 은 적힌 스코프 안에서만 유효하므로, 스크립트에서는 위쪽 준비 블록의 `open` 이 아래 중첩 모듈에 다 미치지만 프로젝트에서는 파일마다 다시 적어야 한다. `Store.fs` 는 `System` 과 `System.Collections.Concurrent` , `Inspections.fs` 는 `System · System.Text.Json · Microsoft.AspNetCore.Http · Giraffe · Giraffe.EndpointRouting` 이 필요하다.
- 코드 주석의 `FSI` 실측 은 13챕터와 같은 방식으로 네임스페이스와 모듈 접두를 벗겨 적었다. `FSI` 는 `Giraffe.Core.HttpFunc` , `Domain.Inspection` , `Giraffe.EndpointRouting.Routers.Endpoint` 처럼 접두를 붙여 찍는다.

## Sample Data — 쓸 수 있는 저장소 (원서 pp.178-179)

- 13챕터의 저장소는 최상위 `let hives = [ ... ]` 였다. 리스트는 불변이므로 요청이 값을 바꿀 수 없다. 쓰기를 받으려면 상태를 담을 그릇이 필요하다.
- 원서는 데이터베이스를 붙이지 않고 `ConcurrentDictionary` 를 클래스 타입으로 감싼다. 웹 서버는 요청을 여러 스레드에서 동시에 처리하므로 그냥 `Dictionary` 를 쓰면 동시 접근에서 깨진다. `ConcurrentDictionary` 는 그 조정을 스스로 한다.
- 프로세스가 끝나면 내용도 사라진다. 저장소를 인터페이스로 추상화하지 않은 것은 이 챕터의 범위를 좁히려는 선택이다. 클래스 하나가 사전을 감싸고 있으므로 그 안을 실제 데이터베이스로 바꾸는 일은 이 파일에서 끝나지만, 구현을 둘 이상 두고 갈아 끼우려면 그때 인터페이스가 필요해진다. 핸들러가 구체 클래스 이름으로 꺼내 쓰고 있다는 점도 그때 걸린다.

`ConcurrentDictionary` 를 F# 에서 쓸 때 알아 둘 관용구가 셋 있다. 저장소 코드를 읽기 전에 따로 확인한다.

```
// 이 단위가 보여주는 것: ConcurrentDictionary 를 F# 에서 다루는 관용구
open System.Collections.Concurrent

type Reading = { Sensor: string; Celsius: float }

let readings = ConcurrentDictionary<string, Reading>()

// 인덱스에 값을 넣는 것은 없으면 추가, 있으면 덮어쓰기다
readings["r1"] <- { Sensor = "H-07"; Celsius = 34.2 }
readings["r1"] <- { Sensor = "H-07"; Celsius = 34.8 }
printfn "%d개 %.1f도" readings.Count readings["r1"].Celsius // 1개 34.8도
```

- 첫째는 인덱서 설정이다. <- 는 13챕터에서 본 할당 연산자이고, 사전에서는 추가와 갱신을 한 번에 처리한다. 원서는 추가에 TryAdd, 갱신에 TryUpdate 를 따로 쓰는데 그러면 호출하는 쪽이 지금 키가 있는지 먼저 알아야 한다.

```
// out 매개변수를 쓰는 .NET 메서드는 F# 에서 튜플을 돌려준다
// FSI 실측: key: string -> Reading option
let tryFind (key: string) =
    match readings.TryGetValue key with
    | true, found -> Some found
    | false, _ -> None

printfn "%A" (tryFind "r1" |> Option.map (fun reading -> reading.Celsius)) // Some 34.8
printfn "%A" (tryFind "r9" |> Option.map (fun reading -> reading.Celsius)) // None
```

- 둘째는 out 매개변수다. C# 에서 dict.TryGetValue(key, out value) 로 쓰는 메서드를 F# 은 bool \* 'Value 튜플을 돌려주는 함수로 본다. 그래서 성공 여부와 값을 한 번의 패턴 매칭으로 갈라낼 수 있고, 그 결과를 3챕터의 option 으로 바꿔 밖으로 내보내면 호출하는 쪽에서 bool 을 들고 다닐 일이 없다.
- 타입 주석 (key: string) 은 장식이 아니다. TryRemove 처럼 오버로드가 둘 이상인 메서드에서는 키 타입이 정해지지 않으면 컴파일러가 어느 것을 부를지 고르지 못한다. 그때 bool 만 돌려주는 오버로드가 잡혀 패턴 매칭에서 error FS0001 이 난다.

```
// FSI 실측: key: string -> Reading option
let tryRemove (key: string) =
    match readings.TryRemove key with
    | true, removed -> Some removed
    | false, _ -> None

printfn "%A" (tryRemove "r1" |> Option.map (fun reading -> reading.Celsius)) // Some 34.8
printfn "%A" (tryRemove "r1" |> Option.map (fun reading -> reading.Celsius)) // None
printfn "%d개" readings.Count // 0개
```

- 셋째는 없는 키의 처리다. TryRemove 는 없는 키에도 false 를 돌려주지만, 인덱서로 읽으면 예외가 난다.

```
// 없는 키를 인덱서로 읽으면 예외가 난다
try
    readings["r9"] |> ignore
with e ->
    printfn "%s" (e.GetType().Name) // KeyNotFoundException

// 반면 TryUpdate 는 없는 키에도 예외 없이 false 를 돌려준다
let stale = { Sensor = "H-11"; Celsius = 0.0 }
printfn "%b" (readings.TryUpdate("r9", stale, stale)) // false
```

- 원서 p.179 의 `update` 멤버는 `data.TryUpdate(todo.Id, todo, data[todo.Id])` 다. 세 번째 인자로 현재 값을 읽는데, 그 키가 없으면 `TryUpdate` 에 닿기 전에 인덱서가 `KeyNotFoundException` 을 던진다. 없는 항목에 PUT 을 보내면 의도한 410 대신 500 이 돌아온다는 뜻이다. 게다가 방금 읽은 값을 그대로 비교 인자로 넘기므로 비교가 거의 언제나 성공한다. 읽은 뒤 갱신에 닿기 전에 다른 스레드가 끼어들면 조용히 `false` 가 되는데, 그 결과를 본문에 실어 200 으로 답하니 호출한 쪽이 알 길도 없다. 이 노트는 갱신을 인덱서 설정 하나로 처리하고, 있는지 없는지는 핸들러에서 먼저 확인한다.

이제 도메인이다. 13챕터의 타입과 조회 함수를 그대로 옮기고, 점검 기록 타입 둘을 더한다.

```
// Domain.fs - 프로젝트에서는 module HiveLog.Domain 으로 시작하는 파일이다
module Domain =
    // 여기까지 13챕터에서 그대로 이어받는다
    type HiveCode = HiveCode of string

    type Hive = {
        Code: HiveCode
        Frames: int
        LastCheckedOn: DateOnly
        QueenSeen: bool
    }

    let hives =
        [ { Code = HiveCode "H-07"; Frames = 8; LastCheckedOn = DateOnly(2026, 4, 12); QueenSeen = true }
          { Code = HiveCode "H-11"; Frames = 10; LastCheckedOn = DateOnly(2026, 4, 19); QueenSeen = false } ]

    // FSI 실측: code: string -> Result<Hive,string>
    let findHive (code: string) : Result<Hive, string> =
        hives
        |> List.tryFind (fun hive -> hive.Code = HiveCode(code.ToUpperInvariant()))
        |> Option.map Ok
        |> Option.defaultWith (fun () -> Error $"등록되지 않은 벌통 코드: {code}")

    // 이 챕터가 더하는 것
    type InspectionId = InspectionId of Guid

    type Inspection = {
        Id: InspectionId
        Hive: HiveCode
        Frames: int
        QueenSeen: bool
        RecordedAt: DateTime
    }
```

- `InspectionId` 는 9챕터의 단일 케이스 판별 유니온이다. 원서는 `type TodoId = Guid` 로 타입 약어를 쓰는데, 약어는 이름만 붙일 뿐이어서 `Guid` 를 받는 자리에 아무 `Guid` 나 들어간다. 감싸 두면 벌통 코드와 점검 기록 id 를 뒤바꿔 넣는 실수가 컴파일에서 걸린다.
- 감싸는 대가는 뒤에서 치른다. `System.Text.Json` 은 `option` 이 아닌 판별 유니온을 다루지 못하므로 이 타입은 응답에도 요청에도 그대로 실을 수 없다.

저장소는 클래스 타입이다. 10챕터에서 본 대로 본문의 `let` 은 밖에서 보이지 않는다.

```
// Store.fs - module HiveLog.Store
module Store =
    open Domain

    // FSI 실측:
    // new: unit -> InspectionStore
    // member All: unit -> Inspection list
    // member Remove: InspectionId -> Inspection option
    // member Save: inspection: Inspection -> unit
    // member TryFind: InspectionId -> Inspection option
    type InspectionStore() =
        let data = ConcurrentDictionary<Guid, Inspection>()

        member _.Save(inspection: Inspection) =
            let (InspectionId key) = inspection.Id
            data[key] <- inspection

        member _.TryFind(InspectionId key) =
            match data.TryGetValue key with
            | true, found -> Some found
            | false, _ -> None

        member _.Remove(InspectionId key) =
            match data.TryRemove key with
            | true, removed -> Some removed
            | false, _ -> None

        member _.All() =
            data.Values |> Seq.sortBy (fun item -> item.RecordedAt) |> Seq.toList
```

- 사전의 키는 `Guid` 이고 멤버가 받는 것은 `InspectionId` 다. 꺾뎀기를 벗기는 자리를 저장소 안으로 모아 두어 밖에서는 감싼 타입만 쓰게 했다. `member _.TryFind(InspectionId key)` 처럼 매개변수 자리에 패턴을 적으면 받는 즉시 꺾뎀기가 벗겨진다.
- `Save` 하나가 추가와 갱신을 겸한다. 그래서 저장소는 `bool` 을 돌려줄 일이 없고, 조회와 삭제만 `Option` 을 돌려준다. HTTP 상태 코드를 고르는 판단은 전부 핸들러 몫이다.
- `All` 이 `Seq` 를 그대로 내보내지 않고 `Seq.toList` 로 확정하는 것은 5챕터의 이야기다. `data.Values` 는 그 순간의 값을 복사해 담은 `ReadOnlyCollection<'T>` 라 열거는 이미 안전하지만, `Seq.sortBy` 가 지연 평가되므로 그 결과를 그대로 내보내면 정렬이 언제 일어나는지 알 수 없는 `seq` 가 저장소 밖으로 나간다. `Seq.toList` 가 그 자리에서 정렬을 끝내고 타입도 `Inspection list` 로 못박는다.
- 평범한 `Dictionary` 의 `Values` 는 반대로 살아 있는 뷰다. 열거 도중에 사전이 바뀌면 `InvalidOperationException` 이 나므로, 요청을 여러 스레드에서 받는 자리에서 `ConcurrentDictionary` 를 고른 이유가 여기서도 확인된다.

## 저장소를 의존성 주입으로 넘긴다 (원서 p.179)

- 저장소 인스턴스는 하나만 있어야 한다. 요청마다 새로 만들면 방금 저장한 기록이 사라진다.
- [ASP.NET Core](#) 의 의존성 주입 컨테이너에 싱글톤(singleton)으로 등록하면 그 하나를 앱 전체가 공유한다. 등록 코드는 뒤의 설정 절에 있다.
- 핸들러는 그 인스턴스를 `HttpContext` 에서 받는다. Giraffe 가 붙여 주는 확장 멤버 `ctx.GetService<'T>()` 가 컨테이너에서 등록된 타입을 찾아 준다.

```
let store = ctx.GetService<InspectionStore>()
```

- 원서는 이 방식을 서비스 로케이션(service location)이라 부르고 의존성 주입의 간단한 형태라고 적는다. 생성자로 받는 대신 필요할 때 컨테이너에 물어보는 쪽이다. 함수가 매개변수로 받지 않은 것을 몸통에서 꺼내 쓰므로 테스트할 때 대신 넣어 주기가 까다롭다는 점은 알아 둘 만하다. 핸들러를 `InspectionStore -> HttpHandler` 로 적고 엔드포인트 목록을 만들 때 저장소를 넘기는 방법도 있고, 그러면 컨테이너 없이도 테스트가 된다.
- 등록하지 않은 타입을 `GetService` 로 꺼내면 실행 중에 `Giraffe.MissingDependencyException` 이 난다. 메시지가 `services.AddGiraffe()` 를 부르라고 안내하지만 실제 원인은 그 타입을 등록하지 않은 것이다. 컴파일은 통과하므로 등록을 잊으면 첫 요청에서 500 을 받는다.

## 요청 본문과 응답 본문을 위한 타입 (원서 p.182 확장)

- POST 와 PUT 은 요청 본문을 받는다. Giraffe 는 `ctx.BindJsonAsync<'T>()` 로 본문을 타입 있는 값으로 되돌려 준다. 이것이 모델 바인딩이다.
- 어떤 타입으로 되돌릴지가 설계 판단이다. 도메인 타입 `Inspection` 을 그대로 쓰면 id 와 기록 시각까지 클라이언트가 보내야 하고, `InspectionId` 가 `Option` 이 아닌 판별 유니온이라 역직렬화 자체가 되지 않는다. 그래서 요청과 응답에는 도메인과 별개인 평평한 타입을 쓴다. 이런 타입을 DTO(Data Transfer Object) 라 부른다.
- 되돌리는 쪽이 실제로 어떻게 움직이는지 먼저 확인한다. Giraffe 의 모델 바인딩은 13챕터에서 등록한 직렬화를 그대로 쓰므로, 아래 결과가 곧 서버가 보는 결과다.

```
// 이 단위가 보여주는 것: 요청 본문을 F# 타입으로 되돌릴 때 실제로 벌어지는 일
open System
open System.Text.Json

// Giraffe 의 BindJsonAsync 가 쓰는 설정과 같다(13챕터에서 등록한 직렬화기)
let options = JsonSerializerOptions(JsonSerializerDefaults.Web)

type InspectionRequest = {
    Hive: string
    Frames: int
    QueenSeen: bool
}

// FSI 실측: body: string -> InspectionRequest
let parse (body: string) = JsonSerializer.Deserialize<InspectionRequest>(body, options)

printfn "%A" (parse ""{"hive":"H-07","frames":9,"queenSeen":true}"" )
// { Hive = "H-07"
//   Frames = 9
//   QueenSeen = true }
```

- F# 레코드는 생성자 매개변수가 곧 필드이므로 `System.Text.Json` 이 그 생성자를 찾아 채운다. 프로퍼티를 설정 가능하게 만들지 않아도 되고 `[<CLIMutable>]` 같은 특성을 붙일 일도 없다.

```
// 필드가 빠진 본문도 예외 없이 통과한다. 빠진 자리에는 타입의 기본값이 들어간다
let partial = parse ""{"queenSeen":true}""
printfn "Hive=%A Frames=%d QueenSeen=%b" partial.Hive partial.Frames partial.QueenSeen
// Hive=<null> Frames=0 QueenSeen=true
printfn "Hive 가 null 인가: %b" (isNull partial.Hive) // Hive 가 null 인가: true
```

- 이 결과가 이 절의 핵심이다. `hive` 를 보내지 않았는데 예외 없이 값이 만들어지고, `Hive` 필드에 `null` 이 들어 있다. F# 안에서만 만든 레코드라면 있을 수 없는 상태이고, 3챕터에서 `null` 을 밀어낸 노력이 바깥에서 들어오는 데이터에는 통하지 않는다는 뜻이다.
- 그래서 모델 바인딩 뒤에는 검증이 반드시 온다. 문자열 필드를 `String.IsNullOrWhiteSpace` 로 보고 숫자 필드의 범위를 보는 일까지 모델 바인딩이 해 주지는 않는다.

```
// 예외를 던지는 경우는 따로 있다. 타입 이름과 첫 문장만 찍는다
let describe (action: unit -> unit) =
    try
        action ()
    with e ->
        let message = e.Message.Split '.' |> Array.head
        // FSI 가 타입 이름 앞에 붙이는 모듈 접두는 지운다
        printfn "%s: %s." (e.GetType().Name) (Text.RegularExpressions.Regex.Replace(message, @"FSI_\d+\+", ""))

type HiveCode = HiveCode of string

describe (fun () -> JsonSerializer.Deserialize<HiveCode>("""["H-07"]""", options) |> ignore)
// NotSupportedException: F# discriminated union serialization is not supported.
describe (fun () -> parse """{"hive":}""") |> ignore
// JsonException: '}' is an invalid start of a value.
describe (fun () -> parse """) |> ignore
// JsonException: The input does not contain any JSON tokens.
describe (fun () -> parse """{"frames":"아홉"}""") |> ignore
// JsonException: The JSON value could not be converted to InspectionRequest.
```

- 판별 유니온은 읽는 방향에서도 막힌다. `System.Text.Json` 의 F# 지원은 레코드 · `list` · `Set` · `Map` · 튜플과 `option` 까지이고, `option` 이 아닌 판별 유니온은 방향에 상관없이 `NotSupportedException` 이다. 13챕터가 응답에서 만난 예외가 요청에서도 그대로 나오고, 본문을 어떤 모양으로 보내든 타입을 보는 순간 막히므로 본문을 고쳐 피할 방법이 없다. 도메인 타입을 요청 타입으로 쓸 수 없는 이유가 이것이다.
- 나머지 셋은 모두 `JsonException` 이다. 깨진 JSON, 빈 본문, 타입이 맞지 않는 값이 한 종류의 예외로 모이므로 핸들러에서 한 번만 잡으면 된다. 잡지 않으면 500 이 나가는데, 잘못된 본문을 보낸 것은 클라이언트 쪽 문제이므로 400 이 맞다.

```
// bool 은 빠진 것과 false 를 구별할 수 없다. Option 은 구별한다
type StrictRequest = {
    Hive: string
    Frames: int option
    QueenSeen: bool option
}

// FSI 실측: body: string -> StrictRequest
let parseStrict (body: string) = JsonSerializer.Deserialize<StrictRequest>(body, options)

printfn "%A" (parseStrict """{"hive":"H-07"}""").QueenSeen // None
printfn "%A" (parseStrict """{"hive":"H-07","queenSeen":false}""").QueenSeen // Some false
printfn "%A" (parseStrict """{"hive":"H-07","frames":9}""").Frames // Some 9
```

- `string` 필드에는 `null` 이라는 표시가 남지만 `bool` 과 `int` 에는 그것이 없다. `queenSeen` 을 보내지 않은 요청과 `false` 를 보낸 요청이 서버에서 똑같이 `false` 로 보이므로 어떤 검증을 붙여도 갈라낼 수 없다. `Frames` 가 0 에서 걸리는 것은 0 이 마침 범위 밖이어서 생긴 우연이다.
- 필드가 빠진 것을 반드시 알아야 한다면 요청 타입의 필드를 `option` 으로 적는다. `System.Text.Json` 은 `option` 만은 특별히 지원해서 필드가 없거나 `null` 이면 `None` , 값이 있으면 `Some` 을 준다. 이 노트의 `InspectionRequest` 는 세 필드가 다 필수이고 빠진 자리를 검증이 400 으로 잡아 주므로 평평한 타입을 유지했다.



요청 타입, 응답 만들기, 검증을 `Inspections.fs` 앞부분에 둔다.

```
// Inspections.fs - module HiveLog.Inspections
module Inspections =
    open Domain
    open Store

    // 요청 본문을 받는 타입. 도메인 타입과 달리 평평하고 감싼 값이 없다
    type InspectionRequest = {
        Hive: string
        Frames: int
        QueenSeen: bool
    }

    // FSI 실행:
    // inspection: Inspection ->
    //   {| Frames: int; Hive: string; Id: Guid; QueenSeen: bool; RecordedAt: string |}
    let toPayload (inspection: Inspection) =
        let (InspectionId id) = inspection.Id
        let (HiveCode hive) = inspection.Hive
        {| Id = id
           Hive = hive
           Frames = inspection.Frames
           QueenSeen = inspection.QueenSeen
           RecordedAt = inspection.RecordedAt.ToString("yyyy-MM-ddTHH:mm:ssZ", CultureInfo.InvariantCulture) |}
```

- `toPayload` 가 감싼 값 둘을 벗기고 시각을 문자열로 바꾼다. 응답을 만드는 자리를 이 함수 하나로 모아 두면 핸들러 다섯 개가 같은 모양의 JSON 을 내보낸다.
- 시각을 찍을 때 `CultureInfo.InvariantCulture` 를 넘긴 것은 이 문자열을 기계가 읽기 때문이다. `ToString` 은 문화권을 주지 않으면 실행 환경의 것을 쓰는데, 사용자 지정 형식의 `:` 는 문화권의 시간 구분 기호이고 연도는 문화권의 달력으로 계산된다. `fi-FI` 로캘에서는 `09.00.00` , `th-TH` 로캘에서는 불교력의 `2569` 년이 응답에 실린다(실측). 사람이 읽을 화면이라면 그것이 맞는 동작이지만 JSON 응답에서는 받는 쪽의 파싱을 깨뜨린다.
- 익명 레코드의 필드는 알파벳 순으로 정렬된다. FSI 가 보여 주는 타입에서 적은 순서와 다른 순서가 나오는 것이 그 때문이고, 응답 JSON 의 키 순서도 같다.

```
// FSI 실행: request: InspectionRequest -> Result<HiveCode,string list>
let validate (request: InspectionRequest) : Result<HiveCode, string list> =
    let hiveResult =
        if String.IsNullOrEmpty request.Hive then Error "hive 를 적어야 한다"
        else findHive request.Hive |> Result.map (fun hive -> hive.Code)
    let frameErrors =
        if request.Frames <= 0 then [ "frames 는 1 이상이어야 한다" ] else []
    match hiveResult, frameErrors with
    | Ok code, [] -> Ok code
    | Ok _, errors -> Error errors
    | Error message, errors -> Error (message :: errors)
```

- 8챕터의 검증이 여기 그대로 쓰인다. 실패를 문자열 리스트로 모으므로 클라이언트가 한 번의 응답으로 잘못 된 곳을 다 볼 수 있다. 첫 실패에서 멈추는 `Result.bind` 사슬과 다른 점이다.
- 벌통 코드가 등록된 것인지는 13챕터의 `findHive` 가 판단한다. 조회 경로를 그 함수 하나로 좁혀 둔 덕에 검증이 새 코드를 쓰지 않는다.
- 검증이 돌려주는 것은 `HiveCode` 다. 문자열을 그대로 넘기지 않고 감싼 값으로 바꿔 내보내면, 검증을 지나지 않은 문자열이 도메인 타입에 들어갈 길이 막힌다.
- 실패 타입이 8챕터의 `ValidationError` 판별 유니온이 아니라 `string list` 인 것은 이 오류를 쓰는 곳이 하나뿐이어서다. 오류는 곧 JSON 응답 본문이 되므로 케이스를 나눠 두고 다시 문자열로 되돌릴 이유가 없다. 오류마다 다른 상태 코드를 붙이거나 화면 쪽에서도 같은 검증을 쓴다면 8챕터처럼 판별 유니온으로 올려야 한다.

## Handlers — 실패 응답과 GET 두 개 (원서 pp.181-182)

- 핸들러 다섯 개가 같은 실패 응답 둘을 나눠 쓴다. 먼저 그 둘을 값으로 뽑아 둔다.

```
module Handlers =

  // FSI 실측: HttpHandler
  let recordMissingHandler : HttpHandler = RequestErrors.notFound (json {| Error = "점검 기록이 없다" |})

  // FSI 실측: errors: string list -> HttpHandler
  let requestInvalidHandler (errors: string list) : HttpHandler =
    RequestErrors.badRequest (json {| Errors = errors |})
```

- 원서는 핸들러를 `Handlers` 라는 중첩 모듈에 담으므로 이름의 `Handler` 꼬리를 떼도 되겠다고 적어 둔다. 이 노트는 13챕터에서 정한 이름 규칙을 지켜 꼬리를 남긴다. 원서는 핸들러를 한 모듈에만 두지만 이 노트는 `Program.fs` 쪽에도 핸들러가 있어 접미사를 떼면 규칙이 반쪽이 된다. 규칙이 챕터마다 흔들리는 것보다는 조금 긴 이름이 낫다.
- 매개변수 타입 주석 `string list` 와 반환 타입 주석 `HttpHandler` 가 둘 다 필요하다. `json` 은 어떤 타입이든 받는 제네릭 함수라서 매개변수 타입 주석을 빼면 `requestInvalidHandler` 가 `errors: 'a -> ...` 로 자동 일반화되고, 반환 타입 주석을 빼면 `recordMissingHandler` 가 매개변수 없는 `let` 바인딩이라 값 제한 오류 FS0030 에 걸린다. 지금 노트에서는 뒤의 핸들러가 이 값을 써서 타입이 정해지지만, 두 줄만 떼어 옮기면 바로 오류가 난다.
- 타입 주석을 달아 두면 FSI 도 타입 약어 `HttpHandler` 를 그대로 찍는다. 13챕터의 `notFoundHandler` 와 같은 모양이고, 펼쳐진 형태로 나오는 것은 `fun _ ctx -> ...` 처럼 람다로 적은 핸들러 쪽이다.

목록과 한 건 조회부터 만든다.

```
// FSI 실측: HttpFunc -> ctx: HttpContext -> HttpFuncResult
let inspectionListHandler : HttpHandler =
    fun _ ctx ->
        let store = ctx.GetService<InspectionStore>()
        store.All() |> List.map toPayload |> ctx.WriteJsonAsync

// FSI 실측: id: Guid -> next: HttpFunc -> ctx: HttpContext -> HttpFuncResult
let inspectionDetailHandler (id: Guid) : HttpHandler =
    fun next ctx ->
        task {
            let store = ctx.GetService<InspectionStore>()
            let handler =
                match store.TryFind(InspectionId id) with
                | Some inspection -> json (toPayload inspection)
                | None -> recordMissingHandler
            return! handler next ctx
        }
```

- 목록 핸들러는 다음 핸들러를 부르지 않고 응답을 끝내므로 첫 매개변수를 `_` 로 버린다. `task` 가 없는 것은 `ctx.WriteJsonAsync` 가 이미 `Task` 를 돌려주기 때문이다.
- 한 건 조회 핸들러는 매개변수로 `Guid` 를 받는다. 뒤에서 `routeof "/%0"` 로 연결할 것이고, 그 `%0` 가 경로 조각을 `Guid` 로 바꿔 넘긴다. 13챕터의 `%s` 와 같은 자리이고 핸들러로 넘어오는 타입만 다르다.
- `match` 로 핸들러를 고르고 마지막에 한 번 부르는 형태를 눈여겨볼 만하다. 분기마다 `return!` 을 적는 것 보다 짧고, 두 분기의 타입이 같아야 한다는 사실이 눈에 보인다. 원서도 같은 형태를 쓰는데 괄호로 감싸 (`match ... with ...`) `next ctx` 로 적는다. 이 노트는 중간 값에 이름을 붙였다.
- `store.TryFind` 가 `Option` 을 돌려주므로 핸들러가 하는 일은 없음을 404 로 옮기는 것뿐이다. 저장소는 HTTP 를 모르고 핸들러만 상태 코드를 안다. 13챕터에서 `findHive` 를 두고 했던 이야기가 그대로 반복된다.

## POST — 모델 바인딩과 201 Created (원서 p.182)

- 본문을 읽는 부분을 따로 뽑는다. POST 와 PUT 이 같은 코드를 쓰고, 예외를 `Result` 로 바꾸는 자리를 한 곳으로 모을 수 있다.

```
// FSI 실측: ctx: HttpContext -> Task<Result<InspectionRequest,string list>>
let bindRequest (ctx: HttpContext) =
    task {
        try
            let! request = ctx.BindJsonAsync<InspectionRequest>()
            return Ok request
        with :? JsonException ->
            return Error [ "본문이 올바른 JSON 이 아니다" ]
    }
```

- `task { ... }` 는 12챕터에서 다룬 계산 식이다. 그 안에서도 `try ... with` 를 평범한 코드처럼 적을 수 있고, `let!` 이 기다리는 동안 난 예외도 여기서 걸린다.
- `:? JsonException` 은 3챕터에서 본 타입 테스트 패턴이다. 앞 절에서 확인한 대로 깨진 본문·빈 본문·타입 불일치가 모두 이 예외로 오므로 한 갈래로 충분하다.
- 예외를 잡아 `Result` 로 바꾸는 이 함수가 경계다. 여기서부터 아래로는 예외가 없고 값만 흐른다.

```
// FSI 실측: next: HttpFunc -> ctx: HttpContext -> HttpFuncResult
let inspectionCreateHandler : HttpHandler =
    fun next ctx ->
        task {
            let store = ctx.GetService<InspectionStore>()
            let! bound = bindRequest ctx
            let handler =
                match bound with
                | Error errors -> requestInvalidHandler errors
                | Ok request ->
                    match validate request with
                    | Error errors -> requestInvalidHandler errors
                    | Ok code ->
                        let inspection =
                            { Id = InspectionId(Guid.NewGuid())
                              Hive = code
                              Frames = request.Frames
                              QueenSeen = request.QueenSeen
                              RecordedAt = DateTime.UtcNow }
                        store.Save inspection
                        let (InspectionId id) = inspection.Id
                        setHttpHeader "Location" $"api/inspections/{id}"
                        ==> Successful.CREATED(toPayload inspection)
            return! handler next ctx
        }
}
```

- 흐름이 세 단계다. 본문을 읽고, 검증하고, 도메인 타입을 만들어 저장한다. 앞의 둘이 실패하면 400 이고 성공하면 201 이다.
- id 와 기록 시각은 서버가 정한다. 클라이언트가 보낸 것을 쓰지 않으므로 요청 타입에 그 두 필드가 없다.
- `Successful.CREATED` 가 상태 코드 201 을 붙인다. 원서는 저장소가 돌려준 `bool` 을 그대로 본문에 실어 200 으로 응답하는데, 그러면 만들기가 실패해도 200 이 나가고 클라이언트는 방금 만든 것의 id 를 알 수 없다. 201 과 함께 만들어진 것을 본문에 실어 주는 편이 낫다.
- `Location` 헤더는 만들어진 것을 어디서 볼 수 있는지 알려 준다. `>=>` 로 헤더 핸들러와 본문 핸들러를 이어 붙이는 것은 13챕터의 `apiSummaryHandler` 와 같은 형태다.

## PUT — 교체와 검증 (원서 p.182)

```
// FSI 실측: id: Guid -> next: HttpFunc -> ctx: HttpContext -> HttpFuncResult
let inspectionUpdateHandler (id: Guid) : HttpHandler =
    fun next ctx ->
        task {
            let store = ctx.GetService<InspectionStore>()
            let! bound = bindRequest ctx
            let handler =
                match store.TryFind(InspectionId id), bound with
                | None, _ -> recordMissingHandler
                | Some _, Error errors -> requestInvalidHandler errors
                | Some existing, Ok request ->
                    match validate request with
                    | Error errors -> requestInvalidHandler errors
                    | Ok code ->
                        let updated =
                            { existing with
                              Hive = code
                              Frames = request.Frames
                              QueenSeen = request.QueenSeen }
                        store.Save updated
                        json (toPayload updated)
            return! handler next ctx
        }
}
```

- 있는지 확인한 결과와 본문을 읽은 결과를 튜플로 묶어 한 번에 매칭했다. 없는 id 면 본문이 잘못됐는지 따지지 않고 404 로 답한다.
- `{ existing with ... }` 는 2챕터의 복사-수정 레코드 식이다. 새 레코드를 만들어 저장하므로 원래 값은 그대로 남고, `Id` 와 `RecordedAt` 은 손대지 않는다. 점검 기록의 최초 기록 시각을 PUT 이 바뀌서는 안 된다.
- 원서는 갱신 결과로 `bool` 을 내보내고 실패에 상태 코드 410 을 쓴다. 410 은 있었는데 영구히 사라졌다는 뜻이라 없는 id 일반에 쓰기에는 좁다. 이 노트는 조회와 같은 404 로 맞췄다.

## DELETE — 204 No Content (원서 p.183)

```
// FSI 실측: id: Guid -> next: HttpFunc -> ctx: HttpContext -> HttpFuncResult
let inspectionDeleteHandler (id: Guid) : HttpHandler =
    fun next ctx ->
        task {
            let store = ctx.GetService<InspectionStore>()
            let handler =
                match store.Remove(InspectionId id) with
                | Some _ -> Successful.NO_CONTENT
                | None -> recordMissingHandler
            return! handler next ctx
        }
```

- 삭제는 저장소의 `Remove` 한 번으로 끝난다. 지운 값을 `Option` 으로 돌려주므로 있었는지 없었는지가 그 결과에 담겨 있고, 원서처럼 먼저 조회하고 그 값으로 다시 지울 필요가 없다. 조회와 삭제를 두 번에 나누면 그 사이에 다른 요청이 끼어들 틈도 생긴다.
- `Successful.NO_CONTENT` 는 상태 코드 204 만 보내고 본문을 쓰지 않는다. 지운 것을 굳이 되돌려줄 이유가 없을 때 쓴다. 값을 받는 다른 핸들러들과 달리 인자 없는 값이다.

## Routes — 메서드별로 묶은 엔드포인트 목록 (원서 pp.179-180)

- 13챕터의 엔드포인트 목록은 `GET [ ... ]` 하나였다. 이제 네 가지 메서드가 붙는다.  
`GET · POST · PUT · DELETE` 는 각각 엔드포인트 목록을 받아 엔드포인트를 만드는 함수이므로, 메서드마다 묶음을 하나씩 적고 그것들을 리스트로 나열하면 된다.

```
// FSI 실측: Endpoint list
let inspectionEndpoints =
    [
        GET [
            route "" Handlers.inspectionListHandler
            routef "%0" Handlers.inspectionDetailHandler
        ]
        POST [
            route "" Handlers.inspectionCreateHandler
        ]
        PUT [
            routef "%0" Handlers.inspectionUpdateHandler
        ]
        DELETE [
            routef "%0" Handlers.inspectionDeleteHandler
        ]
    ]
```

- 같은 경로에 메서드만 다른 핸들러가 붙는다. `route ""` 가 `/api/inspections` 자체이고 `routeef "/%0"` 가 그 아래 한 건이다. 접두는 이 목록을 엮는 쪽에서 붙인다.
- `routeef` 의 서식 지정자는 경로 템플릿(route template)으로 번역되어 [ASP.NET Core](#) 라우팅에 넘어간다. 엔드포인트는 판별 유니온이고 `routeef` 가 만든 값은 `TemplateEndpoint` 케이스에 그 템플릿 문자열을 담고 있으므로, 패턴 매칭으로 꺼내 볼 수 있다. `%0` 가 만드는 것은 이렇다.

```
GET /{00:regex(^[0-9A-Fa-f]{{8}}-[0-9A-Fa-f]{{4}}-[0-9A-Fa-f]{{4}}-[0-9A-Fa-f]{{4}}-[0-9A-Fa-f]{{12}}$|^[0-9A-Fa-f]{{32}}$|
```

- 중괄호가 겹쳐 있는 것은 경로 템플릿의 이스케이프다. [ASP.NET Core](#) 의 템플릿 파서는 `{{` 와 `}}` 를 리터럴 중괄호로 읽으므로, Giraffe 가 미리 겹쳐 둔 것이 파싱 단계에서 한 겹 풀려 실제 제약에는 `{8}` 이 들어간다. 정규식의 반복 횟수 표기가 템플릿의 매개변수 표기와 같은 문자를 쓰는 탓에 생긴 일이다.
- 제약이 있으니 `Guid` 모양이 아닌 조각은 이 엔드포인트에 아예 걸리지 않는다. `/api/inspections/oops` 는 핸들러에 닿지 못하고 13챕터에서 만든 `notFoundHandler` 로 떨어진다. 같은 404 라도 응답 본문이 다른 이유가 이것이다.
- 제약이 허용하는 모양은 셋이다. 하이픈이 들어간 보통 표기, 하이픈 없는 32자리, 그리고 22자리로 줄여 적은 표기다. 하이픈을 뺀 id 로 요청해도 같은 기록을 찾는 것을 뒤에서 확인한다.

## 13챕터의 나머지 조각과 콘텐츠 협상 (원서 p.184 확장)

여기부터 `Program.fs` 다. 뷰와 벌통 핸들러는 13챕터의 것을 그대로 쓴다.

```
// Program.fs
open Domain
open Store

let notFoundHandler : HttpHandler =
    "요청한 경로가 없다"
    |> text
    |> RequestErrors.notFound

let indexView =
    html [] [
        head [] [
            title [] [ str "HiveLog" ]
        ]
        body [] [
            h1 [] [ str "HiveLog" ]
            p [ _class "lede"; _id "intro" ] [ str "옥상 양봉장 점검 기록" ]
        ]
    ]

let hiveListHandler : HttpHandler =
    fun _ ctx ->
        hives
        |> List.map (fun hive ->
            let (HiveCode code) = hive.Code
            { | Hive = code; Frames = hive.Frames | })
        |> ctx.WriteJsonAsync

let hiveStatusHandler (code: string) : HttpHandler =
    fun next ctx ->
        task {
            match findHive code with
            | Ok hive ->
                let (HiveCode found) = hive.Code
                let payload =
                    { | Hive = found
                      Frames = hive.Frames
                      LastCheckedOn = hive.LastCheckedOn.ToString("yyyy-MM-dd", CultureInfo.InvariantCulture)
                      QueenSeen = hive.QueenSeen | }
                return! json payload next ctx
            | Error message ->
                return! RequestErrors.notFound (json { | Error = message | }) next ctx
        }
}
```

- 원서는 챕터를 맺으며 콘텐츠 협상(content negotiation)과 모델 검증을 아직 다루지 않았다고 적는다. 검증은 앞 절에서 했고, 콘텐츠 협상은 요약 핸들러 하나를 바꿔 보면 바로 확인된다.
- 콘텐츠 협상은 클라이언트가 `Accept` 헤더로 원하는 형식을 말하고 서버가 그중 만들 수 있는 것을 골라 응답하는 방식이다. Giraffe의 핸들러 `negotiate`가 그 판단을 한다. `json`을 `negotiate`로 바꾸면 같은 값이 형식만 달라져 나간다.

```
// FSI 실측: apiSummaryHandler: HttpHandler
let apiSummaryHandler : HttpHandler =
    setHttpHeader "X-Apiary" "seongsu-rooftop"
    >=> negotiate { | Apiary = "성수 옥상"; Hives = List.length hives | }
```

- `AddGiraffe()` 가 등록해 두는 기본 규칙이 `Accept` 값과 핸들러를 짝지어 준다. 규칙은 다섯 개다. `application/json` 과 `*/*` 는 JSON, `text/plain` 은 값의 문자열 표현, `application/xml` 과 `text/xml` 은 XML 이다. 어느 규칙에도 맞지 않으면 상태 코드 406 으로 답한다.
- XML 은 기대와 다르다. Giraffe 8 의 기본 XML 직렬화기는 매개변수 없는 생성자를 요구하는데 F# 레코드에는 그것이 없다. `Accept: application/xml` 로 요청하면 `System.InvalidOperationException: ... cannot be serialized because it does not have a parameterless constructor.` 가 나고 500 이 돌아온다. 익명 레코드도 이름을 붙인 레코드도 마찬가지다. XML 을 정말 내보내야 한다면 직렬화기를 갈아 끼우거나, 레코드에 `[<CLIMutable>]` 을 붙여 매개변수 없는 생성자를 만들어 주거나, XML 용 타입을 따로 두어야 한다. 앞 절에서 JSON 에는 이 특성이 필요 없다고 적은 것과 대비되는 자리다. 익명 레코드에는 특성을 붙일 수 없다.
- `Successful.CREATED` 도 콘텐츠 협상을 거친다. POST 에 `Accept: text/plain` 을 붙이면 201 응답 본문이 JSON 이 아니라 문자열 표현으로 온다. 뒤의 확인에서 그 줄을 볼 수 있다.

## subRoute 를 한 층 더 없는다 (원서 p.180)

- 13챕터가 라우팅을 두 층으로 만들어 두었다. HTTP 메서드 묶음을 안쪽 `apiEndpoints` 에 둔 덕에 이 챕터가 손덜 곳은 한 줄이다.

```
// FSI 실측: apiEndpoints: Endpoint list
let apiEndpoints =
    [
        GET [
            route "" apiSummaryHandler
            route "/hives" hiveListHandler
            route "/"hives/%s" hiveStatusHandler
        ]
        subRoute "/inspections" Inspections.inspectionEndpoints
    ]

// 13챕터와 한 글자도 다르지 않다
let endpoints =
    [
        GET [
            route "/" (htmlView indexView)
        ]
        subRoute "/api" apiEndpoints
    ]
```

- `subRoute "/inspections" Inspections.inspectionEndpoints` 한 줄이 다른 파일에서 만든 목록을 여기에 꽂는다. 라우팅이 값이라 이렇게 넘길 수 있다는 것이 13챕터에서 본 성질이고, 이 챕터가 그 값을 실제로 다른 파일에서 만들어 왔다.
- 원서는 새 경로를 바깥 `endpoints` 에 `subRoute "/api/todo" ...` 로 붙이고, 경로 문자열을 한곳에 모아 두는 편이 좋다고 적는다. 취향의 문제이고 원서도 그렇게 말한다. 이 노트는 `/api` 아래의 일을 `apiEndpoints` 안에서 끝내는 쪽을 골랐다. 접두가 한 번만 나오고, `/api` 밖의 경로와 안의 경로를 눈으로 갈라 볼 수 있다.



## 설정 — 저장소를 싱글톤으로 등록한다 (원서 p.179)

```
// FSI 실측: services: IServiceCollection -> unit
let configureServices (services: IServiceCollection) =
    let options = JsonSerializerOptions(JsonSerializerDefaults.Web)
    options.Encoder <- JavaScriptEncoder.UnsafeRelaxedJsonEscaping
    services
        .AddRouting()
        .AddGiraffe()
        .AddSingleton<Json.ISerializer>(Json.Serializer options)
        .AddSingleton<InspectionStore>(InspectionStore())
    |> ignore

// FSI 실측: appBuilder: IApplicationBuilder -> unit
let configureApp (appBuilder: IApplicationBuilder) =
    appBuilder
        .UseRouting()
        .UseGiraffe(endpoints)
        .UseGiraffe(notFoundHandler)
```

- 13챕터의 `configureServices` 에서 달라진 것은 마지막 등록 한 줄이다.  
`AddSingleton<InspectionStore>(InspectionStore())` 가 인스턴스를 직접 만들어 넘긴다. 초기 데이터를 넣어 두고 싶으면 그 생성자에 넘기면 된다.
- 등록 순서는 상관없다. `configureApp` 의 순서는 상관있다. 13챕터에서 본 대로 미들웨어 파이프라인을 정하는 코드라 라우팅을 먼저 켜고, 엔드포인트를 등록하고, 어디에도 걸리지 않은 요청을 받는 핸들러를 마지막에 둔다. 이 챕터가 여기는 건드리지 않는다.
- 진입점도 13챕터와 같다. `configureServices` 와 `configureApp` 을 부르고 `app.Run()` 으로 끝난다.

## Using the API — 실제 요청과 응답 (원서 pp.183-184)

- 원서는 Postman 으로 API 를 만져 보라고 안내하고 목록·생성 응답 두 개를 보여 준다. 프로젝트를 띄워 `curl` 로 확인하면 다음과 같다. 날짜와 전송 관련 헤더는 줄였다.

```
dotnet run
# 다른 터미널에서
curl -i -X POST http://localhost:5291/api/inspections \
  -H "Content-Type: application/json" \
  -d '{"hive":"h-07", "frames":9, "queenSeen":true}'
```

```
HTTP/1.1 201 Created
Content-Type: application/json; charset=utf-8
Location: /api/inspections/75eb055d-28ce-4a8a-89c4-4fe6886732f1

{"frames":9, "hive":"H-07", "id":"75eb055d-28ce-4a8a-89c4-4fe6886732f1", "queenSeen":true, "recordedAt":"2026-09-07T02:48:14Z"}
```

```
# curl -i -X DELETE http://localhost:5291/api/inspections/75eb055d-28ce-4a8a-89c4-4fe6886732f1
HTTP/1.1 204 No Content

# curl -i -X POST ... -d '{"queenSeen":true}'
HTTP/1.1 400 Bad Request
Content-Type: application/json; charset=utf-8

{"errors":["hive 를 적어야 한다", "frames 는 1 이상이어야 한다"]}
```

- 원서 p.184 가 보여 주는 목록 응답은 `{"key": ..., "value": ...}` 를 원소로 담은 배열이다. 그런데 원서의 `GetAll` 은 `data.Values |> Seq.ToArray` 여서 값만 나와야 하고 그 모양이 되지 않는다. 사전을 `KeyValuePair` 의 컬렉션으로 직렬화한 결과처럼 보이며, 코드와 지면의 출력 예가 어긋난 대목이다. 어느 쪽이든 사전의 구현 세부가 API 응답에 새어 나온 모양이므로 클라이언트가 볼 것이 아니다. 이 노트의 `All` 은 값만 담은 리스트를 돌려주므로 응답이 평평한 배열이다.

같은 확인을 스크립트에서 한다. 앞의 `14-server` 블록들이 프로젝트 코드와 같은 정의를 쌓아 두었으므로 남은 것은 호스트를 띄우고 요청을 보내는 부분이다.

```
// 여기부터는 검증용 코드다
let builder = WebApplication.CreateBuilder()
configureServices builder.Services
builder.Logging.ClearProviders() |> ignore

let app = builder.Build()
configureApp app
app.Uris.Add "http://127.0.0.1:0" // 0 = 빈 포트를 골라 달라는 뜻
app.StartAsync() |> Async.AwaitTask |> Async.RunSynchronously

let baseUrl = app.Uris |> Seq.head
let client = new Net.Http.HttpClient()

// 새로 만든 id 와 기록 시각은 실행마다 달라지므로 자리표시자로 바꿔 찍는다
let mask (value: string) =
    let replace (pattern: string) (replacement: string) (input: string) =
        Text.RegularExpressions.Regex.Replace(input, pattern, replacement)
    value
    |> replace "[0-9a-f]{8}-([0-9a-f]{4}-){3}[0-9a-f]{12}" "<id>"
    |> replace "[0-9a-f]{32}" "<id-n>"
    |> replace @"^(\d{4})-(\d{2})-(\d{2})T(\d{2}):(\d{2}):(\d{2})Z$" "<time>"

let jsonBody (content: string) =
    new Net.Http.StringContent(content, Text.Encoding.UTF8, "application/json")

let call (method: string) (path: string) (body: string option) (accept: string option) =
    let request = new Net.Http.HttpRequestMessage(Net.Http.HttpMethod method, baseUrl + path)
    body |> Option.iter (fun content -> request.Content <- jsonBody content)
    accept |> Option.iter (fun value -> request.Headers.TryAddWithoutValidation("Accept", value) |> ignore)
    let response = client.SendAsync request |> Async.AwaitTask |> Async.RunSynchronously
    let text = response.Content.ReadAsStringAsync() |> Async.AwaitTask |> Async.RunSynchronously
    response, (if text = "" then "(본문 없음)" else mask text)

let send method path body =
    let response, text = call method path body None
    printfn "%-6s %-23s %d %s" method (mask path) (int response.StatusCode) text
    response
```

먼저 빈 저장소를 확인하고 한 건을 만든다.

```
send "GET" "/api/inspections" None |> ignore
// GET /api/inspections 200 []
send "GET" "/api/inspections/2f8b0a4e-0000-4000-8000-000000000000" None |> ignore
// GET /api/inspections/<id> 404 {"error":"점검 기록이 없다"}

let created = send "POST" "/api/inspections" (Some ""{"hive":"h-07","frames":9,"queenSeen":true}""")
// POST /api/inspections 201 {"frames":9,"hive":"H-07","id":"<id>","queenSeen":true,"recordedAt":"<time>"}
let createdPath = string created.Headers.Location
printfn "Location: %s" (mask createdPath)
// Location: /api/inspections/<id>
```

- 소문자로 보낸 `h-07` 이 `H-07` 로 저장되었다. 13챕터의 `findHive` 가 대문자로 바꿔 비교하고, 검증이 그 결과의 `HiveCode` 를 돌려주므로 정규화가 한 번만 일어난다.
- 만들기 전에 임의의 id 로 조회한 것이 404 이고 본문이 JSON 이다. 뒤에 나오는 경로 자체가 없는 404 와 비교할 자리다.

만든 것을 다시 읽는다. `Location` 헤더의 경로가 그대로 조회 경로다.

```
send "GET" "/api/inspections" None |> ignore
// GET /api/inspections 200 [{"frames":9,"hive":"H-07","id":"<id>","queenSeen":true,"recordedAt":"<time>"}]
send "GET" createdPath None |> ignore
// GET /api/inspections/<id> 200 {"frames":9,"hive":"H-07","id":"<id>","queenSeen":true,"recordedAt":"<time>"}

// 하이픈을 뺀 32자리 표기도 경로 템플릿의 제약을 지나간다
let createdId = Guid(createdPath.Split('/') |> Array.last)
let dashless = createdId.ToString "N"
send "GET" $" /api/inspections/{dashless}" None |> ignore
// GET /api/inspections/<id-n> 200 {"frames":9,"hive":"H-07","id":"<id>","queenSeen":true,"recordedAt":"<time>"}

```

- `<id>` 와 `<id-n>` 은 자리표시자다. 앞은 하이픈이 들어간 보통 표기, 뒤는 하이픈을 뺀 32자리다. 두 요청이 같은 기록을 찾아 왔다.

교체와 실패 경로를 확인한다.

```
send "PUT" createdPath (Some ""{"hive":"H-11","frames":10,"queenSeen":false}"" ) |> ignore
// PUT /api/inspections/<id> 200 {"frames":10,"hive":"H-11","id":"<id>","queenSeen":false,"recordedAt":"<time>"}
send "POST" "/api/inspections" (Some ""{"queenSeen":true}"" ) |> ignore
// POST /api/inspections 400 {"errors":["hive 를 적어야 한다","frames 는 1 이상이어야 한다"]}
send "POST" "/api/inspections" (Some ""{"hive":"H-99","frames":3}"" ) |> ignore
// POST /api/inspections 400 {"errors":["등록되지 않은 벌통 코드: H-99"]}
send "POST" "/api/inspections" (Some ""{"hive":{}}"" ) |> ignore
// POST /api/inspections 400 {"errors":["본문이 올바른 JSON 이 아니다"]}

```

- PUT 이 `id` 와 `recordedAt` 을 그대로 두고 나머지만 바꿨다. 복사-수정 식에 그 두 필드를 적지 않았기 때문이다.
- 400 세 줄이 각각 다른 자리에서 나왔다. 첫 줄은 검증이 실패를 둘 모아 온 것이고, 둘째 줄은 `findHive` 가 낸 실패이며, 셋째 줄은 `bindRequest` 가 `JsonException` 을 잡아 바꾼 것이다. 세 갈래가 한 모양의 응답으로 모인다.

삭제와 없는 경로를 확인하고 서버를 내린다.

```
send "DELETE" createdPath None |> ignore
// DELETE /api/inspections/<id> 204 (본문 없음)
send "DELETE" createdPath None |> ignore
// DELETE /api/inspections/<id> 404 {"error":"점점 기록이 없다"}
send "GET" "/api/inspections/oops" None |> ignore
// GET /api/inspections/oops 404 요청한 경로가 없다

```

- 같은 요청을 두 번 보내면 204 다음에 404 다. 저장소가 지운 값을 `option` 으로 돌려주므로 핸들러가 그 차이를 상태 코드로 옮길 수 있다.
- 마지막 줄이 앞에서 이야기한 경로 템플릿 제약이다. `oops` 는 `Guid` 모양이 아니어서 엔드포인트에 걸리지 않고, 응답이 `notFoundHandler` 의 평문이다. 같은 404 인데 본문이 JSON 이 아닌 것으로 어느 자리에서 나온 404 인지 구별된다.

콘텐츠 협상은 `Accept` 헤더를 붙여 확인한다.

```
let negotiated method path body accept =
    let response, text = call method path body (Some accept)
    let oneLine = Text.RegularExpressions.Regex.Replace(text, @"\s+", " ")
    printfn "%-4s %-17s %d %-31s %s" method accept (int response.StatusCode)
        (string response.Content.Headers.ContentType) oneLine

negotiated "GET" "/api" None "application/json"
// GET application/json 200 application/json; charset=utf-8 {"apiary":"성수 옥상","hives":2}
negotiated "GET" "/api" None "text/plain"
// GET text/plain 200 text/plain; charset=utf-8 { Apiary = "성수 옥상" Hives = 2 }
negotiated "GET" "/api" None "text/html"
// GET text/html 406 text/plain; charset=utf-8 text/html is unacceptable by the server.
negotiated "POST" "/api/inspections" (Some "{ \"hive\":\"H-11\", \"frames\":7, \"queenSeen\":true }") "text/plain"
// POST text/plain 201 text/plain; charset=utf-8 { Frames = 7 Hive = "H-11" Id = <id> QueenSeen = true Recorded = true }

app.StopAsync() |> Async.AwaitTask |> Async.RunSynchronously
client.Dispose()
printfn "서버를 내렸다" // 서버를 내렸다
```

- 같은 요약 핸들러가 `Accept` 에 따라 세 가지로 답했다. `text/plain` 응답의 본문은 값의 문자열 표현이고, 원래 여러 줄로 오는 것을 위에서는 한 줄로 줄여 찍었다. 마지막 줄은 `Successful.CREATED` 도 같은 협상을 거친다는 것을 보여 준다.
- `Accept: text/html` 만 보내면 규칙에 없으므로 406 이다. 그런데 브라우저가 실제로 보내는 값은 `text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,*/*;q=0.8` 이고, 협상은 품질값 (q)이 큰 것부터 규칙에 있는 첫 형식을 고른다. `text/html` 을 지나 `application/xml` 이 먼저 걸리므로 주소창으로 열면 406 이 아니라 앞에서 본 XML 직렬화 실패로 500 이 온다. `text/html,*/*;q=0.8` 처럼 `*/*` 가 붙은 값이면 200 JSON 이다. 13챕터의 첫 화면처럼 HTML 을 내보내야 하는 경로에는 `htmlView` 를 그대로 쓴다.
- 마지막 두 줄이 중요하다. 서버를 내리고 클라이언트를 해제해야 스크립트가 끝나고, 다음 검증이 막히지 않는다.

## Summary — 원서의 챕터 요약 (원서 p.184)

- 원서는 Giraffe 로 API 를 만드는 방법의 걸만 훑었다고 적고, 콘텐츠 협상과 모델 검증 같은 것이 더 있으니 Giraffe 문서를 읽어 보라고 권한다.
- 이 노트는 그중 둘을 앞당겨 확인했다. 검증은 8챕터의 방식을 그대로 쓰면 되고, 콘텐츠 협상은 핸들러 하나를 바꾸는 것으로 되지만 XML 쪽에는 함정이 있다.
- 다음 챕터에서는 HTML 뷰로 돌아가 View Engine 을 깊게 다룬다.

## 정리 — 이 노트의 요약

- 쓰기를 받는 API에는 상태를 담을 그릇이 필요하다. `ConcurrentDictionary`를 클래스 타입으로 감싸 저장소를 만들고, 의존성 주입 컨테이너에 싱글톤으로 등록해 앱 전체가 하나를 쓰게 한다.
- 핸들러는 `ctx.GetService<InspectionStore>()`로 그 인스턴스를 꺼낸다. 원서가 서비스 로케이션이라 부르는 방식이고, 매개변수로 받지 않으므로 테스트에서 갈아 끼우기는 까다롭다.
- `out` 매개변수를 쓰는 .NET 메서드는 F#에서 튜플을 돌려주므로 `match dict.TryGetValue key with | true, v -> Some v | false, _ -> None`이 관용구다. 오버로드가 둘 이상인 `TryRemove` 같은 메서드는 키 타입을 주석으로 못박아야 한다.
- 저장소가 `bool` 대신 `Option`을 돌려주면 상태 코드 선택이 핸들러 한 곳으로 모인다. 원서의 `Update` 구현은 없는 키에서 인덱서가 `KeyNotFoundException`을 던져 500이 나가므로, 갱신은 인덱서 설정으로 하고 존재 확인은 핸들러가 한다.
- 요청 본문은 `ctx.BindJsonAsync<'T>()`로 받는다. 도메인 타입을 그대로 쓸 수 없다. `Option`이 아닌 판별 유니온은 읽는 방향에서도 `NotSupportedException`이고, 서버가 정하는 id와 시각을 클라이언트가 보낼 이유도 없다. 요청과 응답에는 평평한 DTO를 따로 둔다.
- 모델 바인딩은 검증이 아니다. 필드가 빠진 본문도 예외 없이 통과하고 문자열 필드에 `null`이 들어온다. `bool`과 `int`는 빠진 것과 기본값을 구별할 수 없으므로 그 구별이 필요하다면 요청 타입의 필드를 `Option`으로 적는다. 모델 바인딩 뒤에 검증이 반드시 와야 하고, 검증은 8챕터처럼 실패를 모아 한 번에 400으로 보내면 된다.
- 깨진 본문·빈 본문·타입 불일치는 모두 `JsonException`이다. `task { try ... with :? JsonException -> ... }`로 한 번 잡아 `Result`로 바꾸면, 그 아래로는 예외 없이 값만 흐른다. 잡지 않으면 클라이언트 잘못에 500이 나간다.
- 상태 코드는 `Successful.CREATED (201)`, `Successful.NO_CONTENT (204)`, `RequestErrors.badRequest (400)`, `RequestErrors.notFound (404)`로 고른다. 만든 것의 위치는 `Location` 헤더로 알려 준다.
- `routeof "/%0"`는 `Guid` 모양을 요구하는 정규식 제약이 붙은 경로 템플릿으로 번역된다. 모양이 맞지 않는 조각은 핸들러에 닿지 못하고 `notFoundHandler`로 떨어지며, 하이픈을 뺀 32자리 표기는 제약을 지나간다.
- `negotiate`는 `Accept` 헤더를 보고 형식을 고른다. JSON과 평문은 바로 되지만, Giraffe 8의 기본 XML 직렬화기는 매개변수 없는 생성자를 요구해서 F# 레코드를 다루지 못한다. 규칙에 없는 형식은 406이다.
- 파일을 나누는 대가는 컴파일 순서를 적는 것이다. 참조 방향이 순서를 정하고 `[<EntryPoint>]`는 마지막 파일에 있어야 한다. 라우팅이 값이므로 엔드포인트 목록을 다른 파일에서 만들어 `subRoute`로 꽃을 수 있다.

## 15챕터가 이어받는 것

프로젝트는 `HiveLog`이고 이제 파일이 넷이다. `.fsproj`의 컴파일 순서는 `Domain.fs`, `Store.fs`, `Inspections.fs`, `Program.fs`이며 15챕터가 뷰를 새 파일로 뺀다면 `Program.fs` 앞에 넣어야 한다. 도메인은 `Domain.fs`로 옮겨졌고 `HiveCode · Hive · hives · findHive`에 `InspectionId · Inspection`이 더해졌다. 점검 기록 저장소 `InspectionStore`는 `AddSingleton`으로 등록되어 있어 핸들러가 `ctx.GetService<InspectionStore>()`로 꺼내 쓴다. 라우팅은 세 층이다. 바깥 `endpoints`에 첫 화면과 `subRoute "/api" apiEndpoints`, `apiEndpoints`에 벌통 경로들과 `subRoute "/inspections"` `Inspections.inspectionEndpoints`가 있다. 15챕터가 화면용 경로를 더할 자리는 바깥 `endpoints`이고,

/api 아래는 건드릴 일이 없다. 이름 규칙은 13챕터와 같다. 핸들러 <대상><동작 또는 응답 내용>Handler, 엔드포인트 목록 <영역>Endpoints, 뷰 <이름>View 이며 핸들러를 담는 중첩 모듈은 Handlers 다. 원서에서 온 notFoundHandler 는 대상이 없어 규칙 밖이다. 뷰는 indexView 하나뿐이고 p [ \_class "lede"; \_id "intro" ] 가 CSS 클래스를 하나 쓰고 있는데, 그 클래스를 정의할 /css/hive.css 와 그것을 담을 wwwroot/ 는 아직 없으므로 15챕터가 만든다. 정적 파일을 내보내려면 configureApp 에 UseStaticFiles 계열의 미들웨어가 한 줄 더 들어가야 한다는 점도 그 챕터의 일이다. 응답에 도메인 타입을 그대로 실을 수 없다는 제약은 뷰에는 해당하지 않는다. View Engine 은 F# 값을 직접 읽어 HTML 을 만들므로 Inspection 을 그대로 넘겨도 된다. 검증 방식은 그대로 물려받는다. 14-server 실행 단위의 준비 코드를 쓰고, 포트 0 으로 띄워 HttpClient 로 요청한 뒤 StopAsync 로 내린다.

## 원서 대조 표

| 절                                        | 원서 페이지     | 실행 단위                 |
|------------------------------------------|------------|-----------------------|
| Getting Started · Our Task — 이 챕터가 더하는 것 | p.178      | —                     |
| Handlers — 파일을 넷으로 나눈다                   | p.181      | —                     |
| 스크립트에서 앱을 세우는 준비                         | (노트 보충)    | 14-server             |
| Sample Data — 쓸 수 있는 저장소                 | pp.178-179 | 14-store, 14-server   |
| 저장소를 의존성 주입으로 넘긴다                        | p.179      | —                     |
| 요청 본문과 응답 본문을 위한 타입                      | p.182 확장   | 14-binding, 14-server |
| Handlers — 실패 응답과 GET 두 개                | pp.181-182 | 14-server             |
| POST — 모델 바인딩과 201 Created               | p.182      | 14-server             |
| PUT — 교체와 검증                             | p.182      | 14-server             |
| DELETE — 204 No Content                  | p.183      | 14-server             |
| Routes — 메서드별로 묶은 엔드포인트 목록               | pp.179-180 | 14-server             |
| 13챕터의 나머지 조각과 콘텐츠 협상                     | p.184 확장   | 14-server             |
| subRoute 를 한 층 더 얹는다                     | p.180      | 14-server             |
| 설정 — 저장소를 싱글톤으로 등록한다                     | p.179      | 14-server             |
| Using the API — 실제 요청과 응답                | pp.183-184 | 14-server             |
| Summary — 원서의 챕터 요약                      | p.184      | —                     |

## 15 - Giraffe 로 웹 페이지 만들기 (원서 pp.185-191)

13챕터가 첫 화면 하나를 띄우고 14챕터가 API 를 채웠다. 이 챕터에서는 그 API 가 다루는 데이터를 사람이 볼 화면으로 내보내며 앱을 마무리한다. 새 문법은 거의 없다. 13챕터에서 훑어본 View Engine 의 DSL 을 실제 화면 두 개에 쓰고, 여러 화면이 공유할 공용 레이아웃을 함수 하나로 뽑고, 목록을 리스트 컴프리헨션으로 만들고, CSS·JavaScript 를 내보내려고 `configureApp` 에 미들웨어 한 줄을 더한다. 5챕터의 컬렉션, 9챕터의 단일 케이스 판별 유니온, 4챕터의 컴파일 순서가 화면 코드에서 다시 만난다.

14챕터에서 넘어오는 것은 이렇다. 프로젝트 `HiveLog` 는 파일이 넷이고 컴파일 순서가 `Domain.fs` → `Store.fs` → `Inspections.fs` → `Program.fs` 다. 도메인은 옥상 양봉장의 벌통 점검 기록이고 `HiveCode · Hive · hives · findHive · InspectionId · Inspection` 이 있다. 점검 기록 저장소 `InspectionStore` 는 의존성 주입 컨테이너에 싱글턴으로 등록되어 있어 핸들러가 `ctx.GetService<InspectionStore>()` 로 꺼내 쓴다. 라우팅은 세 층이다 — 바깥 `endpoints`, 그 안의 `subRoute "/api"` `apiEndpoints`, 다시 그 안의 `subRoute "/inspections"` `Inspections.inspectionEndpoints`. 뷰는 `indexView` 하나뿐이고 `wwwroot/` 는 아직 없다.

### Getting Started — 이 챕터가 더하는 것 (원서 p.185)

- 원서는 앞 챕터의 프로젝트를 계속 고쳐 나가고, HTML·CSS 를 직접 짜는 대신 w3schools 의 할 일 목록 예제를 가져와 View Engine 형식으로 옮긴다. 데이터는 서버가 만든 가짜 목록이다.
- 이 노트는 도메인을 13·14챕터에서 이어받으므로 화면도 점검 기록이다. 만드는 것은 셋이다. 여러 화면이 공유할 공용 레이아웃, 점검 기록 한 건을 목록의 한 줄로 바꾸는 부분 뷰, 그리고 저장소의 기록을 그 부분 뷰로 넣어놓는 화면이다.
- 경로는 둘로 갈린다. `/api/inspections` 는 14챕터가 만든 JSON 응답이고, 이 챕터가 더하는 `/inspections` 는 같은 데이터의 HTML 화면이다. 저장소 하나를 두 표현이 나눠 쓴다.

|                      |                                                   |
|----------------------|---------------------------------------------------|
| GET /                | 첫 화면 (13챕터의 <code>indexView</code> 를 레이아웃 위에 올린다) |
| GET /inspections     | 점검 기록 화면 (이 챕터가 더한다)                              |
| GET /css/hive.css    | 정적 파일 (이 챕터가 더한다)                                 |
| GET /js/hive.js      | 정적 파일 (이 챕터가 더한다)                                 |
| GET /api/inspections | 14챕터의 JSON 응답 (그대로 둔다)                            |

- 뷰를 새 파일로 빼면 `Views.fs` 가 `Program.fs` 앞에 와야 한다. 뷰가 참조하는 것은 `Domain.fs` 의 타입 뿐이고, 뷰를 참조하는 것은 `Program.fs` 의 핸들러다. 그래서 자리는 하나로 정해진다.

```
<ItemGroup>
  <Compile Include="Domain.fs" />
  <Compile Include="Store.fs" />
  <Compile Include="Inspections.fs" />
  <Compile Include="Views.fs" />
  <Compile Include="Program.fs" />
</ItemGroup>
```

- 원서는 공용 레이아웃을 `Shared.fs` 에 두고 화면 뷰는 `Todos.fs` 안의 `Views` 모듈에 둔다. 파일을 둘로 갈라야 할 이유가 이 규모에서는 없으므로 이 노트는 `Views.fs` 하나에 레이아웃과 화면을 함께 담는다.
- 14챕터가 응답에서 만난 제약은 화면에는 없다. `System.Text.Json` 이 `option` 이 아닌 판별 유니온을 다루지 못해 API 는 평평한 DTO 를 따로 두어야 했지만, View Engine 은 F# 값을 직접 읽어 HTML 을 만들므로 `Inspection` 을 그대로 뷰 함수에 넘겨도 된다. 껍데기를 벗기는 일은 뷰 안에서 패턴으로 처리한다.

## Configuration — wwwroot 와 부록 2 (원서 p.185)

- 정적 파일(static files)은 서버가 내용을 손대지 않고 그대로 내보내는 파일이다. CSS·JavaScript·이미지·글꼴이 여기 든다. ASP.NET Core 의 기본 규칙은 콘텐츠 루트(content root) 아래 `wwwroot/` 를 웹 루트(web root)로 삼고 그 안의 파일만 내보내는 것이다. 프로젝트 폴더에 `wwwroot/` 를 만들면 따로 설정할 것이 없다.
- 원서는 `wwwroot/css/main.css` 와 `wwwroot/js/main.js` 를 만들고 그 내용을 부록 2 에서 복사해 오라고 안내한다. 부록 2 는 원서 pp.197-201 이고, w3schools 할 일 목록 예제의 CSS 와 JavaScript 가 그대로 실려 있다. CSS 쪽은 목록 항목 줄무늬·완료 표시·삭제 버튼·머리글 색과 입력란 배치를 정하고, JavaScript 쪽은 항목마다 닫기 버튼을 붙이고(누르면 그 항목이 숨는다) 클릭으로 완료 표시를 켜고 끄며 입력란의 값으로 새 항목을 만든다.
- 부록 2 의 코드는 원서 뷰의 `id` 와 클래스 이름( `myUL` · `myInput` · `checked` · `close` · `addBtn` )을 그대로 찾으므로, 뷰에서 그 이름을 바꾸면 CSS 와 JavaScript 도 같이 바뀌어야 한다.
- 이 노트는 그 두 파일을 옮기지 않는다. 대신 벌통 점검 기록 화면에 맞는 최소한의 CSS 와 JavaScript 를 새로 짜서 `wwwroot/css/hive.css` 와 `wwwroot/js/hive.js` 에 둔다. 원서 부록의 내용과는 관계가 없고, 정적 파일이 실제로 내보내지는지 확인할 만큼만 짧다. 원서를 따라 실습한다면 부록 2 의 코드를 원서에서 직접 가져다 쓰면 된다.
- 파일을 두었다고 저절로 나가지는 않는다. `configureApp` 에 정적 파일 미들웨어를 한 줄 더해야 한다. 그 코드는 뒤의 설정 절에 있다.

```
// 이 챕터가 configureApp 에 더하는 한 줄
.UseStaticFiles()
```

- 웹 루트를 `wwwroot` 가 아닌 곳으로 바꾸려면 호스트를 만들 때 `WebApplicationOptions(WebRootPath = "public")` 처럼 옵션으로 넘긴다. 빌더를 만든 뒤에 `builder.WebHost.UseWebRoot` 로 바꾸려 하면 `System.NotSupportedException` 이 난다. 뒤의 검증 코드가 옵션으로 넘기는 쪽을 쓰는데, 스크립트로 실행할 때는 콘텐츠 루트가 프로젝트 폴더가 아니어서 경로를 못박아야 하기 때문이다.
- 이 노트가 확인한 SDK 10.0.111 에는 `MapStaticAssets` 라는 다른 방법도 들어 있다. 빌드 때 만들어지는 자산 목록에 기대어 압축본과 캐시 헤더까지 함께 처리하는 쪽이다. 빌드 산출물이 필요하므로 `dotnet fsi` 스크립트에서는 쓸 수 없고, 이 노트는 원서와 같이 `UseStaticFiles` 를 쓴다.
- 웹 프로젝트 템플릿( `dotnet new web` )으로 만든 프로젝트라면 `UseStaticFiles` 를 쓰는 데 패키지를 더 받을 필요가 없다. 프레임워크 참조에 이미 들어 있다.



## Adding a Master Page — 공용 레이아웃 (원서 pp.186-187)

- 화면이 둘 이상 생기면 `<html>` · `<head>` · 스타일시트 링크가 화면마다 반복된다. 원서는 그 껍데기를 함수 하나로 뽑고 제목과 본문을 넘겨받게 만든다. 다른 뷰 엔진에서 마스터 페이지나 레이아웃이라 부르는 것이고, View Engine에서는 그저 함수다.
- 그래서 상속이나 특별한 규칙이 필요하지 않다. 레이아웃 함수는 `XmlNode list` 를 받아 `XmlNode` 를 돌려 주고, 화면 뷰는 자기 본문 목록을 만들어 그 함수에 넘긴다.

아래 실행 단위는 `Views.fs` 에 들어갈 코드를 서버 없이 문자열로 렌더링해 확인한다. 선행 요구사항은 처음 한번 `Giraffe.ViewEngine 1.4.0` 을 받아 올 네트워크다. 받아 오지 못하면 `error FS0999` 로 실패한다.

```
// 이 단위가 보여주는 것: 공용 레이아웃 함수와 그 위에 올리는 화면 뷰
#r "nuget: Giraffe.ViewEngine, 1.4.0"

open System
open System.Globalization
open Giraffe.ViewEngine

// FSI 실측: heading: string -> content: XmlNode list -> XmlNode
let pageLayout (heading: string) (content: XmlNode list) =
    html [ _lang "ko" ] [
        head [ ] [
            meta [ _charset "utf-8" ]
            title [ ] [ str heading ]
            link [ _rel "stylesheet"; _href "/css/hive.css" ]
        ]
        body [ ] [
            header [ _class "bar" ] [ h1 [ ] [ str heading ] ]
            main [ ] content
            script [ _src "/js/hive.js" ] [ ]
        ]
    ]
```

- 요소 함수 대부분이 리스트 둘을 받는다. 앞은 특성 목록, 뒤는 자식 목록이다. `meta` · `link` 처럼 자식을 담을 수 없는 요소는 특성 목록만 받는다. 자식 목록을 붙이려 하면 컴파일되지 않으므로, 자식을 담을 수 없는 요소에 내용을 넣는 실수가 애초에 생기지 않는다.
- 스타일시트 경로에 앞 슬래시를 붙인 것에 뜻이 있다. 원서는 `_href "css/main.css"` 로 적는데, 앞 슬래시가 없는 경로는 브라우저가 현재 주소를 기준으로 풀기 때문에 `/inspections` 화면에서는 `/inspections/css/main.css` 를 찾는다. 원서 예제는 화면이 `/` 하나뿐이라 드러나지 않는 문제다. 뒤의 확인 절에서 그 경로가 404 인 것을 본다.
- `script` 에는 `_type "text/javascript"` 를 적지 않았다. 원서는 적어 두지만 HTML5 에서 그 값이 기본이라 없어도 같다.

레이아웃을 쓰는 쪽은 본문 목록을 만들어 파이프를 넘긴다. 13챕터의 첫 화면 뷰를 그 형태로 고친다.

```
// FSI 실측: indexView: XmlNode
let indexView =
    [
        p [ _class "lede" ] [ str "육상 양봉장 점검 기록" ]
        nav [ ] [ a [ _href "/inspections" ] [ str "점검 기록 보기" ] ]
    ]
    |> pageLayout "HiveLog"

printfn "%s" (RenderView.AsString.htmlDocument indexView)
// <!DOCTYPE html>
// <html lang="ko"><head><meta charset="utf-8"><title>HiveLog</title><link rel="stylesheet" href="/css/hive.css"></head><bod
```

- 본문 목록을 먼저 적고 `|> pageLayout "HiveLog"` 로 끝내면 읽는 순서가 화면의 내용에서 꺾데기로 흐른다. 원서도 같은 형태를 쓴다.
- 13챕터의 `indexView` 는 `p [ _class "lede" ]` 를 쓰면서 정작 그 클래스를 정의한 CSS 가 없었다. 이 챕터가 `wwwroot/css/hive.css` 를 만들어 그 자리를 채운다. 13챕터의 표 예제가 링크한 파일도 같은 `/css/hive.css` 다.
- 렌더링 함수는 13챕터에서 본 그대로다. 문서 전체는 `RenderView.AsString.htmlDocument` (앞에 `<!DOCTYPE html>` 이 붙는다), 조각은 `RenderView.AsString.htmlNode` 이며 둘 다 `XmlNode -> string` 이다.

## str 이 문자를 이스케이프한다 (노트 보충)

- 원서 p.187 은 View Engine 방식의 이점을 타입 안전성으로 설명한다. 문자열을 찾아 바꾸는 다른 뷰 엔진과 달리 F# 코드로 구조를 만들기 때문이다. 실제로는 이점이 하나 더 있는데, 데이터가 HTML 문법으로 새어 나가지 못한다는 것이다.
- 사람이 적어 넣은 값을 화면에 뿌리는 자리에서 이 성질이 중요하다. 아래 결과가 곧 화면에 나가는 HTML 이다.

```
// 이 단위가 보여주는 것: str 은 이스케이프하고 rawText 는 하지 않는다
#r "nuget: Giraffe.ViewEngine, 1.4.0"

open Giraffe.ViewEngine

// 점검자가 적어 넣은 메모라고 하자. HTML 문법에 쓰이는 문자가 섞여 있다
let memo = "\"벌통 \"H-07\" & <급이기> 교체\""

printfn "%s" (RenderView.AsString.htmlNode (p [] [ str memo ]))
// <p>벌통 &quot;H-07&quot; &amp; &lt;급이기&gt; 교체</p>

printfn "%s" (RenderView.AsString.htmlNode (p [ _title memo ] [ str "메모" ]))
// <p title="벌통 &quot;H-07&quot; &amp; &lt;급이기&gt; 교체">메모</p>
```

- 자식 자리든 특성 값이든 `str` 과 특성 함수를 지나면 `<·>·&·"·'·'` 다섯 문자가 문자 참조(character reference)로 바뀐다. 이렇게 바꾸는 일을 이스케이프(escape)라 한다. 그래서 특성 값에 따옴표가 들어와도 특성이 조기에 닫히지 않고, 바로 아래 블록의 `&#39;` 가 홑따옴표가 바뀐 자리다.

```
printfn "%s" (RenderView.AsString.htmlNode (p [] [ rawText memo ]))
// <p>벌통 "H-07" & <급이기> 교체</p>

// 스크립트를 심으려는 입력도 str 을 지나면 글자가 된다
let attack = "<script>alert('!')</script>"
printfn "%s" (RenderView.AsString.htmlNode (li [] [ str attack ]))
// <li>&lt;script&gt;alert(&#39;!&#39;)&lt;/script&gt;</li>
printfn "%s" (RenderView.AsString.htmlNode (li [] [ rawText attack ]))
// <li><script>alert('!')</script></li>
```

- `rawText` 는 문자열을 그대로 끼워 넣는다. 이미 HTML 인 조각을 넣을 때 쓰는 함수이고, 바깥에서 들어온 값에는 쓰면 안 된다. 마지막 줄이 그 이유다.
- 이 노트의 부분 뷰가 벌통 코드와 소비 개수를 `str` 로 넣는 것은 이 성질에 기댄 것이다. 값이 어디서 왔는지 따지지 않아도 구조가 깨지지 않는다.

## 스크립트에서 앱을 세우는 준비 (노트 보충)

- 검증 방식은 13·14챕터와 같다. `dotnet fsi` 스크립트 안에서 Giraffe 앱을 띄우고, 같은 스크립트의 `HttpClient` 로 요청해 응답을 확인하고, 마지막에 서버를 내린다. 준비 코드도 14챕터의 것을 쓰되 정적 파일 미들웨어가 쓰는 어셈블리 하나를 참조 목록에 더한다.
- 선행 요구사항은 두 가지다. [ASP.NET Core](#) 공유 프레임워크가 포함된 .NET SDK(이 노트는 10.0.111 로 확인했다), 그리고 처음 한 번 `Giraffe 8.3.0` 과 `Giraffe.ViewEngine 1.4.0` 을 받아 올 네트워크다.
- 실제 개발은 프로젝트를 만들어 `dotnet run` 으로 확인하는 것이다. 아래 준비 코드는 노트의 예제를 자동으로 검증하기 위한 장치이고 `Program.fs` 에는 들어가지 않는다.

```
// 이 단위가 보여주는 것: 레이아웃 · 부분 뷰 · 정적 파일을 갖춘 앱을 띄워 HTML 까지 확인하기
// 준비 코드 - 프로젝트 코드에는 들어가지 않는다(14챕터와 같다)
open System
open System.IO
open System.Runtime.Loader

let sharedFramework =
    let core =
        Runtime.InteropServices.RuntimeEnvironment.GetRuntimeDirectory().TrimEnd(Path.DirectorySeparatorChar)
        |> DirectoryInfo
    // 폴더 이름을 문자열로 비교하면 10.0.9 가 10.0.11 보다 크므로 Version 으로 비교한다
    let version (path: string) =
        match Version.TryParse((Path.GetFileName path).Split('-')[0]) with
        | true, v -> v
        | _ -> Version(0, 0)
    let root = Path.Combine(core.Parent.Parent.FullName, "Microsoft.AspNetCore.App")
    if not (Directory.Exists root) then
        failwithf "ASP.NET Core 공유 프레임워크 폴더가 없다: %s. 이 폴더가 포함된 .NET SDK 가 필요하다." root
    let candidates = Directory.GetDirectories root |> Array.sortBy version
    match candidates |> Array.filter (fun d -> (version d).Major = (version core.FullName).Major) with
    | [||] -> Array.last candidates
    | sameLine -> Array.last sameLine

AssemblyLoadContext.Default.add_Resolving(
    Func<AssemblyLoadContext, Reflection.AssemblyName, Reflection.Assembly>(fun context name ->
        let dll = Path.Combine(sharedFramework, name.Name + ".dll")
        if File.Exists dll then context.LoadFromAssemblyPath dll else null))

let referenceDir = Path.Combine(__SOURCE_DIRECTORY__, "aspnetcore")
Directory.CreateDirectory referenceDir |> ignore
for name in [ "Microsoft.AspNetCore"
              "Microsoft.AspNetCore.Hosting"
              "Microsoft.AspNetCore.Hosting.Abstractions"
              "Microsoft.AspNetCore.Http"
              "Microsoft.AspNetCore.Http.Abstractions"
              "Microsoft.AspNetCore.Routing"
              "Microsoft.AspNetCore.StaticFiles"
              "Microsoft.Extensions.DependencyInjection.Abstractions"
              "Microsoft.Extensions.Hosting.Abstractions"
              "Microsoft.Extensions.Logging"
              "Microsoft.Extensions.Logging.Abstractions" ] do
    let source = Path.Combine(sharedFramework, name + ".dll")
    if not (File.Exists source) then
        failwithf "%s.dll 을 %s 에서 찾지 못했다." name sharedFramework
    let target = Path.Combine(referenceDir, name + ".dll")
    if not (File.Exists target) then
        try File.CreateSymbolicLink(target, source) |> ignore
        with _ -> File.Copy(source, target, true)

printfn "ASP.NET Core %s" (Path.GetFileName sharedFramework) // ASP.NET Core 10.0.11
```

- 새로 든 것은 `Microsoft.AspNetCore.StaticFiles` 하나다. `UseStaticFiles` 확장 메서드가 여기 들어 있고, 이 줄을 빼면 `error FS0039` 가 난다. 프로젝트에서는 프레임워크 참조에 들어 있어 적을 일이 없다.

```
#r "aspnetcore/Microsoft.AspNetCore.dll"
#r "aspnetcore/Microsoft.AspNetCore.Hosting.dll"
#r "aspnetcore/Microsoft.AspNetCore.Hosting.Abstractions.dll"
#r "aspnetcore/Microsoft.AspNetCore.Http.dll"
#r "aspnetcore/Microsoft.AspNetCore.Http.Abstractions.dll"
#r "aspnetcore/Microsoft.AspNetCore.Routing.dll"
#r "aspnetcore/Microsoft.AspNetCore.StaticFiles.dll"
#r "aspnetcore/Microsoft.Extensions.DependencyInjection.Abstractions.dll"
#r "aspnetcore/Microsoft.Extensions.Hosting.Abstractions.dll"
#r "aspnetcore/Microsoft.Extensions.Logging.dll"
#r "aspnetcore/Microsoft.Extensions.Logging.Abstractions.dll"
#r "nuget: Giraffe, 8.3.0"
#r "nuget: Giraffe.ViewEngine, 1.4.0"

open System.Collections.Concurrent
open System.Globalization
open System.Text.Encodings.Web
open System.Text.Json
open Microsoft.AspNetCore.Builder
open Microsoft.AspNetCore.Http
open Microsoft.Extensions.DependencyInjection
open Microsoft.Extensions.Logging
open Giraffe
open Giraffe.EndpointRouting
open Giraffe.ViewEngine
```

- 이 프로젝트는 파일마다 최상위 모듈 하나를 선언한다. 스크립트에서는 최상위 `module X.Y` 를 쓸 수 없으므로 그 구조를 중첩 모듈로 본뜨고 선언 순서를 `.fsproj` 의 컴파일 순서에 맞춘다. 도메인과 저장소는 14챕터의 것이므로 이 챕터가 쓰는 부분만 남겨 옮긴다.

```
// Domain.fs · Store.fs - 14챕터에서 이어받는다(이 챕터가 쓰는 부분만 남겼다)
module Domain =
    type HiveCode = HiveCode of string
    type InspectionId = InspectionId of Guid

    type Inspection = {
        Id: InspectionId
        Hive: HiveCode
        Frames: int
        QueenSeen: bool
        RecordedAt: DateTime
    }

module Store =
    open Domain

    type InspectionStore() =
        let data = ConcurrentDictionary<Guid, Inspection>()

        member _.Save(inspection: Inspection) =
            let (InspectionId key) = inspection.Id
            data[key] <- inspection

        member _.All() =
            data.Values |> Seq.sortBy (fun item -> item.RecordedAt) |> Seq.toList
```

## Creating the View — 부분 뷰와 리스트 컴프리헨션 (원서 pp.187-191)

- 원서는 목록 항목 여섯 개를 뷰에 그대로 적었다가, 완료 여부에 따라 클래스가 달라지는 부분을 함수로 뽑는다. 원서가 부분 뷰(partial view)라 부르는 것이고 실체는 `Inspection -> XmlNode` 함수다. 13챕터에서 본 대로 모든 요소가 `XmlNode` 이기 때문에 조각을 함수로 뺄 수 있다.
- 원서는 같은 뷰를 두 번 실행하는데 제목과 완료 표시가 서로 다르다(My To Do List / My ToDo List, `Read a book` 의 `checked` 여부, 여섯째 항목 이름). 화면이 책의 그림과 달라 보이는 것은 독자의 실수가 아니다.
- 이 노트의 부분 뷰는 여왕벌을 봤는지에 따라 특성 목록을 갈아 끼운다. 특성 목록도 그냥 리스트라서 `if` 로 고르면 된다.

```
// Views.fs - 이 챕터가 더하는 파일. module HiveLog.Views 로 시작한다
module Views =
    open Domain

    let pageLayout (heading: string) (content: XmlNode list) =
        html [ _lang "ko" ] [
            head [] [
                meta [ _charset "utf-8" ]
                title [] [ str heading ]
                link [ _rel "stylesheet"; _href "/css/hive.css" ]
            ]
            body [] [
                header [ _class "bar" ] [ h1 [] [ str heading ] ]
                main [] content
                script [ _src "/js/hive.js" ] []
            ]
        ]

    // 부분 뷰 - 점검 기록 한 건이 목록의 한 줄이 된다
    // FSI 실측: inspection: Inspection -> XmlNode
    let private inspectionItemView (inspection: Inspection) =
        let (HiveCode code) = inspection.Hive
        let marks = if inspection.QueenSeen then [ _class "queen-seen" ] else []
        li marks [
            span [ _class "code" ] [ str code ]
            span [ _class "frames" ] [ str $"소비 {inspection.Frames}개" ]
            time [ _datetime (inspection.RecordedAt.ToString("yyyy-MM-ddTHH:mm:ssZ", CultureInfo.InvariantCulture)) ] [
                str (inspection.RecordedAt.ToString("MM-dd HH:mm", CultureInfo.InvariantCulture))
            ]
        ]
    ]
```

- 꺾대기를 벗기는 자리가 `let (HiveCode code) = inspection.Hive` 한 줄이다. 도메인 타입을 그대로 받아 뷰 안에서 벗기므로 API 처럼 DTO 를 따로 둘 필요가 없다.
- `marks` 가 빈 리스트면 `li marks [ ... ]` 는 `li [] [ ... ]` 와 같다. 원서도 같은 방식으로 완료 표시를 켜고 끈다. 클래스 이름을 문자열로 이어 붙이는 것보다 리스트를 고르는 편이 실수하기 어렵다.
- `time` 요소는 사람이 읽을 표기와 기계가 읽을 표기를 따로 담는다. 특성 쪽에 `yyyy-MM-ddTHH:mm:ssZ` 로 정렬도 되고 시간대까지 드러나는 표기를 넣고, 자식 쪽에 짧은 표기를 넣었다. 14챕터의 `toPayload` 가 쓴 형식과 같으므로 같은 기록이 두 챕터에서 같은 문자열로 나간다. 두 표기 모두 불변 문화권으로 찍는다 — 사용자 지정 형식의 `:` 는 문화권의 시간 구분 기호라서 그러지 않으면 `fi-FI` 로컬에서 `09.00` 이 나온다(실측).
- `private` 을 붙였으므로 이 함수는 `Views` 모듈 밖에서 보이지 않는다. 화면 조각을 쓰는 것은 같은 모듈의 화면 뷰뿐이므로 밖으로 내보낼 이유가 없다.

목록을 만드는 부분이 원서가 말하는 리스트 컴프리헨션이다. 5챕터의 그 문법이 자식 목록 자리에 그대로 들어간다.

```
// FSI 실측: indexView: XmlNode
let indexView =
[
  p [ _class "lede" ] [ str "육상 양봉장 점검 기록" ]
  nav [ [ a [ _href "/inspections" ] [ str "점검 기록 보기" ] ] ]
]
|> pageLayout "HiveLog"

// FSI 실측: inspections: Inspection list -> XmlNode
let inspectionBoardView (inspections: Inspection list) =
[
  p [ _class "count" ] [ str $"기록 {List.length inspections}건" ]
  ul [ _id "inspection-list" ] [
    for inspection in inspections do
      inspectionItemView inspection
  ]
]
|> pageLayout "점검 기록"
```

- `for inspection in inspections do` 아래에 값을 적기만 하면 그것이 목록의 원소가 된다. `yield` 를 적지 않아도 되는 암시적 `yield` 다. `inspections |> List.map inspectionItemView` 로 써도 결과는 같고, 13챕터의 표 뷰는 그렇게 적었다. 조건에 따라 어떤 항목을 건너뛰거나 항목 사이에 다른 요소를 끼워 넣어야 할 때는 컴프리헨션 쪽이 편하다.
- 이 함수의 타입 주석은 없어도 시그니처가 같다. 본문의 `List.length` 가 매개변수를 리스트로 못박기 때문에 주석을 지워도 FSI 가 `inspections: Inspection list -> XmlNode` 로 잡는다. 반대로 건수를 세는 줄을 빼거나 `Seq.length` 로 바꾸면 `for ... in` 이 요구하는 것은 열거 가능성뿐이라 `inspections: Inspection seq -> XmlNode` 가 된다. 제약은 컴프리헨션이 아니라 함께 쓴 함수가 정한다.
- 이 화면의 렌더링 결과는 아래 `15-view` 단위에서 문자열로 확인한다.

같은 코드를 서버 없이 확인해 둔다. 앞의 `15-view` 단위에 부분 뷰와 화면 뷰를 이어 붙인 것이다.

```
// Domain.fs 의 타입 둘을 이 단위에서만 다시 적는다.
// Inspection 은 뷰가 쓰는 필드만 남겼다
type HiveCode = HiveCode of string

type Inspection = {
  Hive: HiveCode
  Frames: int
  QueenSeen: bool
  RecordedAt: DateTime
}

// FSI 실측: inspection: Inspection -> XmlNode
let inspectionItemView (inspection: Inspection) =
  let (HiveCode code) = inspection.Hive
  let marks = if inspection.QueenSeen then [ _class "queen-seen" ] else []
  li marks [
    span [ _class "code" ] [ str code ]
    span [ _class "frames" ] [ str $"소비 {inspection.Frames}개" ]
    time [ _datetime (inspection.RecordedAt.ToString("yyyy-MM-ddTHH:mm:ssZ", CultureInfo.InvariantCulture)) ] [
      str (inspection.RecordedAt.ToString("MM-dd HH:mm", CultureInfo.InvariantCulture))
    ]
  ]
]

let samples =
[ { Hive = HiveCode "H-07"; Frames = 8; QueenSeen = true; RecordedAt = DateTime(2026, 5, 3, 9, 0, 0, DateTimeKind.Utc) }
  { Hive = HiveCode "H-11"; Frames = 10; QueenSeen = false; RecordedAt = DateTime(2026, 5, 3, 11, 0, 0, DateTimeKind.Utc) }
  { Hive = HiveCode "H-07"; Frames = 9; QueenSeen = true; RecordedAt = DateTime(2026, 5, 17, 9, 0, 0, DateTimeKind.Utc) } ]

printfn "%s" (RenderView.AsString.htmlNode (inspectionItemView samples[0]))
// <li class="queen-seen"><span class="code">H-07</span><span class="frames">소비 8개</span><time datetime="2026-05-03T09:00:00Z">2026-05-03 09:00:00</time></li>
printfn "%s" (RenderView.AsString.htmlNode (inspectionItemView samples[1]))
// <li><span class="code">H-11</span><span class="frames">소비 10개</span><time datetime="2026-05-03T11:00:00Z">2026-05-03 11:00:00</time></li>
```

- 첫 줄과 둘째 줄의 차이가 `marks` 하나다. 여왕벌을 본 기록에만 클래스가 붙었고, 못 본 기록의 `<li>`에는 특성이 아예 없다.

```
// FSI 실측: inspections: Inspection list -> XmlNode
let inspectionBoardView (inspections: Inspection list) =
    [
        p [ _class "count" ] [ str $"기록 {List.length inspections}건" ]
        ul [ _id "inspection-list" ] [
            for inspection in inspections do
                inspectionItemView inspection
        ]
    ]
    |> pageLayout "점검 기록"

let board = RenderView.AsString.htmlNode (inspectionBoardView samples)
// 읽기 좋게 항목마다 줄을 나눠 찍는다
printfn "%s" (board.Replace("</li>", "</li>\n"))
// <html lang="ko"><head><meta charset="utf-8"><title>점검 기록</title><link rel="stylesheet" href="/css/hive.css"></head><body>
// <li><span class="code">H-11</span><span class="frames">소비 10개</span><time datetime="2026-05-03T11:00:00Z">05-03 11:00</time></li>
// <li class="queen-seen"><span class="code">H-07</span><span class="frames">소비 9개</span><time datetime="2026-05-17T09:00:00Z">05-17 09:00</time></li>
// </ul></main><script src="/js/hive.js"></script></body></html>
```

- 이 블록이 앞 절의 `pageLayout` 을 그대로 쓸 수 있는 것은 같은 `id` 를 쓰는 블록들이 문서 순서대로 이어 붙어 하나의 스크립트가 되기 때문이다. `views.fs` 안에서 레이아웃과 화면 뷰가 이웃해 있는 것과 같은 모양이다.
- `RenderView.AsString.htmlNode` 로 찍었으므로 `<!DOCTYPE html>` 이 없다. 문서로 내보낼 때는 `htmlDocument` 쪽을 쓰거나, 서버에서는 `htmlView` 가 알아서 붙인다.

## 뷰 핸들러와 화면용 경로 (원서 p.189)

- 원서는 목록을 하드코딩한 뷰를 먼저 만들고 나중에 데이터를 넘기도록 고친다. 이 노트는 처음부터 저장소를 읽는다. 14챕터가 `InspectionStore` 를 싱글턴으로 등록해 두었으므로 핸들러가 `ctx.GetService` 로 꺼내면 된다.
- 뷰를 응답으로 내보내는 핸들러는 `htmlView` 다. 데이터를 받지 않는 화면은 `htmlView Views.indexView` 를 경로에 그대로 붙이고, 데이터를 읽어야 하는 화면은 핸들러를 하나 만들어 그 안에서 `htmlView` 를 부른다.
- 프로젝트에서는 `Program.fs` 에 `open HiveLog` 를 적어 `Views.` 접두를 남긴다. 14챕터가 엔드포인트 목록에 쓴 방식과 같다.

```
// Program.fs
open Domain
open Store

let notFoundHandler : HttpHandler =
    "요청한 경로가 없다" |> text |> RequestErrors.notFound

// FSI 실측: next: HttpFunc -> ctx: HttpContext -> HttpFuncResult
let inspectionBoardHandler : HttpHandler =
    fun next ctx ->
        let store = ctx.GetService<InspectionStore>()
        htmlView (Views.inspectionBoardView (store.All())) next ctx
```

- `htmlView` 는 `XmlNode -> HttpHandler` 다. 화면 뷰가 만든 값을 넘겨 핸들러를 얻고, 그 핸들러에 `next` 와 `ctx` 를 넘겨 부른다. 14챕터의 핸들러들이 `match` 로 핸들러를 고른 뒤 마지막에 부른 것과 같은 형태이고, 여기는 고를 것이 없어 한 줄이다.
- 이 핸들러에는 `task { ... }` 가 없다. `htmlView` 가 돌려주는 핸들러를 부른 결과가 이미 `HttpFuncResult` 이므로 감쌀 것이 없다.
- 저장소가 비어 있어도 이 핸들러는 실패하지 않는다. `store.All()` 이 빈 리스트를 돌려주고 컴프리헨션이 빈 `<ul>` 을 만든다. API 의 404 판단과 달리 목록 화면에는 없음을 알릴 상태 코드가 필요하지 않다.

경로를 더할 자리는 바깥 `endpoints` 다. 13챕터가 HTTP 메서드 묶음을 안쪽 목록에 몰아 둔 덕에 `/api` 아래는 손대지 않는다.

```
// 14챕터의 점검 기록 핸들러 다섯 개 가운데 목록 하나만 남기고, 요약·별통 핸들러와
// subRoute "/inspections" 한 층도 접었다. 화면 쪽에 집중하기 위한 축약이고 프로젝트에서는
// Inspections.fs 의 다섯 개와 Program.fs 의 요약·별통 핸들러, 세 층 라우팅이 그대로 있다
let inspectionListHandler : HttpHandler =
    fun _ ctx ->
        let store = ctx.GetService<InspectionStore>()
        store.All()
        |> List.map (fun item ->
            let (HiveCode code) = item.Hive
            {| Hive = code; Frames = item.Frames |})
        |> ctx.WriteJsonAsync

let apiEndpoints =
    [
        GET [
            route "/inspections" inspectionListHandler
        ]
    ]

// FSI 실측: endpoints: Endpoint list
let endpoints =
    [
        GET [
            route "/" (htmlView Views.indexView)
            route "/inspections" inspectionBoardHandler
        ]
        subRoute "/api" apiEndpoints
    ]
```

- `/inspections` 와 `/api/inspections` 가 같은 저장소를 읽어 다른 형식으로 답한다. 앞은 사람이 볼 HTML, 뒤는 기계가 읽을 JSON 이다. 14챕터에서 본 콘텐츠 협상으로 한 경로에 둘을 겹칠 수도 있지만, `negotiate` 의 기본 규칙에는 `text/html` 이 없어 HTML 을 골라 줄 방법이 없다. `Accept: text/html` 만 보내면 406 이고, 브라우저가 보내는 긴 `Accept` 헤더는 품질값 0.9 의 `application/xml` 규칙에 걸려 14챕터에서 본 XML 직렬화 실패로 500 이 된다. 화면과 API 를 경로로 갈라 두는 편이 단순하다.
- 정적 파일 경로는 이 목록에 적지 않는다. 라우팅이 아니라 미들웨어가 처리하기 때문이다. `wwwroot/` 아래 파일을 하나 더 두면 경로가 자동으로 늘어난다.
- 원서 p.191 의 마지막 라우팅 코드에는 괄호가 하나 빠져 있다( `route "/" (htmlView (Todos.Views.todoView Todos.Data.todoList)` ). 그대로 옮겨 적으면 컴파일되지 않으니 짝을 맞춰야 한다.



## Loading Data on Startup — 표본 데이터 (원서 pp.189-190)

- 원서는 `Todos.fs` 안에 `Data` 모듈을 만들어 항목 여섯 개를 최상위 리스트로 적는다. 그러고는 레코드 필드가 반복되는 것을 보고 `create` 도우미 함수를 만들어 튜플 목록을 `List.map` 으로 옮기는 형태로 고친다.
- 이 노트는 그 리스트를 저장소에 넣는다. 화면과 API 가 같은 저장소를 읽어야 하고, 저장소는 14챕터에서 이미 등록되어 있으므로 데이터가 들어갈 자리도 거기다. 최상위 리스트를 따로 두면 화면은 그것을 보고 API 는 저장소를 봐서 둘이 어긋난다.

```
// FSI 실행: unit -> InspectionStore
let storeWithSamples () =
    let store = InspectionStore()
    let entry hive frames queenSeen day hour =
        { Id = InspectionId(Guid.NewGuid())
          Hive = HiveCode hive
          Frames = frames
          QueenSeen = queenSeen
          RecordedAt = DateTime(2026, 5, day, hour, 0, 0, DateTimeKind.Utc) }
    [ entry "H-07" 8 true 3 9
      entry "H-11" 10 false 3 11
      entry "H-07" 9 true 17 9 ]
    |> List.iter store.Save
    store
```

- `entry` 가 원서의 `create` 에 해당한다. 반복되는 필드 이름을 한 번만 적고 달라지는 값만 매개변수로 받는다. 원서는 튜플 목록을 `List.map` 으로 옮기지만, 매개변수가 다섯이면 커링된 함수를 그대로 부르는 편이 짧다.
- 기록 시각을 `DateTime.UtcNow` 가 아니라 못박은 값으로 넣었다. 원서는 표본 여섯 건 모두에 `DateTime.UtcNow` 를 쓰는데, 그러면 실행할 때마다 화면이 달라져 예제 출력과 대조할 수 없다. 정렬 순서를 확인하려면 서로 다른 시각이 필요하기도 하다.
- 시그니처가 `unit -> InspectionStore` 인 것에 뜻이 있다. `let storeWithSamples = ...` 로 적어 값으로 두면 스크립트를 읽는 시점에 인스턴스가 하나 만들어진다. 함수로 두면 부르는 쪽이 시점을 정한다.

## Configuration — UseStaticFiles 한 줄 (원서 p.185)

- 서비스 등록은 14챕터와 같다. 달라지는 것은 저장소 인스턴스를 만드는 방법 하나뿐이다.
- 미들웨어 쪽에 `UseStaticFiles()` 가 들어간다. 이것이 이 챕터가 설정에 더하는 전부다.

```
// FSI 실행: services: IServiceCollection -> unit
let configureServices (services: IServiceCollection) =
    let options = JsonSerializerOptions(JsonSerializerDefaults.Web)
    options.Encoder <- JavaScriptEncoder.UnsafeRelaxedJsonEscaping
    services
        .AddRouting()
        .AddGiraffe()
        .AddSingleton<Json.ISerializer>(Json.Serializer options)
        .AddSingleton<InspectionStore>(storeWithSamples ())
    |> ignore

// FSI 실행: appBuilder: IApplicationBuilder -> unit
let configureApp (appBuilder: IApplicationBuilder) =
    appBuilder
        .UseStaticFiles()
        .UseRouting()
        .UseGiraffe(endpoints)
        .UseGiraffe(notFoundHandler)
```

- 원서는 `UseStaticFiles()` 를 `UseRouting()` 뒤에 적는다. 이 노트는 앞에 두었다. 두 순서 모두 이 앱에서는 같은 결과를 내는데, 정적 파일 경로와 겹치는 엔드포인트가 없기 때문이다. 겹치면 결과가 갈린다 — 확인 절 마지막에서 실제로 갈리는 것을 본다.
- 정적 파일 미들웨어를 앞에 두는 것이 관례이고 이유는 두 가지다. 파일 요청이 라우팅 표를 뒤지지 않고 바로 끝나고, 실수로 같은 경로에 엔드포인트를 만들었을 때 파일 쪽이 이긴다.
- `configureApp` 의 순서가 곧 동작이라는 13챕터의 이야기를 이 절이 실제 응답으로 확인한다. `configureServices` 의 등록 순서는 상관없다.

## 정적 파일을 만들고 앱을 띄워 확인한다 (노트 보충)

- 프로젝트에서는 `wwwroot/css/hive.css` 와 `wwwroot/js/hive.js` 를 편집기로 만들면 된다. 스크립트로 검증하려면 그 두 파일을 스크립트가 직접 만들어야 하므로, 아래 코드가 임시 경로 안에 웹 루트를 만들어 쓴다.
- 내용은 이 노트가 새로 짰 것이고 원서 부록 2 와는 관계가 없다. CSS 는 머리글 색과 목록 모양, 여왕벌을 본 기록에 표시를 붙이는 규칙 셋이고, JavaScript 는 항목을 클릭하면 클래스가 켜지고 꺼지게 하는 몇 줄이다.

```
// ---- 여기부터는 검증용 코드다
let webRoot = Path.Combine(__SOURCE_DIRECTORY__, "wwwroot")
Directory.CreateDirectory(Path.Combine(webRoot, "css")) |> ignore
Directory.CreateDirectory(Path.Combine(webRoot, "js")) |> ignore

File.WriteAllText(
    Path.Combine(webRoot, "css", "hive.css"),
    """.bar { background: #f5c451; padding: 12px }
#inspection-list { list-style: none; padding: 0 }
.queen-seen .code::after { content: " (여왕 확인)" }
""")

File.WriteAllText(
    Path.Combine(webRoot, "js", "hive.js"),
    ""document.querySelectorAll("#inspection-list li").forEach(function (item) {
item.addEventListener("click", function () { item.classList.toggle("open"); });
});
""")
```

- CSS 의 `.queen-seen .code::after` 는 부분 뷰가 붙인 클래스를 받는다. 클래스 이름 하나를 F# 코드와 CSS 파일이 나눠 쓰는 자리이고, View Engine 의 타입 안전성이 닿지 않는 경계이기도 하다. 오타가 나면 컴파일은 통과하고 화면만 어긋난다.
- 호스트를 세우는 부분은 13·14챕터와 같고 웹 루트를 못박는 한 줄만 다르다. 스크립트는 콘텐츠 루트가 프로젝트 폴더가 아니어서 기본 규칙으로 `wwwroot/` 를 찾지 못한다.

```

let builder =
    WebApplication.CreateBuilder(
        WebApplicationOptions(ContentRootPath = __SOURCE_DIRECTORY__, WebRootPath = "wwwroot"))
configureServices builder.Services
builder.Logging.ClearProviders() |> ignore

let app = builder.Build()
configureApp app
app.Uris.Add "http://127.0.0.1:0" // 0 = 빈 포트를 골라 달라는 뜻
app.StartAsync() |> Async.AwaitTask |> Async.RunSynchronously

let baseUrl = app.Uris |> Seq.head
let client = new Net.Http.HttpClient()

// 경로 · 상태 코드 · Content-Type 을 찍고 본문을 돌려준다
let get (path: string) =
    let response = client.GetAsync(baseUrl + path) |> Async.AwaitTask |> Async.RunSynchronously
    let body = response.Content.ReadAsStringAsync() |> Async.AwaitTask |> Async.RunSynchronously
    printfn "%-26s %d %s" path (int response.StatusCode) (string response.Content.Headers.ContentType)
    body

let indexBody = get "/"
// / 200 text/html; charset=utf-8
let boardBody = get "/inspections"
// /inspections 200 text/html; charset=utf-8

```

- `htmlview` 가 `Content-Type` 을 `text/html; charset=utf-8` 로 정한다. 14챕터의 JSON 응답과 같리는 지점이고, 브라우저 주소창에 이 경로를 넣으면 화면이 그려진다.

응답 본문이 정말 기대한 HTML 인지 조각을 찾아 확인한다.

```

// FSI 실측: label: string -> fragment: string -> body: string -> unit
let check (label: string) (fragment: string) (body: string) =
    printfn " %-5b %s" (body.Contains fragment) label

check "문서 선언" "<!DOCTYPE html>\n<html lang=\"ko\">" indexBody
check "레이아웃의 stylesheet" "<link rel=\"stylesheet\" href=\"/css/hive.css\">" indexBody
check "레이아웃의 script" "<script src=\"/js/hive.js\"></script>" indexBody
check "목록 링크" "<a href=\"/inspections\">" indexBody
check "건수" "<p class=\"count\">기록 3건</p>" boardBody
check "여왕벌을 본 기록" "<li class=\"queen-seen\">" boardBody
check "여왕벌을 못 본 기록" "<li><span class=\"code\">H-11</span>" boardBody
check "부분 뷰가 낸 시각" "<time datetime=\"2026-05-17T09:00:00Z\">" boardBody
check "항목 순서" "</time></li><li><span class=\"code\">H-11</span>" boardBody
// true 문서 선언
// true 레이아웃의 stylesheet
// true 레이아웃의 script
// true 목록 링크
// true 건수
// true 여왕벌을 본 기록
// true 여왕벌을 못 본 기록
// true 부분 뷰가 낸 시각
// true 항목 순서

```

- 문서 선언 은 `htmlview` 가 응답 앞에 `<!DOCTYPE html>` 을 붙였다는 것을, 이어지는 셋은 공용 레이아웃 이 두 화면에 같은 껍데기를 씌웠다는 것을, 건수 부터 항목 순서 까지 다섯은 부분 뷰와 컴프리헨션이 저장소의 세 건을 순서대로 옮겼다는 것을 말한다. 여왕벌을 본 기록에만 클래스가 붙고 못 본 기록의 `<li>` 에는 특성이 없다는 차이도 확인된다.
- 표본 데이터의 기록 시각을 못박아 둔 덕에 부분 뷰가 낸 시각 검사처럼 시각까지 대조할 수 있다.  
`DateTime.UtcNow` 로 채웠다면 이 검사는 쓸 수 없다.
- `body.Contains` 는 조각이 있지만 본다. 항목 순서와 중첩은 항목 순서 검사처럼 이웃한 두 조각을 이어 붙여야 잡히고, 부분 뷰를 두 번 붙여 항목이 여섯이 되는 중복은 이 검사들이 잡지 못한다. 문서 전체의 모양은 `15-view` 단위가 문자열로 찍어 대조하는 쪽이 맞는다. 두 단위가 역할을 나눠 맡는다.

정적 파일 차례다.

```
printfn "%s" (get "/css/hive.css")
// /css/hive.css 200 text/css
// .bar { background: #f5c451; padding: 12px }
// #inspection-list { list-style: none; padding: 0 }
// .queen-seen .code::after { content: " (여왕 확인)" }
//
get "/js/hive.js" |> ignore
// /js/hive.js 200 text/javascript
get "/css/nope.css" |> ignore
// /css/nope.css 404 text/plain; charset=utf-8
get "/inspections/css/hive.css" |> ignore
// /inspections/css/hive.css 404 text/plain; charset=utf-8
printfn "%s" (get "/api/inspections")
// /api/inspections 200 application/json; charset=utf-8
// [{"frames":8,"hive":"H-07"}, {"frames":10,"hive":"H-11"}, {"frames":9,"hive":"H-07"}]
```

- 확장자를 보고 Content-Type 이 정해진다. `.css` 는 `text/css` , `.js` 는 `text/javascript` 이고 둘 다 문자 집합이 붙지 않는다. 이 대응 표를 들고 있는 것이 정적 파일 미들웨어이며, 표에 없는 확장자는 기본적으로 내보내지 않는다.
- 없는 파일 요청은 미들웨어가 그냥 흘려보내고, 라우팅에도 걸리지 않아 마지막 `notFoundHandler` 로 떨어진다. 그래서 응답이 평문 404 다. 정적 파일 전용 404 화면이 따로 나오지는 않는다.
- 넷째 줄이 앞에서 이야기한 상대 경로 문제다. 원서처럼 `_href "css/hive.css"` 로 적었다면 `/inspections` 화면의 브라우저가 이 경로를 찾고, 보다시피 404 다. 앞 슬래시를 붙이면 어느 화면에서 열어도 같은 파일을 가리킨다.
- 마지막 줄이 14챕터의 목록 경로다. 화면을 더하는 동안 API 는 손대지 않았다. 다만 이 단위는 핸들러를 축약했으므로 필드가 둘뿐이고, 프로젝트에서는 14챕터의 `toPayload` 가 만든 다섯 필드가 그대로 나간다. 같은 저장소의 세 건이 두 형식으로 나갔다.

미들웨어 파이프라인의 순서가 결과를 갈라 놓는 경우를 마지막에 확인한다. 앞의 앱을 내리고, 정적 파일 경로와 같은 엔드포인트를 하나 만들어 순서만 바꿔 두 번 띄운다.

```

app.StopAsync() |> Async.AwaitTask |> Async.RunSynchronously
client.Dispose()

// 정적 파일과 라우팅이 같은 경로를 노릴 때 순서가 결과를 정한다
let raceEndpoints = [ GET [ route "/css/hive.css" (text "라우팅이 응답했다") ] ]

let probeOrder (label: string) (configure: IApplicationBuilder -> unit) =
    let raceBuilder =
        WebApplication.CreateBuilder(
            WebApplicationOptions(ContentRootPath = __SOURCE_DIRECTORY__, WebRootPath = "wwwroot"))
    configureServices raceBuilder.Services
    raceBuilder.Logging.ClearProviders() |> ignore
    let raceApp = raceBuilder.Build()
    configure raceApp
    raceApp.Urls.Add "http://127.0.0.1:0"
    raceApp.StartAsync() |> Async.AwaitTask |> Async.RunSynchronously
    use probe = new Net.Http.HttpClient()
    let body =
        probe.GetStringAsync((raceApp.Urls |> Seq.head) + "/css/hive.css")
    |> Async.AwaitTask
    |> Async.RunSynchronously
    printfn "%s = %s" label (body.Split '\n' |> Array.head)
    raceApp.StopAsync() |> Async.AwaitTask |> Async.RunSynchronously

probeOrder "정적 파일 먼저" (fun appBuilder ->
    appBuilder.UseStaticFiles().UseRouting().UseGiraffe(raceEndpoints).UseGiraffe(notFoundHandler))
// 정적 파일 먼저 = .bar { background: #f5c451; padding: 12px }

probeOrder "라우팅 먼저" (fun appBuilder ->
    appBuilder.UseRouting().UseStaticFiles().UseGiraffe(raceEndpoints).UseGiraffe(notFoundHandler))
// 라우팅 먼저 = 라우팅이 응답했다

printfn "서버를 내렸다" // 서버를 내렸다

```

- 순서가 결과를 갈라 놓는 이유는 `UseRouting` 이 응답을 쓰지 않고 엔드포인트만 골라 둔다는 데 있다. 정적 파일 미들웨어는 이미 골라진 엔드포인트가 있으면 파일을 내보내지 않고 요청을 그냥 넘긴다. 그래서 라우팅을 먼저 켜면 경로가 겹치는 순간 엔드포인트가 이기고, 정적 파일을 먼저 두면 라우팅에 닿기 전에 파일로 응답이 끝난다. 겹치지 않는 경로라면 어느 순서든 파일이 나간다.
- `configureApp` 이 값을 조립하는 코드가 아니라 순서를 적는 코드라는 것이 여기서 드러난다. 라우팅과 달리 우선순위 규칙이 따로 없고 적은 순서가 그대로 규칙이다.
- 두 앱 모두 `StopAsync` 로 내렸고 `HttpClient` 도 해제했다. 남은 프로세스나 포트가 없어야 이 노트를 반복해서 검증할 수 있다.

## Summary — 원서의 챕터 요약 (원서 p.191)

- 원서는 Giraffe 와 View Engine 으로 할 수 있는 일의 절반 훑었다고 적고 View Engine 문서를 읽어 보라고 권한다.
- JavaScript 를 직접 쓰는 것이 싫으면 SAFE Stack 을 보라고도 권한다. 서버와 브라우저 양쪽을 F# 로 쓰는 조합이고 React 기반 단일 페이지 앱에도 쓴다는 소개다.
- 원서 본문은 여기서 끝난다. 다음 챕터는 맺음말과 이어서 읽을 자료 목록이다.

## 정리 — 이 노트의 요약

- 공용 레이아웃은 문법이 아니라 함수다. `heading: string -> content: XmlNode list -> XmlNode` 로 껍데기를 뽑아 두면 화면 뷰가 본문 목록을 만들어 파이프로 넘긴다. 상속이나 특별한 규칙이 필요하지 않다.
- 부분 뷰도 함수다. 모든 요소가 `XmlNode` 라서 `Inspection -> XmlNode` 함수를 자식 목록에 끼워 넣을 수 있다. 특성 목록도 리스트이므로 `if` 로 골라 클래스를 켜고 끈다.
- 목록은 자식 자리의 리스트 컴프리헨션으로 만든다. `for x in items do` 아래 값을 적으면 그대로 원소가 되고, `List.map` 으로 써도 결과는 같다. 타입 주석이 없으면 `for ... in` 이 요구하는 것은 열거 가능성뿐이라 매개변수가 `seq` 로 잡히고, 같은 본문에서 `List.length` 를 부르면 그 호출이 매개변수를 리스트로 못박는다.
- `str` 과 특성 함수는 `<·>·&·"·"·'` 다섯 문자를 문자 참조로 바꾼다. 바깥에서 들어온 값을 그대로 넣어도 HTML 구조가 깨지지 않는다. `rawText` 는 이스케이프하지 않으므로 이미 HTML 인 조각에만 쓴다.
- API 의 직렬화 제약은 화면에 해당하지 않는다. View Engine 은 F# 값을 직접 읽으므로 판별 유니온이 든 도 메인 타입을 뷰 함수에 그대로 넘기고, 껍데기는 뷰 안에서 패턴으로 벗긴다. DTO 를 따로 둘 이유가 없다.
- 데이터를 읽는 화면은 핸들러를 하나 만들어 `ctx.GetService<InspectionStore>()` 로 저장소를 꺼내고 `htmlView` 에 뷰의 결과를 넘긴다. 데이터를 받지 않는 화면은 `htmlView Views.indexView` 를 경로에 바로 붙인다.
- 정적 파일은 콘텐츠 루트 아래 `wwwroot/` 에 두고 `configureApp` 에 `UseStaticFiles()` 를 더하면 나간다. 확장자에 따라 `Content-Type` 이 정해지고( `.css` → `text/css` , `.js` → `text/javascript` ), 없는 파일은 미들웨어를 지나쳐 마지막 핸들러의 404 로 떨어진다.
- 미들웨어 파이프라인의 순서는 취향이 아니다. 정적 파일 경로와 엔드포인트 경로가 겹치면 `configureApp` 에 정적 파일을 먼저 적었을 때만 파일이 나간다. 정적 파일 미들웨어는 이미 골라진 엔드포인트가 있으면 요청을 그냥 넘기기 때문이다. 정적 파일을 앞에 두는 것이 관례다.
- 스타일시트와 스크립트 경로에는 앞 슬래시를 붙인다. 상대 경로는 브라우저가 현재 주소를 기준으로 풀어서 중첩된 경로의 화면에서 404 가 된다.
- 뷰를 새 파일로 빼면 컴파일 순서에서 `Program.fs` 앞에 와야 한다. 뷰는 `Domain.fs` 만 참조하고 핸들러가 뷰를 참조하므로 자리가 하나로 정해진다.

## 세 챕터를 지난 앱의 최종 모습

HiveLog 는 `dotnet new web -lang "F#"` 로 만든 단일 프로젝트이고 파일이 다섯이다. `Domain.fs` 가 `HiveCode` · `Hive` · `hives` · `findHive` · `InspectionId` · `Inspection` 을, `Store.fs` 가 `ConcurrentDictionary` 를 감싼 `InspectionStore` 를 담는다. `Inspections.fs` 에는 요청 타입·검증·API 핸들러 다섯 개와 `inspectionEndpoints` 가, `Views.fs` 에는 `pageLayout` ·부분 뷰 ·`indexView` ·`inspectionBoardView` 가, `Program.fs` 에는 화면 핸들러·요약 핸들러·벌통 핸들러·마지막 핸들러·라우팅·설정·진입점이 있다. 밖으로 드러난 경로는 세 갈래다. 화면 쪽에 `/` 와 `/inspections` , API 쪽에 `/api` 아래의 요약 하나와 벌통 조회 둘, 점검 기록 다섯(GET 두 개, POST, PUT, DELETE), 그리고 라우팅을 거치지 않는 `wwwroot/` 의 정적 파일이다. 미들웨어 파이프라인은 정적 파일 → 라우팅 → Giraffe 엔드포인트 → 어디에도 걸리지 않은 요청을 받는 핸들러 순이고, 서비스 등록은 라우팅·Giraffe·JSON 직렬화기·점검 기록 저장소 넷이다. 저장소 하나를 화면과 API 가 나눠 쓰므로 POST 로 만든 기록이 곧 화면에 나타난다. 데이터는 메모리에만 있어 프로세스가 끝나면 사라지고, 여기서 더 나아가려면 저장소 클래스 뒤에 실제 데이터베이스를 붙이는 것이 다음 걸음이다.

## 원서 대조 표

| 절                                   | 원서 페이지     | 실행 단위               |
|-------------------------------------|------------|---------------------|
| Getting Started — 이 챕터가 더하는 것       | p.185      | —                   |
| Configuration — wwwroot 와 부록 2      | p.185      | —                   |
| Adding a Master Page — 공용 레이아웃      | pp.186-187 | 15-view             |
| str 이 문자를 이스케이프한다                   | (노트 보충)    | 15-escape           |
| 스크립트에서 앱을 세우는 준비                    | (노트 보충)    | 15-server           |
| Creating the View — 부분 뷰와 리스트 컴프리헨션 | pp.187-191 | 15-server , 15-view |
| 뷰 핸들러와 화면용 경로                       | p.189      | 15-server           |
| Loading Data on Startup — 표본 데이터    | pp.189-190 | 15-server           |
| Configuration — UseStaticFiles 한 줄  | p.185      | 15-server           |
| 정적 파일을 만들고 앱을 띄워 확인한다               | (노트 보충)    | 15-server           |
| Summary — 원서의 챕터 요약                 | p.191      | —                   |

## 16 - 맺음말과 다음 걸음 (원서 pp.192-194)

원서 본문은 15챕터에서 끝나고, 이 세 쪽은 저자의 맺음말이다. 다룬 것을 한 번 훑고, F# Software Foundation 가입을 권하고, 이어서 읽을 자료를 묶고, 마지막 한마디를 남긴다. F# 문법은 하나도 나오지 않으므로 이 노트에도 실행 단위가 없다. 대신 이 시점의 독자에게 실제로 쓸모가 있는 두 가지를 챙긴다. 열다섯 챕터에서 손에 남는 것이 무엇인지 되짚는 표, 그리고 여기서 어디로 갈지에 관한 정리다. 원서의 자료 목록은 2023년 1월 기준이라 링크가 지금도 그대로인지 하나씩 확인해 결과를 적었다.

### Summary — 200쪽 가까이 지나간 자리 (원서 pp.192-193)

- 저자가 먼저 하는 이야기는 F# 로 일할 기회가 없더라도 이 책의 생각과 습관이 코드를 쓰는 방식에 남는다는 것이다. 얻는 것이 언어 하나가 아니라는 뜻이다.
- C# 로 일하는 것과 F# 로 일하는 것의 차이도 언어 기능 목록이나 패러다임 이름의 차이만은 아니라고 적는다. 같은 문제에 다른 풀이를 여러 개 시도해 보는 비용이 싸고, 그래서 실제로 그렇게 하게 된다는 쪽이 저자가 말하는 차이다. 5챕터의 “Solving a Problem in Many Ways” 절이 그 감각을 그대로 실습으로 옮겨 놓은 자리다.
- 저자가 자신의 신조로 내놓는 한 줄이 있다. Erlang 을 만든 사람 중 하나인 Joe Armstrong 에게서 처음 들은 것으로 기억한다고 덧붙인다.

먼저 돌아가게, 그다음 보기 좋게, 마지막으로 필요하다면 빠르게. (원서 p.192, 저자의 신조를 옮김)

- 이 순서를 F# 에서 실행하는 도구가 F# Interactive(FSI) 다. 서문 챕터의 F# Interactive 절에 저자가 중단점을 걸어 본 일이 없다는 이야기가 있고(원서 p.6), 열다섯 챕터의 실행 단위 전부가 `dotnet fsi` 로 검증돼 있다. 먼저 돌아가는 것을 FSF 에서 확인하고, 그다음 파이프라인과 타입을 정리하고, 성능은 필요할 때 손대는 순서다.
- 원서는 이어서 다룬 항목을 목차 형태의 목록으로 늘어놓는다. FSF, 대수적 타입 시스템, 패턴 매칭, `let` 바인딩, 불변성, 함수, `unit`, 효과, 계산 식, 컬렉션, 재귀, 객체 프로그래밍, 예외, 단위 테스트, 네임스페이스와 모듈이다. 몇몇 항목에는 하위 항목이 달려 있다 — 함수 아래의 커링·부분 적용, 컬렉션 아래의 `Set` 이 그렇다. 아래 색인은 그 하위 항목까지 함께 짚는다.
- 목록 뒤에는 독자가 Giraffe 로 API 와 웹사이트를 쓰는 여정도 시작했다는 한 줄이 붙는다. 아래 표의 13~15 챕터가 그 자리다.

## 열다섯 챕터에서 손에 남는 것

서문 챕터의 표가 "각 챕터가 무엇을 다루는가"이므로 여기서는 각도를 바꿔 "그 챕터를 지난 뒤 무엇을 판단할 수 있게 되는가"로 적는다. 되짚을 챕터를 고르는 데 쓰면 된다.



| 챕터                       | 지나고 나면 판단할 수 있게 되는 것                                                                                                                            |
|--------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------|
| 1 도메인 모델링                | 명세를 읽으면서 그 내용을 AND 타입(튜플·레코드)과 OR 타입(판별 유니온)으로 옮긴다. <code>bool</code> 플래그를 판별 유니온 케이스로 올려 있을 수 없는 상태를 표현 자체로 막을 시점을 안다                          |
| 2 함수                     | 시그니처만 보고 그 함수를 합성할지 부분 적용할지 정한다. <code> &gt;</code> 를 기본으로 쓰고 <code>&gt;&gt;</code> 가 맞는 자리를 구분한다                                               |
| 3 <code>null</code> 과 예외 | 없음과 실패를 반환 타입에 적는다. 이유를 실어야 하면 <code>Result</code> , 아니면 <code>Option</code> 이다. <code>null</code> 은 다른 .NET 코드와 맞는 경계에서만 만난다                   |
| 4 코드 구성과 테스트             | <code>.fsproj</code> 의 컴파일 순서가 곧 의존 방향이라는 전제로 프로젝트를 나눈다. xUnit 과 FsUnit 으로 테스트를 짠다                                                              |
| 5 컬렉션                    | 반복문 대신 파이프라인으로 데이터를 변환한다. <code>List</code> · <code>Array</code> · <code>Seq</code> 중 무엇을 고를지, 기존 함수로 풀리지 않아 <code>fold</code> 로 내려갈 때가 언제인지 안다 |
| 6 파일에서 데이터 읽기            | 바깥 세계의 입력을 <code>Result</code> 로 감싼다. 지연 평가 때문에 <code>try/with</code> 가 예외를 놓치는 자리를 피하고, 데이터 출처를 매개변수로 빼 테스트한다                                  |
| 7 액티브 패턴                 | 조건 분기와 파싱을 이름 붙인 패턴으로 옮긴다. 네 종류를 반환 타입으로 구분한다                                                                                                   |
| 8 함수형 검증                 | 검증을 확인이 아니라 변환으로 짠다. 첫 오류에서 멈출지 오류를 모을지 고르고, 그에 맞는 조립 방식을 쓴다                                                                                    |
| 9 단일 케이스 판별 유니온          | 뜻이 다른 원시 타입이 서로 뒤바뀌는 실수를 컴파일 시점에 막는다. 타입 약어와 새 타입의 차이를 안다                                                                                       |
| 10 객체 프로그래밍              | 클래스 타입·인터페이스·객체 식이 각각 필요한 자리를 구분한다. 가변 상태를 타입 안에 감춰 불변식을 지킨다                                                                                    |
| 11 재귀                    | 기저 경우에 실제로 닿는지 확인하고, 꼬리 호출인지 본다. 누적값을 나눌지 <code>fold</code> 로 접을지 고른다                                                                           |
| 12 계산 식                  | 효과가 낀 코드에서 해피 패스만 보이게 정리한다. 빌더를 직접 짜 봤으므로 문법 설탕 안쪽에서 <code>Bind</code> 가 무엇을 하는지 안다                                                             |
| 13 Giraffe 입문            | 라우팅이 값이고 핸들러가 함수라는 전제로 웹 애플리케이션의 최소 골격을 세운다. View Engine 으로 HTML 을 F# 코드로 짠다                                                                    |
| 14 Giraffe API           | 읽기와 쓰기 경로를 나누고 상태 코드를 정한다. 요청 본문은 모델 바인딩으로 받고 그 뒤에 검증을 세운다. 저장소는 컨테이너에 싱글톤으로 등록해 핸들러에서 꺼내 쓴다                                                    |
| 15 Giraffe 웹 페이지         | 뷰가 함수라는 점을 살려 레이아웃과 부분 뷰를 함수로 뽑아 조립한다. 정적 파일과 미들웨어 순서를 다룬다                                                                                      |

원서 목록의 항목 중 챕터 제목에 드러나지 않는 것들의 자리는 이렇다. FSI 는 서문 챕터의 F# Interactive 절, 대수적 타입 시스템은 1챕터, 불변성과 `let` 바인딩은 1챕터와 2챕터, `unit` · 커링·부분 적용은 2챕터, 가드 절은 1·7·8챕터, `set` 은 5챕터, 단위 테스트는 4챕터, 네임스페이스와 모듈은 4챕터와 9챕터다. 패턴 매칭은 1챕터가 잡고 7챕터가 넓힌다. 효과라는 이름은 12챕터가 붙이고, 그 아래의 `Option` 과 `Result` 는 3챕터, `Async` 와 `Task` 는 12챕터가 개념을 잡고 13~15챕터가 실습으로 쓴다. 원서 목록에 없지만 함께 찾게 되는 두 가지도 적어 둔다. 익명 함수는 2챕터에 있고, 가변 바인딩은 1챕터가 `<-` 와 함께 처음 다루고 10챕터가 클래스 안에 감춘 가변 상태로 다시 쓰며 13챕터의 Mutability 절이 설정 코드에서 쓰는 자리를 보여 준다.

## F# Software Foundation — 가입 권유 (원서 p.193)

---

- 저자는 이 책의 인세 전액을 F# Software Foundation 에 보낸다고 적는다. 재단이 하는 언어와 커뮤니티 홍보 활동을 지원한다는 뜻이다.
- 가입은 무료다. 가입하면 재단 Slack 에 들어갈 수 있다. 입문자용 채널이 따로 갖춰져 있다는 것이 저자가 드는 실질적인 이점이다. 같은 권유가 서문 챕터에도 한 번 나온다(원서 pp.4-5).
- 링크는 이렇게 확인했다. 재단 사이트 `fsharp.org/` 와 `foundation.fsharp.org/` 는 지금도 응답한다 (2026년 9월 확인). 원서가 각주로 건 가입 경로 `foundation.fsharp.org/join` 은 명령줄에서 받아 오면 403 이 돌아와 확인하지 못했다. 브라우저에서는 열릴 가능성이 높지만 확인한 사실이 아니므로, 재단 사이트에서 가입 항목을 찾아 들어가는 쪽이 확실하다.

## Resources — 원서가 원하는 자료 (원서 pp.193-194)

---

원서는 책 둘, 사이트 다섯, 강의 하나를 든다. 아래 표는 원서 나열 순서를 그대로 두고 확인 결과만 덧붙인 것이다. 확인 시점은 2026년 9월이고, 표에는 원서에 없는 자료를 더하지 않았다. 표 아래 설명에서 두 가지만 보탠다.

| 자료                                               | 종류  | 원서가 건 링크                                                                                                                                      | 확인 결과                                                      |
|--------------------------------------------------|-----|-----------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------|
| Stylish F# 6 (Kit Eason)                         | 책   | <a href="https://link.springer.com/book/10.1007/978-1-4842-7205-3">link.springer.com/book/10.1007/978-1-4842-7205-3</a>                       | 응답은 오지만 봇 확인 화면이 떠서 내용까지는 확인하지 못했다. 저자 이름도 링크에서 확인한 것이 아니다 |
| Domain Modeling Made Functional (Scott Wlaschin) | 책   | <a href="https://pragprog.com/titles/swdddf/domain-modeling-made-functional/">pragprog.com/titles/swdddf/domain-modeling-made-functional/</a> | 살아 있다. 제목과 저자를 확인했다                                        |
| F# Docs                                          | 사이트 | <a href="https://fsharp.github.io/fsharp-core-docs">fsharp.github.io/fsharp-core-docs</a>                                                     | 살아 있다(끝에 슬래시를 붙인 주소로 넘어간다). FSharp.Core API 참조 문서다         |
| F# Software Foundation                           | 사이트 | <a href="https://fsharp.org/">fsharp.org/</a>                                                                                                 | 살아 있다                                                      |
| F# For Fun and Profit                            | 사이트 | <a href="https://fsharpforfunandprofit.com/">fsharpforfunandprofit.com/</a>                                                                   | 살아 있다                                                      |
| F# Weekly                                        | 사이트 | <a href="https://sergeytihon.com/category/f-weekly/">sergeytihon.com/category/f-weekly/</a>                                                   | 살아 있다. Sergey Tihon 의 블로그 안에 있다                            |
| Compositional-IT Blog                            | 사이트 | <a href="https://compositional-it.com/news-blog/">compositional-it.com/news-blog/</a>                                                         | 그 회사 사이트가 아니게 됐다. 자세한 것은 표 아래 설명에 있다                       |
| F# From the Ground Up                            | 강의  | <a href="https://udemy.com/course/fsharp-from-the-ground-up/">udemy.com/course/fsharp-from-the-ground-up/</a>                                 | 봇 확인 화면이 떠서 확인하지 못했다                                       |

- 표에서 손이 가는 항목은 사이트 다섯 중 마지막인 Compositional-IT Blog 다. [compositional-it.com](https://compositional-it.com) 은 지금 그 회사와 무관한 도메인으로 넘어간다. 확인하는 동안 넘어가는 목적지가 한 번 바뀌기도 했다. 도메인이 원래 주인의 손을 떠났을 때 나타나는 모습이다. 원서 각주를 따라 들어갈 자리가 아니다. 같은 팀이 남긴 자료 중 확인한 것은 GitHub 조직 [github.com/CompositionalIT](https://github.com/CompositionalIT) 와 YouTube 채널 [@CompositionalIT](https://www.youtube.com/channel/UCCompositionalIT) 두 곳이다. 원서가 권한 블로그 글 자체는 그 주소에 없다.
- F# Docs 라는 이름은 오해를 사기 쉽다. 원서가 건 주소는 코어 라이브러리 API 참조이고, 언어 안내서와 튜토리얼이 모인 쪽은 Microsoft Learn 의 F# 문서다. 후자는 한국어 번역본도 응답한다 ([learn.microsoft.com/ko-kr/dotnet/fsharp/](https://learn.microsoft.com/ko-kr/dotnet/fsharp/) , 확인했다). 다만 이 노트가 확인한 것은 링크가 살아 있다는 사실까지이고, 번역 품질을 평가한 것은 아니다.
- 원서는 두 번째 책의 제목을 Domain Modelling Made Functional 로 적는데 실제 제목은 Domain Modeling Made Functional 이다. 표는 확인한 제목 쪽을 적었다.
- 원서의 이 목록에는 없지만 15챕터가 각주로 이미 권한 자료가 하나 더 있다. 서버와 브라우저 양쪽을 F# 로 쓰는 SAFE Stack 이다(원서 p.191 각주, [safe-stack.github.io](https://safe-stack.github.io) , 확인했다).

## And Finally — 맺음말 한마디 (원서 p.194)

---

- 마지막 절의 요지는 F# 을 좋은 언어로 만드는 것이 개별 기능만이 아니라 기능들이 서로 맞물리는 방식이라는 것이다. 근거로 전체는 부분의 합보다 크다는 아리스토텔레스의 널리 알려진 문장을 든다.
- 같은 이야기가 원서 서문에도 있다. 원서는 F# 을 고를 다섯 가지 이유를 꼽은 다음 곧바로 개별 기능보다 맞물림이 중요하다고 덧붙이고(원서 pp.1-2), 책을 닫으면서 그 문장으로 돌아온다. 열다섯 챕터가 그 사이의 증명이었다는 구성이다.
- 실제로 열다섯 챕터를 되짚어 보면 한 챕터의 기능만으로 굴러가는 예제가 거의 없다. 8챕터의 검증은 3챕터의 `Result` 와 7챕터의 부분 액티브 패턴과 1챕터의 판별 유니온이 동시에 필요하고, 14챕터의 API 핸들러에는 거기에 10챕터의 클래스 타입과 4챕터의 컴파일 순서까지 얹힌다. 기능이 하나씩 늘 때마다 조합이 배로 늘어나는 것이 저자가 말하는 맞물림이다.

## 다음 걸음 (노트 보충)

- 원서가 남긴 갈래는 셋이다. 첫째는 12챕터가 계산 식을 더 파려면 다른 자료로 넘어가라고 적어 둔 지점이다 (원서 p.164). 둘째는 15챕터가 View Engine 은 걸만 훑었다며 문서를 읽어 보라고 한 지점, 그리고 JavaScript 를 직접 쓰기 싫으면 SAFE Stack 을 보라는 권유다(원서 p.191). 셋째는 이 챕터의 자료 목록이다.
- 계산 식을 더 파고 싶으면 12챕터의 Further Reading 절이 가리키는 두 방향이 있다. 계산 식으로 도메인 특화 언어(DSL, Domain-Specific Language)를 짜는 Saturn 과 Farmer, 그리고 Async 와 Task 의 관계·취소·병행 실행을 다루는 Microsoft Learn 의 비동기 프로그래밍 문서다.
- 언어 쪽을 더 다지고 싶으면 자료 목록의 책 둘 중 하나를 고르는 것이 무난하다. Domain Modeling Made Functional 은 1챕터와 9챕터가 맛만 보여 준 타입 중심 설계를 한 권 분량으로 밀고 나간 책이고, Stylish F# 6 은 이 노트가 곳곳에서 "관례다"라고만 적고 지나간 스타일 판단을 다룬다.
- 실습 쪽으로 이어 가려면 손에 남은 코드가 이미 있다. 13~15챕터에서 세운 HiveLog 다. 세 챕터를 지난 뒤 이 앱의 상태는 두 곳에 있다. 점검 기록은 ConcurrentDictionary 를 감싼 저장소 클래스 안에, 벌통 목록은 Domain.fs 의 최상위 hives 리스트에 있다. 점검 기록은 요청이 바꿀 수 있는 데이터이고 벌통 목록은 표본 데이터이며, 둘 다 메모리에만 있어 프로세스가 끝나면 사라진다.
- 15챕터 노트가 정리 절에서 다음 걸음으로 지목한 것이 이 자리다. 저장소 클래스 뒤에 실제 데이터베이스를 붙이는 일이다.
- 그 작업이 이 시점에 맞는 이유는 저장소의 모양에 있다. InspectionStore 는 밖으로 멤버 넷만 열어 두고 안쪽 사전을 감춘 클래스다(10챕터의 캡슐화). 핸들러는 그 인스턴스를 컨테이너에서 꺼내 쓴다(14챕터의 서비스 로케이션). 그래서 멤버의 시그니처가 그대로라면 안쪽 구현을 SQL 이든 문서 데이터베이스든 다른 것으로 바꿔도 핸들러에는 손댈 곳이 없다.
- 다만 실제 데이터베이스는 호출이 비동기이고 실패할 수 있으므로 시그니처가 그대로 남지 않는다. TryFind 의 반환 타입이 Inspection option 에서 Task<Inspection option> 이나 Task<Result<Inspection, DbError>> 로 바뀐다. 핸들러는 이미 task { ... } 안에서 돌고 있으니 겹데기가 새로 필요한 것은 아니고, 저장소를 부르는 줄에 let! 이 붙는다. 실패까지 담기면 효과가 둘 겹치므로 12챕터의 복합 계산 식 자리가 여기다. Async<Result<...>> 에는 12챕터에서 쓴 asyncResult , Task<Result<...>> 에는 같은 패키지( FsToolkit.ErrorHandling )의 taskResult 를 쓴다. 구현을 둘 이상 두고 같아 끼우려면 저장소를 인터페이스로 뽑는 일이 먼저인데, 14챕터가 그 일을 남겨 두었다.
- 규모를 더 키우기 전에 4챕터로 한 번 돌아가 보는 것도 해 볼 만하다. 13~15챕터의 예제에는 테스트가 없다. HiveLog 의 검증 함수와 저장소에 테스트를 붙여 보면 4챕터에서 배운 것이 실제 프로젝트에서 어떻게 자리를 잡는지 확인할 수 있다.

## 정리

- 이 챕터에서 챙길 것은 되짚기와 방향이다. 새 문법은 없고, 그래서 이 노트에도 실행 단위가 없다.
- 저자의 신조는 순서에 관한 것이다. 돌아가게 만드는 것이 먼저이고 성능은 마지막이다. F# 에서 그 순서를 지탱하는 도구가 FSI 다.
- 원서의 마지막 주장은 원서 서문의 주장과 같다. F# 의 값은 기능 목록에만 있지 않고 기능들이 맞물리는 방식에 있으며, 열다섯 챕터의 예제가 뒤로 갈수록 여러 챕터의 내용을 동시에 요구한 것이 그 증거다.
- 원서 자료 목록은 2023년 1월 기준이다. 여덟 항목 중 다섯은 지금도 살아 있음을 확인했고, 둘은 못 확인 하면 때문에 확인하지 못했으며, [compositional-it.com](https://compositional-it.com) 은 Compositional IT 의 사이트가 아니게 됐다.
- 실습을 이어 갈 자리는 [HiveLog](#) 의 저장소 뒤에 실제 데이터베이스를 붙이는 일이다. 캡슐화와 서비스 로케이션 덕에 멤버의 시그니처가 그대로면 핸들러에 손댈 곳이 없고, 반환 타입이 비동기와 실패를 담게 되면 그 호출 줄에 `let!` 이 붙으면서 12챕터의 복합 계산 식이 필요해진다.

## 원서 대조 표

| 절                              | 원서 페이지     | 실행 단위 |
|--------------------------------|------------|-------|
| Summary — 200쪽 가까이 지나간 자리      | pp.192-193 | —     |
| F# Software Foundation — 가입 권유 | p.193      | —     |
| Resources — 원서가 권하는 자료         | pp.193-194 | —     |
| And Finally — 맺음말 한마디          | p.194      | —     |
| 다음 걸음                          | (노트 보충)    | —     |

## 17 - 부록 1: VS Code 에서 솔루션과 프로젝트 만들기 (원서 pp.195-196)

원서 4챕터(p.52)는 솔루션(solution)과 프로젝트(project)를 먼저 만들어 두라고만 하고 실제 명령은 이 부록으로 넘긴다. 그래서 이 부록은 개념을 설명하는 곳이 아니라 명령을 순서대로 늘어놓은 곳이다. 이 노트도 그 성격을 그대로 지킨다. 솔루션과 프로젝트가 각각 무엇이고 F# 의 컴파일 순서가 왜 `.fsproj` 에 적히는지는 4챕터 노트에 있고, 편집기와 SDK 설치의 서문 챕터에 있다. 여기서는 셸에 무엇을 치면 4챕터 실습 환경이 만들어지는지만 다룬다. 아래 명령은 전부 .NET SDK 10.0.111 에서 실제로 돌려 확인했다.

## 이 부록의 자리 (원서 p.195)

- 원서가 만드는 구조는 솔루션 하나에 프로젝트 둘이다. 코드용 콘솔 프로젝트와 테스트용 xUnit 프로젝트를 각각 `src/` 와 `tests/` 아래에 둔다.
- 원서는 이 절차를 편집기 조작으로 안내한다. 텍스트 파일에 명령을 적어 두고 그것을 선택해 통합 터미널로 보내는 방식이다. 명령 자체는 `dotnet` CLI 이므로 편집기 없이 셸에서 그대로 돌려도 결과가 같다.
- 이 노트는 프로젝트 이름을 4챕터 노트와 맞춰 `ShopSolution` , `Shop` , `ShopTests` 로 쓴다. 원서의 `MySolution` , `MyProject` , `MyProjectTests` 자리에 그대로 대응한다. 이름은 무엇으로 잡아도 되지만, 콘솔 프로젝트와 테스트 프로젝트의 디렉터리 이름이 곧 프로젝트 이름이 되고 그 이름이 어셈블리 이름까지 결정한다는 점만 기억하면 된다.

## VS Code 쪽 조작 — 실행으로 확인할 수 없는 부분 (원서 pp.195-196)

아래 항목은 편집기 기능이라 셸에서 확인할 수 없다. 절차는 원서를 따라 적었고, 적어 둔 단축키는 현재 VS Code 문서의 기본값이다.

- 빈 디렉터리에서 시작한다. 그 디렉터리에서 `code .` 로 열면 거기가 VS Code 의 작업 폴더가 된다.
- 통합 터미널을 새로 연다. VS Code 의 기본값은 `CTRL+SHIFT+백틱` 이고, 이미 열린 패널을 접었다 펴는 것은 `CTRL+백틱` 이다. 원서가 적은 것은 `CTRL+SHIFT+'` 로 백틱이 아니라 따옴표다. 원서가 어느 키를 가리켰는지는 확인할 방법이 없으니 편집기 기본값을 따르면 된다.
- 명령을 적어 둔 파일에서 프로젝트 이름을 한꺼번에 바꿀 때 `CTRL+F2` 를 쓴다. 선택한 낱말과 같은 낱말 전부를 동시에 편집하는 Change All Occurrences 기능이다.
- 선택한 텍스트를 터미널로 보내는 길은 둘이다. Terminal 메뉴의 Run Selected Text 항목을 고르거나, `CTRL+SHIFT+P` 로 명령 팔레트를 열고 Terminal: Run Selected Text in Active Terminal 을 고른다.
- Ionide 확장 설치의 서문 챕터에서 다뤘으므로 여기서 반복하지 않는다. 부록 1 자체는 확장을 언급하지 않는다.

## 한 번에 돌리는 스크립트 (원서 pp.195-196)

원서처럼 명령을 파일에 모아 두는 방식을 쓴다면 확장자를 `.txt` 대신 `.sh` 로 잡고 셸에 파일째로 넘기는 편이 안전하다. 선택 텍스트를 터미널로 보내는 방식은 앞 명령이 실패해도 뒤 명령이 계속 실행되기 때문이다. 맨 앞의 `set -euo pipefail` 이 그 문제를 막는다. 다만 이 줄은 파일에 넣어 `bash setup.sh` 로 돌릴 때만 쓴다. 통합 터미널에 골라 보내면 뒤이어 실패하는 명령 하나에 그 터미널이 닫힌다.

```
# setup.sh - 빈 디렉터리에서 bash setup.sh 로 실행한다
set -euo pipefail

dotnet new sln -o ShopSolution
cd ShopSolution
mkdir src tests

dotnet new console -lang "F#" -o src/Shop
dotnet new xunit -lang "F#" -o tests/ShopTests
dotnet sln add src/Shop/Shop.fsproj tests/ShopTests/ShopTests.fsproj

dotnet add tests/ShopTests/ShopTests.fsproj reference src/Shop/Shop.fsproj
dotnet add tests/ShopTests/ShopTests.fsproj package FsUnit.xUnit --version 7.1.1

dotnet build
dotnet test
```

- 원서 스크립트와 달라진 점 가운데 짚어 둘 것이 셋이다. 첫째, 테스트 프로젝트로 `cd` 해 들어가는 대신 `dotnet add <프로젝트 경로> reference|package` 형태를 썼다. 스크립트가 끝난 뒤 셸이 어디에 서 있는지 헷갈릴 일이 없다. 둘째, `FsUnit` 패키지를 넣지 않고 `FsUnit.xUnit` 하나만 넣었다(이유는 4챕터 노트에 적었다). 셋째, 버전을 고정했다.
- 이 밖에 `mkdir` 두 줄과 `dotnet sln add` 두 줄을 각각 한 줄로 묶었다(`dotnet sln add` 를 한 번만 불러도 되는 이유는 아래 절에 적었다).
- 스크립트를 파일로 만들었다면 실행한 뒤 지워도 된다. 만들어진 것은 솔루션과 프로젝트 쪽이고 스크립트 자체는 남을 필요가 없다.
- 이 스크립트는 같은 자리에서 두 번 돌릴 수 없다. `dotnet new sln` 이 기존 `.slnx` 를 덮어쓰겠다는 확인을 요구하며 종료 코드 73 으로 멈추고, `set -e` 때문에 뒤 명령은 실행되지 않는다. 메시지가 안내하는 `--force` 를 붙이면 `dotnet new sln` 이 기존 파일을 덮어쓰고 넘어간다. 스크립트를 부분적으로 골라 다시 돌리면 `mkdir src tests` 도 실패한다. 다시 만들 때는 솔루션 디렉터리를 지우고 처음부터 돌리는 것이 깔끔하다.
- CLI 메시지는 셸 로케일을 따라 번역돼 나온다. 아래 인용은 `DOTNET_CLI_UI_LANGUAGE=en` 을 붙여 얻은 영어 출력이다.

## 명령을 하나씩 확인하기 (원서 pp.195-196)

솔루션 파일부터 만든다.

```
dotnet new sln -o ShopSolution
cd ShopSolution
```

- 만들어지는 파일은 `ShopSolution/ShopSolution.slnx` 다. 내용은 빈 껍데기 두 줄이다.

```
<Solution>
</Solution>
```

- 예전 `.sln` 형식이 필요하면 `dotnet new sln -f sln -o ShopSolution` 처럼 형식을 지정한다. 그러면 `ShopSolution.sln` 이 생긴다. 두 형식 중 무엇을 써도 이 부록의 나머지 명령은 같다.
- 템플릿 짧은 이름은 `dotnet new list --language "F#"` 출력의 Short Name 칸에서 확인한다. 이 부록에서 쓰는 것은 `console` 과 `xunit` 이다.



디렉터리 둘과 프로젝트 둘을 만들어 솔루션에 넣는다.

```
mkdir src tests
dotnet new console -lang "F#" -o src/Shop
dotnet new xunit -lang "F#" -o tests/ShopTests
dotnet sln add src/Shop/Shop.fsproj tests/ShopTests/ShopTests.fsproj
```

- 여기까지 생긴 파일은 넷이다. `src/Shop/Program.fs`, `src/Shop/Shop.fsproj`, `tests/ShopTests/Tests.fs`, `tests/ShopTests/ShopTests.fsproj`.
- `dotnet sln add` 는 프로젝트 경로를 여러 개 받으므로 한 번만 호출하면 된다. 넣고 나면 `.slnx` 가 이렇게 채워진다. 경로의 `src`, `tests` 를 보고 같은 이름의 솔루션 폴더를 만들어 준다.

```
<Solution>
  <Folder Name="/src/">
    <Project Path="src/Shop/Shop.fsproj" />
  </Folder>
  <Folder Name="/tests/">
    <Project Path="tests/ShopTests/ShopTests.fsproj" />
  </Folder>
</Solution>
```

- 이 폴더 항목은 편집기의 솔루션 탐색기에 보이는 분류일 뿐이다. F# 의 컴파일 순서를 결정하는 것은 각 프로젝트의 `.fsproj` 이고, 솔루션에 프로젝트를 적은 순서와는 무관하다.
- 경로에서 솔루션 폴더를 만드는 것은 SDK 10 에서 새로 생긴 동작이 아니다. `dotnet sln add` 의 `--in-root` 가 이 동작을 끄는 옵션이고 기본값은 끄지 않는 쪽이다. `-f sln` 으로 만든 `.sln` 에도 같은 폴더가 `SolutionFolder` 항목과 `NestedProjects` 절로 들어간다. SDK 10 에서 달라진 것은 그 폴더를 적는 표기뿐이다.
- 등록 결과는 `dotnet sln list` 로 확인한다. 같은 프로젝트를 두 번 넣으면 `already contains` 메시지만 찍히고 종료 코드는 0 이다. 스크립트를 부분적으로 골라 다시 돌려도 이 명령은 걸림돌이 되지 않는다.

테스트 프로젝트에 참조와 어서션(assertion) 라이브러리를 붙인다.

```
dotnet add tests/ShopTests/ShopTests.fsproj reference src/Shop/Shop.fsproj
dotnet add tests/ShopTests/ShopTests.fsproj package FsUnit.xUnit --version 7.1.1
```

- 두 명령을 돌린 뒤 `tests/ShopTests/ShopTests.fsproj` 에서 패키지 `<ItemGroup>` 과 프로젝트 참조 `<ItemGroup>` 이 이렇게 된다. `FsUnit.xUnit` 은 템플릿이 만들어 둔 패키지 `<ItemGroup>` 안에 이름 순으로 끼어들고, 새 `<ItemGroup>` 을 얻는 것은 프로젝트 참조 쪽뿐이다. 파일 앞쪽의 `<Compile Include="Tests.fs" />` 항목은 그대로다.

```
<ItemGroup>
  <PackageReference Include="coverlet.collector" Version="6.0.4" />
  <PackageReference Include="FsUnit.xUnit" Version="7.1.1" />
  <PackageReference Include="Microsoft.NET.Test.Sdk" Version="17.14.1" />
  <PackageReference Include="xunit" Version="2.9.3" />
  <PackageReference Include="xunit.runner.visualstudio" Version="3.1.4" />
</ItemGroup>

<ItemGroup>
  <ProjectReference Include="..\..\src\Shop\Shop.fsproj" />
</ItemGroup>
```

- 블록에 적힌 템플릿 패키지 버전은 SDK 10.0.111 의 xUnit 템플릿 기준이다.
- 원서 스크립트는 패키지 이름을 `FsUnit.XUnit` 으로 적었다. NuGet 은 패키지 이름의 대소문자를 구분하지 않으므로 그대로 써도 복원은 되고, CLI 가 등록된 이름인 `FsUnit.xUnit` 으로 바꿔 적어 준다. 이 노트와 4 챕터 노트는 등록된 이름 쪽을 쓴다.
- 참조 방향은 테스트 프로젝트에서 코드 프로젝트로 한 방향뿐이다. 반대로 걸면 어떻게 되는지는 4챕터 노트에 적었다.

빌드하고 테스트를 돌린다. 둘 다 솔루션 디렉터리에서 실행하면 솔루션에 든 프로젝트 전부가 대상이 된다.

```
dotnet build
dotnet test
```

- 빌드는 경고 없이 끝나고 어셈블리 둘이 나온다. `src/Shop/bin/Debug/net10.0/Shop.dll` 과 `tests/ShopTests/bin/Debug/net10.0/ShopTests.dll` 이다.
- 잡히는 테스트는 xUnit 템플릿이 만들어 둔 `My test` 하나다. 여기까지 오면 환경이 완성된 것이다.

```
Passed! - Failed: 0, Passed: 1, Skipped: 0, Total: 1, Duration: 29 ms - ShopTests.dll (net10.0)
```

- 콘솔 프로젝트를 돌리려면 `dotnet run --project src/Shop` 을 쓴다. 앞의 `dotnet add` 와 같은 이유로 `cd` 를 피한 형태이고, 4챕터 노트가 쓴 `cd src/Shop` 뒤 `dotnet run` 과 결과가 같다. 템플릿이 넣은 한 줄이 `Hello from F#` 을 출력한다.

## 만든 환경에 파일을 엮어 보기 (노트 보충)

절차가 제대로 끝났는지는 파일을 하나씩 더해 보면 확실해진다. 4챕터의 실습이 바로 이 단계에서 시작한다. 아래 함수는 그 자리에 넣어 볼 가장 작은 예다.

```
// 이 단위가 보여주는 것: src/Shop/Shipping.fs 에 넣을 순수 함수와 그 결과 확인
// 실제 파일이라면 첫 줄이 module Shop.Shipping 이지만, FSI 로 검증하려고 선언 없이 적었다

// float -> int
let feeFor weightKg =
    if weightKg <= 0.5 then 2500
    elif weightKg <= 5.0 then 4000
    else 4000 + int (ceil (weightKg - 5.0)) * 800

printfn "%-7s %d" "0.4kg" (feeFor 0.4) // 기대: 0.4kg 2500
printfn "%-7s %d" "3.0kg" (feeFor 3.0) // 기대: 3.0kg 4000
printfn "%-7s %d" "7.2kg" (feeFor 7.2) // 기대: 7.2kg 6400
```

- 마지막 값은 이렇게 나온다. 5kg 초과분 2.2kg 을 `ceil` 로 3kg 으로 올리고 800 을 곱해 2400 을 얻은 뒤, 기본 요금 4000 에 더한다.

이 코드를 `src/Shop/Shipping.fs` 로 저장했다면 `Shop.fsproj` 의 컴파일 목록에 적어야 한다. 자리는 `Program.fs` 앞이다. `Program.fs` 에서 `feeFor` 를 부른다면 정의가 앞에 와야 하고, 부르지 않아도 순서를 뒤집으면 빌드가 실패한다. 템플릿이 만든 `Program.fs` 에는 모듈 선언이 없다. 선언을 생략할 수 있는 파일은 마지막 하나뿐이므로 `Shipping.fs` 를 뒤에 적으면 오류 FS0222 가 난다. 이 규칙은 4챕터 노트에 있다.

```
<ItemGroup>
  <Compile Include="Shipping.fs" />
  <Compile Include="Program.fs" />
</ItemGroup>
```

- 템플릿이 만든 `.fsproj` 와 `.fs` 는 BOM 이 붙은 UTF-8 이다( `.slnx` 는 BOM 이 없고 `-f sln` 으로 만든 `.sln` 에는 있다). BOM 을 떼도 빌드는 성공하므로 인코딩 자체가 문제가 되지는 않는다. 걸리는 자리는 스크립트로 문자열을 치환하는 대목이다. BOM 세 바이트가 첫 줄 맨 앞에 있어서 `<Project` 처럼 줄 앞을 잡는 패턴이 아무 말 없이 안 맞는다. 편집기로 고칠 때는 이 문제가 없다.
- 파일 생성과 목록 등록을 한 번에 하려면 Ionide 의 F# 솔루션 탐색기를 쓴다. 이 기능과 컴파일 순서 규칙은 4챕터 노트에서 다뤘다.

테스트 쪽도 같다. `tests/ShopTests/ShippingTests.fs` 를 만들고 `ShopTests.fsproj` 의 목록에 `Tests.fs` 다음으로 적는다. 이 블록은 `namespace` 로 시작하므로 `.fsx` 로는 돌릴 수 없다(오류 FS0010). 그래서 실행 단위에서 빼고 파일 모양만 적었다.

```
namespace ShopTests

open Xunit
open FsUnit.Xunit
open Shop.Shipping

module ``배송비를 계산하면`` =

    [<Fact>]
    let ``0.5kg 이하는 소형 요금이다`` () =
        feeFor 0.4 |> should equal 2500

    [<Fact>]
    let ``5kg 를 넘으면 초과분에 kg 당 요금이 붙는다`` () =
        feeFor 7.2 |> should equal 6400
```

- `open` 할 이름은 `FsUnit.Xunit` 이다. 원서 본문의 `open FsUnit` 은 NUnit 쪽이라 이 프로젝트에서는 `should` 가 보이지 않는다. 4챕터 노트에 자세히 적었다.
- 다시 `dotnet test` 를 돌리면 템플릿 테스트까지 셋이 통과한다. 여기까지 확인했으면 부록의 역할은 끝이다.

```
Passed! - Failed: 0, Passed: 3, Skipped: 0, Total: 3, Duration: 62 ms - ShopTests.dll (net10.0)
```

## 원서와 달라진 점 (노트 보충)

원서는 2023년 1월 판이고 아래 표는 SDK 10.0.111 에서 실측한 결과다. 원서 이후 달라진 것과 이 노트가 달리 잡은 것을 함께 적었다. 명령 이름과 순서는 원서 그대로 유효하다.

| 항목                                | 원서                                                                              | SDK 10.0.111                                                  |
|-----------------------------------|---------------------------------------------------------------------------------|---------------------------------------------------------------|
| <code>dotnet new sln</code> 산출물   | <code>.sln</code>                                                               | <code>.slnx</code> (XML). 예전 형식은 <code>-f sln</code>          |
| <code>dotnet sln add</code> 결과 표기 | <code>.sln</code> 의 SolutionFolder 항목과 NestedProjects 절                         | <code>.slnx</code> 의 <code>&lt;Folder Name="/src/"&gt;</code> |
| 대상 프레임워크                          | <code>net5.0</code> (원서가 인쇄한 유일한 값. 원서가 쓴 SDK 6.0.x 를 따르면 <code>net6.0</code> ) | <code>net10.0</code>                                          |
| 어서션 패키지                           | <code>FsUnit</code> 과 <code>FsUnit.XUnit</code> 둘                               | <code>FsUnit.xUnit</code> 7.1.1 하나로 충분하다                      |

## 정리

- 이 부록의 알맹이는 여덟 단계다. 솔루션 만들기, 디렉터리 둘 만들기, 프로젝트 둘 만들기, 솔루션에 넣기, 참조 걸기, 패키지 넣기, 빌드, 테스트.
- 원서처럼 명령을 파일에 모아 둘 때는 `.sh` 로 만들어 `set -euo pipefail` 을 맨 앞에 두고 파일째로 실행한다. 선택 텍스트를 터미널로 보내는 방식은 중간에 실패해도 멈추지 않는다.
- `cd` 대신 `dotnet add <프로젝트 경로> reference|package` 형태를 쓰면 스크립트가 끝난 뒤 셸 위치가 그대로 남는다.
- SDK 10 에서 달라진 것은 `dotnet new sln` 의 산출물이 `.slnx` 라는 점이다. `dotnet sln add` 가 경로에서 솔루션 폴더를 만드는 것은 예전 `.sln` 에서도 같고 적는 표기만 다르다. 이 폴더 항목은 분류일 뿐이고 F# 컴파일 순서와 관계가 없다.
- 어서션 패키지는 `FsUnit.xUnit` 하나로 충분하고, xUnit 테스트에서 여는 이름은 `FsUnit.Xunit` 이다.
- 이 부록은 개념을 설명하는 곳이 아니다. 산출물을 짚는 데 필요한 만큼만 규칙을 언급했다. 솔루션과 프로젝트의 경계, `.fsproj` 의 컴파일 순서, 네임스페이스와 모듈, 테스트 작성은 4챕터 노트로 간다.

## 원서 대조 표

| 절                               | 원서 페이지     | 실행 단위                    |
|---------------------------------|------------|--------------------------|
| 이 부록의 자리                        | p.195      | —                        |
| VS Code 쪽 조작 — 실행으로 확인할 수 없는 부분 | pp.195-196 | —                        |
| 한 번에 돌리는 스크립트                   | pp.195-196 | —                        |
| 명령을 하나씩 확인하기                    | pp.195-196 | —                        |
| 만든 환경에 파일을 엮어 보기                | (노트 보충)    | <code>17-shipping</code> |
| 원서와 달라진 점                       | (노트 보충)    | —                        |